

VISI Documentation

VISI

April 2026

Technical documentation VISI 1.8

<https://teec2.nl/visi>

DigiGO Standard

1 May 2025

Last published version:	https://nl-digigo.github.io/visi/visi1.8/
Last work version:	https://nl-digigo.github.io/visi
Author:	Visi committees (digiGO), translation TEEC2 BV
Join the discussion:	GitHub https://github.com/nl-digigo/visi Report an issue Revision history Pull requests

Table 1: Version information

This document is licensed under a Creative Commons Attribution 4.0 License

Abstract

This document contains the technical documentation of VISI 1.8, under translation by TEEC2.

Status of this document

Working version 2026

Contents

1	Summary	7
2	Introduction	8
2.1	Purpose of this documentation	8
2.2	Reader guide	9
2.3	VISI Basics	9
2.3.1	VISI Framework	9
2.3.2	Transaction	10
2.3.3	Role	11
2.3.4	Organisation	11
2.3.5	Project	11
2.3.6	Message	11
2.3.7	Appendices of VISI-messages	12
3	Conceptual	12
3.1	Why Use the VISI Standard	12
3.2	Management structure	13
3.2.1	Management structure VISI Open Standard TC130607-02	13
3.2.2	Backwards compatibility	15
3.3	User perspective	16
3.3.1	Execution	16
3.3.2	Preparation	17
3.4	VISI structures	17
3.4.1	Examples of a project by means of VISI	18
3.5	VISI terminology	20
3.5.1	Transaction	20
3.5.2	Organisation	25
3.5.3	Project	25
3.5.4	Role	25
3.5.5	Message	26
3.6	Communicating based on a VISI framework	27
3.7	Details of the framework	28
3.8	User guarantees	29
4	Project initiation	30
4.1	Bookmark	30
4.2	Project specific file	30
4.2.1	Project	30
4.2.2	Person	31
4.2.3	Organisation	31
4.2.4	Role	32
4.2.5	Relationship between Person, Role and Organisation	32
4.2.6	Personnel changes	33
4.2.7	Project Setup	34
4.3	Framework/Communication template of a project	34
4.3.1	Role Type (RoleType)	34
4.3.2	Information transfer conforms to transaction type (TransactionType)	35
4.3.3	Message Type (MessageType)	36
4.3.4	Message Fields (SimpleElementType)	36
4.3.5	Special Fields	37
4.3.6	Data type of an element (UserDefinedType)	38
4.3.7	Table as element of a message type	39
4.3.8	Grouping of elements (ComplexElementType)	39

4.3.9	Order of message types (MessageInTransactionType)	40
4.3.10	Conditions for elements of message type (ElementCondition)	45
4.3.11	Conditions for subsequent message types (MessageInTransactionTypeCondition)	45
4.3.12	Order of messages	47
4.3.13	Appendix type (AppendixType)	48
4.3.14	Organisation type (OrganisationType)	49
4.3.15	Person Type (PersonType)	49
4.3.16	Project Type (ProjectType)	49
4.3.17	Transaction Phase Type (TransactionPhaseType)	50
4.3.18	Group Type (GroupType)	50
4.4	VISI Attributes	50
4.4.1	id	50
4.4.2	description	51
4.4.3	state	51
4.4.4	dateLaMu	51
4.4.5	userLaMuo	51
4.4.6	language	51
4.4.7	helpInfo	51
4.4.8	code	52
4.4.9	appendixMandatory	52
4.4.10	appendixTypes	52
4.4.11	baseType	52
4.4.12	complexElements	52
4.4.13	condition	53
4.4.14	executor	53
4.4.15	firstMessage	53
4.4.16	initiator	53
4.4.17	initiatorToExecutor	53
4.4.18	maxOccurs	53
4.4.19	minOccurs	53
4.4.20	namespace	53
4.4.21	openSecondaryTransactionsAllowed	53
4.4.22	received	54
4.4.23	responsibilityFeedback	54
4.4.24	responsibilitySupportTask	54
4.4.25	responsibilityTask	54
4.4.26	sequence	54
4.4.27	xsdRestriction	54
4.5	Guidance for Element Conditions	55
4.5.1	Implementation of element conditions	55
4.6	Delegation and mandating	57
5	Project execution	58
5.1	Message communication	58
5.1.1	Order	58
5.1.2	MessageType	58
5.1.3	Message from person to person	58
5.1.4	Sending a message	59
5.2	Attachments	60
5.2.1	Mandatory Attributes	60
5.2.2	Optional properties	60
5.2.3	Additional properties	60
5.2.4	Is an appendix required?	62
5.2.5	Transport of an attachment	62
5.2.6	Validation	62
5.2.7	Behavior message fields	63

5.2.8	Changes to a framework	63
5.3	Changing framework	65
5.4	Behaviour of software for initiating a transaction or replying to a message	65
5.4.1	Behaviour of software around connected message flow	65
5.4.2	Behaviour around offering and not offering messages based on framework/role	65
5.5	Archiving VISI Projects	66
5.5.1	The requirements for the VISI archive	66
5.5.2	Project archive application	66
5.5.3	VISI export	67
5.6	META FRAMEWORK	71
5.6.1	The available roles and associated responsibilities	71
5.6.2	The available transactions within the META framework	72
5.6.3	Application of the META framework	72
5.7	Appendix 6 Metaframework – Rationale	72
5.7.1	Introduction	72
5.7.2	Framework modification constraints	73
5.7.3	Design rationale	73
5.8	Appendix 6 Metaframework – Normative specification	73
5.8.1	Scope	73
5.8.2	Transactions	74
5.8.3	Behavioural rules	74
5.8.4	Message flow (high level)	74
5.8.5	Meta-framework XML	75
5.8.6	Scenario for using meta framework and meta project specific message	85
5.9	Interactive information model	87
5.10	Appendix 7: Guidelines for Successor	87
5.10.1	Rules for ‘Successor’	87
5.10.2	Examples	88
6	Technical	89
6.1	TBD	89
6.2	Appendix 10 Additional functional requirements	89
6.2.1	Additional functions and requirements for the VISI quality mark	89
6.3	Appendix 13 Reducing frames using optional fields	89
6.3.1	Part 1 (_2.exp): VISI-framework	91
6.3.2	Part 2 (_5.exp): VISI messages and project-specific message	91
6.3.3	Test result	91
6.4	Appendix 14: Validating frameworks	93
6.4.1	General principles:	93
6.4.2	With passive messages	93
6.4.3	Without Passive messages	94
6.5	Guideline for implementation of the complete DEMO transaction pattern (with VISI Standard version 1.6)	94
6.5.1	Implementation of the complete DEMO transaction pattern with the VISI Standard 1.6	95
6.6	DEMO transaction pattern	95
6.7	Definition of Framework	99
6.7.1	DEMO or VISI framework?	99
6.7.2	DEMO transaction pattern in VISI: default 4 states	99
6.7.3	DEMO revokes in VISI: separate transactions	99
6.7.4	Asynchronous & revokes	99
6.8	DEMO transaction with open standard VISI	99
6.8.1	Fixed pattern for a VISI transaction	99
6.8.2	Mandatory transactionPhase and predefined values	102
6.8.3	Skip messages/transaction phases	102
6.8.4	Sub transations	104

6.8.5	Fact Bank per transaction	105
6.9	Revokes in a Transaction Tree	106
6.9.1	Guideline for framework designers	107
6.9.2	Software Guidelines	108
6.9.3	Revoke of main transaction and open sub transactions	108
6.9.4	Content of revoke messages	109
6.9.5	Revokes & sub transactions	109
6.9.6	Summary	109
6.10	DEMO Crossing Messages (and order of events)	110
6.10.1	Crossing revoke and message of main transaction	110
6.10.2	Crossing revokes	111
6.11	Getting started with DEMO Transaction Pattern	112
6.11.1	Prerequisite	112
6.11.2	Start with a VISI framework	112
6.11.3	Start from scratch	113
6.12	Attachment A: Translate VISI to DST method	113
6.12.1	Method for converting a VISI framework to the DEMO standard transaction pattern	113
6.13	Attachment B Translate VISI to DST example	116
6.13.1	Example: Converting UAV2012 transaction changes to DEMO standard transaction pattern	116
6.14	DEMO	123
6.14.1	DEMO transaction pattern (#60) research	123
6.14.2	Revoke Message (#76)	126
6.15	Data Model	130
6.15.1	Attributes	131
6.16	Systematics Part 2 Messages	134
6.16.1	Templates	134
6.16.2	Elements	155
6.16.3	References	162
7	Message Communication	165
7.1	Introduction	165
7.2	Goal	166
7.3	Architecture	166
7.4	Protocols	167
7.5	Message exchange	168
7.5.1	Message exchange of fixed elements at framework level	168
7.5.2	Message exchange of fixed elements at message level	169
7.5.3	Consequences	169
7.5.4	Message exchange initialisation	169
7.5.5	Message exchange scenario	169
7.6	Disclosure of the framework and update scenario	172
7.7	Disclosure of the configuration scenario	172
7.8	Attachments, Ref's en Id's	173
7.9	SOAP function calls	173
7.10	Configuration	173
7.11	Optional implementation suggestions	173
7.12	Visi werk sessie 16 mei 2023	173
7.12.1	leidraad 1.7	173
7.13	Message Signing	174
8	4. Implementation using an example	179
8.1	VISI version 1.6	179
8.1.1	4.1 Introduction	179
8.1.2	4.2 Example speciality restaurant	179
8.2	Appendix 4 Detailed Implementation Example	182

8.2.1	Introduction	182
8.2.2	Description of example (file “ 7.xml”)	182
8.2.3	Elaboration of messages	212
8.3	Procedure for applying for a VISI quality mark	251
8.3.1	Application for quality mark	251
8.3.2	Test	252
8.3.3	VISI quality mark and VISI certificate	253
8.3.4	Time expenditure and costs	253
8.4	VISI Quality mark Test	254
8.4.1	VISI Software conformance test	254
8.4.2	Test Environment	254
8.4.3	Test	254
8.4.4	Scenario’s	255
8.4.5	VISI 1.4 features	256
8.4.6	VISI 1.5 features	256
8.4.7	VISI 1.6 features	256
8.4.8	Additional requirements	257
8.5	5. VISI Quality mark	258
9	Testprotocol	259
9.1	Test framework VISI 1.8 - Scenario overview	259
9.2	TransactionTypes in TestFramework	260
10	Conformity	275
10.1	Phasing out versions of the VISI Standard	275
10.1.1	Binding Directive	275
10.1.2	Background	276
10.1.3	Practical situations	276
10.1.4	Certification and management	277
10.1.5	Implementation	277
10.2	Version Management	278
10.3	Important changes in the VISI system	278
10.3.1	Introduction; Release Cycle	278
10.3.2	Important changes in version 1.2	278
10.3.3	Changes in version 1.3 vs version 1.2	278
10.3.4	Changes in version 1.4 vs version 1.3	280
10.3.5	Changes in version 1.6 vs version 1.4	280
10.3.6	Changes in version 1.7 vs version 1.6	281
11	Release Policy	282
11.1	Governance bodies	282
11.1.1	User Committee	282
11.1.2	Expert Committee	283
11.1.3	Steering Committee	284
11.2	Proof of Concept	285
11.3	Release Process	285
11.4	Process Constraints	288
A	Index	288
A.1	Terms defined by this specification	288
A.2	Terms defined by reference	292
B	VISI Glossary	292
B.1	Terms used in VISI standard	292
B.1.1	Message (Bericht)	292
B.1.2	Message Category (Berichtcategorie)	292

B.1.3	Security Certificate (Beveiligingscertificaat)	292
B.1.4	Complex (Data) Element (Complex (gegevens)element)	292
B.1.5	DEMO-method	292
B.1.6	Executor	292
B.1.7	Generic Framework (Generiek raamwerk)	292
B.1.8	Initiator	292
B.1.9	Interaction Diagram (Interactiekaart)	292
B.1.10	Project Specific Framework (Projectspecifiek raamwerk)	293
B.1.11	Promotor	293
B.1.12	New promotor (#55)	293
B.1.13	Framework validator (#10)	293
B.1.14	REST (#148)	293
B.1.15	TestServer	293
B.1.16	Framework (Raamwerk)	293
B.1.17	Role Description (Rolbeschrijving)	293
B.1.18	Role (Rol)	293
B.1.19	Assigning Roles (Rolverdeling)	293
B.1.20	Cooperation Concept (Samenwerkingsconcept)	293
B.1.21	Simple (Data) Element (Simpel (gegevens)element)	294
B.1.22	SSL	294
B.1.23	Transaction (Transactie)	294
B.1.24	Transaction Model (Transactieschema)	294
B.1.25	Transaction State (Transactietoestand)	294
B.1.26	VISI	294
B.1.27	VISI Communication (VISI-communicatie)	294
B.1.28	VISI-compatible software	294
B.1.29	VISI interaction diagram (VISI-interactiekaart)	294
B.1.30	(VISI-certificate (VISI-keurmerk)	294
B.1.31	VISI Supplier (VISI-leverancier)	294
B.1.32	VISI-Framework (VISI-raamwerk)	294
B.1.33	VISI-standard (VISI-standaard)	295
B.1.34	VISI-system (VISI-systematiek)	295
B.1.35	VISI-website	295
B.1.36	W3C	295
B.1.37	XML	295
B.1.38	XSD	295

C References 295

C.1	Informative References	295
C.2	FAQ	295
C.2.1	What is the first MessageType in a TransactionType?	295
C.2.2	What is the last MessageType in a TransactionType?	295
C.2.3	If a message has been sent in a transaction, is it allowed to send another message?	296
C.2.4	How is the DATETIME data type handled when no time zone is specified?	296
C.2.5	If I have promoted a framework and generated a message schema, do I have a guarantee that this framework will work in practice?	296
C.2.6	If I relate transactions to each other, is the traceability of communication always guaranteed?	296
C.2.7	7. How do I indicate in a framework whether a message is a start message or not?	297

1 Summary

VISI is the open standard for organising digital communication about your collaboration. People agree in advance on when and what messages they will communicate with each other. They then exchange these agreed-upon messages during the collaboration. If several organisations work together, they can each use

their own VISI-certified software. Message exchange between software vendors is guaranteed. In this way, each party can, in its own software, fully understand mutual communication.

A characteristic of VISI is its flexibility: both the workflow and the message content can be set up specifically for each organisation, project or collaboration. In the event of a dispute, it is immediately clear how the collaboration went. VISI offers an audit trail because agreements are recorded digitally in messages. Decisions are recorded in the context of a document version or provided information.

2 Introduction

VISI: What is it and what is its goal?

VISI is a set of agreements that supports collaboration through digital communication.

The philosophy behind VISI:

1. Joint work agreements. Always ensures clarity about who should do what or who decided what at what time;
2. Up-to-date and complete communication files. This provides insight into the collaboration's progress and prevents communication disruptions.
3. Organisations shift from document-driven work to data- and process-driven working.

With the help of VISI, organisations gain more control over and become more mature in managing and structuring projects and collaborations.

Construction project example: We do this by thinking in advance about the communication that takes place between the Client and the Contractor in accordance with, among other things, the UAV and UAVgc contract forms, with the addition of the contract documents containing agreements about communication. This is translated into messages in a pre-agreed workflow with associated response times. This allows all parties to concentrate on the content and implementation of the project during the project.

2.1 Purpose of this documentation

The construction sector is the owner of the open standard VISI. DigiGO manages the VISI standard. DigiGO is advised by an expert committee. This expert committee includes software vendors who have implemented VISI. They are in close contact with the users, who in turn ask for new functionalities, which may lead to adjustments to the standard. The various software developers are each responsible for (and owners of) their own implementation of the VISI standard within their software applications. The VISI standard ensures that software packages can communicate with each other, and DigiGO guarantees this. In a project or collaboration, organisations can work with different software packages. By making the documentation accessible, more software vendors can implement the VISI standard.

Target audience

The conceptual documentation is for software vendors or system administrators who want to learn more about the VISI standard and how it is applied. End users of organisations that use VISI for communication and other interested parties can visit the website of the DigiGO for more user-oriented documentation and application examples.

The functional and technical documentation of the VISI standard is intended for parties who want to implement the standard in their software. Software vendors provide the information systems that are suitable for communicating based on VISI. In order to make the documentation suitable for readers at the product management level as well as the software implementation level, it has been decided to divide the documentation into a conceptual, functional, and technical part.

2.2 Reader guide

The documentation is divided into three parts: conceptual, functional, and technical. These parts are explained in the following paragraphs. Two examples are used at each level in the documentation: a construction project, with a collaboration between a client and a contractor; and a pizzeria, with a customer, a waiter, and a cook.

The conceptual part describes the following:

1. How does VISI work for the user (both in execution and preparation);
2. structure using VISI;
3. main VISI terms;
4. communication based on a VISI framework;
5. deepening of the VISI framework;
6. guarantees to the user.

2.3 VISI Basics

When partners work together (for example, in a project), clear mutual communication is an essential condition for smooth progress. VISI provides tools for making communication agreements.

The basic principles of VISI are based on the DEMO method developed by Prof. Dr. ir. J.L.G. Dietz of the TU Delft see *“Introductie tot DEMO”, 1996, Dietz, J.L.G., Samson, Alphen aan den Rijn; ISBN 9789014053271*. Many VISI concepts are derived from this method.

Communication agreements are laid down in a structure called a 'framework'. The following section describes the components of the VISI framework. Each part of the framework is explained.

2.3.1 VISI Framework

In accordance with the VISI system, communication agreements are laid down in a structure called the 'VISI framework'. The terms used in the VISI frameworks follow the VISI jargon for clarity. Important terms in the VISI system are the following:

- Transaction
- Transaction scheme
- Transaction State
- Interaction map
- Organisation
- Project
- Role
- Message
- Appendix

The building process can be represented as a network of interrelated roles between which transactions take place. The schematic representation of this network, in fact, the 'framework', is also referred to as the 'interaction map' ¹.

It's possible to distinguish a pattern of recurring transactions that can be applied to all construction projects in a certain application area. For example, contracts based on uniform administrative conditions. Knowing this, a model VISI framework can be created for each application. Some standard frameworks can be used as a starting point, such as UAV, UAVgc, management & maintenance. Standard frameworks can also be created for other processes (the number of possible frameworks is, in fact, unlimited). These generic frameworks can be tailored to a specific use case/application/project.

¹The notation of the VISI interaction map is based on Prof. Dietz's model. The notation differs from the Business Process Modelling Notation (BPMN, a standard for process modelling), on the one hand, to keep interaction maps as simple as possible and, on the other hand, because the concept of 'transaction' is not defined within BPMN. The latter is a common, but not an official Open Standard.

2.3.2 Transaction

When two parties agree that work must be done, or that something else must be done, there is a "transaction". Each transaction has an initiator (the role that initiates the transaction) and an executor (the role that executes the transaction). fig. 1 shows a transaction. The arrow always runs from the initiator to the executor.

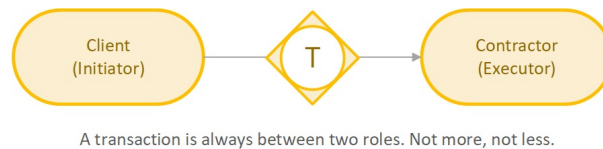


Figure 1: There are always two roles involved in a transaction: never more, never less

An example is the situation in which a municipality asks an engineering firm to design a viaduct. The municipality performs the role of commissioning party and is the initiator of the transaction; after all, it initiates the transaction. The engineering firm takes on the role of designer; it executes the transaction and is thus an executor.

A transaction goes through a life cycle. The status of a transaction, say, how far the transaction has progressed, is indicated by transaction states. Transactions are initiated, requested, promised, executed, completed, and accepted in sequence.

Within a transaction, messages are included to handle formal communication between roles. For example, messages are developed to give orders, report changes or deviations, report progress, and send readiness notifications.

2.3.2.1 Transaction scheme

A transaction scheme describes how communication between involved parties is structured to achieve a specific objective. It outlines the interactions that take place during a transaction, providing a high-level view of how participants coordinate and exchange information, rather than detailing the full conceptual model. For example, a customer places an order with the waiter; the waiter passes it to the cook; the cook prepares the pizza; and the waiter notifies the customer when it is ready.

2.3.2.2 Transaction state

Transactions typically progress through a series of predefined states that describe their lifecycle. These states provide a structured way to track the status of a transaction from initiation to completion.

- Start
- Requested
- Promised/Execution
- Change/Hold
- Declare/Ready
- Accept/End

A transaction begins in the 'start' state and evolves through subsequent states as parties exchange messages and perform actions. Each state reflects a particular phase in the process, such as requesting, executing, adjusting, or completing the transaction.

Transitions between states are triggered by communication between the involved parties. Depending on the situation, a transaction may move forward, pause for changes, or return to a previous state before continuing.

Ultimately, once the transaction has been completed and accepted, it reaches its final state.

2.3.2.3 Interaction Map

A VISI interaction map is a simple diagram that shows the roles in a process and how they interact through transactions. Each transaction always involves one role that starts the transaction (initiator) and one role that carries it out (executor).

The main purpose of the map is to focus on how roles connect and interact, not on the detailed steps within each role. Because the roles are described abstractly, the map can be used for all construction projects, regardless of contracts or collaboration styles.

The map includes all necessary roles and transactions needed for the project to work. Each role and transaction has its own name and number. Role names are based on their main task in the process

2.3.3 Role

In the VISI system, various responsibilities and associated tasks occur within (construction) projects and forms of cooperation. Think of a simple agreement with a Contractor and Client with agreed tasks and responsibilities. These responsibilities with associated tasks are bundled into logical roles. The concept of how parties interact and the agreement they have with each other influences the distribution of roles among the parties.

2.3.4 Organisation

An organisation is uniquely identified within a project by a globally unique identifier. In addition to this identifier, an organisation has a descriptive name intended for human recognition and a three-letter abbreviation. The abbreviation is unique within the project and serves both a functional and technical purpose. It can, for example, be used to derive the organisational source of communication and to distinguish organisations in automated processing and integrations.

2.3.5 Project

A project forms the central context for all communication within VISI. Because projects involve coordination between multiple participants, communication plays a key role in managing activities, responsibilities, quality, planning, and resources throughout the project lifecycle.

2.3.6 Message

A transaction consists of messages that are necessary to handle the formal communication between the roles. A message is, in fact, a completed 'form' with data, where the data in the fields of the form is stored in a database. A framework specifies for each message which data elements it contains. A message is of a specific message type and is inextricably linked to a transaction (although a message type can be associated with several transaction types).

A message contains the data structure prescribed in the VISI framework. The message has a "header" that includes details of the involved roles, the transaction, and the work to which the transaction relates. The body of the message contains the information to be transferred, consisting of completed data elements, attached documents and references to documents. The content of a message (or attachment) can contribute to the BIM of a project.

By working with VISI, this information transfer is brought into the (formal) process context.

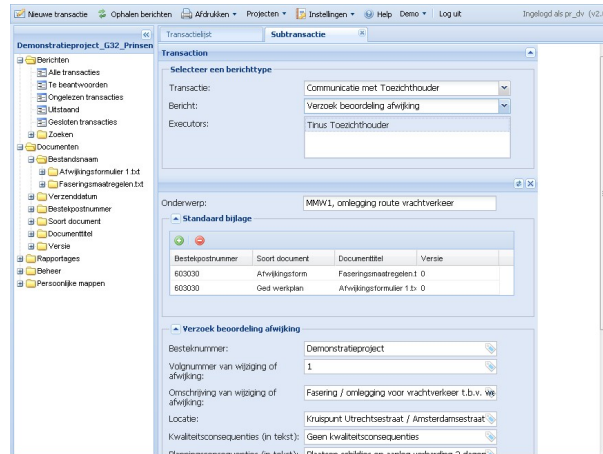


Figure 2: Example of a message from a transaction scheme

2.3.7 Appendices of VISI-messages

Attachments can be linked to VISI messages. This is very similar to an 'attachment' to an e-mail. An important difference is that an attachment of a VISI message is provided with a specification. This is done by entering several data elements. In addition, VISI messages are archived with their attachments, so that they can always be found.

VISI messages are an important tool for transferring documents and files. The type of message and the content of a message determine why the document is transferred from one to another (meaning in the process). The specification of the attachments is important for document management.

There are frameworks in which multiple attachments ('AppendixTypes') are created for different types of attachments. Because there can be many types, and the user should not be burdened with the choice of the correct AppendixType, it is possible to indicate in a framework which AppendixType may be selected and when.

3 Conceptual

3.1 Why Use the VISI Standard

In projects that span multiple phases, it often becomes difficult to keep track of what has been done, which version is the most recent, and what has actually been agreed upon. Miscommunication can easily arise, especially when different parties rely on different versions of documents or have conflicting interpretations of prior agreements.

The VISI standard addresses this issue by introducing a structured messaging system. This system defines the communication framework in advance, ensuring that all parties agree on how information will be exchanged before the project begins. Within this predefined framework, every message is formally recorded, organised, and traceable.

A key principle of VISI is that communication is structured in two steps:

1. First, the parties define what topics will be communicated about.
2. Then, the actual communication takes place within that agreed structure.

All messages exchanged are:

- Stored systematically

- Recorded in chronological order
- Linked to specific subjects or agreements
- Clearly version-controlled, with predefined rules about which version is authoritative

The VISI standard originates from a legal perspective, where disputes often arise due to ambiguity in agreements. For example, in construction projects, parties may disagree about responsibilities or previously made decisions, often due to missing, outdated, or inconsistent documentation. VISI helps prevent such conflicts by ensuring that every communication can be verified and traced.

Although similar communication could take place via email, email lacks structure, consistency, and enforceable rules. VISI, on the other hand, provides a formalised and structured environment where:

- Communication is predefined
- Content and responsibilities are clearly defined
- All interactions are documented and legally supportable

In summary, VISI is a communication standard that ensures transparent, structured, and verifiable communication throughout a project. It enables all stakeholders to clearly understand what has been agreed upon, when it was agreed, and by whom. By providing a traceable record of communication, it supports legal clarity, reduces the risk of disputes, and improves overall project coordination.

3.2 Management structure

3.2.1 Management structure VISI Open Standard TC130607-02

In recent years, the VISI Standard has developed into a mature, operational Open Standard. This includes a demand-driven approach for further development, management and maintenance. This approach has been introduced since 2009 in accordance with the BOMOS model. DigiGO is the administrator of the VISI Standard and monitors this approach on behalf of the sector. As the administrator, DigiGO also coordinates updates to the VISI Standard. The VISI Technical Committee (VISI TC) plays an important substantive role in this. The content of (the scope of each new release of) the sector representation determines the Standard.

Role of the Technical Committee

The VISI TC contributes to the construction-wide development of the VISI system and ensures the connection of the VISI system to user wishes. The TC consists of experts who participate in the committee free of charge (namely: VISI-certified software suppliers, VISI advisors, and a representative/liaison from the User Group). The committee is chaired by DigiGO. This facilitates the committee and supplements it with a representative of the independent testing body (TNO) that operates on behalf of DigiGO.

The tasks of the TC are not sector-related and consist of:

- Technical development, documentation (requirements, guidelines) and implementation of user wishes in the VISI Standard
- Technical advice on development and management of the VISI Standard.

The standard will be formally published by DigiGO after adoption by the sector representation (i.e. the VISI Steering Committee).

The output of the TC consists of technical adjustments/improvements to the Open Standard (VISI system, requirements and/or guidelines; normative or informative) that are implemented by the Software suppliers in their VISI-compatible software. After (a new release of) this Standard has been implemented

in the software, it is tested by DigiGO and 'certified' after a successful test. TNO is the independent testing body commissioned by DigiGO. The costs of the test are borne by the relevant software supplier ².

Release Policy

Users can continuously express wishes and needs. DigiGO collects the points for improvement and bundles them. Improvements to the Open Standard are implemented in new releases. At the request of VISI users and VISI suppliers, DigiGO uses an annual release cycle, where the scope is flexible, but the deadlines are strictly observed (this will be elaborated further below). In this way, expectations are well managed.

The scope of each new release is determined based on the input from VISI Users, Advisors and Suppliers. The User Group channels the wishes and needs of users, which are ultimately bundled and submitted to DigiGO. Users are involved in consultations. The User Group consists of representatives of organisations that apply the VISI Standard with conviction, operates on a non-sector basis, and ensures the exchange of knowledge between the various users of the VISI Standard.

The collection of improvement points is prioritized for each release, from which the scope of the new release follows. The TC members assign a priority to each improvement point for each release. The resulting 'average value' determines the priority that the TC applies to each improvement point. In this way, the points for improvement are given the best possible balanced order according to importance. Depending on the feasibility, this results in the subset of improvement points that forms the scope of the release. This subset is developed by the TC into a new version of the VISI Standard (after consultation with the Users).

The sector representation (the VISI Steering Committee) ultimately determines the scope of each new release. Improvement points that cannot be included in the new release can, depending on the priority, already be considered by the TC for inclusion in a subsequent release.

Process

The creation of a new Version of the VISI Standard takes place in the following steps:

1. DigiGO collects and registers improvement points from Users, Software Suppliers and Advisors. Points for improvement can be reported directly to DigiGO.
2. The User Committee channels the wishes and needs of the users wherever possible.
3. In consultation with the TC, DigiGO compiles a subset of the reported improvement points. This subset is prioritized in such a way that there is a distinction in importance between all improvement points in the subset. In addition, the complexity of all points is also determined.
4. The Technical Committee is technically developing the subset into a new version of the VISI Standard. This also includes the complete and unambiguous technical documentation for the VISI Guidelines.
5. The Technical Committee categorizes this list and examines whether the points fit within the knowledge and expertise of the committee, and/or need to be split into smaller points for improvement. If necessary - if there really is no other option, for example because feasibility is at stake - the TC can re-prioritize the improvement points. The Technical Committee provides explicit feedback to DigiGO of the final work list with points for improvement.
6. The Technical Committee addresses the improvement points of the prioritized subset one by one in order of the established priority. An improvement point is only completed if the documentation of the VISI Standard has been completely updated, including complete technical documentation in the Guidelines (exp, xsd), and an associated test scenario.
7. The progress of the Technical Committee is transparent to the Users.

²<http://www.crow.nl/nl/VISI/VISI-Systematiek/VISI-Keurmerk-en-Certificaat.html>

8. After six months, the Technical Committee bundles the completed improvement points into Release Candidate 1 (RC1).
9. DigiGO links the contents of RC1 back to the Users.
10. After one year, the Technical Committee bundles all completed improvement points into Release Candidate 2 (RC2).
11. DigiGO links the content of RC2 back to the Users.
12. The sector representation determines the scope of RC2 as the scope of the new Release. DigiGO provides feedback on the content of this formal Version to the Users and publishes the formal version in the VISI Guidelines.
13. The Software Suppliers will implement the full RC2 within six months and must also successfully pass the inspection by the testing authority within this period. The findings when implementing the improvement points are fed back to the Technical Committee by the Software Suppliers on a point-by-point basis.
14. The Technical Committee discusses the findings in the context of the next release(s).

The principle of 'time-boxing' applies to the above process (see figures 1 and 2). There are fixed delivery dates that occur annually.

All areas for improvement are worked on in a central, publicly accessible work environment. The following is recorded for each improvement point: (unique) ID number, name, description, priority, status, date of first report, by whom reported, planned version, and a brief description of the solution. Comments can be made for each improvement point and attachments can be administered. The complete solution is unambiguously technically documented in the VISI Guidelines.

The work environment provides insight into the points for improvement, functional and technical documentation (including guidelines), and source files (exp, xsd) for each Version.

Finally, it is important that the testing scenario is adjusted if necessary, so that the VISI-compatible software can also be tested for the implemented improvement points.

Planning

The overall timetable for the annual release cycles is as follows:

3.2.2 Backwards compatibility

Backwards compatibility relates to the functionality/operation of the VISI Standard (and/or frameworks according to this standard)

Definition

The "backwards compatibility" of a new VISI Standard means that the operation of the new VISI Standard (and/or a framework according to this standard) has at least the same operation as the previous version of the VISI Standard (and/or a framework according to this standard). framework according to this standard).

With others, if functionality is lost, or if behavior is changed, there is no backwards compatibility.

A new VISI Standard (and/or a framework according to this standard) does not have to be backwards compatible. This is a choice, a deliberate choice.

Process

Date	Responsible	Actions
April 1	Product Manager	Compiling a prioritized list of Stakeholders' improvement points for the upcoming Version
May 31	Working group	Discussing, dividing, determining complexity of list of improvement points - Updating list
June 1	Product Manager	Public consultation with Stakeholders on improvements to be implemented
September 30	Working group	Developing points for improvement into functional specifications, designs and technical solutions
October 1	Working group	Publication of completed improvement points up to Release Candidate 1
March 31	Working group	Developing points for improvement into functional specifications, designs and technical solutions
March 31	Working group	Publication of completed improvement points up to Release Candidate 2
August 31	Software Suppliers	Adjusting end user software and framework editor software in accordance with Release Candidate 2 - Feedback of findings per improvement point
August 31	Advisors	Adjusting framework editor software in accordance with Release Candidate 2 - Feedback of findings per improvement point
September 1	Working group	Compiling advice to the Administrator for final Version
September 30	Product Manager	Publishing Formal Version
September 30	Software Suppliers	Obtaining VISI quality mark in accordance with the testing procedure for the VISI quality mark (by inspection body)
September 30	Group of Stakeholders	Commissioning of formal Version

Figure 3: Yearly Release Cycle

For each improvement point, the topic of "backwards compatibility" is included and it is analyzed whether the technical solution has an impact on the backwards compatibility of the VISI Standard.

It is not a prerequisite that a technical solution must necessarily be backwards compatible.

3.3 User perspective

For VISI application users, VISI functions as an inbox or task manager, displaying which messages require responses, what responses are anticipated, and agreed-upon deadlines.

3.3.1 Execution

Send messages and keep appointments

1. VISI focuses on fulfilling agreements.

Users can easily identify their required actions and initiate message flows for specific transactions if they have the correct role. They can also view response options and anticipate incoming messages.

2. It is clear whether a message has arrived.

Due to the agreed-upon message traffic protocol, users can confirm message delivery.

3. Transferring work to a new employee is seamless.

Because an employee has a role, a successor can be appointed during a collaboration. From that moment on, this successor takes over the message flow. It is expected that, in a VISI application, the successor also has the messages of the predecessor available.

4. The communication can be found without mutual dependence.

If several organisations work together, they can each use their own VISI software. Due to the implemented message traffic protocol, two people in different organisations each have access to the information within their organisation and the message traffic at the intersection of the other organisation in their own VISI software.

5. With VISI, changes in the collaboration agreements are always traceable.

During the project, the agreements about the workflow, message content, roles and responsibilities can be changed.

3.3.2 Preparation

Flexibility of VISI in making collaboration agreements

1. Agreeing on tasks and responsibilities in advance clarifies expectations.

During the preparation of a collaboration, it is important to determine the roles of each team member and the tasks they are responsible for. This includes clearly assigning responsibilities so that everyone knows what is expected of them and how they contribute to the overall objective. By setting these agreements beforehand, confusion is reduced, and the collaboration can proceed more efficiently.

2. Agreeing on communication flows in advance allows you to anticipate responses and decisions.

During the preparation of a collaboration, it is important to determine the roles of each team member and their communication channels. This includes deciding on the message flow and the type of information that is expected from each other. By setting agreements and expectations beforehand, everyone involved knows their responsibilities and can work more effectively. Moreover, it is essential to decide on the content of each message, including any attachments that need to be shared. This ensures that the messages are clear, concise, and relevant to the collaboration.

3. Parties have the freedom to shape the cooperation as desired.

VISI is very flexible: The parties have the freedom to structure the collaboration as is suitable for their project. VISI standardises the way in which you digitally record roles, workflows and messages. This requires software packages to be prepared to be able to read and activate these flexible agreements per project or collaboration, so that VISI-based communication can be started.

3.4 VISI structures

In order to allow the exchange of information to take place in a structured manner, digital communication agreements have been established within VISI. The ease of use is comparable to, for example, sending e-mails or offering documents on a shared SharePoint environment. The difference is that the contents of the messages are predefined. For example, a message contains editable fields, such as a form. The sender can add attachments to the messages. At the end of a project, it is still clear which version of an attachment/file has been approved and which ones have been denied.

It is also considered in advance who communicates with whom based on which responsibility. This is expressed in so-called roles.

Transactions take place between those roles to achieve a certain result. For this, we use the term 'transaction'. We record all of this in a model called the VISI framework. During the implementation phase, project participants can focus on communication content rather than contract agreements.

- Be sure that you are in the right place at the right time correct process.
- Immediately see what information is available (who already said something)
- Immediately see what information is needed and how the process continues.
- Immediately see how much time you have to respond

- Ensure that you are the right person to participate in the process
- Certainty that everything is done with the right mandate handled with a digital signature
- Certainty that everything is secured for the future in accordance with the archive law

3.4.1 Examples of a project by means of VISI

3.4.1.1 Example 1: Request for Change fig. 4

The interaction starts with the Contractor. The Client may reject or approve the proposal with a request for a quote.

If the Contractor receives a rejection, the process to record a contract change is terminated.

If the proposal is approved, the Contractor will provide an offer with the consequences of the contract amendment.

After this, the Client can send a rejection again, or approval, after which the contract change is stopped or accepted, respectively.

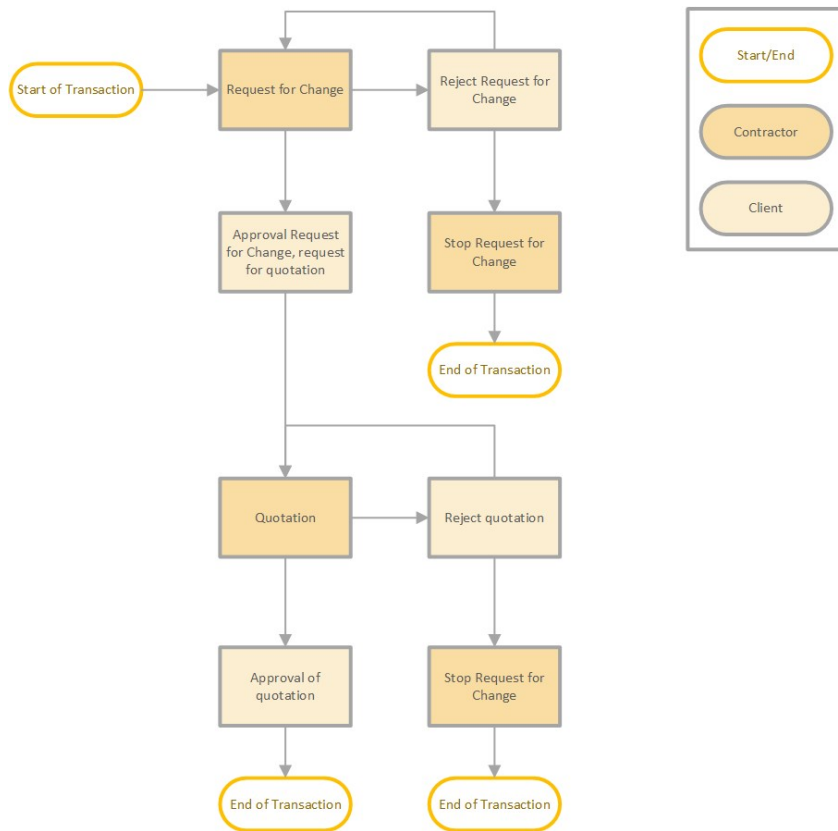


Figure 4: Contract change

3.4.1.2 Example 2: Pizzeria Waiter-Customer fig. 5

The interaction starts with the Waiter. The Customer has a limited number of response options.

If the Customer does not wish to see a menu, the interaction will stop.

If the Customer does want to see a menu, the transaction continues.

The scope of this transaction is to place an order.

Another transaction could be that the Customer asks for the bill from the Waiter, after which the Waiter brings the bill.

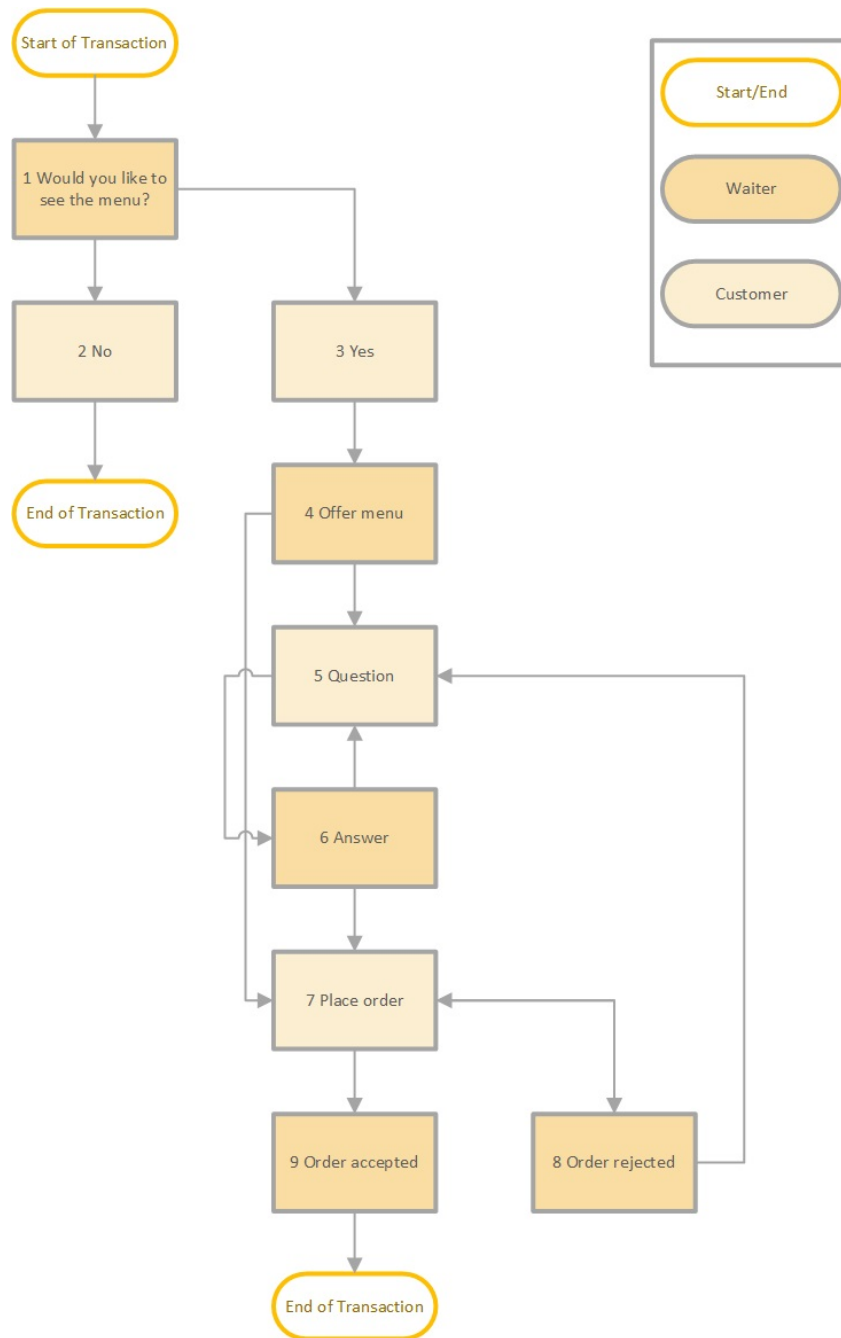


Figure 5: Pizzeria Customer Waiter

3.4.1.3 Example 3: Pizzeria Waiter-Cook fig. 6 The waiter places the order with the cook. The Cook can accept or refuse the order. Depending on the response of the Cook, a response is given to the Customer. For example, a question about order. It is structured in such a way that the Customer cannot communicate directly with the Cook.

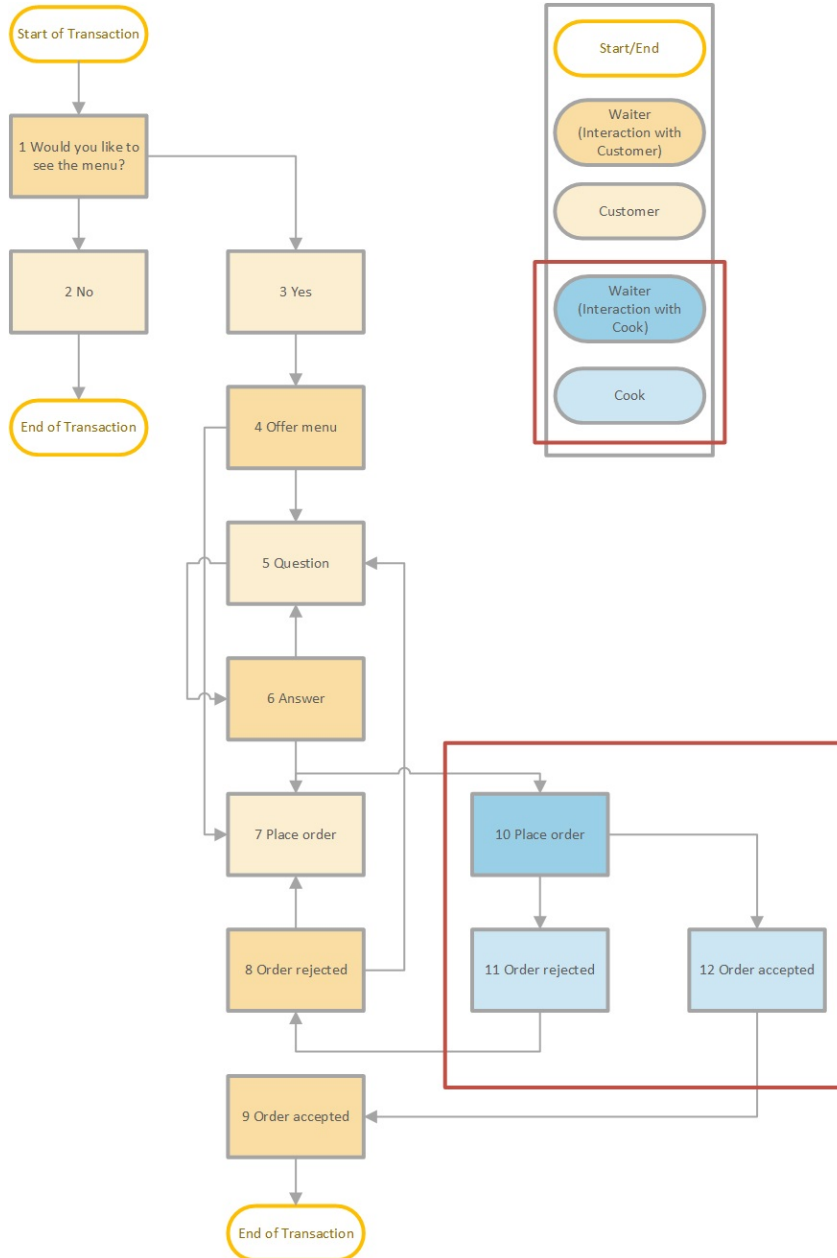


Figure 6: Pizzeria Waiter Cook

3.5 VISI terminology

3.5.1 Transaction

The messages are also grouped by subject and thus always form small transactions. Procedures, i.e. the subject about which one wants to communicate, can in general always be chosen independently of each other. In VISI, we call such a procedure a transaction. The word transaction here refers to 'an agreement'. A transaction can be a request to do or deliver something, often on the basis of pre-arranged agreements. Within VISI, the so-called transaction is central: a binding agreement between two roles that agree on what must be done. In practice, a transaction is a series of messages between two roles, resulting in a certain result. fig. 1

The framework defines which order of messages can be used when communicating about the transaction. The order of messages always goes between roles: usually after a message from role A, a message from

role B follows, after which it is role A's turn again, until the end of the transaction. With the application of DEMO patterns and transaction phases (fig. 11), it is possible to send two messages as role A to role B. For example, in the event of a promise message and a declare message. The VISI-compatible software will interpret the possible routes from the framework.

Example 4: Question-Response

fig. 7 shows a basic Question/Response transaction. Role A sends a Question to role B. Role B responds with either Response X or Response Y.

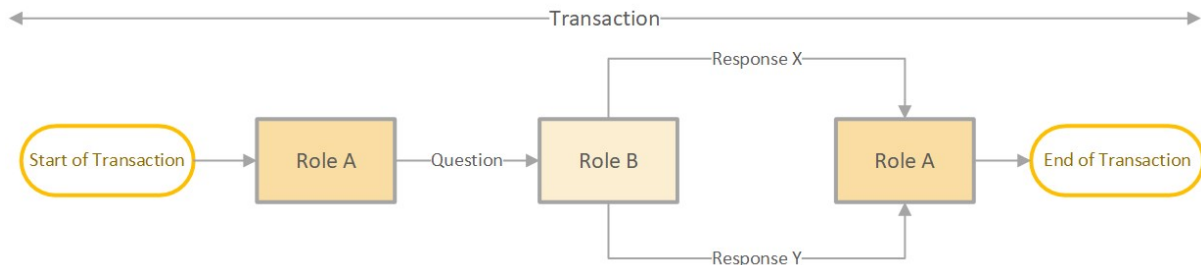


Figure 7: Transaction structure

Example 5: Construction project

The Project Provider submits a document for approval, as agreed in the contract. The Client may reject the document, after which a new document will be submitted. If the Client approves the document for approval, the transaction is completed.

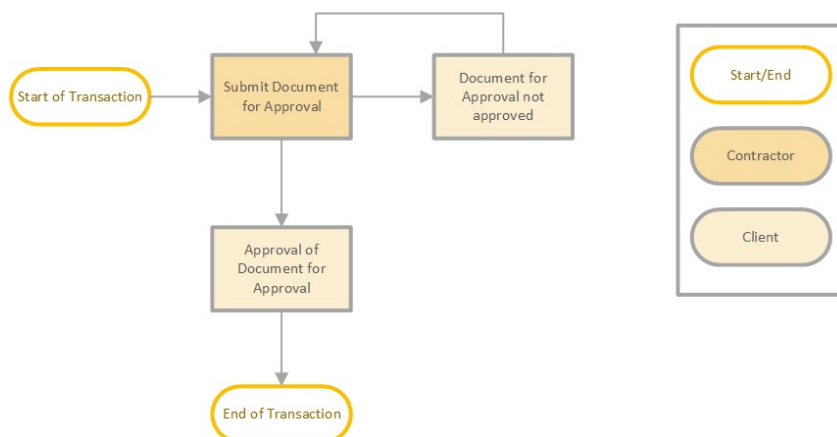


Figure 8: Document for approval

Example 6: Construction project

The Project provider must submit a document for approval as agreed in the contract. The Client wishes to ask one or more advisers to assess the document. We call this a connected transaction or sub-transaction. This transaction again takes place between two roles (no more, no less).

Example 7: Pizzeria

Another example is a waiter's request to a cook to prepare a certain pizza (Ask), after which the cook goes to work, and the relevant pizza is reported to the waiter as ready (Answer X) or it is reported that the pizza is ready. ingredients are not available (Answer Y), after which the transaction is completed.

3.5.1.1 Transaction scheme

Transactions are performed by roles sending each other messages. A transaction contains a set of messages that are exchanged for a specific reason. The transaction also leaves undisputed which roles participate

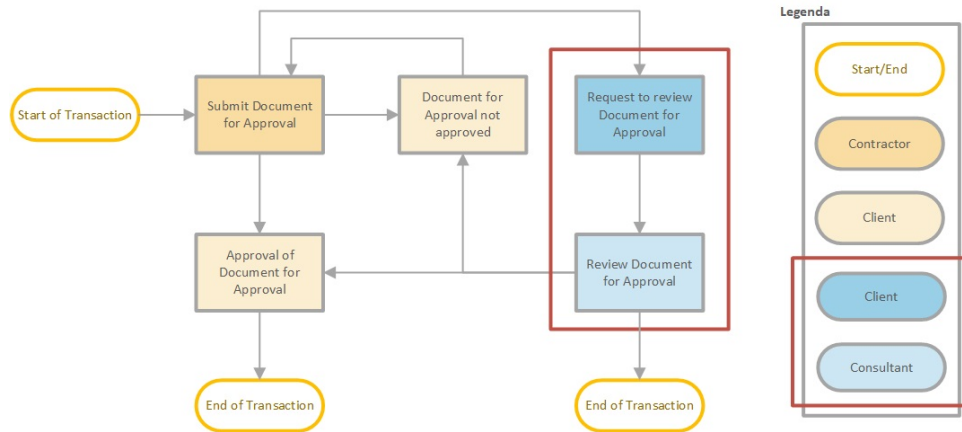


Figure 9: Document for approval- internal

(and who is initiator or executor), at which point of the life cycle communication takes place and – if applicable – in which order the messages should be sent. The list of messages that can be sent as part of a transaction is called a transaction scheme. The transaction scheme naturally contains the name, description and desired outcome of the transaction.

As an example of a transaction scheme (in UML notation), fig. 10 deals with the request for a 3D model. The transaction can only be initiated by role R1 with the message 'Request for 3D model'. The 3D engineer (role R3) can reply with the message 'Work done and request for approval'. After the message 'Work approved' or 'Work not approved', the transaction is completed.

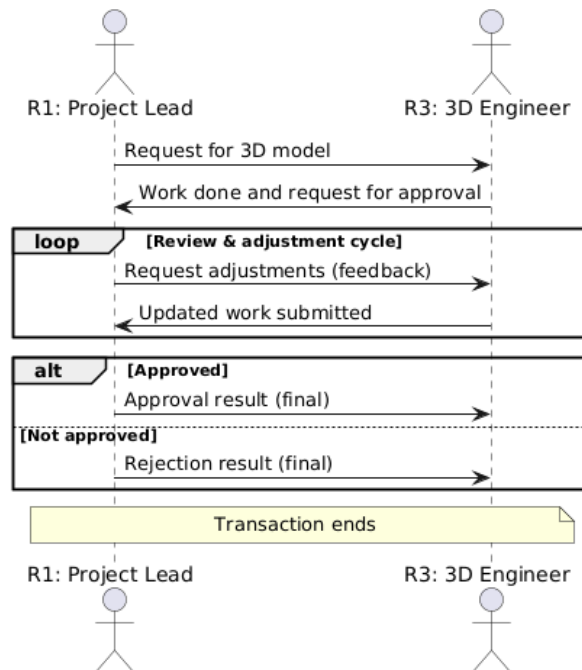


Figure 10: Example of the Interaction Map of a Simplified Design Office

The order of messages is such that, as far as possible, related messages are placed in a logical order. For example, before a confirmation of the order can be sent, the order must first have been issued.

The settlement of transactions takes place according to a fixed pattern and has six different states. Certain

messages can put the transaction in a different state. Arrows indicate in which direction the message is going (from initiator to executor role or vice versa).

The schemes for the different transactions are widely applicable: the messages usually cover a fair share of the communication required between roles. However, the parties have the freedom to adjust transactions (and therefore transaction schemes) to their specific needs by mutual agreement. More about this in the section 'messages'.

3.5.1.2 Transaction state

The settlement of a transaction takes place by means of messages and according to a fixed pattern. Six states are distinguished in the settlement of most transactions:

- Start
- Requested
- Promised/Execution
- Change/Hold
- Declare/Ready
- Accept/End

A transaction enters 'start' when the initiator sends the first message to the executor. This is preceded by a process in which both parties agreed upon the content and conditions of the transaction. The initiator sends the executor an instruction (1) requesting the status of the transaction. With the sending of an order confirmation (2) by the executor, the status changes to 'promised / execution'. Within 'promised / execution', various messages (3) are available for, for example, reporting changes and deviations, delivering progress reports, as well as requests for and feedback from quality tests (fig. 11).

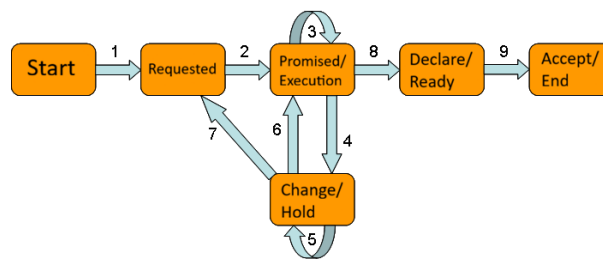


Figure 11: The different transaction states

With the notification of order changes (by the initiator) or deviations and consequences (by the executor), the transaction status changes to 'change/hold' (4). When the initiator has reported a change, the executor can respond by reporting the consequences of the order change (5). The initiator can reject this (6) or approve it with an additional command (7), which is then confirmed by the executor (2). In either case, the transaction will return to the 'promised/executed' state.

With a ready report (8), the executor indicates that the order has been completed, and the transaction ends up in the 'report ready' state. After acceptance by the initiator (9), the transaction enters the 'accepted/finished' state, and the transaction is completed.

3.5.1.3 Interaction Map

A (VISI) interaction map is the schematic representation of a framework. It is intended to map the relevant roles and transactions for an existing process. As mentioned, VISI distinguishes between roles that initiate a transaction (initiator) and roles that execute a transaction (executor). A transaction has only one initiator and one executor.

The added value of a transaction map is that it focuses on the interfaces between roles. The details of the interactions between those roles have not yet been discussed. That will come later. The often complex

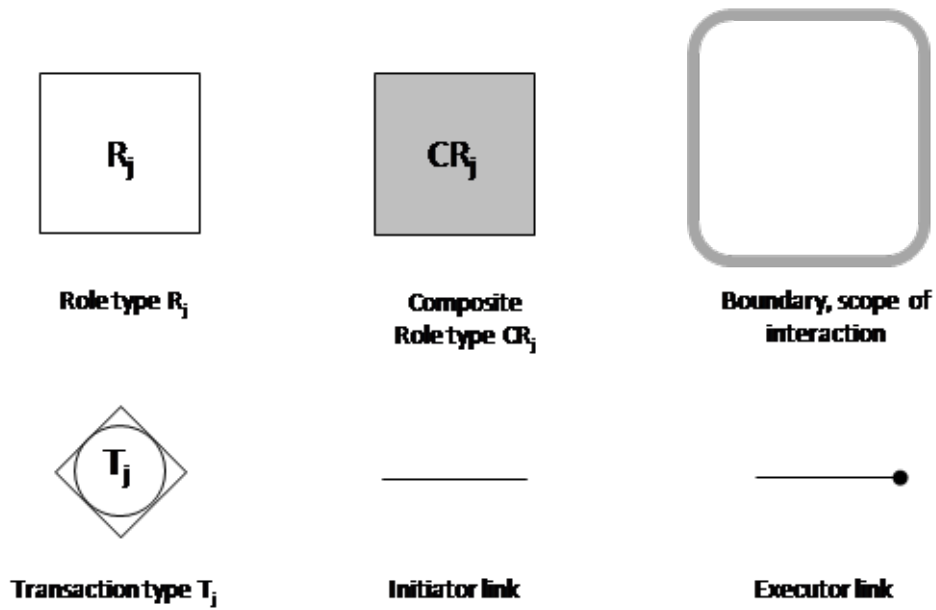


Figure 12: Legend for an interaction map

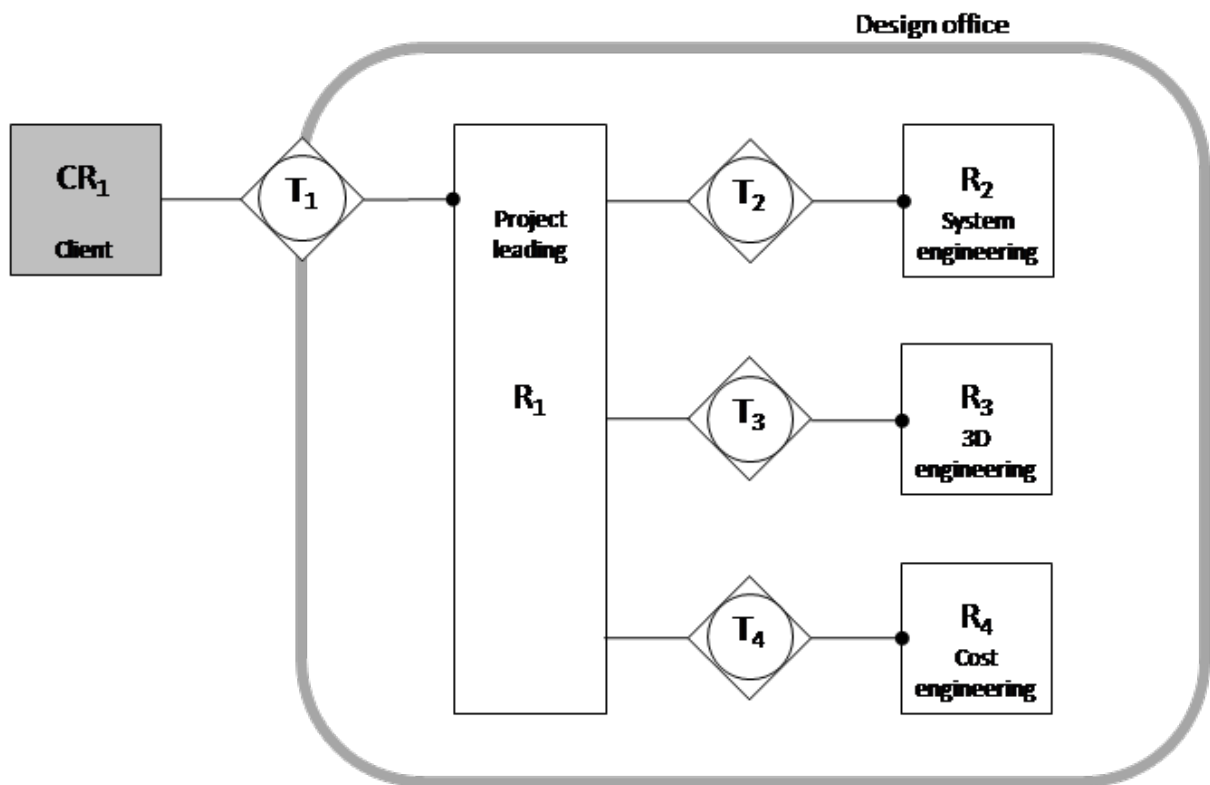


Figure 13: Example Interaction map

processes within a role are also not considered. By using 'abstract' roles, the transaction map can be applied to all construction projects, regardless of the collaboration concept or contract form. It is a valuable tool to analyse and define the essential elements of the production process.

An interaction map contains all transactions that are necessary for the acting roles in that project to make their contributions to the process. All roles and transactions on the map have a unique identity and name. The numbering is arbitrary. The name of a role is derived from the main activity that the role performs in the process. The interactions in fig. 13 are summarised in table 2.

Transaction Type	Transaction Result
T1 Design delivering	Design has been delivered
T2 Deliver specifications	Specification has been delivered
T3 Deliver 3D model	3D model is delivered
T4 Provide Cost Estimate	Cost Estimate has been delivered

Table 2: Summary of transactions and corresponding transaction results in the interaction map

3.5.2 Organisation

An organisation is uniquely identified within a project by a globally unique identifier. In addition to this identifier, an organisation has a descriptive name intended for human recognition and a three-letter abbreviation. The abbreviation is unique within the project and serves both a functional and technical purpose. It can, for example, be used to derive the organisational source of communication and to distinguish organisations in automated processing and integrations.

3.5.3 Project

A project is defined as an activity with a limited duration and constrained resources, aimed at achieving a specific outcome. Projects are characterised by their temporary and unique nature, distinguishing them from continuous operational processes or programmes. Within VISI, a project provides the organisational and communicative framework in which interactions between participants take place.

Each project is uniquely identified by a globally unique identifier, ensuring that references to a project are always unambiguous. In addition to this identifier, a project contains a name and a description to support human recognition and understanding. These properties are not required to be unique, since their primary purpose is descriptive rather than technical.

To define the project's temporal scope, both a start date and an end date are recorded. These dates indicate the planned or actual duration of the project and provide context for communication and coordination activities performed within the project environment.

3.5.4 Role

In VISI, the responsibilities and tasks are not assigned to *officials* within parties, but to *roles*. This was done because, although the same roles always occur in construction projects, officials can often perform several roles or switch roles. The cooperation concept or contract form has *no* influence on the existence of the roles. The cooperation concept *does* influence the assignment of roles to parties; the *role division*. Parties can, for example, perform the role of 'initiator', 'client', 'builder' or 'designer'.

There is by no means always a one-to-one relationship between parties and roles. There are construction projects in which one party takes on more than one role, and there are also construction projects in which several parties have the same role. Examples include a municipality that is both the initiator and the client, or a groundwork contractor and an installation company that both serve as the builder on a project. The same party can also perform different roles within different construction projects.

Tasks and responsibilities can thus be divided based on the division of roles between the parties. Examples of roles are 'commissioning' and 'building'. This then refers to the party that has the specific task of, in this case, 'commissioning' or 'building'.

A role is fundamentally different from a function. An essential difference is that roles are a reflection of the work being organised, while functions are a reflection of people being organised (in an organisation).

A VISI project does not have the function of Head of Department, but does have the roles of Assessment and Advisory.

3.5.5 Message

Messages are a means of transferring information or exchanging data. The message name equals a certain status, depending on the position within a process (delivery document or rejection). A message consists of several fields, depending on the information required. Some parts of a message are fixed, such as the sender and the time of sending.

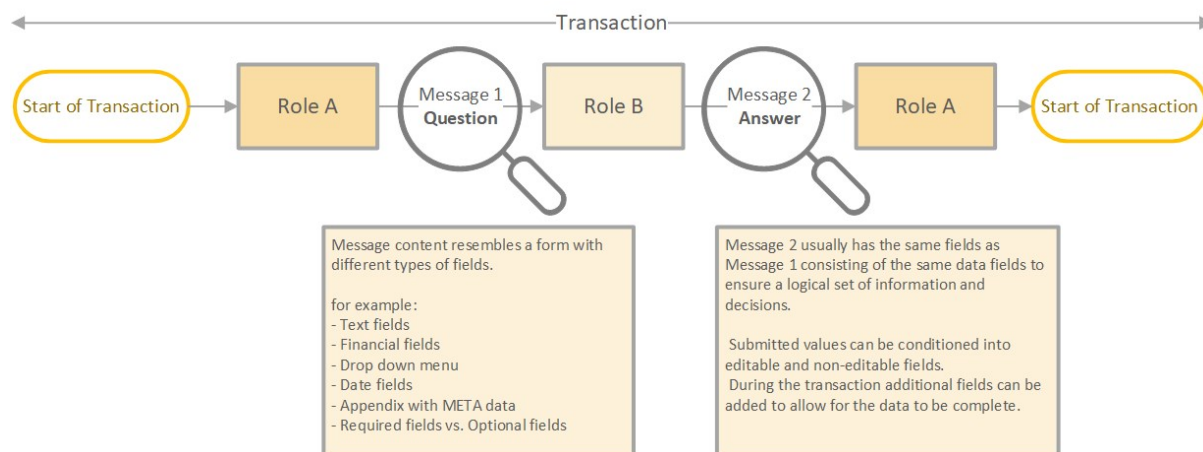


Figure 14: Message In Transaction content

Information in a message

The content of a message can be compared to a form with fields. Such as: text fields, dates, amounts, Yes/No, list boxes, and/or tables. This is where inheritance takes place: data entered in a previous message is automatically carried over to a subsequent message, unless a condition in the framework specifies otherwise.

Appendices and meta information

One or more appendices can be added to VISI messages. This is very similar to an 'attachment' of an email. An appendix of a VISI message can also be provided with metadata. These elements can be predefined. For example, the version number, document date, and status. Or the metadata that are necessary in an organisation-specific document management, such as file name, file date and file type.

Appendices (one or more)

Because appendices are linked to a message, they can always be interpreted in context afterwards, including when they were sent and the state of the transaction at that moment.

1. Approved/Rejected/Notification
2. Draft/Definitive
3. Date/Version

Message sequence

The messages are not only fillable, but the sequence of the messages is also predetermined. After receiving a message, you can choose one or more answers. In this way, it is determined in advance what the communication will look like and also how the course will proceed. In other words, the workflow is fixed.

3.5.5.1 Data Element Each message consists of a number of data elements. A data element has an identifier, a name and a description. Conditions can be set for the format and value of the data element.

Examples of data elements are:

- Transaction Description;
- Message Description;
- Sender Name;
- Sender Organisation;
- Sender Role Description;
- Recipient Name;
- Recipient Organisation;
- Recipient Role Description;
- Transaction Start Date;
- Message Send Date and;
- Message Read Date.

There can be two types of data elements in a VISI framework. First, the “simple data elements.” Elements of this type form the basis. In addition, there are the "complex data elements" that are used to group data elements, for example the data of a contact person.

For example, the complex data element "Contact" could consist of the following simple data elements:

- Initials
- First Name
- Middle Name
- Last Name
- Mobile Phone
- etc.

Although VISI was born out of a need for standardisation, it is possible to omit or add data elements to messages. After all, the communication needs can differ per project. The starting point is that the parties determine in mutual consultation which data will be exchanged, based on the templates provided in VISI.

3.6 Communicating based on a VISI framework

The whole of descriptions of roles, transactions, messages and data, which comprise the agreements and/or communication that can and may be used in a project/collaboration, is called a framework. In order to be able to communicate based on a VISI framework, several things need to be set.

Project specific message

The project-specific message puts the framework in the context of a project. This means that this file indicates which specific project and/or contract the framework represents. It also determines which persons from which organisation participate in the project in which role(s).

Connecting persons to roles

In VISI, responsibilities and tasks are not assigned to parties or persons, but to “roles”. This was done because, for example, in construction projects, the same “roles” always appear, but parties can often change. Other parties can then fulfil that role

Think of a situation in which a Client wants to have certain responsibilities and tasks fulfilled by an engineering firm or a situation in which a pizzeria does not have the delivery done by its own deliverers,

but concludes a contract with an external party, such as Deliveroo or Uber Eats. A person can be associated with an organisation and a role using the project-specific message. But several people can also be linked to a role, and a person can fulfil several roles.

See the example below for additional explanation.

Example 8: Pizzeria

The role of "waiter" can be fulfilled by several people. A person can also be a waiter and dishwasher.

	Person1	Person2	Person3	Person4	Person5	Person6
Client	X	X				
Waiter			X	X		
Cook					X	
Kitchen help				X	X	X

Table 3: "Waiter" role fulfilment

Example 9: Construction project

In a construction project, the distribution could look like this.

Client	Adviser	Tester/Acceptor	Project execution	Organisation	Name	Function
X				OG	Martijn Gerards	Client
X				OG	Marjon van Hagen	Project manager
	X			OG	WimSchilperoort	Controller
	X			OG	Hans Lijmbach	Contract manager
	X			OG	Iris Adegeest	Adviser
		X		OG	Mario van Bommel	Tester
		X		OG	Peter Bloemers	Tester
	X	X		OG	Raj Dhaker	Cost expert
		X		OG	Jose de Kwade	Technical Adviser
			X	ON	Arjen van der Hof	Project leader
			X	ON	Klaas Slot	Contractor
			X	ON	Jan van Geest	Planner

Table 4: Construction project roles

3.7 Details of the framework

The framework as a file to read in VISI compatible software

The VISI framework is recorded on the basis of the systematic rules in a file that can be read into VISI-compatible software. This should be done in combination with the previously discussed project-specific message and a so-called message schedule with which an extra check can be performed whether the messages that are sent and received conform to the rules of the system and the defined framework.

Supporting a business process with connected transactions

A business process or procedure often involves more than two roles. However, it is not possible to support an entire business process with a single transaction between two roles. The standard, therefore, offers the possibility to connect transactions. For example, a business process can be supported using a set of connected transactions.

Example 10: Pizzeria

Think of a customer of the pizzeria who enters into a transaction with a waiter to obtain a delicious pizza. Based on the transaction with the customer, the waiter can, in turn, initiate a transaction towards a cook to make the pizza. In this example, the customer does not have the mandate to walk into the kitchen

and ask a cook to make the pizza themselves. The waiter can also not initiate a transaction with the cook to make a pizza if there is no customer request.

An important basic principle is that a transaction always yields something. If a role from one transaction initiates one or more transactions with another role, something must always come back from these secondary transactions. A secondary transaction must always end in the transaction from which it was initiated.

Conditions within a transaction

In order to enforce certain dependencies in a transaction, conditions can be set at certain moments when creating a transaction. Think of the following:

- Enforce message sequence. This means that messages can only be sent after other messages have been received, or that messages may only be sent when other messages have not yet been received or that a message may only be sent once.
- You may or may not be allowed to initiate multiple secondary transactions.
- You may or may not be allowed to edit or delete certain fields in a message at some point in the transaction.

Can a framework be supplemented or changed during a project?

While using a VISI framework in a project or other collaboration, it is possible to adapt the VISI framework. For example, if business processes or process requirements change or if predefined elements, such as selection lists in messages, need to be adjusted.

Several things should always be kept in mind, namely:

- Pending transactions are completed in the version of the framework in which they were initiated. This means that a change to a framework does not affect current VISI communication.
- A new version of the framework must be imported for all project partners.

3.8 User guarantees

VISI en ISO 29481-1 Building information models - Information delivery manual

VISI interprets and extends the international standard ISO 29481-1 Building information models - Information delivery manual.

Transferring projects from one system to another

If a VISI user wants to switch to another supplier during a project, it is expected that they can read the messages already sent, to allow the user to continue the project from another application.

Archiving

After the project has been completed, the agreements and the messages are archived together. A VISI application is expected to be able to read in a project so that a user can read the message traffic again and offer it as an offline project archive where the messages and appendices are preserved in the context of the workflow.

Control information: Monitoring cooperation progress

You can see the progress of the collaboration for the entire project because you can check whether messages have been sent in accordance with the agreements. The delivery of monitoring data or control information across projects is not part of the standard, but is expected by users from the VISI applications.

Messages are not editable after sending

Certification VISI compatible software

Control questions:

- Do reports for control information also have to be supported by software, or is this software dependent?
- How do you know the message content is correct?
- How do you know your message has arrived?
- How do you know that the right person has received the message?
- How do you know that an authorised person wrote the message?

4 Project initiation

The functional documentation is intended for a software product manager or a software developer.

4.1 Bookmark

In the Conceptual chapter, the concept of VISI is introduced. In this Functional chapter, the concept is explained further and in more depth, so that the functionality and underlying building blocks of VISI can be fully understood.

The functional explanation is also structured into chapters, where each chapter describes a specific functional level of abstraction together with its corresponding building blocks. Each subsequent chapter expands and deepens the previous one by introducing additional building blocks and functional details. Skipping chapters may result in missing essential building blocks and disrupting the logical sequence of understanding. Therefore, the chapter structure is designed to gradually guide the reader from beginner level to expert level.

4.2 Project specific file

Before communication can take place in accordance with the open standard VISI, a project must be set up with a project-specific file (psb), a framework and a message schedule generated from said framework.

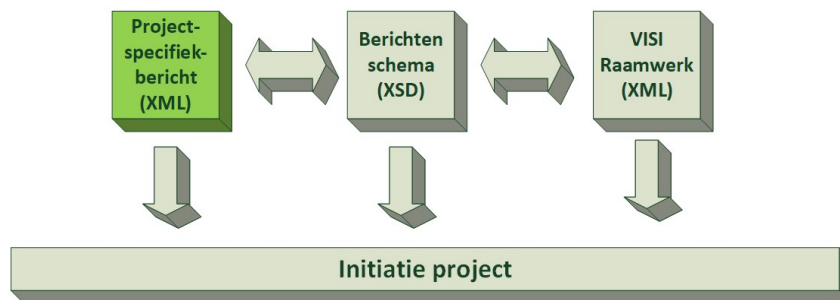


Figure 15: Initiation - Projectspecific message

4.2.1 Project

A project is central to VISI communication. The definition of a project in this case is an activity, limited in time and resources, to create something. What is created is a service/product/result. A project is carried out in collaboration between different people and organisations. The resources can be both human efforts and materials. It is distinguished by the one-time nature of a program or process. A project requires a specific management environment to coordinate activities and steer them on quality, time and money. Communication is necessary for coordination.

Limited, mandatory properties are recorded for a project. Each project has a unique identifier that can be used to refer to the project, without misunderstanding which project is meant. In addition, a project has a name and a description. The name and description don't have to be unique, and are in addition to the unique identifier. A name and description allow easier recognition of a project (by people who are communicating within a project). In addition, the moment of start and end of the project is recorded with start and end dates.

```

1 Unique ID: _badc9dab-807e-4319-bd56-60c57605e109
2 Name: Realisation of the West/East line
3 Description: Realisation of metro line 47, including 7 new stations with 9.7 kilometres
               of track, of which 7.1 kilometres are underground
4 Startdate: 23-12-2020 14:59
5 Enddate: 20-05-2022 07:08

```

4.2.2 Person

The communication within a project is carried out by people. The people who participate in a project are called persons. Each person in a project has a unique identifier that refers to the person in the project, without misunderstanding which person is meant.

In addition to identification, a person has a name and a username. The name, in addition to the unique identifier, is intended for easier recognition (by humans). The username is not mandatory and can be given a (technical) interpretation of your choice. Username can - for example - be used to link to a user in an (existing) software system.

```

1 Person
2 Unique ID: _0c585186-fa97-4de7-8fe0-5c4bee3afead
3 Name: Urbain Servranckx
4 Username: userservranckx
5
6 Person
7 Unique ID: _974e4822-4944-45ad-ab2f-161370662e30
8 Name: Georges Remi
9 Username: gremi
10
11 Person
12 Unique ID: _d4e02752-2ac6-4700-ae67-d33f428f78fb
13 Name: Luc Dupanloup
14 Username: ldupanloup

```

Note:

If a person works on two projects, the unique identification of this person can be different for each project. There is no global unique identifier, but a unique identifier within a project.

4.2.3 Organisation

People perform an activity in a project on behalf of an organisation. In a business context, the people are working on a project for an organisation, so recording the organisation is important. Every organisation in a project also has a unique identification. In addition to the identification, an organisation has a name and an abbreviation. The name is, in addition to the unique identifier, and is intended for easier recognition (by humans). The abbreviation has a functional and technical purpose. This property is a three-letter abbreviation of the organisation, which is unique within the project.

The organisational source of communication can be derived from the abbreviation, see ... (Reference to explanation of TransactionTemplate property). For an organisation, a contact person is defined by reference to a person's unique identifier. This reference has no functional meaning and is for administrative purposes only.

```

1 Organisation
2 Unique ID: _87e9144d-04d6-4a5f-bdcb-b50e96c2b498
3 Name: Gemeente Rommelmegem
4 Abbreviation: GRG
5 Contactperson: _0c585186-fa97-4de7-8fe0-5c4bee3afead

```

Note: The organisation has an address The address is also recorded for each organisation. This is not a physical or postal address, but a digital address. This digital address has a technical purpose, see 7.4 section about the SOAP protocol.

4.2.4 Role

The differentiation in authorisation, tasks and responsibilities in a project is made explicit by defining different roles. A role represents coherent authorisation, tasks and responsibilities to perform activities, which a person usually assumes temporarily, and which can be performed (alternately) by different persons.

A very limited number of properties are recorded for a role. Each role has a unique identification, supplemented with a name and description. This name and description need not be unique, and are an addition to the unique identifier intended to explain and give meaning to the role. The name and description make it easier for people to recognise the role.

```

1 Role
2 Unique ID: _1252a67a-5f69-4a9c-b4ca-506a4459c7c3
3 Name: Beoordelende rol
4 Description: This role provides an independent assessment based on competences and
               expertise within the framework of the project. During the assessment, the role goes
               through a number of fixed steps. Role makes an inventory of data, weighs, motivates
               and assesses this information and then makes the judgement.

```

Note: Role versus function

A role is fundamentally different from a function. Responsibilities and tasks are ****not**** assigned to officials of an organisation, but to roles. An essential difference is that roles are a reflection of **the work** being organised, while functions are a reflection of **people** being organised (in an organisation). A project does not have a "Department Head" function, but does have the "Assessment" and "Advisory" roles.

There is not always a one-to-one relationship between organisations and roles. There are projects in which one party fulfils several roles, and there are also projects in which different organisations have the same role. An example would be a municipality that has both the initiating and commissioning role. Or an engineering firm that fulfils the advisory role. With an accountancy firm in the same project, which also fulfils the advisory role, but in a different field.

Based on the division of roles between organisations, tasks and responsibilities can be divided among individuals.

4.2.5 Relationship between Person, Role and Organisation

In a project, people always act from a role on behalf of an organisation. A person is not directly linked to the organisation. The person has a relationship with a role and an organisation in the project. This relationship can be seen as a unique key that connects person, role and organisation. We call this key "Person in Role of Organisation". Very limited, mandatory properties are recorded for this key. For the key, person, role and organisation are defined by reference to the unique identifier of a person, role and organisation.

```

1 Person in Role of Organisation
2 Unique ID: _e0ad4954-7986-417c-96d6-813efa36e86b
3 Contactperson: _0c585186-fa97-4de7-8fe0-5c4bee3afead

```

4 Role: _1252a67a-5f69-4a9c-b4ca-506a4459c7c3
5 Organisation: _87e9144d-04d6-4a5f-bdcb-b50e96c2b498

Note: Relationship between person and role

A person can have multiple roles. A role can be fulfilled by more than one person.

4.2.6 Personnel changes

In almost every project, there are changes with regard to the staffing. This can be in the form of a temporary replacement due to absence or holidays. Or a permanent replacement if the project employees go to work for another organisation or retire.

4.2.6.1 Permanent replacement

Permanent replacement is also known as succession. The successor is an authorised person who takes over the role, tasks and responsibility of the (initially) authorised person. For traceability and transparency, authorised persons can never be removed. That is why the succession is made explicit.

Permanent replacement is always at the role level. With the aforementioned key of Person in Role of Organisation, this permanent replacement is recorded by means of a reference to the unique identification of a person, a role and an organisation. The relationship is from authorised to successor. The roles must match exactly.

1 Person in Role of Organisation
2 Unique ID: _e0ad4954-7986-417c-96d6-813efa36e86b
3 Contactperson: _0c585186-fa97-4de7-8fe0-5c4bee3afead
4 Role: _1252a67a-5f69-4a9c-b4ca-506a4459c7c3
5 Organisation: _87e9144d-04d6-4a5f-bdcb-b50e96c2b498
6 Successor: _51693b0e-84e5-4a9a-9fb6-ec4a7dea6edb
7
8 Person in Role of Organisation
9 Unique ID: _51693b0e-84e5-4a9a-9fb6-ec4a7dea6edb
10 Contactperson: _974e4822-4944-45ad-ab2f-161370662e30
11 Role: _1252a67a-5f69-4a9c-b4ca-506a4459c7c3
12 Organisation: _2ba76318-83ab-4a6e-9257-ea91045717d2

If a person (in a role of an organisation) has a successor, this means that the person is no longer authorised to act from this role. Only the successor is authorised. A person (in a role of an organisation) can have a maximum of one (1) successor.

4.2.6.2 Temporary replacement

A temporary replacement can also be read as a (temporary) authorisation. The temporary replacement is an authorisation for a third person to act on behalf of the authorised person. This can be of a temporary nature or an authorisation for the duration of the project.

The temporary replacement is always at the role level. With the aforementioned key of Person in Role of Organisation, this temporary authorisation is recorded by means of a reference to the unique identification of the authorised person, role and organisation. The relationship is from an authorised person to an authorised person. The roles must match exactly.

1 Person in Role of Organisation
2 Unique ID: _752b1dfd-dd91-4404-9708-39b91047ca66
3 Contactperson: _d4e02752-2ac6-4700-ae67-d33f428f78fb
4 Role: _1252a67a-5f69-4a9c-b4ca-506a4459c7c3
5 Organisation: _87e9144d-04d6-4a5f-bdcb-b50e96c2b498
6 Replaces: _51693b0e-84e5-4a9a-9fb6-ec4a7dea6edb

If an authorised person (in a role of an organisation) acts, this is always on behalf of the authorised person. With an action, the authorised person is recorded and the authorised person. The authorised/delegate can reply to transactions from the authorised's role, regardless of who initiated the transaction. If an authorisation is undone, only the authorised person can complete the action. An authorisation may be revoked, because authorisations can have a temporary character.

If an authorised representative sends something, it must be traceable that the authorised representative has sent something, and on whose behalf it was sent.

A person can be authorised to act on behalf of several authorised persons (in role of organisation). Multiple persons can be authorised to act on behalf of one authorised person (in role of organisation).

4.2.7 Project Setup

Project, persons, organisations, roles and replacements must be known and equal to all project participants. Therefore, this setup of a project is recorded in an interchangeable format: a project-specific file. This file is distributed among the participating organisations. This is usually per organisation due to the aforementioned digital address of an organisation. This digital address, therefore, knows about the existence of a project. In the example used, "Realisation West/East Line" with unique identifier _badc9dab-807e-4319-bd56-60c57605e109 is known to all organisations of the project. Project communication by sending and replying to messages can start, see 5.1.4.

4.3 Framework/Communication template of a project

The framework is the communication template of a project. All communication must follow this template. The template defines what type of information can be transferred between people. The template defines which type of information can be transferred (what), between which types of roles (who), and when (how).

The template also serves to validate the communication. As the name indicates, it is a template that the communication must comply with. Based on this template, it can be verified whether the information provided complies with the agreements.

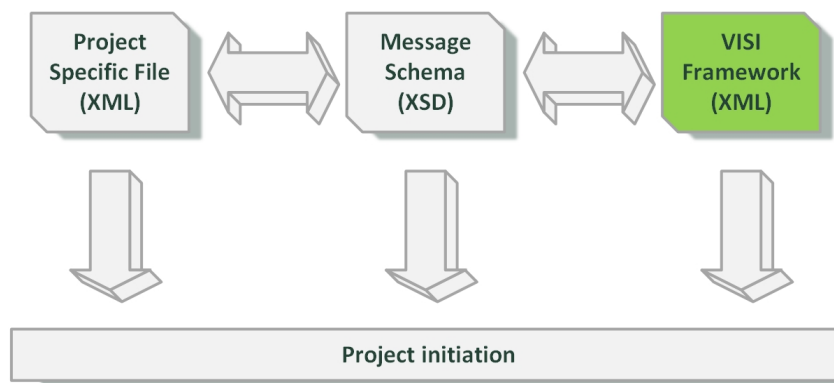


Figure 16: Initiation - VISI Framework

4.3.1 Role Type (RoleType)

Responsibilities and tasks are not assigned to functions/functionaries within organisations, but to types of roles. The definition of a role type includes a unique identifier and a textual description. Therefore, a role type has the unique identifier and description properties.

¹ Role type
² id : RT_Becoordelende_rol

Attribute	RoleType	TransactionType	MessageType	SimpleElementType	UserDefinedType	ComplexElementType	MessageInTransactionType	ElementCondition	MessageInTransactionTypeCondition	AppendixType	OrganisationType	PersonType	ProjectType	TransactionPhaseType	GroupType
id	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
description	X	X	X	X	X	X		X		X	X	X	X	X	X
state	X	X	X	X	X	X	X		X	X	X	X	X	X	X
dateLaMu	X	X	X	X	X	X	X		X	X	X	X	X	X	X
userLaMu	X	X	X	X	X	X	X		X	X	X	X	X	X	X
language	X	X	X	X	X	X				X	X	X	X	X	X
helpInfo	X	X	X	X	X	X		X	X	X	X	X	X	X	X
code	X	X	X		X					X	X	X	X	X	
appendixMandatory			X												
appendixTypes		X	X												
baseType					X										
complexElements			X												
condition								X							
executor		X													
firstMessage							X								
initiator		X													
initiatorToExecutor							X								
fileChecksum(1.8)										X					
maxOccurs						X									
minOccurs						X									
namespace													X		
openSecondaryTransactionsAllowed							X								
responsibilityFeedback	X														
responsibilityScope	X														
responsibilitySupportTask	X														
responsibilityTask	X														
sequence(1.8)					X										
xsdRestriction					X										

Table 5: Every VISI Type's attributes

³ `description: Beoordelende rol [startDate](#startdate)`

4.3.2 Information transfer conforms to transaction type (TransactionType)

Communication in a project always takes place based on a type of transaction. A transaction type is an arrangement of possibly successive messages between two roles. A transaction in a project, therefore, proceeds according to the rules of the transaction type from the framework. This refers to the initiating

role (initiator) and the executing role (executor).

A person in the initiating role may initiate a transaction and send the first message of this transaction to one (1) person in the executing role. After that, it is always the recipient (executor or initiator) of the last message who has to respond with a message until there are no more reply options. Then the transaction is completed. The possible start and reply messages are defined as the order of the type of messages of a transaction (see 5.1).

Several properties are recorded for a type of transaction. Not all properties are equally important; a division follows whether properties are mandatory or optional. An optional property can be important, although the property is not required. An example of this is the help text property (helpInfo). helpInfo is available to provide a more detailed explanation of the purpose and use of the transaction type. It can also describe why certain roles communicate with each other.

```
1 id: TT_Contractwijziging_door_opdrachtgever
2 description: Contractwijziging (door de opdrachtgever)
3 helpinfo: With this transaction type, the client can report a contract change to the
    contractor, with the intermediate goals of obtaining a technical proposal from the
    contractor, a technical agreement from the client, a quotation from the contractor,
    an acceptance of that quotation by the client and ultimately the acceptance of one
    or more multiple completion notifications of the work carried out through the
    delivery of performance declarations.
4 initiating role: Opdrachtgevende_contractueel
5 executing role: Opdrachtnemende_contractueel
```

4.3.3 Message Type (MessageType)

Message type defines which information can be exchanged. The type is recorded with a unique identifier (id) and a textual description (description). This description implicitly indicates the status of a type of transaction.

```
1 Message type
2 id: MT_Acceptatie_technisch_voorstel_contractwijziging
3 Description: Acceptance of the technical proposal (for contract modification)
4 helpinfo: With this message the client accepts the technical proposal, after which the
    contractor can continue with a financial proposal/quote for the contract change.
5 elementen: CE_Contractwijziging_Basis, CE_Contractwijziging_Voorstel,
    CE_Contractwijziging_Opmerkingen
```

4.3.4 Message Fields (SimpleElementType)

Message fields are defined within the message. A field property (UDT)(4.3.6) is assigned to a message field.

```
1 Simple element type
2 id: SE_DatumUitvoering
3 description: Gepland moment start uitvoering
4 helpinfo: Kies hier de beoogde startdatum waarop de hier benoemde werkzaamheden beginnen
5 User defined type: UDT_Datumveld
```

Note: Relation between person and role

A person can have multiple roles. A role can be fulfilled by more than one person. Reserved Field ID: The field with ID “SOAPServerURL” is reserved exclusively for the message chapter associated with the organisation type. This is so that the SOAP address of an organisation can always be found for the VISI software.

4.3.5 Special Fields

4.3.5.1 Sequence Field Automated sequence numbers within framework fields can now be realised by creating a user-defined type that configures the behaviour of the sequence in the field of this type. Prefixes that are related to organisations are stored in the configuration. The sequence number is a flexible format type. The sequence generated by the configuration in the UDT and project-specific message has to comply with the xsdRestriction base type for that UDT to avoid XML validation errors. The baseType field in the sequence UDT can be STRING or INTEGER. Other baseType values will be rejected.

```
1 Framework :
2
3 <UserDefinedType id="Udt_transaction_sequence">
4   <description>Transaction Sequence</description>
5   ...
6   <baseType>STRING</baseType>
7   <sequenceSyntax>{organisationSequencePrefix}_{sequence}_ABC</sequenceSyntax>
8   <sequenceStartNumber>0</sequenceStartNumber>
9   <sequenceNumberMinLength>4</sequenceNumberMinLength>
10  <organisationSequencePrefix>
11    <OrganisationSequencePrefixTypeRef idref="StGeneric"/>
12  </organisationSequencePrefix>
13  ...
14 </UserDefinedType>
15
16 <OrganisationSequencePrefixType id="StGeneric">
17   <description>Generic prefix</description>
18   <dateLaMu>2024-04-05T00:00:00</dateLaMu>
19   <userLaMu>xyz</userLaMu>
20 </OrganisationSequencePrefixType>
21
22 <OrganisationSequencePrefixType id="StChange">
23   <description>Change prefix</description>
24   <dateLaMu>2024-04-05T00:00:00</dateLaMu>
25   <userLaMu>xyz</userLaMu>
26 </OrganisationSequencePrefixType>
27
28 Configuration :
29
30 <StandardOrganisation id="ORG_5c3268e2bc9647488fb5ba38b864991d">
31   <name>Company This</name>
32   ...
33   <organisationSequencePrefixes>
34     <StGenericRef idref="st_11"/>
35     <StChangeRef idref="st_21"/>
36   </organisationSequencePrefixes>
37   ...
38 </StandardOrganisation>
39
40 <StGeneric id="st_11">
41   <name>Algemene prefix</name>
42   <organisationPrefix>OG</organisationPrefix>
43   ...
44 </StGeneric>
45
46 <StChange id="st_21">
47   <name>Wijziging prefix</name>
48   <organisationPrefix>6</organisationPrefix>
49   ...
50 </StChange>
```

Figure 17: Sequence user-defined type

A sequence is a way to assign increasing numbers to a field. The sequence field is used to define the format of the sequence number. {variable} can be used in the pattern value to target the known information. A list of supported variables is listed below.

After the variable, a semicolon can be placed to indicate the length of the previous variable. When the

Variable	part	Description
sequence_x	id	The name of the sequence that is used for this User Defined Type.
organisation		The organisation in the configuration.
project		The id of the organisation The project

Table 6: Variables for sequences

length is preceded by a 0, zeros will precede the result until the size has been reached, e.g. {number:5} would give '123', where {number:05} would give '00123'. A variable that contains a period '.' refers to the sub-variable within that structure. Sequences do not have sub-variables.

- Examples of sequence number settings without a prefix, with and without leading zeros,
- prefixes per organisation per sequence number, setting all of the above with XML examples
- An explanation that sequence numbers without a prefix are only workable if only 1 organisation can initiate messages with that sequence number, because otherwise identical sequence numbers with different content will arise

Variables within the sequences will be dependent on the starter of the sequence. Therefore, each organisation can have its own sequence. For VISI 1.8, no sequence communication will take place between servers. The storage of sequence numbers will be proprietary within the software and will be standard in the next version. Consequently, we will not yet describe how to initiate the sequence and how to change the sequence number from a standards perspective. Because the generated number is a user-defined type, the user's change of the sequence number is defined in the field conditions. Whether the software wants to register these changes is out of scope for the standard.

4.3.5.2 Response Date

Date on which the next message has to be sent. Not currently present in the framework. Standard setting will be a couple of days from the first send. A recommendation has to be written for the framework standard. Response date should be optional (on last message always unwanted)

The field reacts in the software:

- becomes read when later than the date
- shows in a separate column in the transaction overview
- always be a date field
- send an email when too late

customers want to do a standard increase in x days per term, sometimes differing per message type.
Response date: framework field name (date field) per organisation: (internal/external) standard increase +now()

4.3.6 Data type of an element (UserDefinedType)

This determines the behaviour of the input field and how it should be validated.

A field property (UDT) is assigned to a message field. For example, a field can be given a date/-time/currency/mandatory text/optional text/an exact number of characters and so on as a requirement. VISI software usually offers custom input functionality for the most common input types, such as a date selection window with a date field and a drop-down function if a choice from a list of values is required.

This is how a field gets the properties associated:

```

1 Framework:
2
3 <UserDefinedType id="Udt_transaction_duedate">
4   <description>Transaction Due Date</description>
5   <state>active</state>
6   <dateLaMu>2024-04-05T00:00:00</dateLaMu>
7   <userLaMu>xyz</userLaMu>
8   <baseType>DATUM</baseType>
9   <reactionDueDate>true</reactionDueDate>
10  <xsdRestriction></xsdRestriction>
11 </UserDefinedType>

```

Figure 18: Sequence user-defined type

```

1 User defined type
2
3 id: UDT_Datumveld
4 description: Datumveld
5 basetype: DATE
6
7 id: UDT_VerplichteTekst
8 description: Verplichte tekst veld
9 basetype: STRING
10 xsdRestriction: xs:minLength value="1"
11
12 id: UDT_Volnummer
13 description: Opvolgend nummer
14 basetype: STRING
15 sequence: {volnummer_contract}

```

4.3.7 Table as element of a message type

A special element is a table.

4.3.8 Grouping of elements (ComplexElementType)

The message chapters are used to easily manage input fields in messages in a framework and to make logical subdivisions in messages. Because only the different chapters are named in messages, a certain field can easily be added to or removed from an entire group of messages that contain that chapter. The order in which the message fields appear in the chapter determines the order in which the VISI software presents these chapters in the chapters and thus also in the messages.

Specifically for message chapters, there is often no support in the available software. Yet the market tends to demand this. This is so that it can be clearly explained what purpose this part of the message serves and what route this information takes, or where this information does not go, for example. (think of information that must remain within a certain domain)

Message Chapters (Table Forming)

It is permitted to name another chapter in a chapter in addition to message fields. The agreed effect of this is that the chapter is displayed in the “parent” chapter as a table. The message fields in the inserted chapter will then be presented as columns.

In this way, a chapter can consist of one or more tables, possibly also supplemented with normal message fields.

NB1: At this time, it is not yet technically possible to indicate at which location the tables in a chapter should be shown. As a result, they are currently sometimes above and sometimes below the normal fields, and it is not possible to create a chapter in which fields and tables alternate. This issue is being worked

on in issue 37.

NB2: Technically, it is possible to create a chapter within a chapter within a chapter, etc., which could, for example, produce a 3D table. There are no agreements yet in the system about such applications, and it has not yet been applied anywhere. This is considered to be room for unknown future applications.

Message fields

Placing message fields in a certain order in a chapter creates the structure of that chapter, as it appears in all messages containing this chapter.

ProjectType

As far as we know, it is technically possible to link a chapter to a project type, but there is no known application yet.

PersonType

This allows additional information to be entered for persons in the project-specific message, such as a telephone number or other personal information relevant to the project.

OrganisationType

This allows additional information to be entered for organisations in the project-specific message, such as an address or other information relevant to the project.

AppendixType

In this way, a different set of message fields with metadata about the attachment can be entered per attachment type.

MessageType

For building messages

Message field restrictions

This allows you to say about a field that it should not be editable or that it should be editable.

```
1 Complex element type
2 id: CeContractwijzigingVoorstel
3 description: Technisch voorstel
4 Simple elements:
5 SE_Wijzigingsnummer
6 SE_BeschrijvingTechnisch
7 SE_DatumUitvoering
8 SE_KostenramingEUR
9 CE_Tabel_Werkpakketten
10 Complex element type in Complex element type (tabel)
11 id: CE_Tabel_Werkpakketten
12 description: Werkpakketten
13 Simple elements:
14 SE_Werkpakket
15 SE_WPkostenpercentage
```

4.3.9 Order of message types (MessageInTransactionType)

The design of the message follow-up so that a flow chart can be followed is regulated in the Message In Transaction (MITT) type .

The MITTs connect message types within and between transactions.

If a message type occurs in more than one place in a transaction, for example, 1 time as a start message

and 1 time as a response, then there are 2 MITTs. Both refer to the transaction and the message type; only the connection between the MITTs differs.

References from the MITT:

Messages

Determines the message name and content of the element in the message flow

previous message

In this way, the flow of messages is determined. Because MITTs have the previous MITTs here, you know from which message this message can be selected as the next message. There are roughly 4 types of previous connections that look the same in definition (this MITT is a previous to this MITT), but are different in operation:

- Start of a new main transaction. Then the value of the previous is empty. This MITT may have the property “firstmessage”, but since this message is by definition a first message, this is not necessary.
- An answer within the same transaction (The previous MITT is also connected to the same transaction)
- A start of a new transaction based on the MITT in this transaction. We also call this starting a “Sub-transaction”. This MITT must always have the property “firstmessage”, otherwise no new transaction can be started, and the previous connection probably won’t do anything.
- A return from the sub-transaction to the parent transaction. This option is only allowed from a message in a subtransaction back to the transaction from which the subtransaction was initiated. This message in the parent transaction must not have the property “firstmessage”, otherwise a subtransaction will be created under the subtransaction.

Transaction

A MITT is always connected to a transaction. So, at a minimum, an MITT places a message type in a transaction.

Transactionphase

This property is intended to indicate the transaction phases of the DEMO universal transaction pattern. Little use is made of this yet. The first DEMO software on the VISI system does use this feature as an essential part.

Grouptypes

A group can be entered, but nothing has been done with this until now.

Appendixtypes

This is the most specific way to name which metadata set (message chapter) of message fields must be filled in for an appendix that is added to a message.

MITT conditions

Conditionality when you get to choose this message.

References to the MITT type:

Message field conditions (elementcondition)

MITT conditions Conditionality when you are allowed to choose another message based on the presence of this message.

Previous MITT. If this message is a previous message to the message it is set to.

¹ MessageInTransactionType

² id: mit40170_T7

```

3 initiatorToExecutor: true
4 firstMessage: false
5 messageType: MS_AcceptatieVoorstelWijziging
6 previous: mitt563
7 transactionType: TR_ContractwijzigingOG
8 MessageInTransactionTypeCondition: mtcl

```

See the decision matrix to determine if a transaction should be considered open or closed. ??

4.3.9.1 Order of MessageInTransactionType transcending one TransactionType

4.3.9 describes that a transaction type consists of message types, and that the order of the messages in a transaction is defined with an MITT. Each MITT has a reference to the previous MITT. However, this reference is not limited to a transactiontype. In other words, the previous MITT does not have to be a message within the same transaction; it could also be a message in another transaction. As a result, transactions are actually connected to each other. There are, however, specific rules on how this linking of transactions should be modelled in the framework.

How a connection is made between two transactions is explained by means of a change procedure between three roles. These are the roles: drafting role, approving role and advisory role. There is a transaction between the drafting role and the approving role. The purpose of this transaction is to achieve an approved change. There is a transaction between the approving role and the advisory role. The purpose of this transaction is to arrive at an opinion on the drafted amendment. The approving role benefits from a connection between these two transactions, creating one procedure for drafting, advising and approving a change between three roles: drafting, advisory and approving role.

The transaction Approve_Change between drafting role and approving role consists of three messages:

- Proposal_to_Change
- Agree_Change
- Disagree_Change

The message Proposal_to_Change is the first message of the transaction. The MITT of Proposal_to_Change has NO previous MITT. The MITTS of Agree_Change and Disagree_Change do have a previous MITT. This is the MITT of Proposal_to_Change.

```

1 Bericht in Transactie
2 Unieke identificatie: _666f01b3-6714-44cc-8865-0a9ae1f938ce
3 Vorig: Bericht: Voorstel_tot_Wijziging
4 Transactie: Accorderen\_Wijziging
5
6 Bericht in Transactie
7 Unieke identificatie: _6bdae158-a6dc-491b-a9ea-692fd419a828
8 Vorig: _666f01b3-6714-44cc-8865-0a9ae1f938ceBericht: Akkoord_Wijziging
9 Transactie: Accorderen_Wijziging
10
11 Bericht in Transactie
12 Unieke identificatie: _4e5569b7-0c6a-4976-811d-9d9189b8b1df
13 Vorig: _666f01b3-6714-44cc-8865-0a9ae1f938ceBericht: Niet_Akkord_Wijziging
14 Transactie: Accorderen_Wijziging

```

The transaction Advise_Change between the approving role and the advisory role consists of two messages:

- Request_Advice_Change
- Advice_Change

The message Request_Advice_Change is the first MITT. The MITT of Request_Advice_Change has NO previous MITT. The MITT of Advise_Change does have a previous MITT. This is the MITT of Request_Advice_Change.

```

1 Bericht in Transactie
2 Unieke identificatie: _a7e9fa9c-13ce-49f7-bbd6-f5eeb4688228
3 Vorig:
4 Bericht: Verzoek_Advies_Wijziging
5 Transactie: Adviseren_Wijziging
6
7 Bericht in Transactie
8 Unieke identificatie: _f75a35c7-30f3-4b3b-8776-120cd7811fd4
9 Vorig: _a7e9fa9c-13ce-49f7-bbd6-f5eeb4688228
10 Bericht: Advies_Wijziging
11 Transactie: Adviseren_Wijziging

```

How is the advisory transaction linked to the approval transaction? With reference to the previous MITT. However, because the property of the first MITT Adviseren_Wijziging an empty reference to the previous MITT should flag the first message (True/False) that this is the first message of the transaction.

```

1 Bericht in Transactie
2 Unieke identificatie: _a7e9fa9c-13ce-49f7-bbd6-f5eeb4688228
3 Vorig: _666f01b3-6714-44cc-8865-0a9aef938ce
4 Eerste bericht: Waar
5 Bericht: Verzoek_Advies_Wijziging
6 Transactie: Adviseren_Wijziging

```

After receiving Voorstel_tot_Wijziging, the approving role can choose three response messages:

- Akkoord_Wijziging
- Niet_Akkoord
- Verzoek_Advies_Wijziging

However, after receiving the message Advies_Wijziging, there is no response message. The connection back from the advisory transaction to the approving transaction has not yet been made. This is also modelled with reference to the previous MITT. In this case, the MITT of Akkoord_Wijziging and Niet_Akkoord_Wijziging.

```

1 Bericht in Transactie
2 Unieke identificatie: _6bdae158-a6dc-491b-a9ea-692fd419a828
3 Vorig: _666f01b3-6714-44cc-8865-0a9aef938ce , _f75a35c7-30f3-4b3b-8776-120cd7811fd4
4 Bericht: Akkoord_Wijziging
5 Transactie: Accorderen_Wijziging
6
7 Bericht in Transactie
8 Unieke identificatie: _4e5569b7-0c6a-4976-811d-9d9189b8b1df
9 Vorig: _666f01b3-6714-44cc-8865-0a9aef938ce , _f75a35c7-30f3-4b3b-8776-120cd7811fd4
10 Bericht: Niet_Akkoord_Wijziging
11 Transactie: Accorderen_Wijziging

```

For the correct definition of one procedure (made up of main and sub-transactions), two additional functional basic rules apply, which cannot be technically enforced.

Uitzoeken of het klopt dat ontvanger (niet per se executor) van bericht in T1 ook initiator moet zijn van startbericht T2. Fiodor "IS DIT WEL EEN REGEL???". In theorie zou het voldoende moeten zijn dat een Persoon beide rollen bekleedt, om bericht te ontvangen en van daaruit een bericht te versturen in gerelateerde transactie. Roles can be different between main and sub transactions.

Rule 1: the roles in the main and sub-transaction must match. This means that the receiving role in the main transaction must be the same as the sending role in the subtransaction. The definition of sending and receiving role is not at the message level. The roles are defined as initiator and executor by type of (main and sub) transaction.

```

1 Transactietype
2 id: Accorderen_Wijziging
3 description: Accorderen wijziging
4 initiator rol: Opdrachtnemende_contractueel
5 executor rol: Opdrachtgevende_contractueel
6
7 Transactietype
8 id: Adviseren_Wijziging
9 description: Adviseren wijziging
10 initiator rol: Opdrachtgevende_contractueel
11 executor rol: Adviserende_contractueel

```

For a type of message in a type of transaction (MITT), a direction is defined. The direction is from initiator to executor or from executor to initiator. This direction then moves from the initiating role to the executing role (of a type of transaction), respectively.

```

1 Bericht in Transactie
2 Unieke identificatie: _a7e9fa9c-13ce-49f7-bbd6-f5eeb4688228
3 Vorig: _666f01b3-6714-44cc-8865-0a9aef938ce
4 Eerste bericht: Waar
5 Richting: Van initiator naar executor
6 Bericht: Verzoek_Advies_Wijziging
7 Transactie: Adviseren_Wijziging

```

Uitzoeken. Fiodor "IS DIT WEL EEN REGEL???". Initiator to Executor is optional in the framework.

Rule 2: The direction of the messages must match.

By direction is meant the value or a message from the initiator to the executor of a transaction, or from the executor to the initiator.

```

1 Bericht in Transactie
2 Unieke identificatie: _666f01b3-6714-44cc-8865-0a9aef938ce
3 Vorig:
4 Bericht: Voorstel_tot_Wijziging
5 Transactie: Accorderen_Wijziging
6
7 Bericht in Transactie
8 Unieke identificatie: _a7e9fa9c-13ce-49f7-bbd6-f5eeb4688228
9 Vorig: _666f01b3-6714-44cc-8865-0a9aef938ce
10 Eerste bericht: Waar
11 Bericht: Verzoek_Advies_Wijziging
12 Transactie: Adviseren_Wijziging

```

If an instance of MITT is sent, openSecondaryTransactionsAllowed must also be sent. A message that does not appear as previous is required, so that it is clear that a transaction has been completed.

From a (1) type, a reference is made to the transaction type. This is the MITT. Although message types belong to a transaction type, there is no reference from message type to transaction type. As the name suggests, the reference is from a MITT to the message type and transaction type. This means that message types can be (re)used in different transaction types.

4.3.10 Conditions for elements of message type (ElementCondition)

This controls when a user may or may not edit a field, or whether the field should be cleared by the software for the next message.

References from the Message field conditions:

By naming one or more of the elements below, the situation to which the condition applies is defined.

complexElement
simpleElement
messageInTransaction
The basic rules are:

- A field that is offered for the first time, so does not yet exist in the previous message, can always be filled in and is never blocked by this setting.
- By default, a field that already exists in the previous message is blocked in subsequent messages. Even if the field happened to be “empty” in the previous message. So, only if the field is given the property “FREE” can it be modified.
- The value of a field from a previous message is filled in the next message when it’s the same SimpleElementType, and there is no condition ‘EMPTY’.
- Field restrictions can be applied to ComplexElementTypes, SimpleElementTypes and MessageInTransactionTypes, or a combination thereof.

4.3.10.1 Element Condition

1	ElementCondition
2	id: EC_116
3	description: New element condition
4	condition: FREE
5	ComplexElement: CeContractwijzigingVoorstel
6	ComplexElement: CE_Tabel_Werkpakketten
7	simpleElement: SE_WPkostenpercentage
8	messageInTransaction: mitt_265

The 4.3.10 indicates which restriction to use if there are multiple restrictions on a field in a message: The highest number in the first column “wins”. So if a condition is set on a CE and SE (5) then this condition applies relative to a setting only on a SE (1).

4.3.11 Conditions for subsequent message types (MessageInTransactionTypeCondition)

Sometimes it is necessary to ensure that you can only send messages after other messages have been received, that messages may only be sent when other messages have not yet been received, or that a message may only be sent once.

We do this by enforcing the sequence of messages to be sent.

The aim is to limit possible continuations based on a certain state in the message flow within a transaction and its subtransactions. The terms "transaction", "parent transaction", "sub-transaction" and "affiliated transactions" mean the following: From VISI system version 1.2, transactions are linked to each other via the "previous mitt method" so that a message from another transaction can be sent based on the message in one transaction.

- transaction: the transaction we are currently looking at regarding whether or not to offer messages. This transaction can have 1 or more transactions "before" and also one or more "after".

- direct parent transaction: The transaction from which "the transaction" we are looking at started (via a previous mitt link). By this, we do not mean other transactions that are at the same level next to or above this "parent transaction".
- direct subtransaction: which we are currently viewing. Any transaction started directly from "the transaction" we are looking at (via a previous mitt link).
- affiliated transactions: "the transaction" together with its direct parent transaction and its direct sub-transactions.

There are three reasons why certain continuations may not be offered:

1. A message may only be sent after one or more specific messages are present (already received or sent) in the transaction or its *direct* subtransactions, so there is a dependency between two (or more) already present messages.
2. A message may only be sent when one or more specific messages are not present (not yet received or sent) in the transaction or its *direct* subtransactions, so there is a dependency between two (or more) messages that are not yet present.
3. A message that can only be sent once has already been sent, so there is a restriction on the number of times a message can be sent.

References from MITT restrictions:

If there is more than one of the references below in a condition, there is an OR situation. So if one of the named messages is found, the condition is met, and the message may be chosen. By connecting multiple conditions to a message with, for example, only 1 message type in the options below, an AND situation arises. All named conditions must be met before you can use this message.

sendAfter MITT See situation 1 above

sendBefore MITT See situations 2 and 3 above

For situation 3: The effect of being allowed to send a message only once is achieved by placing the MITT of the message in question in the sendBefore list of that same MITT. In this specific case, a message may not be sent to multiple executors.

Allowed references to "sendAfter" and "sendBefore" are "MITT types" that can be sent in the current transaction or of the connected transactions, where the person handling the current message is the initiator or executor.

This limitation exists because a VISI Framework plus PSB must always behave in the same way, regardless of whether all actors work on the same VISI server or whether each actor works on a different VISI server. In this way, the actor in question always sees the messages that influence his choices, because he sends or receives them.

As an example: the cook may only use Before or After relationships with transactions in which he himself participates as an initiator or executor, and these transactions must be directly connected to the transaction that the cook currently wants to handle.

References to MITT conditions:

MITT

It is set in the MITT to be restricted by referencing in it to the restricting MITT condition.

¹ MessageInTransactionTypeCondition
² id: mtcl
³ sendAfter: mit_VTWoordeelpositief_1

4.3.12 Order of messages

In practice, it sometimes appears desirable to only be able to send messages after other messages have been received, that messages may only be sent when other messages have not yet been received, or that a message may only be sent once. To make this possible, it is necessary to enforce the order of messages to be sent.

The aim is to limit possible continuations based on a certain state in the message flow within a transaction and its subtransactions. There are three reasons why certain continuations may not be offered:

- A message may only be sent after one or more specific messages are present (already received or sent) in the transaction or its subtransactions, so there is a dependency between two (or more) already present messages.
- A message may only be sent when one or more specific messages are not present (not yet received or sent) in the transaction or its subtransactions, so there is a dependency between two (or more) not yet present messages.
- A message that can only be sent once has already been sent, so there is a restriction on the number of times a message can be sent.

4.3.12.1 Adjustments compared to the previous system

- The existing class “MessageInTransactionType” is extended with an additional relationship “conditions” of the type “OPTIONAL SET [1:?] OF MessageInTransactionTypeCondition”;
- A new class “MessageInTransactionTypeCondition” is created. This determines the type of set relationship specified in the conditions mentioned above. If a “conditions” relationship is specified, then this set must contain at least one element. If the “conditions” set contains more than one element, the elements involved are evaluated according to intermediate logical AND operators;
- A relationship "sendAfter" of type “OPTIONAL SET [1:?] MessageInTransactionType” is added to the class “MessageInTransactionTypeCondition”. This relationship is optional; if present, the set must contain at least one MITT. If the “sendAfter” set contains more than one MITT, the MITTs involved are evaluated according to intermediate logical OR operators;
- A relationship "sendBefore" of type “OPTIONAL SET [1:?] MessageInTransactionType” is added to the class “MessageInTransactionTypeCondition”. This relationship is optional; if present, the set must contain at least one MITT. If the “sendBefore” set contains more than one MITT, the MITTs involved are evaluated according to intermediate logical OR operators;
- Allowed references to "sendAfter" and "sendBefore" are "MessageInTransactionTypes" that can be received in the current transaction or from the connected transactions, where the person handling the current message is the initiator or executor. Connected transactions refer to the transaction from which a transaction was initiated and the direct subtransactions. Direct subtransactions refer to transactions that are initiated from the current transaction, not subtransactions of subtransactions.
- This limitation is introduced because a VISI Framework plus PSB must always behave the same way, regardless of whether all actors are running on the same VISI server, or whether each actor is running on a different VISI server. This is a basic principle that already works this way up to and including the 1.3 standard and is therefore also respected in this adjustment. In this way, the actor in question always sees the messages that influence his choices, because he sends or receives them.
- This limitation is introduced because a VISI Framework plus PSB must always behave the same way, regardless of whether all actors are running on the same VISI server, or whether each actor is running on a different VISI server. This is a basic principle that already works this way up to and including the 1.3 standard and is therefore also respected in this adjustment. In this way, the actor in question always sees the messages that influence his choices, because he sends or receives them.

- As an example: the cook may only use Before or After relationships with transactions in which he himself participates as initiator or executor, and these transactions must be directly linked to the transaction that the cook currently wants to handle.
- The effect of only being allowed to send a message once is achieved by placing the MITT of the message in question in the sendBefore list of the same MITT. In this specific case, a message should `_not_` be sent to multiple executors.

Addition to existing class

```

1 MessageInTransactionType;
2 conditions : OPTIONAL SET [1:?] OF MessageInTransactionTypeCondition; — AND

```

New class

```

1 MessageInTransactionTypeCondition;
2 sendAfter: OPTIONAL SET [1:?] MessageInTransactionType; — OR
3 sendBefore: OPTIONAL SET [1:?] MessageInTransactionType; — OR

```

4.3.13 Appendix type (AppendixType)

Files can be attached to each VISI message. A file must always be connected to an appendix type. A framework must therefore always have at least one appendix type. If a framework has multiple appendix types and no specific appendix type is set on a message, you can choose which appendix type an attachment is connected to in a message.

An appendix type is usually linked to a message chapter, so that when entering an attachment, metadata fields are available to fill in associated data with the attachment.

References from appendix types:

The message chapter provides the message fields with which the metadata is connected to the attachment.

References to appendix types: It is possible to limit the number of appendix types in a message if the framework contains more than one appendix type. If nothing is set on the message, the user must choose from all available types.

Transaction types

An appendix type connected at this level is offered in all messages in that transaction.

Message types

An appendix type connected at this level is offered in this message type in all transactions (where this message appears).

MITT

An appendix type that is connected at this level is only offered in this specific message at this place in the transaction.

```

1 AppendixType
2 id: AT_WijzigingGereed
3 description: Bijlagen bij gereedmelding wijziging
4 ComplexElementType: CE_BijlagenBijWijzigingGereed

```

4.3.14 Organisation type (OrganisationType)

An organisational type is needed in every framework. This is used when creating organisations in the project-specific message. Normally, a framework contains only 1 organisation type. Frameworks with more than 1 variant have not yet been applied.

References from organisation types: [complexElements] see 4.3.8

An organisation type can contain metadata elements, such as address, telephone number, etc.

For projects with SOAP communication, at least the Message field with the exact Id “SOAPServerURL” must be linked to the organisation type via a message chapter, so that the SOAP address can be set.

```
1 OrganisationType
2 id: StandaardOrganisatie
3 description: Standaard organisatie
4 ComplexElementType: CeOrganisatieSOAPServer
```

4.3.15 Person Type (PersonType)

A person type is needed in every framework. This is used when creating people in the project-specific message. Normally, a framework contains only 1 person type. Frameworks with more than 1 variant have not yet been applied.

The basic fields of a person in a project-specific message, such as Name, username and ID abbreviation, etc., are not in the framework. See [#verwijzing naar psb werking#](#).

References from person types:

complexElements

A person type can contain metadata elements, such as address, telephone number, etc. Due to, among other things, new privacy legislation, this is being entered less and less.

```
1 PersonType
2 id: StandaardPersoon
3 description: Standaard persoon
4 ComplexElementType: CE_Persoonspecifieke_informatie
```

4.3.16 Project Type (ProjectType)

A project type is needed in every framework. This is used to record, for example, the project name and the framework namespace in the project-specific message. Frameworks with more than 1 variant are not allowed.

The basic fields of a project type in a project-specific message, such as the project name, project ID, etc., are not in the framework.

References from project types:

Complex element

As far as we know, no metadata fields are applied to the project type in the VISI software, so it is not linked.

References to project types:

```

1 ProjectType
2 id: RaamwerkUAVGCCGemLutjebroek
3 namespace: http://www.visi.nl/schemas/20160331/RaamwerkUAVGCCGemLutjebroek_v2
4 description: Standaard raamwerk UAV GC Gemeente Lutjebroek

```

4.3.17 Transaction Phase Type (TransactionPhaseType)

The DEMO methodology has the transaction phases requested, promised, declared, accepted, declined, rejected, revoked, allowed and refused. It is possible to connect these transaction phases to messages in transaction types, so that VISI software could use them for additional support or information.

Out of scope: When using the DEMO software of the system, at least the transaction phase Promised and the following revoked types are actively used for specific software behaviour: Revoke Acceptance Allow, Revoke Promise Allow, Revoke Request Allow and Revoke Statement Allow

Specific properties to be set:
References from transaction phases:
None

References to transaction phases: With a message in a transaction, a message in a transaction can be connected to a certain transaction phase.

4.3.18 Group Type (GroupType)

The definition of the group for storing appendices sent with a message within a transaction. In practice, no functionality is assigned to this element by suppliers at the moment. However, a GroupType is part of the structure of a framework.

References from GroupTypes:
None

References to GroupTypes:
MITT

4.4 VISI Attributes

All attributes that can occur. By clicking on the name of the property, you can see where in the documentation the attribute is referenced.

4.4.1 id

Each element in a framework has a unique identifier (id). In this way, the relationships are placed technically inside and outside the framework. A key requirement is that an ID is unique within the framework. An example of relationships within the framework is, for example, linking a field (SimpleElement) to a message chapter (ComplexElement).

An example of a connection outside a framework is the Role that is created in a Project-specific message based on the role in the framework or a field name in a VISI message, etc.

An identifier can be a non-readable value, such as TR_532856857120. At the moment, almost all framework writers make something understandable out of this, such as TR_Acceptatiedocument, so that, among other things, the original XML VISI messages can also be read and understood without VISI software.

4.4.2 description

This is the human-readable name of an element. This value is usually shown to the users.

AppendixType: With this naming convention, users choose which appendix type to associate the attachment with.

4.4.3 state

This attribute can be optionally set on many elements in a framework. Possible values for this field are: “passive” and “active”. At the moment, there are applications from software suppliers that have not (yet) achieved implementation in the documentation. Examples are whether or not to show/offer elements or transactions, and to (partially) disable a role of a person in the project-specific message if there is no successor.

4.4.4 dateLaMu

This attribute can be optionally set on many elements in a framework. Framework writers use this to indicate in the XML file when a change was last made to an element. (date Last Mutation) Until now, there are no expectations or known applications in the VISI software, but there are in framework drafting software to update this field automatically. So if a framework writer enters something in this field, it will never show up to the user.

4.4.5 userLaMuo

This attribute can be optionally set on many elements in a framework. Framework writers use this to indicate in the XML file who last made a change to an element. (user Last Mutation) Until now, there are no expectations or known applications in the VISI software, but there are in framework drafting software to update this field automatically. So if a framework writer enters something in this field, it will never show up to the user.

4.4.6 language

This attribute can be optionally set on many elements in a framework. The idea is to define the language of the element. When the field is filled, the coding is according to the ISO 639-1³ followed by a dash and the ISO 3166-alpha-2⁴ country code (e.g. The Netherlands ‘nl-NL’, Belgium ‘nl-BE’, America ‘en-US’, and Britain ‘en-GB’).

4.4.7 helpInfo

This attribute can be optionally set on many elements in a framework. This is frequently used to provide users with additional explanations about framework elements. Certain elements require functionality in the VISI software. See the different elements for examples.

The information in this field must be made visible for the user in the VISI software.

SimpleElementType:

Specifically for message fields, the help text is often used to explain to the author of a message what exactly is expected when filling in that field. This is both on an abstract level, such as: “Enter the length of the element here, without taking into account protruding parts such as reinforcement and the like”. But also specifically in the form of an explanation of which number notation is checked: “Note the length with a comma and two decimal places”.

The help text also helps to better interpret the contents of a field when reading a message.

UserDefinedType:

³https://en.wikipedia.org/wiki/ISO_639-1

⁴https://en.wikipedia.org/wiki/ISO_3166-1_alpha-2

For field properties, no application is known yet for the help info. It is conceivable that it could be displayed next to the help info, to manage the fill instruction here instead of in the help info of multiple field types that have this property.

ProjectType:

With project type, the help info is sometimes used by framework builders for notes or explanations about the framework. There is no further application known in the VISI software.

Other Types: No (software) application is known for help info of role types, transaction types, complex element types, element conditions, message-in-transaction conditions, appendix types, organisation types, person types, transaction phase types, and group types.

4.4.8 code

This attribute can be optionally set on many elements in a framework. The code field is used to describe the combination of the transaction name and the transaction phase (e.g. T1_request. This code is needed to have a transparent coding of translated DEMO models in VISI.

4.4.9 appendixMandatory

With this property, it can be set that it is mandatory to add at least one attachment to a message of this type. This prevents a message type with the description "Document for acceptance" from being sent without an attachment.

4.4.10 appendixTypes

TransactionType: With this optional property, a restriction can be imposed on which appendix types may be added to the message types of this transaction type. This property is a reference to one or more appendix types. If this property is missing for a transaction type, any appendix type may be included.

MessageType: Because it is possible within a framework to specify different metadata forms for an attachment to be attached, it is also possible to set per message type which metadata form(s) can be chosen for an attachment to a message. If no appendix types are connected, any appendix type from the framework can be chosen by the user.

4.4.11 baseType

For each Field Property, you must choose from several basic XML field types to determine the basic behaviour of a field. The choice is limited to the following basic XML field types:

- BOOLEAN
- DATE
- DATETIME
- TIME
- DECIMAL
- INTEGER
- STRING

4.4.12 complexElements

This records the content of a message type, because those message chapters contain the information fields. The order in which the chapters appear in the message type determines in which order the VISI software presents these chapters in the messages. It is technically possible with the current control mechanisms to send a message in which one or more message chapters are missing, but this is not the intention. Studies are happening to improve security for this.

4.4.13 condition

Determines the behaviour of a field. You can choose from the following conditions:

- FREE
 - The field may be modified.
- FIXED
 - The field may not be modified.
- EMPTY
 - The field is cleared by the software for the user.

4.4.14 executor

This is a reference to a role type. This property defines what type of role a person must have in order to receive a first message in this type of transaction. In fact, this is the type of role that will realise/execute the result of the transaction.

4.4.15 firstMessage

A Boolean value is intended to establish indisputably that a message can only be used as the first message of a subtransaction. Using a MITT with this setting to true will therefore always result in a new sub-transaction. When omitted, false will be assumed.

4.4.16 initiator

This is a reference to a role type. This property defines what type of role a person must have to initiate a transaction of this type. In this context, starting means the same as initiating.

4.4.17 initiatorToExecutor

Boolean value indicating which direction the message has in the transaction. With True, the message goes from the person who initiated the transaction to whoever received the first message of the transaction. OPTIONAL??? See exp_2

4.4.18 maxOccurs

With this, it is possible to indicate that a table may not contain more than x lines. By setting Min and Max to the same number, an exact number of lines to be created can also be set.

4.4.19 minOccurs

It is explained below that a chapter can be inserted into another chapter to create a table. For example, by filling in this setting with 1 or 3 on the inserted chapter (the table), a user must create at least 1 or 3 lines in this table.

4.4.20 namespace

Here, there must be a unique namespace in the framework for each version of the framework. This is so that the VISI software can identify which framework a VISI transaction must be handled in.

4.4.21 openSecondaryTransactionsAllowed

An optional Boolean value indicating whether secondary transactions do not need to be completed before the primary transaction can continue. The interpretation for "TRUE" is that not all instances of secondary transactions need to be completed before the primary transaction can proceed. If the value is "FALSE", all instances of secondary transactions must be completed before the primary transaction can be resumed. If openSecondaryTransactionsAllowed is not defined, it will be interpreted as "TRUE".

As a primary transaction, we mean the transaction in which we want to send the mitt that has this openSecondaryTransactionsAllowed setting. The secondary transactions referred to in this rule are

transactions that are initiated from the last message connected in the primary transaction and that refer the secondary transaction to the new message to be sent via a previous link from the last message sent. This means that if the last message of the primary transaction itself is also linked to the message to be sent in the primary transaction, then this way of responding is always allowed. So this rule restricts only the mandatory routes through sub-transactions.

4.4.22 received

Boolean value indicating whether the previous message should have been received. In practice, this element is not used.

4.4.23 responsibilityFeedback

Feedback is expected from the responsibility of the role towards other roles. This includes responses in the accept or rejection of the work.

4.4.24 responsibilitySupportTask

Tasks performed to support other roles. Think of delegated responsibilities. This describes the purpose of the task for other tasks to have an understanding of why the task has to be executed in a specific way.

4.4.25 responsibilityTask

Duties arising from the responsibilities of the relevant role. This can encompass the description of the work that needs to be done to complete the work that needs to be transmitted to another party.

4.4.26 sequence

Sequence text from the separate 1.8 documentation

4.4.27 xsdRestriction

This allows very exact behaviour to be defined for a field of a certain base type. A regular expression can be defined for the base type STRING.

Some examples that are now widely used for base type STRING are:

- 1 choice, so the field always has this value:

```
1 <xs:enumeration value="Automatisch gevuld"/>
2
```

- Selection list with multiple values

```
1 <xs:enumeration value="k1"/><xs:enumeration value="k2"/><xs:enumeration value="
k3"/>
2
```

- Optional dropdown list:

```
1 <xs:enumeration value=""/><xs:enumeration value="Keuze 1"/><xs:enumeration
value="Keuze 2"/>
2
```

- Mandatory text of at least 1 character

```

1  <xs:minLength value="1"/>
2

```

- Optional text no xsd restriction
- For example, a year number

```

1  <xs:pattern value="(20\d\d)"/>
2

```

An example that is often used for base type DECIMAL is:

- Amount with two decimal places

```

1  <xs:fractionDigits value="2"/><xs:fractionDigits value="2"/>
2

```

4.5 Guidance for Element Conditions

4.5.1 Implementation of element conditions

During the implementation of 'element conditions', a discussion arose about the correct way to apply these conditions. The main reason is that the application of a condition at a certain level can affect the behaviour of Simple Elements at other levels (precedence rules).

To clarify things, all possible conditions are shown in a table below. From no condition at all to conditions at the MITT, CE and SE levels. This table is then interpreted in pseudo-code.

Since a condition element can be placed on 2 CE's in tables, a distinction is made in the table below between parent and child CE. The element condition in the XML framework is simply set to 1 or 2 ce's; parent or child is determined per message situation and is not recorded in the element condition.

If there are 2 CEs in an element condition, the first is always the parent and the second the child CE. For example:

- The SE "Comments" must always be EMPTY, you could represent this as: Condition(-, -, -, 'Comments') = EMPTY
- In MITT "MITT01" the entire MITT must always be FREE, you could represent this as: Condition('MITT01', -, -, -) = FREE
- If 1 column of a table is "FIXED", no lines may be added or deleted, even if it is an empty table.
- The CE "verifications" must always be FREE, regardless of whether it exists in a table or as a normal CE: Condition(-, 'verifications', -, -) = FREE
- The CE "checks" must only be FREE as a table within CE "advice": Condition(-, 'advice', 'checks', -) = FREE
- A message exists containing a parent CE "CE1" and a child CE "CE2". For the element condition examples below, the location is indicated in the table above:
 - (-, 'CE1', -, -) : Position 4
 - (-, 'CE2', -, -) : Position 2

	MITT	Ce normal/parent C1	Cechild C2	SE
0	-	-	-	-
1	-	-	-	X
2	-	-	X	-
3	-	-	X	X
4	-	X	-	-
5	-	X	-	X
6	-	X	X	-
7	-	X	X	X
8	X	-	-	-
9	X	-	-	X
10	X	-	X	-
11	X	-	X	X
12	X	X	-	-
13	X	X	-	X
14	X	X	X	-
15	X	X	X	X
16	First Message, or Simple Element Type not present in previous message			

Table 7: All options for Element condition (- absent, x defined)

- (-, 'CE1', 'CE2', -) : Position 6
- (-, 'CE2', 'CE1', -): No position: 1 is not a child of 2. This element condition does nothing.

Proposed pseudo code for determining the condition:

```

1 If the first message or element is not present in the previous message
2 return Free
3
4 if Condition found(MITT, Ce normal/parent, Cechild, SE )
5   <Condition(MITT, Ce normal/parent, Cechild, SE)>
6
7 if Condition found(MITT, Ce normal/parent, Cechild, -)
8   <Condition(MITT, Ce normal/parent, Cechild, -)>
9
10 if Condition found(MITT, Ce normal/parent, -, SE )
11   <Condition(MITT, Ce normal/parent, -, SE )>
12
13 if Condition found(MITT, Ce normal/parent, -, - )
14   <Condition(MITT, Ce normal/parent, -, - )>
15
16 if Condition found(MITT, -, Cechild, SE )
17   <Condition(MITT, -, Cechild, SE )>
18
19 if Condition found(MITT, -, Cechild, -)
20   <Condition(MITT, -, Cechild, -)>
21
22 if Condition found(MITT, -, -, SE )
23   <Condition(MITT, -, -, SE )>
24
25 if Condition found(MITT, -, -, -)
26   <Condition(MITT, -, -, -)>
27
28 if Condition found(-, Ce normal/parent, Cechild, SE )
29   <Condition(-, Ce normal/parent, Cechild, SE )>
30
31 if Condition found(-, Ce normal/parent, Cechild, -)
32   <Condition(-, Ce normal/parent, Cechild, -)>
33
34 if Condition found(-, Ce normal/parent, -, SE )
35   <Condition(-, Ce normal/parent, -, SE )>
36
37 if Condition found(-, Ce normal/parent, -, -)

```

```

38 <Condition(-, Ce normal/parent, -, -)>
39
40 if Condition found(-, -, Cechild, SE )
41 <Condition(-, -, Cechild, SE )>
42
43 if Condition found(-, -, Cechild, -)
44 <Condition(-, -, Cechild, -)>
45
46 if Condition found(-, -, -, SE )
47 <Condition(-, -, -, SE )>
48
49 if Condition found(-, -, -, - ) (no element condition found)
50 <Condition(-, -, -, - )>
51
52 return Fixed

```

- Is the SE“ Comments” in MITT “MiTT01” now EMPTY or FREE?
 - According to this definition, “Comments” in “MiTT01” is FREE.
 - To make “Comments” in “MiTT01” now EMPTY, Condition(‘MiTT01’, -, ‘Comments’) must be added.

Backwards compatibility: Because old frameworks have element conditions in the same way, with the only difference that never more than 1 ce is specified, all old element conditions in frameworks continue to work when the framework is converted to 1.6. In 1.6, you can mainly arrange additional specifications. The element conditions existing in the framework are, of course, evaluated with the above priority matrix from 1.6 onwards.

4.6 Delegation and mandating

Delegation and mandate describe different ways in which authority and responsibility can be assigned within an organisation.

Delegation means that an administrative body transfers its decision-making authority to another party, who then exercises this authority under their own responsibility. The delegated authority is exercised in the delegate’s own name, and the original authority holder no longer exercises that power.

Mandate means that a person or organisation is authorised to act on behalf of another administrative body. In this case, the authority remains with the original authority holder (the mandator). Decisions taken by the mandate holder are legally considered decisions of the mandator, who therefore remains responsible and accountable.

In DEMO, responsibility for initiating and executing transactions is assigned to roles. DEMO defines which role is responsible for a transaction, but does not define how this responsibility is implemented at the operational level.

VISI operationalises this responsibility model through the **PersonInRole** concept. A person assigned to a role within an organisation can initiate or execute transactions on behalf of that role.

VISI supports the following mechanisms for operational execution:

- **Representative**: a person may act on behalf of another **PersonInRole**. Actions performed by the representative are performed under the responsibility of the original role holder.
- **Successor (substitute)**: a person may temporarily take over the activities associated with a role. The original initiator and executor assignments within the transaction remain unchanged.

These mechanisms support mandate-like behaviour within VISI transactions.

VISI does not explicitly support delegation as a transfer of responsibility from one role to another. Responsibility remains assigned to the original role as defined in the transaction model.

VISI also does not support limited mandates. It is not possible to formally define restrictions on the scope of authority of a representative or successor.

For this reason, mandate-related behaviour within VISI is implemented using the existing concepts `PersonInRole`, `representative`, and `successor`.

5 Project execution

Use Framework and Configuration in project

5.1 Message communication

The project communication via sending and replying to messages can start. All participating organisations are aware of the project. If a message is sent by person Urbain Servranckx (in the assessing role of the Municipality of Rommelgem organisation) to Georges Remi (in the advisory role of the organisation De Blauwe Lotus), the digital addressing and distributed Configuration ensure that the message is actually delivered to Georges Remi.

5.1.1 Order

Messages are not independent of each other, but are related. Messages have an order that can be seen as a chain of messages. The chain has a beginning and an end. There is a start message and a final message. For example, the sequence is:

start -> request -> delivery -> rejection -> redelivery -> approval -> end

For a message, it is indicated what the previous message is. The start message has no preceding message, and this is the only way to distinguish a start message from the other messages. Likewise, the final message does not appear anywhere as a previous message. This is the only way to distinguish a final message from the other messages. The reference to the foregoing is the link in the chain of messages.

A message can have several previous messages. After all, a message approval can follow a delivery, but also a renewed delivery. The final message has a reject and an approval as the previous message. The chain of messages is therefore not linear, but is a mathematical graph. Most simple graph is:

start -> rejection -> approval -> end

5.1.2 MessageType

Not every message is the same. A distinction is made between message types. In the example above, rejection is a different message type than approval. This is not just another name for a message, the content of the message is different between a disapproval and an approval. For example, a disapproval will contain a motivation. In an approval, motivation might not be necessary and therefore not part of the message type 'approval'.

5.1.3 Message from person to person

The chain/count of messages always goes between two people. It is not allowed to send a message to random persons, or to send it to an organisation. These persons are recorded in the project setup (section

1.7). There is a condition for sending a start message from person Urbain Servranckx to person Georges Remi.

This condition is that the persons must fulfill a certain role for their organisation. In fact, a message is sent from "Person in Role of Organisation" to "Person in Role of Organisation". So person Urbain Servranckx (in the Assessment role of the organisation Municipality of Rommelmegem) sends a start message to person Georges Remi (in the Advisory role of the organisation De Blauwe Lotus).

5.1.4 Sending a message

Before a person can send a message, certain rules must be met. A start message may only be sent by people who have been assigned a certain role in the project specific file. A rule also applies to the receiving person. First of all, there can only be one receiving person. This person must also have been assigned a certain role in the project specific file. In the example, these are the initiating role Assessment and receiving role Advisory. Person Urbain Servranckx sends the start message of type "Assignment" to person Georges Remi.

Example 11: Example message (Task)

```
1 Bericht
2 Type: Opdracht
3 Unieke identificatie: _9a163e84-a83e-47ea-b5f3-220c403da98b
4 Verzenddatum: 2021-12-23T14:38:56
5 Richting (Van-Naar): TRUE
6 Transactie: _f8275cfe-3d8d-4bda-a05b-164a5087a1af
7 Inhoud: _170ef962-685f-4fc8-8728-df1bfe2317ba
```

The receiving person can only send a response message to the initiating person. So Georges Remi sends a disapproval (or approval) to Urbain Servranckx.

Example 12: Example message (Rejection)

```
1 Bericht
2 Type: Afkeuring
3 Unieke identificatie: _2bbecc10-948e-4b1a-b6ce-3a133576297c
4 Verzenddatum: 2021-12-27T10:13:42
5 Richting (Van-Naar): FALSE
6 Transactie: _f8275cfe-3d8d-4bda-a05b-164a5087a1af
7 Inhoud: _657ad592-5d76-4248-b959-68af07465cd6
```

After this, Urbain sends another response message to Georges. This goes back and forth until the final message is sent.

Note: You cannot send a copy of a message (as a cc). When sending a first message, the order of the response messages must also be determined, so that the messages follow a fixed pattern. For example the pattern:

start -> request -> delivery -> rejection -> redelivery -> approval -> end

This enforces that a new delivery is made after a rejection. An example of another pattern:

start -> for information -> acknowledgment of receipt -> end

Simplified, you could say that this pattern (or the order of message type) is determined by the type of the transaction that is started. Each message has a reference to the transaction. This transaction - just like a message - is of a certain type.

Example 13: Example message (Assignment)

```
1 Transactie
2 Type: Opdrachtverstrekking
3 Unieke identificatie: _f8275cfe-3d8d-4bda-a05b-164a5087a1af
4 Nummer: GRG000282
```

5 Omschrijving: Opdracht voor project Realisatie West-/Oostlijn
6 Van: _e0ad4954-7986-417c-96d6-813efa36e86b
7 Naar: _752b1dfd-dd91-4404-9708-39b91047ca66
8 Project: _badc9dab-807e-4319-bd56-60c57605e109;

The regulations and rules that the message communication must comply with are recorded in a project communication template. This template contains the definition of roll types, message types and transaction types. The content of a message type is also defined in this template. This template is called a framework. The term template already indicates that it is a blueprint that the transactions and messages must comply with.

5.2 Attachments

Each message can contain 0, 1 or more attachments. Attachment here means a binary file, such as a photo or document. The form is, for example, a jpg, pdf or zip file. The purpose of an attachment is to substantiate and/or clarify the information in the message.

5.2.1 Mandatory Attributes

Any binary file smaller than 10GB can be sent with a message. An additional requirement applies to the sum of all attachments of one message, namely that the combined size of the attachments is less than 10GB. There are no further requirements for the physical file. However, requirements are set for the properties/metadata of the appendix in accordance with the framework (see paragraaf 2.10 about AppendixType).

In addition to the type, an attachment has a fixed set of mandatory default properties of the file, such as name, location, type, version and date. The name of the property indicates which information of the file should be recorded.

Example 14: Example message (Assignment receipt)

1 Bijlage
2 Type: Opdrachtbon
3 Unieke identificatie: _a0186218-82d9-4b0f-a616-8aa1f162128a
4 Naam: Bon_9601.pdf
5 Bestandslocatie: ???
6 Bestandstype: application/pdf
7 Bestandsversie: DF43111
8 Datum: 2021-10-23T08:31:58
9 Bericht: _9a163e84-a83e-47ea-b5f3-220c403da98b
10 Inhoud: _04cdb00c-fd4d-4e58-b0ad-0cc19a3b07c9, _a1cd32d-ae5ed41-e0bc-1ee28b1c08d1

An appendix has a reference to the message of which it is a substantiation and/or clarification.

5.2.2 Optional properties

In addition to the mandatory properties, an appendix has many optional standard properties, which indicate a document as an attachment, such as: document identifier, document version, document reference.

5.2.3 Additional properties

It follows from the characterisation in the project communication template (or framework) whether the appendix has additional properties. These additional properties are not mandatory. The structure of the additional properties is similar to the structure of a message. Where a message refers to 1 or more complex elements, an attachment refers to 0 or more complex elements. A complex element consists of simple elements. And a simple element has a value.

Example 15: Example appendix with optional properties

```
1 Bijlage
2 Type: Bon_van_de_opdracht
3 Unieke identificatie: _a0186218-82d9-4b0f-a616-8aa1f162128a
4 Naam: purchase-order-9601.pdf
5 Inhoud: _04cdb00c-fd4d-4e58-b0ad-0cc19a3b07c9, _a1cd32d-ae5ed41-e0bc-1ee28b1c08d1
6 Complex Element
7 Type: Gegevens_van_de_opdracht
8 Unieke identificatie: _04cdb00c-fd4d-4e58-b0ad-0cc19a3b07c9
9 Elementen:
10 Simpel Element
11 Type: nummer_van_de_opdracht
12 Waarde: ROM/2022-666
13 Simpel Element
14 Type: naam_opdrachtgever
15 Waarde: Gemeente Rommelgem
16 Complex Element
17 Type: Betalingsgegevens
18 Unieke identificatie: _a1cd32d-ae5ed41-e0bc-1ee28b1c08d1
19 Elementen:
20 Simpel Element
21 Type: iBAN
22 Waarde: RABO28932392
23 Simpel Element
24 Type: naam_van_de_rekeninghouder
25 Waarde: Afdeling Inkoop, Gemeente Rommelgem
26 Simpel Element
27 Type: BTW_Nummer
28 Waarde: 2933092392809B01
```

The available types of appendices are defined in the project communication template (or framework) and which additional properties a particular type can have. The structure is the same as the structure of the message type. An appendix type has references to 0 or more complex element types. A complex element type consists of 1 or more simple element type. And a simple element type has one reference to a value. N.B. There is no table definition (i.e. no complex in complex element, see paragraph ???) per appendix.

Example 16: Example appendix with optional properties

```
1 Bijlage
2 Unieke identificatie: Bon_van_de_opdracht
3 Omschrijving: Opdrachtbon ten behoeve van opdrachtverstrekking
4 Complex Elementen: Opdrachtgegevens, Betalingsgegevens
5 Complex Element
6 Unieke identificatie: Gegevens_van_de_opdracht
7 Omschrijving: Gegevens van de opdracht
8 SimpelElementen: Opdrachtnummer, Naam_opdrachtgever
9 Complex Element
10 Unieke identificatie: Betalingsgegevens
11 Omschrijving: Gegevens voor betaling
12 Simpel Elementen: Naam_van_de_rekeninghouder, BTW_Nummer
13 Simpel Element
14 Unieke identificatie: opdrachtnummer
15 Omschrijving: Het unieke identificatie van de opdracht
16 Type: Tekst
17 Simpel Element
18 Unieke identificatie: naam_opdrachtgever
19 Omschrijving: Naam van de organisatie, die opdracht verstrekt
20 Type: Tekst
21 Simpel Element
22 Unieke identificatie: iBAN
23 Omschrijving: IBAN nummer
24 Type: Tekst_verplicht
25 Simpel Element
26 Unieke identificatie: naam_van_de_rekeninghouder
27 Omschrijving: Naam van de rekeninghouder
```

```

28 Type: Tekst_verplicht
29 Simpel Element
30 Unieke identificatie: bTW_Nummer
31 Omschrijving: BIW Nummer van de organisatie , die opdracht verstrekt
32 Type: Nummer
33 Waarde
34 Unieke identificatie: Tekst
35 Basis XML Type: STRING
36 Waarde
37 Unieke identificatie: Tekst_verplicht
38 Basis XML Type: STRING
39 XSD Restrictie: minlength=1
40 Waarde
41 Unieke identificatie: Nummer
42 Basis XML Type: INTEGER

```

5.2.4 Is an appendix required?

For a certain type of message it can be enforced that at least 1 attachment must be added. Also which appendix type this should be can be enforced. These conditions are included in the project communication template (or framework) (zie paragraaf 2.1, 2.2, 2.7). A type of message has a flag (Yes/No) whether an appendix is required (zie paragraaf 2.2).

Example 17: Example message with mandatory appendix (Proposal for Amendment)

```

1 Bericht
2 Unieke identificatie: Voorstel_tot_Wijziging
3 Omschrijving: Voorstel tot een wijziging
4 Bijlage verplicht: Ja
5 Bijlage typen: Foto , Offerte , Berekening
6 Complex Elementen: Voorstel , Wijziging , Planning , Opmerkingen

```

The maximum number of appendices cannot be prescribed. The minimum number is 0 or 1 (appendix required: Yes/No)

5.2.5 Transport of an attachment

The transport protocol for messages and attachments is SOAP. The attachments (binaries) are first transported from sending to receiving software. Due to the potential size of an attachment (maximum 10GB), each attachment is sent in chunks. The receiving software builds the original attachment with the received chunks. There is a checking mechanism (checksum) to check whether all chunks have been received and whether all chunks are correct.

After sending all attachments, the sending software sends the message (the metadata). The receiving software relates metadata to previously received binaries so that the exact same information is available on the sending and receiving side. The transport is successful.

5.2.6 Validation

The software must validate attachments and message before sending.

The software must validate received attachments and message. The result of the validation is sent back to the sending software. Validation takes place against:

1. W3C Standaarden waarop VISI gebaseerd is (XML, XSD, SOAP)

2. VISI Standaard
3. VISI Raamwerk (cq. Projectsjabloon)
4. VISI Project Specifiek Bestand

The received message contains the identification of the project and of the addressed 'person in role' (of organisation). With this data, the project and project specific file can be traced, and the content of the message can be validated against the content of the project specific file. The message received has a reference to the framework (or project template). The common functionality of XML is used for the reference: namespace. With this reference, the framework can be traced, and the content of the message can be validated against the definitions in the framework. For validation against the framework, the xsd version of the framework can be used.

The validation process is asynchronous. This means that after receiving the attachments and the message in the receiving software, the validation does not have to take place immediately. The result of the validation is returned to the sending software with a reference to which the validation result belongs. The validation result has two options: no error (0), error (1).

Optionally, additional text can be sent with the error code of the validation (0 or 1) to clarify the error. For example: that an attachment is missing, or the message does not conform to the project template (or framework), or the project is unknown.

5.2.7 Behavior message fields

text

5.2.8 Changes to a framework

Two types of changes can be made during the duration of a project. The first change concerns changes to the project, such as changed project name, changed roles. For example, an extra project employee or a new project employee as a permanent replacement for a current employee. The second change concerns changes to the project communication template, such as changed transaction type, changed message type.

Any change must be implemented at all organisations involved. This means that the modified files (project specific file and/or framework) must be provided to all organisations involved in the project. Because changes can affect the content of a project, the content of the change must be coordinated in advance between project employees.

Example 18: Example change in the framework

Compare the first change to a personnel change in a project. This change is agreed in advance between project leaders. And compare the second change with a process-based change of, for example, the approval process of documents. Any such change will be agreed in advance between contract managers with perhaps support from the legal department.

5.2.8.1 Versioning Open processes

Supporting old frameworks and transitioning to new versions of the framework

5.2.8.2 Authorisation/Temporary Substitution

The change with the least impact is the authorisation change. As described in paragraph 1.6.2, provision is made for appointing one or more authorized representatives for a person in role (of organisation). If an authorized person performs an action, this is always on behalf of the authorized person. An authorisation can be removed, because the authorized person can always complete the action. When an authorisation is added, the authorized person can act directly on existing transactions of the authorized person, and initiate new transactions (on behalf of the authorized person). An authorized person can never be removed (zie paragraaf).

5.2.8.3 New people

New people can be added to a project. These people can also be assigned a role. This may be a role that is already being fulfilled. For example, the advisory role is filled by several people in a project.

Please note that new individuals cannot a) participate in existing transactions and messages, or b) take over existing communications. If desired, use the authorisation/temporary or permanent replacement options.

5.2.8.4 Permanent replacement

If done properly, changing the permanent replacement has a relatively small impact. As described in paragraph 1.6.1, provision is made to appoint one permanent replacement for a person in role (of organisation). For traceability and transparency, a) authorized persons can never be removed, and b) authorized persons' follow-ups can never be changed. This always creates an unbroken chain of successors of person in role (of organisation). In other words, no circular reference or branches. That is why succession is made explicit by appointing a permanent successor.

Example 19: Permanent replacement

Peter has Role A and Role B (PIR1 and PIR2). Trudy replaces role A (PIR3 successor from PIR1) and Oscar replaces role B (PIR4 successor from PIR2). At a certain moment Hank takes over responsibility for role A from Trudy (PIR5 successor from PIR3). Later on, Peter returns to the project and takes on both roles again (PIR6 successor from PIR5 and PIR7 successor from PIR4)

If you now look at role A, there is an unbroken chain from the first person responsible to the current person responsible. From that moment on, Peter handles the transactions of PIR1 (himself), PIR3 (Trudy), PIR5 (Hank) and PIR6, (the current PIR of himself)

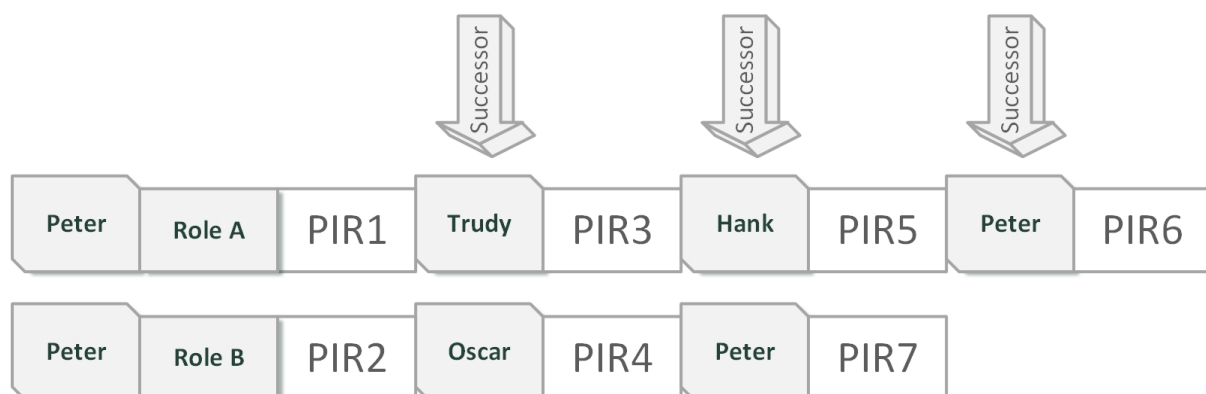


Figure 19: Permanent replacement - successor

5.2.8.5 Removing people

With the exception of a permission, a person cannot be removed from a project. Except if a person in role (of organisation) has not connected any transactions to his role (or to any successor roles). In which case removal is possible.

The only option to deactivate a person for a project is to appoint a permanent replacement for this person. For this, the property is successor of a person in role (of organisation). Should a person with an advisory role be removed from a project, then this person in role (of organisation) will receive a successor (with the same advisory role). This can also be an existing person (with the same advisory role).

Example 20: Removing people

```

2 Unieke identificatie: _b16d7aeb-6a5e-4326-9283-aef843684f5c
3 Contactpersoon: _d4e02752-2ac6-4700-ae67-d33f428f78fb
4 Rol: _1252a67a-5f69-4a9c-b4ca-506a4459c7c3
5 Organisatie: _87e9144d-04d6-4a5f-bdcb-b50e96c2b498
6 Opvolger: _666f01b3-6714-44cc-8865-0a9aelf938ce
7
8 Persoon in Rol van Organisatie
9 Unieke identificatie: _666f01b3-6714-44cc-8865-0a9aelf938ce
10 Contactpersoon: _c6e4430e-65aa-4e55-a8db-1beb6d3f4250
11 Rol: _1252a67a-5f69-4a9c-b4ca-506a4459c7c3
12 Organisatie: _87e9144d-04d6-4a5f-bdcb-b50e96c2b498
13
14 Rol v
15 Unieke identificatie: _1252a67a-5f69-4a9c-b4ca-506a4459c7c3
16 Naam: Adviserende rol
17 Omschrijving: Deze rol geeft een onafhankelijk advies op basis van competenties en
    deskundigheid binnen het kader van project. Bij het advies doorloopt de rol een
    aantal vaste stappen. Rol inventariseert gegevens, weegt, motiveert en beoordeelt
    deze informatie en geeft vervolgens het advies.

```

5.3 Changing framework

This is the change with the greatest impact. In addition to changing the project communication template (or framework), the project setup in the project-specific file must also be updated in accordance with the framework. The project specific file has a reference to the framework. If the framework changes, this reference must also change. So minimal updating is modifying the reference to the framework. The usual functionality of XML is used for this reference: namespace. A modified project communication template must contain a new unique namespace.

Example 21: Adjusting framework

```

1 Project
2 Unieke identificatie: UAV
3 Omschrijving: Uniforme Administratieve Voorwaarden
4 Namespace: http://www.visi.nl/schemas/20160331/uav_20-10-2020\_1723

```

If an amended project communication template (or framework) is supplied without a modified project-specific file, then the "previous" project-specific file also applies to the modified project communication template (or framework).

The reference to the associated framework is also included in each message. This makes it possible to find out for each message which framework the message has been drawn up in accordance with, and to check whether the message is valid in accordance with this framework. And which subsequent messages are possible as a next step.

5.4 Behaviour of software for initiating a transaction or replying to a message

5.4.1 Behaviour of software around connected message flow

text

5.4.2 Behaviour around offering and not offering messages based on framework/role

text

5.5 Archiving VISI Projects

The exchange and archiving format for (completed) VISI projects is referred to in this documentation as a "VISI archive". The following paragraphs describe which requirements the archive must meet and how it is used.

5.5.1 The requirements for the VISI archive

1.) The information that must be present in the archive includes all data from a maximum and a minimum of 1 project, namely:
 - All frameworks of the project.
 - All project-specific messages from the project.
 - All sent and received VISI messages in the format xml.
 - All sent and received attachments in the original format.
2.) Information from the archive must be transparent, without exotic/VISI (viewer) software:
 - Attachments are archived in the original format in which they were sent by the end user/organisation. So a PDF file can be found as a PDF file. So archiving of attachments is not done in BASE64, MTOM or other similar formats.
 - The archive must have a structure so that the communication can be navigated in a logical manner.
3.) The archive must be importable:
 - The information present in the VISI archive must be complete enough to reproduce/read the entire communication into a VISI solution of your choice.
 - The connection between a message and an attachment must be retrievable.

N.B. In a previous document the following was rightly described: "The correct reconstruction of messages, framework(s) and project specific message(s) can only be done with projects that run under the 1.2 system because there all messages work with a DateTime value. In the 1.1 system, only a Date value was used, which will make it difficult to restore messages per day in the correct order." Therefore in addition, the requirement:

 - If the VISI software has the time at which a VISI message was sent, this information is also archived. If this information is not available, the first message within a transaction on that day will be timed as '00:01', and for each subsequent message, 'one minute later' will be used. Justification: Allows 1440 messages within a transaction to be numbered within a day.
4.) Importing archived projects must be possible:
 - After the import, the VISI project must again function entirely in accordance with the VISI System.

Figure 20:

5.5.2 Project archive application

The table below gives an example of the application in practice and provides insight into the structure of a VISI project archive. In the example, a VISI project is archived in a file system structure.

```

1 Communication
2 |
3 |--- 2011/ // Year in which the transaction started (yyyy)
4 | |--- 02/ // Month in which the transaction started (MM)
5 | | |--- 28/ // Day the transaction started (dd)
6 | | | |--- Weekly report no. 1\_tc1b35f17-8351- 4bb9-a4cb- f2fb53a16018/
7 | | | |--- 20110228\_1524\_Delivery\_weekly report\_m42f15ce5-c97a-4c34-9eb0-
   | | | |ef9894777f36.xml // VISI Messages
8 | | | \--- 20110228\_1524\_10A06 attachment to weekly report \_a1e8d4d55-3a7a- 4508-897c-
   | | | a1a261c4c629.pdf // Attachments
9 |
10 Project
11 |
12 |--- 2009-01-01T00:00:00.0Z |--- Raamwerk.xml
13 | | \--- Project specific message.xml
14 |
15 |--- 2010-12-31T00:00:00.0Z |--- Raamwerk.xml
16 | | \--- Project specific message.xml

```

- Weekly report no. 1_tc1b35f17-8351- 4bb9-a4cb- f2fb53a16018: First 50 characters ⁵ of the subject of a transaction, followed by id (=GUID) of the transaction
- VISI Messages: Date sent of the message in the format 'yyyyMMdd_HH:mm ', followed by the message type, followed by the message GUID. Extension is always .xml
- Attachment(s): Date sent of the message in the format 'yyyyMMdd_HH:mm ', followed by the original file name followed by the GUID of the attachment and the extension of the original sent file. Files are therefore included in the same (transaction) folder when the message is stored.

Figure 21: VISI project archive

VISI messages are structured by date. Optionally, the complete archive can be saved in ZIP format.

Visual example:

Figure 22: VISI archive structure

5.5.3 VISI export

5.5.3.1 Changes In Exports VISI 1.8 Changes overview

- Prior to VISI 1.8, the export was a list of folders labelled per date, which contained folders per transaction. The transaction folder name would be the subject of the transaction. The files within the transaction folder consisting of the messages within that transaction would also be named based on the subject. This resulted in undesirably long path names, which have become unmanageable since at least Windows has a maximum path length it can handle. To combat this, the date folder structure is deprecated starting from VISI 1.8, and transactions will be directly placed in the root folder. The transaction folders are named based on the date plus their VISI Transaction ID instead of using their subject. The messages of a transaction are contained within the transaction folder. An example of this new structure is T_20240725_134500000_T01. For a detailed explanation, see the detailed changes below.

⁵Only alphanumeric characters and spaces are allowed. Non-alphanumeric characters must be converted to spaces.

- To improve the readability and create a better overview for users browsing an archive, an HTML page will be added with a table containing an overview pointing to the underlying transaction folders. The path in the table will be relative.
- Since VISI 1.7 added the signing of messages to prove their authenticity, this signing is also added to the exporting procedure. Therefore, 1.8 will support exporting messages, including their signature.

Detailed changes

The export file will contain transactions, project information, and an HTML file to navigate the export file. Prefixes are used to indicate the type of the folder/file. This is done to provide clarity to the reader. A letter followed by an underscore indicates the type. The following prefixes are used:

- **T**: Transaction
- **M**: Message
- **A**: Attachment
- **F**: Framework
- **P**: Project Specific Message
- **I**: Information regarding the project

Transactions are stored as a folder and use the prefix "T_", followed by the start date/time of that transaction, followed by the unique ID of this transaction in VISI. The generic structure: T_[DateTime]_[transactionID]. The date/time uses the following format: YYYYMMDD_HHmmsnnn.

- YYYY = Year notation. With four(4) digits.
- MM = Month number. With leading zero when below 10.
- DD = Day of the month. With leading zero when below 10.
- HH = Hours in 24h format. With leading zero when below 10.
- mm = Minutes. With leading zero when below 10.
- ss = Seconds. With leading zero when below 10.
- nnn = till a thousandth precision of a second.

For example, 13:45 on the 23rd of July 2024 would be: "20240323_134500000.

The unique ID in the folder name can take any form, as long as it is unique and points to the transaction within the framework.

An example of a complete folder name could be:

1 T_20240723_134500000_T01

Within this folder, all messages tied to transaction T01, which started at 13:45 on July 23rd, 2024, are present.

Messages in a folder also have a specific prefix structure to help identify them. Messages can have attachments and attributes such as confirmation messages. To lower the number of characters needed, only the message will have the message ID, and the sequence number of appendices is used to refer back to that message. For example: Message uses the format: "M_[SequenceNumber]_[DateTime]_[messageID].xml" Appendixes within this message would have the format: "M_[SequenceNumber]_A_[AppendixID].*" The appendix can be of any file extension; therefore, an asterisk is used as a wildcard operator. The confirmation message that tells the other soap server the message has been received has the format:

"M_[SequenceNumber]_C_[DateTime]".

Besides transactions containing messages, the framework and configuration settings are provided in the Information folder: "I_[DateTime]". This structure allows for multiple versions of the framework and configuration. In cases where multiple "I_" files are present, it means there have been numerous versions of the framework and settings. The folder with the earliest dateTime in the name was superseded by the newer version. The newer version became active at the dateTime in the second file's name. This principle can be repeated when several versions are present.

The framework files within the "Information" folder follow the following naming format: "F_[Framework_Namespace]". The same is used for the configurations with the format: "P_[Framework_Namespace]". The framework namespace in the archive is the part after the "http://www.visi.nl/schemas/version/" part of the Project-Type namespace attribute.

Lastly, the export will contain an HTML that helps the reader navigate between the folder structures and files. This file is called Export.html.

The HTML page has a table with the following columns:

- Name of Transaction (message/name or message/description) [when transaction line]
- Subject of Transaction/Message (Framework/TransactionType/description or Framework/MessageType/description)
- DateTime of Sent Message (message/dateSend)
- Message Sequence Number (derived??? of message/<PersonType>/name + message/<TransactionType>/number)
- Sender (message/<PersonType>/name)
- Receiver (message/<PersonType>/name)
- Confirmation time [optional]
- Link field (each field links to the folder, message, or appendix)
 - Transaction ID [on transaction line] show (message/<TransactionType>/id
 - Message ID [on message line] show (message/<MessageType>/id
 - Appendix ID [???] show (message/<AppendixType>/id

Clicking the Transaction ID will open the relative path to the transaction. Clicking the Message ID will open the relative path to the message within the Transaction. Clicking on the Appendix will guide you to the appendix.

Optional: Clicking sender/receiver (person) will display all messages in which that person is in that role.

Below is a complete overview of the file/folder format with the listed options:

```
1 root/
2   T_[DateTime]_[transactionID]/
3     M_[SequenceNumber]_[DateTime]_[messageID].xml
4     M_[SequenceNumber]_A_[AppendixID].*
5     M_[SequenceNumber]_C_[DateTime].xml
6   I_[DateTime]/
7     F_[Framework\_Namespace].xml
8     P_[Framework\_Namespace].xml
9   Export.html
```

Note: The confirmation message is optional in the archive.

5.5.3.2 Archive structure (#72), confirmations(#67), maps (#70), transfer (#151) <https://github.com/bimloket/visi/wiki/Bijlage-11-Richtlijn-voor-het-archiveren-van-VISI--projecten>

Changing the structure

- Soap (soap envelope body) messages get added to the archive one by one
- - Check whether the archive is correct (transactions (follow number), messages, attachments in a XML)
 - > include in the xml validator, own issue. -> Check whether the vision is still the same after reading (checklist) -> Check the number of messages and the status of the messages.

In order to make the VISI archive more workable, it is necessary to adjust the layout of this archive. The current method as prescribed in the standard quickly causes problems for the user when using Microsoft Windows, since the maximum path length (260 characters) can easily be exceeded.

Full dates are not allowed for directory names: 2010-12-31T00:00:00.OZ is not a valid directory name.

Date and time sent should not be used to distinguish soap messages because there are too many duplicates. This might be possible if we also included milliseconds.

- Date/time of the transaction is the same as that of the first message of that transaction.
- Information map is for context information of the messages.
- Same applies to the names of frame and psb. These can be recognized by their prefix.
- The order of the frames can be determined by date_time.
- Sequence number indicates the order of messages in the transaction, a sequence number is a maximum of 2 digits. The attachments get the same sequence number as the message they belong to
- The namespace of the message determines in which framework it runs.
- What is in the 'inbox' is automatically determined based on the order in the framework. Is not explicitly included in the archive. item M and A are related based on a GUID. The A GUID comes from the appendix reference of the message M.
- Confirmations and Errors also store in the archive
- Due to signing from 1.7, issue #70 has been resolved in this archiving. - transfer #151 is covered by this archiving setup.

In this way we provide the following:

- The files can be found consecutively in time.
- The file name shows what type of file it is (message, attachment, framework, psb).
- The maximum path length can be predicted and remains well below 260 characters. The total path length then becomes 1 letter + '_' + 2 for the sequence number + '_' + 4 for the year + 2 for the month + 2 for the day + '_' + 9 for the time + '_' + 36 for the guid = 60 per subdirectory. Total 57+'/' + 60+'.' + '???' = 122.
- Include the sequence number in the archive; Extra file with sequence numbers of the framework. This must become part of the psb.

For the user convenience outside the software, the following will be in the root of the archive. An xml describing the transactions and messages and all soap messages with their soap guid and all attachments with their soapguid and id as in the soap message. The structure is then no longer as easy to read as in the current export, but there are also no problems with file names that are too long. The maximum file name is the name of the folder (projectguid) + soapguid + attachmentID. Searching must then be done via the export.xml. But if there is a standard export xml, it is quite easy to create a viewer with an xslt and css file. The export xml could for example be per 1000 messages per xml. Whether this should be an HTML/MD is still being investigated.

Once this is processed in the standard, exporting/importing a project should also become part of certification. For this, it is necessary that there is a representative project to export/import. This should be facilitated by one of the stakeholders.

Use case:

We have the option to transfer a transaction to a newer framework/of a later date. This gives you, for example, transactions that started in framework 2 of the project, but later continued in framework 5. In itself, it is visible in the archive that this transaction works with framework 5, but there is a chance of errors if the importing software were to assume that this transaction still has to be processed in framework 2. Importing software can then, for example, because the date of the first message is between the reading date of framework 2 and 3, link it to 2, while the intention is that the transaction is linked to framework 5, which is in the namespace of the last message of the transaction. The old messages are therefore still in framework 2, the new ones in framework 5. This transaction therefore goes over more frameworks. The software must be able to understand this.

Comments on mutual exchange of archives:

- namespace of framework is missing in PSBs in the export
- according to Technia a minlength may not be used together with enumerations in an XSD restriction of a UDT.

Build converter between multiple archive types.

Old archive has IDs. The formatting of the IDs must be added to the standard. Dates are not stored correctly in the transaction information. This must be stated in the documentation.

— Where in the documentation is it stated how to deal with deleted persons. — What about the successor and substitute of the successor.. do they have the correct rights? (incl. old messages) — Both PSBs are adjusted via authorisations. Can this be done more efficiently. Sometimes internal roles can be adjusted.

5.6 META FRAMEWORK

The VISI META framework is a standard part of the system and is intended for the exchange of a new contract-specific framework and the exchange of a modified framework or project-specific message between project partners. The META framework itself is project-transcending. After all, the META framework must be available before a VISI environment is set up for a specific project.

To allow project partners to communicate via the META framework, both project partners must have VISI compatible software into which a META project containing the universal META framework and a valid META project-specific message is loaded. This message contains the server addresses of the project partners and the organisation and personal data. Each organisation manages its own META project-specific message and supplements this with the data of project partners.

5.6.1 The available roles and associated responsibilities

The VISI META framework contains four roles. The roles project initiator and project executor are available for initiating and ending a project. As a rule, the project initiator is the Contracting Party and

the project executor is the Contracted Party.

For changing a framework or a project-specific message, the roles VISI communication manager and Change initiator are included. The VISI communication manager has the responsibility to authorize an adjustment. The Change Initiator is responsible for initiating a change. Both roles can be filled by all parties involved. In this way, any party involved can initiate or accept a change.

5.6.2 The available transactions within the META framework

The META framework contains two transactions with the goal of:

- Spreading an initial framework and project specific message (VISI project initiation).
- Spreading a modified framework and/or project-specific message (New or modified framework or project-specific message transaction).

The communication per transaction is shown below in a transaction diagram.

VISI project initiation

Figure 23: projectinitiatie

Spreading a modified framework and/or project-specific message

Figure 24: berichtverspreiding

5.6.3 Application of the META framework

The META framework offers software suppliers the option of (semi)automatically deploying a new or modified framework and/or project-specific message. In addition, the META framework benefits interoperability because there is more certainty whether the correct version of a framework and/or project-specific message is in use.

5.7 Appendix 6 Metaframework – Rationale

5.7.1 Introduction

In 2010/2011, the VISI Technical Committee submitted a proposal on behalf of the VISI User Group to address traceability of messages after changes to a framework during a running project.

The motivation for this proposal was that long-term projects may require adjustments to the framework during execution. Such adjustments may include:

- A. adding transactions;
- B. changing transactions;
- C. deleting transactions.

At that time, framework changes were only allowed when no outstanding transactions existed. This restriction was considered too strict for practical use in long-running projects.

A distinction was therefore introduced between:

- **Compatible changes**, such as adding new transactions or data types, which do not affect existing message flows.
- **Incompatible changes**, such as removing or restructuring message flows, which require multiple framework versions to coexist.

During analysis it was also observed that VISI communication management was often handled via informal channels (telephone, e-mail), and not through a formal system mechanism. This led to the concept of using VISI itself to manage communication about VISI communication, referred to as meta communication. To support this concept, the idea of a META framework was introduced. In addition, a later proposal (2013) introduced automated distribution of project data (frameworks and project-specific messages). The META framework is the resulting technical concept enabling both controlled framework evolution and structured distribution of project data.

5.7.2 Framework modification constraints

Until now, the framework of a project could only be changed if there were no outstanding transactions. This restriction was considered strict and not feasible in many practical long-term project situations. A requirement therefore emerged that the framework should also be modifiable while transactions are still outstanding.

With regard to framework modifications, a distinction is made between changes that are compatible with the message flow of the original framework and changes that are not. In general, extensions (such as new transaction types, new message types, or new complex/simple data types) do not pose problems, meaning that messages initiated under the original framework can still be completed under an updated framework. In contrast, structural changes (such as deleting or moving MessageInTransaction types) are generally incompatible with the original message flow. These changes require that different versions of a framework coexist within the same project environment, each with its own message flows, to ensure continuity of ongoing transactions.

The META framework addresses this problem by enabling controlled propagation of updated frameworks and ensuring consistency across project participants.

5.7.3 Design rationale

The META framework is included as a standard component of VISI-certified software.

A key design question was how the META framework itself should be distributed. Introducing a higher-level meta-meta framework was considered but rejected, as this would only shift the same structural problem to a higher level.

The chosen solution was therefore to include the META framework as a fixed, non-project-specific part of the VISI Standard implementation.

Consequently:

- the META framework is not project-configurable;
- it is not expected to change within a project context;
- changes only occur when the VISI Standard itself is updated.

This design ensures stability and interoperability across implementations while avoiding recursive framework dependency problems.

5.8 Appendix 6 Metaframework – Normative specification

5.8.1 Scope

This appendix defines the normative behaviour of the VISI META framework for implementation in VISI-compatible software.

The META framework is used for:

- distribution of an initial framework and project-specific message;
- distribution of modified frameworks and/or project-specific messages.

A meta project-specific message contains the data of the parties/organisations involved (including the SOAP Servers).

5.8.1.1 Roles

The META framework defines the following roles:

- **Project initiator** – responsible for initiating a VISI project.
- **Project executor** – responsible for executing and closing the project communication.
- **VISI communication manager** – responsible for authorising changes to framework or project-specific messages.
- **Change initiator** – responsible for initiating a change proposal.

The roles VISI communication manager and Change initiator may be fulfilled by any participating organisation.

5.8.2 Transactions

The META framework supports two transactions:

1. Distribution of an initial framework and project-specific message.
2. Distribution of a modified framework and/or project-specific message.

5.8.3 Behavioural rules

- The META framework SHALL be available in certified VISI software.
- The META framework SHALL NOT be modified on a per-project basis.
- Framework and project-specific message updates SHALL be distributed via META transactions.

5.8.4 Message flow (high level)

The META framework governs:

- initiation of VISI projects;
- propagation of updated frameworks;
- propagation of updated project-specific messages.

Figure 25: Distributing an initial framework and project-specific message

Figure 26: Distributing a modified framework and/or project-specific message

A transaction to add a new or modified framework or project-specific message to the VISI communication. ('i—>e' means 'initiator to executor'; vice versa)

1. Message (i—>e): *Proposal start of a new or modified framework or project-specific message.* This message can be sent to all partners for whom this framework or project-specific message is of interest. The new or amended framework or project-specific message is added as an appendix, together with any other documents that may be important for the assessment of the framework or project-specific message.
2. Message (e—>i): *Agree with proposal of a new or modified framework or project-specific message.* This message allows the recipient of message 1 to accept the proposal.

3. Message (e- >i): *Disagree with proposal of a new or modified framework or project-specific message.* This message allows the recipient of message 1 to reject the proposal. If necessary, a further explanation for the rejection can be added as an appendix. Because a proposal must be accepted unanimously, the partner in question already knows that the proposal has not been successful and can therefore regard the transaction as terminated.
4. Message (i- >e): *Proposal to start a new or modified framework not accepted.* If the proposal has not been accepted unanimously, this message can be used to distribute this outcome to the partners who did agree, possibly with a further explanation as an appendix.
5. Message (i- >e): *Proposed effective date of an accepted new or amended framework or project-specific message.* If the proposal has been accepted unanimously, the effective date can be determined with this message. The message therefore implicitly informs that the proposal has been unanimously accepted. In the case of an amended framework, the effective date determines the moment that new transactions transfer to the amended framework. Transactions that are already in progress must be processed according to the original framework. The effective date is sent in a non-changeable simple-typed date/time field.
6. Message (e- >i): *Agree with proposed effective date of a new or amended framework or project-specific message.* This message allows the recipient of message 5 to accept the proposal.
7. Message (e- >i): *Disagree with proposed effective date of a new or amended framework or project-specific message.* This message allows the recipient of message 5 to reject the proposal. An explanation of that decision can be included if necessary. However, because the proposal for a new or modified framework had already been unanimously accepted, a loop is continued with new proposals for effective dates until everyone has accepted it.
8. Message (i- >e): *Confirmation of effective date of an accepted new or modified framework or project-specific message.* If the effective date proposal has been unanimously accepted, the effective date can be confirmed with this message. In the case of a changed framework, the software will no longer allow transactions under the previous framework after the confirmed effective date.

5.8.5 Meta-framework XML

The meta framework is defined in XML as follows.

```

1 <visiXML_VISI_Systematics xmlns="http://www.visi.nl/schemas/20140331"
2   xmlns:xsd="http://www.w3.org/2001/XMLSchema"
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
4   <AppendixType id="APT-NieuwOfGewijzigdRaamwerkOfProjectspecifiekBericht">
5     <description>Nieuw of gewijzigd raamwerk en/of projectspecifiek bericht</
6     description>
7   </AppendixType>
8   <ComplexElementType id="C00-Algemeen">
9     <description>Algemeen</description>
10    <simpleElements>
11      <SimpleElementTypeRef idref="S00-Algemeen" />
12    </simpleElements>
13  </ComplexElementType>
14  <ComplexElementType id="C01-MotivatieVoorstelNietAkkoord">
15    <description>Motivatie voorstel niet akkoord</description>
16    <simpleElements>
17      <SimpleElementTypeRef idref="S01-MotivatieVoorstelNietAkkoord" />
18    </simpleElements>
19  </ComplexElementType>
20  <ComplexElementType id="C02-MotivatieVoorstelGecancelld">
21    <description>Motivatie voorstel gecancelld</description>
22    <simpleElements>
23      <SimpleElementTypeRef idref="S02-MotivatieVoorstelGecancelld" />
24    </simpleElements>
25  </ComplexElementType>
26  <ComplexElementType id="C03-Ingangsdatum">
27    <description>Ingangsdatum</description>
28    <simpleElements>

```

```

28         <SimpleElementTypeRef idref="S03-Ingangsdatum" />
29     </simpleElements>
30 </ComplexElementType>
31 <ComplexElementType id="C04-Start-project">
32     <description>Start project</description>
33     <simpleElements>
34         <SimpleElementTypeRef idref="S04-Moment-start-project" />
35     </simpleElements>
36 </ComplexElementType>
37 <ComplexElementType id="C05-Projectgegevens">
38     <description>Projectgegevens</description>
39     <simpleElements>
40         <SimpleElementTypeRef idref="S05-Project-ID" />
41         <SimpleElementTypeRef idref="S06-Projectnaam" />
42     </simpleElements>
43 </ComplexElementType>
44 <ComplexElementType id="C06-Beeindiging-project">
45     <description>Beeindiging project</description>
46     <simpleElements>
47         <SimpleElementTypeRef idref="S04-Moment-beeindiging-project" />
48     </simpleElements>
49 </ComplexElementType>
50 <ComplexElementType id="CeOrganisatieSOAPServer">
51     <description>Gegevens van SOAP server</description>
52     <helpInfo>Hier kan een omschrijving worden weergegeven</helpInfo>
53     <simpleElements>
54         <SimpleElementTypeRef idref="SOAPServerURL" />
55     </simpleElements>
56 </ComplexElementType>
57 <ElementCondition id="EL-S00-Algemeen">
58     <description></description>
59     <condition>FREE</condition>
60     <simpleElement>
61         <SimpleElementTypeRef idref="S00-Algemeen" />
62     </simpleElement>
63 </ElementCondition>
64 <ElementCondition id="EL-S04-Moment-start-project">
65     <description></description>
66     <condition>FREE</condition>
67     <simpleElement>
68         <SimpleElementTypeRef idref="S04-Moment-start-project" />
69     </simpleElement>
70 </ElementCondition>
71 <GroupType id="StandardGroupType">
72     <description>Standaard Groep</description>
73 </GroupType>
74 <MessageInTransactionType id="Mitt_1">
75     <initiatorToExecutor>true</initiatorToExecutor>
76     <message>
77         <MessageTypeRef idref="M01-
VoorstelNieuwOfGewijzigdRaamwerkOfProjectSpeciekBericht" />
78     </message>
79     <transaction>
80         <TransactionTypeRef idref="T01-
NieuwOfGewijzigdRaamwerkOfProjectSpeciekBerichtTransactie" />
81     </transaction>
82 </MessageInTransactionType>
83 <MessageInTransactionType id="Mitt_10">
84     <initiatorToExecutor>true</initiatorToExecutor>
85     <message>
86         <MessageTypeRef idref="M10-VoorstelNieuwVISIproject" />
87     </message>
88     <transaction>
89         <TransactionTypeRef idref="T02-" />
90     </transaction>
91 </MessageInTransactionType>
92 <MessageInTransactionType id="Mitt_11">
93     <initiatorToExecutor>false</initiatorToExecutor>
94     <message>
95         <MessageTypeRef idref="M11-AfwijzingVoorgesteldVISIproject" />

```



```

96     </message>
97     <previous>
98         <MessageInTransactionTypeRef idref="Mitt_10" />
99     </previous>
100     <transaction>
101         <TransactionTypeRef idref="T02-" />
102     </transaction>
103 </MessageInTransactionType>
104 <MessageInTransactionType id="Mitt_12">
105     <initiatorToExecutor>false</initiatorToExecutor>
106     <message>
107         <MessageTypeRef idref="M12-AcceptatieNieuwVISIproject" />
108     </message>
109     <previous>
110         <MessageInTransactionTypeRef idref="Mitt_10" />
111     </previous>
112     <transaction>
113         <TransactionTypeRef idref="T02-" />
114     </transaction>
115 </MessageInTransactionType>
116 <MessageInTransactionType id="Mitt_13">
117     <initiatorToExecutor>true</initiatorToExecutor>
118     <message>
119         <MessageTypeRef idref="M13-AnnuleringProjectInitiatie" />
120     </message>
121     <previous>
122         <MessageInTransactionTypeRef idref="Mitt_12" />
123     </previous>
124     <transaction>
125         <TransactionTypeRef idref="T02-" />
126     </transaction>
127 </MessageInTransactionType>
128 <MessageInTransactionType id="Mitt_14">
129     <initiatorToExecutor>true</initiatorToExecutor>
130     <message>
131         <MessageTypeRef idref="M14-BevestigingProjectInitiatie" />
132     </message>
133     <previous>
134         <MessageInTransactionTypeRef idref="Mitt_12" />
135     </previous>
136     <transaction>
137         <TransactionTypeRef idref="T02-" />
138     </transaction>
139 </MessageInTransactionType>
140 <MessageInTransactionType id="Mitt_15">
141     <initiatorToExecutor>false</initiatorToExecutor>
142     <message>
143         <MessageTypeRef idref="M15-ProjectIngericht" />
144     </message>
145     <previous>
146         <MessageInTransactionTypeRef idref="Mitt_14" />
147     </previous>
148     <transaction>
149         <TransactionTypeRef idref="T02-" />
150     </transaction>
151 </MessageInTransactionType>
152 <MessageInTransactionType id="Mitt_16">
153     <initiatorToExecutor>true</initiatorToExecutor>
154     <message>
155         <MessageTypeRef idref="M16-VerzoekBeeindigingProject" />
156     </message>
157     <previous>
158         <MessageInTransactionTypeRef idref="Mitt_15" />
159         <MessageInTransactionTypeRef idref="Mitt_18" />
160         <MessageInTransactionTypeRef idref="Mitt_21" />
161     </previous>
162     <transaction>
163         <TransactionTypeRef idref="T02-" />
164     </transaction>
165 </MessageInTransactionType>

```

```

166 <MessageInTransactionType id="Mitt_17">
167   <initiatorToExecutor>false</initiatorToExecutor>
168   <message>
169     <MessageTypeRef idref="M17-AkkoordBeeindigingProject" />
170   </message>
171   <previous>
172     <MessageInTransactionTypeRef idref="Mitt_16" />
173   </previous>
174   <transaction>
175     <TransactionTypeRef idref="T02-" />
176   </transaction>
177 </MessageInTransactionType>
178 <MessageInTransactionType id="Mitt_18">
179   <initiatorToExecutor>false</initiatorToExecutor>
180   <message>
181     <MessageTypeRef idref="M18-AfwijzingBeeindigingProject" />
182   </message>
183   <previous>
184     <MessageInTransactionTypeRef idref="Mitt_16" />
185   </previous>
186   <transaction>
187     <TransactionTypeRef idref="T02-" />
188   </transaction>
189 </MessageInTransactionType>
190 <MessageInTransactionType id="Mitt_19">
191   <initiatorToExecutor>true</initiatorToExecutor>
192   <message>
193     <MessageTypeRef idref="M19-AnnuleringProjectbeeindiging" />
194   </message>
195   <previous>
196     <MessageInTransactionTypeRef idref="Mitt_17" />
197   </previous>
198   <transaction>
199     <TransactionTypeRef idref="T02-" />
200   </transaction>
201 </MessageInTransactionType>
202 <MessageInTransactionType id="Mitt_2">
203   <initiatorToExecutor>false</initiatorToExecutor>
204   <message>
205     <MessageTypeRef idref="M02-VoorstelAkkoord" />
206   </message>
207   <previous>
208     <MessageInTransactionTypeRef idref="Mitt_1" />
209   </previous>
210   <transaction>
211     <TransactionTypeRef idref="T01-
212     NieuwOfGewijzigdRaamwerkOfProjectSpeciekBerichtTransactie" />
213   </transaction>
214 </MessageInTransactionType>
215 <MessageInTransactionType id="Mitt_20">
216   <initiatorToExecutor>true</initiatorToExecutor>
217   <message>
218     <MessageTypeRef idref="M20-BevestigingBeeindigingProject" />
219   </message>
220   <previous>
221     <MessageInTransactionTypeRef idref="Mitt_17" />
222   </previous>
223   <transaction>
224     <TransactionTypeRef idref="T02-" />
225   </transaction>
226 </MessageInTransactionType>
227 <MessageInTransactionType id="Mitt_21">
228   <initiatorToExecutor>false</initiatorToExecutor>
229   <message>
230     <MessageTypeRef idref="M21-BevestigingVoortzettingProject" />
231   </message>
232   <previous>
233     <MessageInTransactionTypeRef idref="Mitt_19" />
234   </previous>
235   <transaction>

```

```

235         <TransactionTypeRef idref="T02-" />
236     </transaction>
237 </MessageInTransactionType>
238 <MessageInTransactionType id="Mitt_3">
239     <initiatorToExecutor>false</initiatorToExecutor>
240     <message>
241         <MessageTypeRef idref="M03-VoorstelNietAkkoord" />
242     </message>
243     <previous>
244         <MessageInTransactionTypeRef idref="Mitt_1" />
245     </previous>
246     <transaction>
247         <TransactionTypeRef idref="T01-
NieuwOfGewijzigdRaamwerkOfProjectSpeciekBerichtTransactie" />
248     </transaction>
249 </MessageInTransactionType>
250 <MessageInTransactionType id="Mitt_4">
251     <initiatorToExecutor>true</initiatorToExecutor>
252     <message>
253         <MessageTypeRef idref="M04-VoorstelGecancelld" />
254     </message>
255     <previous>
256         <MessageInTransactionTypeRef idref="Mitt_3" />
257         <MessageInTransactionTypeRef idref="Mitt_2" />
258     </previous>
259     <transaction>
260         <TransactionTypeRef idref="T01-
NieuwOfGewijzigdRaamwerkOfProjectSpeciekBerichtTransactie" />
261     </transaction>
262 </MessageInTransactionType>
263 <MessageInTransactionType id="Mitt_5">
264     <initiatorToExecutor>true</initiatorToExecutor>
265     <message>
266         <MessageTypeRef idref="M05-VoorstelIngangsdatum" />
267     </message>
268     <previous>
269         <MessageInTransactionTypeRef idref="Mitt_2" />
270         <MessageInTransactionTypeRef idref="Mitt_7" />
271         <MessageInTransactionTypeRef idref="Mitt_6" />
272     </previous>
273     <transaction>
274         <TransactionTypeRef idref="T01-
NieuwOfGewijzigdRaamwerkOfProjectSpeciekBerichtTransactie" />
275     </transaction>
276 </MessageInTransactionType>
277 <MessageInTransactionType id="Mitt_6">
278     <initiatorToExecutor>false</initiatorToExecutor>
279     <message>
280         <MessageTypeRef idref="M06-IngangsdatumAkkoord" />
281     </message>
282     <previous>
283         <MessageInTransactionTypeRef idref="Mitt_5" />
284     </previous>
285     <transaction>
286         <TransactionTypeRef idref="T01-
NieuwOfGewijzigdRaamwerkOfProjectSpeciekBerichtTransactie" />
287     </transaction>
288 </MessageInTransactionType>
289 <MessageInTransactionType id="Mitt_7">
290     <initiatorToExecutor>false</initiatorToExecutor>
291     <message>
292         <MessageTypeRef idref="M07-IngangsdatumNietAkkoord" />
293     </message>
294     <previous>
295         <MessageInTransactionTypeRef idref="Mitt_5" />
296     </previous>
297     <transaction>
298         <TransactionTypeRef idref="T01-
NieuwOfGewijzigdRaamwerkOfProjectSpeciekBerichtTransactie" />
299     </transaction>

```

```

300 </MessageInTransactionType>
301 <MessageInTransactionType id="Mitt_8">
302   <initiatorToExecutor>true</initiatorToExecutor>
303   <message>
304     <MessageTypeRef idref="M08-BevestigingIngangsdatum" />
305   </message>
306   <previous>
307     <MessageInTransactionTypeRef idref="Mitt_6" />
308   </previous>
309   <transaction>
310     <TransactionTypeRef idref="T01-
NieuwOfGewijzigdRaamwerkOfProjectSpeciekBerichtTransactie" />
311   </transaction>
312 </MessageInTransactionType>
313 <MessageType id="M01-VoorstelNieuwOfGewijzigdRaamwerkOfProjectSpeciekBericht">
314   <description>Voorstel voor een nieuw of gewijzigd raamwerk of
projectspecifiekbericht </description>
315   <complexElements>
316     <ComplexElementTypeRef idref="C00-Algemeen" />
317     <ComplexElementTypeRef idref="C05-Projectgegevens" />
318   </complexElements>
319 </MessageType>
320 <MessageType id="M02-VoorstelAkkoord">
321   <description>Voorstel akkoord</description>
322   <complexElements>
323     <ComplexElementTypeRef idref="C00-Algemeen" />
324     <ComplexElementTypeRef idref="C05-Projectgegevens" />
325   </complexElements>
326 </MessageType>
327 <MessageType id="M03-VoorstelNietAkkoord">
328   <description>Voorstel niet akkoord</description>
329   <complexElements>
330     <ComplexElementTypeRef idref="C00-Algemeen" />
331     <ComplexElementTypeRef idref="C01-MotivatatieVoorstelNietAkkoord" />
332     <ComplexElementTypeRef idref="C05-Projectgegevens" />
333   </complexElements>
334 </MessageType>
335 <MessageType id="M04-VoorstelGecancelld">
336   <description>Voorstel is gecancelld</description>
337   <complexElements>
338     <ComplexElementTypeRef idref="C00-Algemeen" />
339     <ComplexElementTypeRef idref="C02-MotivatatieVoorstelGecancelld" />
340     <ComplexElementTypeRef idref="C05-Projectgegevens" />
341   </complexElements>
342 </MessageType>
343 <MessageType id="M05-VoorstelIngangsdatum">
344   <description>Voorstel ingangsdatum</description>
345   <complexElements>
346     <ComplexElementTypeRef idref="C00-Algemeen" />
347     <ComplexElementTypeRef idref="C03-Ingangsdatum" />
348     <ComplexElementTypeRef idref="C05-Projectgegevens" />
349   </complexElements>
350 </MessageType>
351 <MessageType id="M06-IngangsdatumAkkoord">
352   <description>Ingangsdatum akkoord</description>
353   <complexElements>
354     <ComplexElementTypeRef idref="C00-Algemeen" />
355     <ComplexElementTypeRef idref="C03-Ingangsdatum" />
356     <ComplexElementTypeRef idref="C05-Projectgegevens" />
357   </complexElements>
358 </MessageType>
359 <MessageType id="M07-IngangsdatumNietAkkoord">
360   <description>Ingangsdatum niet akkoord</description>
361   <complexElements>
362     <ComplexElementTypeRef idref="C00-Algemeen" />
363     <ComplexElementTypeRef idref="C01-MotivatatieVoorstelNietAkkoord" />
364     <ComplexElementTypeRef idref="C05-Projectgegevens" />
365   </complexElements>
366 </MessageType>
367 <MessageType id="M08-BevestigingIngangsdatum">

```

```

368     <description>Bevestiging ingangsdatum</description>
369     <complexType>
370         <ComplexElementBase idref="C00-Algemeen" />
371         <ComplexElementBase idref="C03-Ingangsdatum" />
372         <ComplexElementBase idref="C05-Projectgegevens" />
373     </complexType>
374 </Message>
375 <Message id="M10-VoorstelNieuwVISIproject">
376     <description>Voorstel voor een nieuw VISI project</description>
377     <complexType>
378         <ComplexElementBase idref="C00-Algemeen" />
379         <ComplexElementBase idref="C04-Start-project" />
380         <ComplexElementBase idref="C05-Projectgegevens" />
381     </complexType>
382 </Message>
383 <Message id="M11-AfwijzingVoorgesteldVISIproject">
384     <description>Afwijzing voorgesteld VISI project</description>
385     <complexType>
386         <ComplexElementBase idref="C00-Algemeen" />
387         <ComplexElementBase idref="C05-Projectgegevens" />
388     </complexType>
389 </Message>
390 <Message id="M12-AcceptatieNieuwVISIproject">
391     <description>Acceptatie nieuw VISI project</description>
392     <complexType>
393         <ComplexElementBase idref="C00-Algemeen" />
394         <ComplexElementBase idref="C04-Start-project" />
395         <ComplexElementBase idref="C05-Projectgegevens" />
396     </complexType>
397 </Message>
398 <Message id="M13-AnnuleringProjectInitiatie">
399     <description>Annulering project initiatie</description>
400     <complexType>
401         <ComplexElementBase idref="C00-Algemeen" />
402         <ComplexElementBase idref="C05-Projectgegevens" />
403     </complexType>
404 </Message>
405 <Message id="M14-BevestigingProjectInitiatie">
406     <description>Bevestiging project initiatie</description>
407     <complexType>
408         <ComplexElementBase idref="C00-Algemeen" />
409         <ComplexElementBase idref="C04-Start-project" />
410         <ComplexElementBase idref="C05-Projectgegevens" />
411     </complexType>
412 </Message>
413 <Message id="M15-ProjectIngericht">
414     <description>Project is ingericht</description>
415     <complexType>
416         <ComplexElementBase idref="C00-Algemeen" />
417         <ComplexElementBase idref="C05-Projectgegevens" />
418     </complexType>
419 </Message>
420 <Message id="M16-VerzoekBeeindigingProject">
421     <description>Verzoek tot beeindiging project</description>
422     <complexType>
423         <ComplexElementBase idref="C06-Beeindiging-project" />
424         <ComplexElementBase idref="C00-Algemeen" />
425         <ComplexElementBase idref="C05-Projectgegevens" />
426     </complexType>
427 </Message>
428 <Message id="M17-AkkoordBeeindigingProject">
429     <description>Akkoord beeindiging project</description>
430     <complexType>
431         <ComplexElementBase idref="C06-Beeindiging-project" />
432         <ComplexElementBase idref="C00-Algemeen" />
433         <ComplexElementBase idref="C05-Projectgegevens" />
434     </complexType>
435 </Message>
436 <Message id="M18-AfwijzingBeeindigingProject">
437     <description>Afwijzing beeindiging project</description>

```

```

438     <complexContent>
439         <ComplexElementTypeRef idref="C00-Algemeen" />
440         <ComplexElementTypeRef idref="C05-Projectgegevens" />
441     </complexContent>
442 </MessageType>
443 <MessageType id="M19-AnnulerenProjectbeeindiging">
444     <description>Annuleren projectbeeindiging</description>
445     <complexContent>
446         <ComplexElementTypeRef idref="C00-Algemeen" />
447         <ComplexElementTypeRef idref="C05-Projectgegevens" />
448     </complexContent>
449 </MessageType>
450 <MessageType id="M20-BevestigingBeeindigingProject">
451     <description>Beverstiging beeindiging project</description>
452     <complexContent>
453         <ComplexElementTypeRef idref="C06-Beeindiging-project" />
454         <ComplexElementTypeRef idref="C00-Algemeen" />
455         <ComplexElementTypeRef idref="C05-Projectgegevens" />
456     </complexContent>
457 </MessageType>
458 <MessageType id="M21-BevestigingVoortzettingProject">
459     <description>Beverstiging voortzetting project</description>
460     <complexContent>
461         <ComplexElementTypeRef idref="C00-Algemeen" />
462         <ComplexElementTypeRef idref="C05-Projectgegevens" />
463     </complexContent>
464 </MessageType>
465 <OrganisationType id="standaardOrganisatie">
466     <description>Standaard Organisatie</description>
467     <complexContent>
468         <ComplexElementTypeRef idref="CeOrganisatieSOAPServer" />
469     </complexContent>
470 </OrganisationType>
471 <PersonType id="standaardPersoon">
472     <description>Standaard Persoon</description>
473 </PersonType>
474 <ProjectType id="PRT-Meta-raamwerk">
475     <namespace>http://www.visi.nl/schemas/20140331/metaframework</namespace>
476     <description>Meta-raamwerk projecttype</description>
477 </ProjectType>
478 <RoleType id="R01-VISI-communicatiemanager">
479     <description>VISI Communicatiemanager</description>
480 </RoleType>
481 <RoleType id="R02-Wijzigingsinitiator">
482     <description>Wijzigingsinitiator</description>
483 </RoleType>
484 <RoleType id="R03-Initiator-VISI-project">
485     <description>Project-initiator</description>
486 </RoleType>
487 <RoleType id="R04-Executor-VISI-project">
488     <description>Project-executor</description>
489 </RoleType>
490 <SimpleElementType id="S00-Algemeen">
491     <description>Eventuele opmerkingen</description>
492     <userDefinedType>
493         <UserDefinedTypeRef idref="String" />
494     </userDefinedType>
495 </SimpleElementType>
496 <SimpleElementType id="S01-MotivatieVoorstelNietAkkoord">
497     <description>Motivatie voorstel niet akkoord</description>
498     <userDefinedType>
499         <UserDefinedTypeRef idref="String" />
500     </userDefinedType>
501 </SimpleElementType>
502 <SimpleElementType id="S02-MotivatieVoorstelGecancelld">
503     <description>Motivatie voorstel gecancelld</description>
504     <userDefinedType>
505         <UserDefinedTypeRef idref="String" />
506     </userDefinedType>
507 </SimpleElementType>

```

```

508 <SimpleElementType id="S03-Ingangsdatum">
509   <description>Ingangsdatum actualisering</description>
510   <userDefinedType>
511     <UserDefinedTypeRef idref="Datumtijd" />
512   </userDefinedType>
513 </SimpleElementType>
514 <SimpleElementType id="S04-Moment-beeindiging-project">
515   <description>Moment van beeindiging project</description>
516   <userDefinedType>
517     <UserDefinedTypeRef idref="Datumtijd" />
518   </userDefinedType>
519 </SimpleElementType>
520 <SimpleElementType id="S04-Moment-start-project">
521   <description>Moment van start project</description>
522   <userDefinedType>
523     <UserDefinedTypeRef idref="Datumtijd" />
524   </userDefinedType>
525 </SimpleElementType>
526 <SimpleElementType id="S05-Project-ID">
527   <description>Project-ID</description>
528   <userDefinedType>
529     <UserDefinedTypeRef idref="ProjectID" />
530   </userDefinedType>
531 </SimpleElementType>
532 <SimpleElementType id="S06-Projectnaam">
533   <description>Projectnaam</description>
534   <userDefinedType>
535     <UserDefinedTypeRef idref="StringVerplicht" />
536   </userDefinedType>
537 </SimpleElementType>
538 <SimpleElementType id="SOAPServerURL">
539   <description>SOAPServerURL</description>
540   <userDefinedType>
541     <UserDefinedTypeRef idref="String" />
542   </userDefinedType>
543 </SimpleElementType>
544 <TransactionPhaseType id="AanvaardEinde">
545   <description>Aanvaard/Einde</description>
546 </TransactionPhaseType>
547 <TransactionPhaseType id="BeloofdExecutie">
548   <description>Beloofd/Executie</description>
549 </TransactionPhaseType>
550 <TransactionPhaseType id="MeldingGereed">
551   <description>Melding Gereed</description>
552 </TransactionPhaseType>
553 <TransactionPhaseType id="Start">
554   <description>Start</description>
555 </TransactionPhaseType>
556 <TransactionPhaseType id="Verzocht">
557   <description>Verzocht</description>
558 </TransactionPhaseType>
559 <TransactionPhaseType id="WijzigingHold">
560   <description>Wijziging/Hold</description>
561 </TransactionPhaseType>
562 <TransactionType id="T01-NieuwOfGewijzigdRaamwerkOfProjectSpeciekBerichtTransactie">
563   <description>Nieuw of gewijzigd raamwerk of projectspecifiekbericht transactie</
description>
564   <initiator>
565     <RoleTypeRef idref="R02-Wijzigingsinitiator" />
566   </initiator>
567   <executor>
568     <RoleTypeRef idref="R01-VISI-communicatiemanager" />
569   </executor>
570 </TransactionType>
571 <TransactionType id="T02-">
572   <description>VISI project initiatie</description>
573   <initiator>
574     <RoleTypeRef idref="R03-Initiator-VISI-project" />
575   </initiator>
576   <executor>

```



```

577         <RoleTypeRef idref="R04-Executor-VISI-project" />
578     </executor>
579 </TransactionType>
580 <UserDefinedType id="Datum">
581     <description>Datum</description>
582     <baseType>DATE</baseType>
583 </UserDefinedType>
584 <UserDefinedType id="Datumtijd">
585     <description>Datum en tijd</description>
586     <baseType>DATETIME</baseType>
587 </UserDefinedType>
588 <UserDefinedType id="ProjectID">
589     <description>Project ID</description>
590     <baseType>STRING</baseType>
591     <xsdRestriction>\&lt;xs:pattern value="[a-zA-Z][\w-]*"/&gt;</xsdRestriction>
592 </UserDefinedType>
593 <UserDefinedType id="String">
594     <description>String</description>
595     <baseType>STRING</baseType>
596 </UserDefinedType>
597 <UserDefinedType id="StringVerplicht">
598     <description>String Verplicht</description>
599     <baseType>STRING</baseType>
600     <xsdRestriction>\&lt;xs:minLength value="1"/&gt;</xsdRestriction>
601 </UserDefinedType>
602 </visiXML_VISI_Systematics>

```

5.8.5.1 Example project message

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/20140331/metaframework">
3     <PRT-Meta-raamwerk id="PRO-META-RW">
4         <name>Meta Raamwerk Project</name>
5         <description>Meta Raamwerk Project</description>
6         <startDate>2011-12-22T12:30:23.0Z</startDate>
7         <endDate>2012-12-22T12:30:23.0Z</endDate>
8     </PRT-Meta-raamwerk>
9     <standaardPersoon id="PER-COMG">
10         <userName>comg@visi.nl</userName>
11         <name>VISI communicatie manager</name>
12     </standaardPersoon>
13     <R01-VISI-communicatiemanager id="ROL-COMG">
14         <name>R01-VISI-communicatiemanager</name>
15         <description>R01-VISI-communicatiemanager</description>
16         <category></category>
17     </R01-VISI-communicatiemanager>
18     <standaardOrganisatie id="ORG-INF">
19         <name>org1</name>
20         <abbreviation>INF</abbreviation>
21         <contactPerson>
22             <standaardPersoonRef idref="PER-COMG" />
23         </contactPerson>
24         <ceOrganisatieSOAPServer>
25             <CeOrganisatieSOAPServerRef idref="SCE-INF" />
26         </ceOrganisatieSOAPServer>
27     </standaardOrganisatie>
28     <CeOrganisatieSOAPServer id="SCE-INF">
29         <sOAPServerURL>http://visi.nl/soapservice.asmx</sOAPServerURL>
30     </CeOrganisatieSOAPServer>
31     <PersonInRole id="PIR-COMG">
32         <contactPerson>
33             <standaardPersoonRef idref="PER-COMG" />
34         </contactPerson>
35         <organisation>
36             <standaardOrganisatieRef idref="ORG-INF" />
37         </organisation>

```



```

38     <role>
39         <R01-VISI-communicatiemanagerRef idref="ROL-COMG" />
40     </role>
41 </PersonInRole>
42 <standaardPersoon id="PER-WZGI">
43     <userName>wzgi@visi.nl</userName>
44     <name>Wijzigingsinitiator</name>
45 </standaardPersoon>
46 <R02-Wijzigingsinitiator id="ROL-WZGI">
47     <name>R02-Wijzigingsinitiator</name>
48     <description>R02-Wijzigingsinitiator</description>
49     <category></category>
50 </R02-Wijzigingsinitiator>
51 <PersonInRole id="PIR-WZGI">
52     <contactPerson>
53         <standaardPersoonRef idref="PER-WZGI" />
54     </contactPerson>
55     <organisation>
56         <standaardOrganisatieRef idref="ORG-INF" />
57     </organisation>
58     <role>
59         <R02-WijzigingsinitiatorRef idref="ROL-WZGI" />
60     </role>
61 </PersonInRole>
62 </visiXML_MessageSchema>

```

5.8.6 Scenario for using meta framework and meta project specific message

5.8.6.1 Introduction

A scenario has been developed to clarify the operation of the meta-framework and meta-project-specific message between the various organisations.

5.8.6.2 Scenario

STEP 0

Organisation A has VISI software -A-0).

Organisation B has VISI software Y with SOAP server 4444.

Organisation C has VISI software Y with SOAP server 4444.

Organisation R has VISI software Z with SOAP server 8888.

This party has access to a meta-project (m-p-R) containing the universal meta-framework (m-rw) and a meta-project-specific message from this organisation (m-psb -R-0).

Step 1

Figure 27:

Organisation B passes on to Organisation A: SOAP server and the person acting as VISI communications manager.

Organisation A extends the meta-project-specific message with, among other things, the SOAP server of Organisation B (m-psb-A-1).

Organisation B receives the meta-project-specific message (m-psb-A-1) by email from Organisation A. Organisation B sets up a meta-project with this meta-project-specific message.

Organisation A sets up a VISI project (p-A-555), and sends the project framework (rw-A-555-0) and project-specific message (psb-A-555-0) by means of a message in the meta-project.) to Organisation B.

Organisation B receives the data from the new VISI project in the meta-project and sets up this project.

Step 2

Figure 28:

Organisation A adjusts the framework and/or project-specific message of the VISI project (p-A-555), and sends the project framework (rw-A-555-1) by means of a message in the meta-project (m-p-A) and/or project-specific message (psb-A-555-1) to Organisation B.

Organisation B receives the adjusted data from the VISI project in the meta-project and updates this existing VISI project.

Step 3

Figure 29:

Organisation C passes on to Organisation A: SOAP server and the person acting as VISI communication Manager.

Organisation A extends the meta project-specific message with, among other things, the SOAP server of Organisation C (m psb A 2).

Optional: Organisation B receives the adjusted meta-project-specific message (m psb A 2) by e-mail from Organisation A. Organisation B updates the corresponding meta project with this meta project-specific message.

Organisation C receives the meta project-specific message (m psb A 2) by e-mail from Organisation A. Organisation C sets up a meta project with this meta project-specific message.

Step 4

Figure 30:

Organisation A sets up a VISI project (p A 777), and sends the project framework (rw A 777 0) and project project-specific message (psb A 777 0) to Organisation C by means of a message in the metaproject.

Organisation C receives the data from the new VISI project in the meta project and sets up this project.

Step 5

Figure 31:

Organisation C passes on to Organisation R: SOAP server and the person acting as VISI communications manager.

Organisation R extends the meta project-specific message with, among other things, the SOAP server of Organisation C (m psb R 1).

Step 6

Figure 32:

Organisation C receives the meta project-specific message (m psb R 1) by e-mail from Organisation R. Organisation B sets up a meta project with this meta project-specific message.

Organisation R sets up a VISI project (p R 999), and sends the project framework (rw R 999 0) and project project-specific message (psb R 999 0) to Organisation C by means of a message in the metaproject.

Organisation C receives the data from the new VISI project in the meta project and sets up this project.

Step 7

Figure 33:

5.9 Interactive information model

1 Interactive information model of _2.exp frameworks VISI frameworks - relations with info. ProjectType ComplexElementType MessageType AppendixType ElementCondition SimpleElementType MessageInTransactionType UserDefinedType TransactionType TransactionPhaseType MessageInTransactionType-Condition GroupType PersonType OrganisationType RoleType Go back to interactive information model

5.10 Appendix 7: Guidelines for Successor

5.10.1 Rules for 'Successor'

For a correct interpretation of all the possibilities of a successor certain rules need to be followed. First the do's and don'ts are presented, followed by some examples.

5.10.1.1 A successor has always the SAME ROLE

If 'B' is a successor of 'A' then 'B' must have the SAME role as 'A'.

5.10.1.2 A successor can NEVER be changed

If 'A' has a successor 'B' then 'B' will ALWAYS be the successor of 'A'. Later on 'C' cannot be the successor of 'A', but can become the successor of 'B'.

5.10.1.3 NO LOOP of successors

If 'B' is a successor of 'A' then 'A' cannot be a successor of 'B'.

5.10.1.4 A successor of a successor is allowed

At first 'B' is a successor of 'A'. But after some time 'B' can also have a successor. Theoretically such a chain of successors can be unlimited, but it can never become a loop.

Keep in mind that following situation can occur:

1. 'A' starts transaction T1;
2. 'B' becomes a successor of 'A';
3. 'B' replies and sends a messages in transaction T1;
4. 'B' starts transaction T2;
5. 'C' becomes a successor of 'B';
6. 'C' replies and sends messages in transaction T1 and T2.

Maybe later on 'D' will become a successor of 'C' then 'D' will be responsible for T1 and T2 (if T1 and T2 are not finished). 'A' cannot be the successor of 'C'.

5.10.1.5 A successor of several persons (in role)

'B' can become the successor of several persons (in role). In this case 'B' will be responsible for all open transactions of all predecessors.

5.10.1.6 A predecessor can NOT start a transaction

A PersonInRole with a successor (=predecessor) is not an active member of the project and therefore cannot start a new transaction.

5.10.1.7 A PersonInRole with a successor can NOT send a message

A PersonInRole with a successor (=predecessor) is not an active member of the project and therefore cannot send a message.

5.10.1.8 An initiator and executor of a transaction will NEVER change

For example 'A' starts a new transaction T1 and sends a message to 'B'. The VISI xml message will contain 'A' as the initiator and 'B' as the executor.

When 'C' becomes the successor of 'B' and replies on behalf of 'B'. The VISI xml message will contain 'A' as the initiator and 'B' as the executor. 'C' is also included, but only as the successor of 'B'.

5.10.2 Examples

5.10.2.1 Example 1

Most simple case is when 'B' is a successor of 'A'. It can be displayed like this:

Figure 34:

The following notation is used to show that 'B' is a successor of 'A':

'A → B'

'B' also can have a successor and the diagram will look like this:

Figure 35:

This can be written like:

'A → B → C'

In common case the "successor predecessor" diagram can be displayed like:

Figure 36:

Or: 'A1 → A2 → ... → An'

5.10.2.2 Example 2

'C' can become the successor of several persons (in role). In a diagram:

Figure 37:

Or: '(A; B) → C'

In common case the "successor predecessor" diagram can be displayed like:

Figure 38:

Or: $(A_1; A_2; \dots; A_n) \rightarrow B$

5.10.2.3 Example 3

A final example which shows how complex "successor predecessor" relations can be:

Figure 39:

Or: $((A \rightarrow B; C; D) \rightarrow E; (F; G) \rightarrow H \rightarrow I) \rightarrow J$

In this example 'J' is responsible for all open transaction of 'A', 'B', ..., 'I'.

6 Technical

6.1 TBD

Systematiek Part II
AppendixGroup

6.2 Appendix 10 Additional functional requirements

The documents Systematics Part 1 (frameworks) and Part 2 (messages) together describe the VISI system. Because these descriptions do not discuss how the system should be interpreted, an implementation guideline has been drawn up with a number of separate guidelines that deal with specific topics. In addition, some additional functional requirements have been drawn up, which are explained in this appendix.

6.2.1 Additional functions and requirements for the VISI quality mark

In addition to the requirements described in the above documents and guidelines, the present document describes a number of functions and requirements that users of products with the VISI quality mark can at least expect from the product in question, namely:

- A. Ensuring the authenticity of exchanged VISI messages and associated files.
- B. Presenting the communication structure laid down in a VISI framework in a recognisable manner.
- C. Being able to reproduce VISI communications afterwards.

The requirements derived from these functions are listed below for each function. These 'additional requirements' only apply if the VISI product contains a user interface for the end user. The additional requirements also relate to obtaining the VISI quality mark.

6.3 Appendix 13 Reducing frames using optional fields

The aim is to make frameworks (XML and XSD) more compact. This is beneficial for VISI users because a bulky and inefficient framework leads to delays in the handling of VISI messages and the use of VISI software. A more compact and efficient framework results in increased scalability and speed of the VISI software. An additional advantage is that the readability of frameworks and VISI messages is increased.

In practice, little changes:

	Ensuring the authenticity of exchanged VISI messages and associated files
A.1	VISI messages must comply with the system as shown when sent from a VISI product(*). described in the documentation associated with the current system. (*) A VISI product is software that a supplier offers in combination with the VISI quality mark.
A.2	A VISI message with any linked file(s) must be included in the VISI product as such stored so that the message and associated file(s) can be retrieved in relation to each other at all times.
A.3	It should not be possible to retrieve originally sent and/or received VISI messages and associated change or delete files in the VISI product. (N.B. This does not include copying to create a new message)
A.4	The VISI product must check whether a user is authorised to access the system. Gaining access must be done using means that the person in question under his/her sole control.
A.5	Only the persons listed in the project-specific message may use the VISI product Perform VISI communications in accordance with the VISI roles assigned to them.

Table 8: Additional requirements for ensuring the authenticity of exchanged VISI messages and associated files

	Presenting in a recognizable manner the communication structure established in a VISI framework
B.1	The software may only show a user those transactions for which the user is responsible based on his/her role(s). N.B. An exception must be made for the so-called 'viewing function'. This is a facility for the administrator of the VISI product and offers the possibility to make all message traffic visible (without the possibility of influencing the flow of messages).
B.2	For a user, after selecting a VISI message to have insight into which files are linked to the message.
B.3	Once a message has been selected, the following information should be available: - The transaction type with associated 'description' - The naming of the message type - The initiator - The executor - The date the message was sent
B.4	It must be possible to show transactions in relation to each other on the manner as laid down in the underlying VISI framework and where the user is involved as 'Person-in-Role' (initiator or executor).

Table 9: Additional requirements for presenting the communication structure established in a VISI framework

- In VISI Standard version 1.2, many mandatory properties can ultimately be left empty, because they are a property of type string (for example <interfaceType></interfaceType>).
- In VISI Standard version 1.2, a large number of properties are optional (e.g. basePoint, category, helpInfo, language: OPTIONAL STRING). These optional features can be used.
- In VISI Standard version 1.3, the number of optional features can be expanded.

It is important to distinguish between VISI Standard Part 1 and Part 2 (layers 2 and 5). Part 1 contains the contents of the VISI framework. Part 2 concerns the content of the VISI messages and project-specific messages.

¹This change was added late in version 1.3. Unfortunately too late to be included in the DIS ballot of IDM-2 (ISO 29481-2). This means that VISI 1.3 and IDM-2 now differ on this point. Software implementations will therefore have to take this into account.

	The afterwards (i.e. after closing the project) being able to reproduce conducted VISI communications
C.1	To perform an audit trail, messages must be able to be reproduced in the order in which they were sent or received, within the framework of the own organisation of a VISI user, and the system used.
C.2	Linked files must be able to be exported in the format in which they were sent.
C.3	Documents (files) exchanged with VISI – within the frameworks set by the Archives Act – must remain retrievable and readable at all times (**), regardless of changes to software or otherwise. http://wetten.overheid.nl/BWBR0007376/geldigheidsdatum_30-11-2010 (**) N.B. Testing of this item is not possible; agreements can be made about uniform archiving

Table 10: Additional requirements for reproducing VISI communications after project completion

6.3.1 Part 1 (_2.exp): VISI-framework

Type	Optional properties
All elements	userLaMu
	dateLaMu
	state
	startDate
	endDate
SimpleElementType	interfaceType
MessageInTransactionType	requiredNotify
	received
	send
	group

Table 11: Optional properties in the VISI framework (Part 1)

6.3.2 Part 2 (_5.exp): VISI messages and project-specific message

6.3.3 Test result

6.3.3.1 Framework before

- 600 KB
- Fragment regarding readability

```

1 <ComplexElementType id="CeDocumentAttributen">
2   <description>Documentgegevens</description>
3   <startDate>2008-10-08</startDate>
4   <endDate>2008-10-08</endDate>
5   <state></state>
6   <dateLaMu>2009-11-10</dateLaMu>
7   <userLaMu>ALFAmail</userLaMu>
8   <language>onbekend</language>
9   <category></category>
10  <helpInfo>Gegevens van een document</helpInfo>
11  <simpleElements>
12    <SimpleElementTypeRef idref="documentNaam" />
13    <SimpleElementTypeRef idref="documentKenmerk" />

```

Type	Properties to be made optional
All elements	userLaMu
	dateLaMu
	state
AppendixTemplate	documentIdentification
	documentVersion
	documentReference
	startDate
	endDate
	appendixGroup
MessageTemplate	dateRead
MessageInTransactionTemplate	dateSend
	dateRead
AppendixGroup	group

Table 12: Properties to be made optional in VISI messages and project-specific messages (Part 2)

```

14     <SimpleElementTypeRef idref="versieNummer" />
15     <SimpleElementTypeRef idref="documentSoort" />
16     <SimpleElementTypeRef idref="disciplineType" />
17 </simpleElements>
18 </ComplexElementType>
19
20 <MessageInTransactionType id="MITT1_Verzoek_advies_korting">
21   <requiredNotify>0</requiredNotify>
22   <dateLaMu>2009-11-10</dateLaMu>
23   <userLaMu>ALFAmail</userLaMu>
24   <received>0</received>
25   <send>0</send>
26   <state>active</state>
27   <initiatorToExecutor>true</initiatorToExecutor>
28   <message>
29     <MessageTypeRef idref="Verzoek_advies_korting" />
30   </message>
31   <transaction>
32     <TransactionTypeRef idref="T8_04_Verzoek_Advies_ADV" />
33   </transaction>
34   <transactionPhase>
35     <TransactionPhaseTypeRef idref="verzocht" />
36   </transactionPhase>
37   <group>
38     <GroupTypeRef idref="standaardGroep" />
39   </group>
40 </MessageInTransactionType>

```

6.3.3.2 Framework after

- 380 KB (37% less)
- Fragment regarding readability

```

1 <ComplexElementType id="CeDocumentAttributen">
2   <description>Documentgegevens</description>
3   <simpleElements>
4     <SimpleElementTypeRef idref="documentNaam" />
5     <SimpleElementTypeRef idref="documentKenmerk" />
6     <SimpleElementTypeRef idref="versieNummer" />
7     <SimpleElementTypeRef idref="documentSoort" />

```



```

8      <SimpleElementTypeRef idref="disciplineType" />
9    </simpleElements>
10 </ComplexElementType>
11
12 <MessageInTransactionType id="MITT1_Verzoek_advies_korting">
13   <initiatorToExecutor>true</initiatorToExecutor>
14   <message>
15     <MessageTypeRef idref="Aanbod_weekrapport" />
16   </message>
17   <transaction>
18     <TransactionTypeRef idref="T2_06_Weekrapporten_PLB-AN" />
19   </transaction>
20 </MessageInTransactionType>

```

6.4 Appendix 14: Validating frameworks

This appendix contains a number of proposals for validating frameworks that are DEMO-based. There are 2 sets of rules: 1 set of rules that assumes a framework with passive messages and 1 set of rules for frameworks in which there are no passive messages, because these messages are not present in the framework.

6.4.1 General principles:

- A MITT does not contain a (starting message), one or more MITT. previous
- check all MITT's

6.4.2 With passive messages

- check per TransactionType whether all statuses (Request, Decline, etc) are present
- check per MessageInTransactionType (MITT), whether each MITT.previous has a correct value
- * do this by checking whether both conditions per line are true or both conditions per line are false
- * MITT = Request and MITT.previous = Decline
- * MITT = Decline and MITT.previous = Request
- * MITT = Quit and MITT.previous = Decline
- * MITT = Promise and MITT.previous = Request
- * MITT = State and MITT.previous = Promise
- * MITT = Reject and MITT.previous = State
- * MITT = Stop and MITT.previous = Reject
- * MITT = Accept and MITT.previous = State

6.4.2.1 Rules to check the Passive MITT's:

- There must always be at least 1 Active message
- If the MITT = Request is passive and the Request has no MITTPrevious within the same TransactionType, then the MITT of Decline must be Passive and the MITT of the Quit must be Passive
- If the MITT = Decline Passive and MITTPrevious = Request and the Request has no MITTPrevious within the same TransactionType, then the MITT of Request must be Passive

- If the MITT = State is Passive and the State of MITTPrevious is not Promise, then the MITT of Reject must be Passive and the MITT of Stop must be Passive
- If the MITT = Reject Passive and MITTPrevious = State, and the State if MITTPrevious does not have Promise, then the MITT of Reject Passive must be

Figure 40: Shortened transaction pattern with passive messages

6.4.3 Without Passive messages

- view each MITT. Previous line per MITT
- check whether an MITT of type State is present
- check if all the following lines are true
- If the MITT = Request and (MITTPrevious = Nothing or MITTPrevious = Decline)
- If the MITT = Decline and MITTPrevious = Request
- If the MITT = Quit and MITTPrevious = Decline
- If the MITT = Promise and (MITTPrevious = Nothing or MITTPrevious = Request)
- If the MITT = State and (MITTPrevious = Nothing or MITTPrevious = Request)
- If the MITT = State and MITTPrevious = Promise
- If the MITT = Reject and MITTPrevious = State
- If the MITT = Stop and MITTPrevious = Reject
- If the MITT = Accept and MITTPrevious = State

Figure 41: Allowed message transitions in a VISI transaction without passive messages

6.5 Guideline for implementation of the complete DEMO transaction pattern (with VISI Standard version 1.6)

The VISI Standard is based on [Design & Engineering Methodology for Organisations](https://en.wikipedia.org/wiki/Design_%26_Engineering_Methodology_for_Organisations) (DEMO), a methodology based on scientific theories and developed by Prof. dr. ir. J.L.G. Dietz at TU Delft (Dietz, Jan L.G., [Enterprise Ontology - Theory and Methodology](<http://www.springer.com/la/book/9783540291695>), Springer-Berlin Berlin Heidelberg 2006). DEMO creates a model of cooperation in terms of roles, responsibilities and transactions, see [characteristics](<http://www.ee-institute.org/en/demo/characteristics>). When there is a collaboration between two people in their roles, there is a transaction: according to one's role, a person wants the other person to do something. This is called the DEMO transaction pattern. For each bit of communication within the pattern, it is clear what the meaning is (intention) and how it can be responded to. A DEMO model forms a network of roles and transactions. In the VISI Standard, a network of roles and transactions is called a framework ("[raamwerk](<http://www.ee-institute.org/download.php?id=113&type=doc>)"). A framework fully describes what the communication possibilities are. Communication is exchanged by sending and receiving digital messages. These messages are according to the VISI Standard and the framework of a project.

In the following sections, the DEMO transaction pattern is described as far as is necessary to understand the framework. In addition, figures from the original training material for DEMO are used with permission from the author J. Dietz. In [Chapter 3](<https://github.com/nl-digigo/visi/wiki/3.-Definition-of-Framework>), the VISI method is described, including the specifications for VISI software. [Chapter 4](<https://github.com/bimloket/visi/wiki/Data-Model>) describes a method of reaching the desired framework and messages in software that supports VISI communication. [Chapter 5](<https://github.com/bimloket/visi/wiki/Method>) lists the technical requirements that VISI software must comply with.

6.5.1 Implementation of the complete DEMO transaction pattern with the VISI Standard 1.6

The VISI Standard is based on “Design and Engineering Methodology” (DEMO).

DEMO is used to clearly record collaboration/processes in and between organisations. This involves using roles and transactions. A transaction always takes place between an initiator and an executor. Every completed transaction results in a decision or production fact. The communication required to realise these facts is described in DEMO in a number of transaction states. A step in the realisation of a fact is recorded as a change in transaction status.

VISI is used to record (formal) communication in and between organisations. VISI also works with roles and transactions, in which only 2 people are involved as initiator and executor. The communication required to record the facts that result from the transactions is not recorded in changes in transaction states but in freely definable messages. An important agreement is that communication takes place alternately. So if the initiator has sent a message, the executor must first respond and only then can the initiator send another message. Unlike DEMO, with VISI, messages are sent from one party to another.

In VISI, due to alternating messages, some transaction states used in DEMO are not possible. In DEMO, a transaction state PROMISE is used, with which the executor indicates that he promises that he will realise a transaction fact. The transaction fact is then transferred to the initiator with a STATE. In VISI, this is not possible because the executor would have to send a message twice in succession. In addition, it is also not possible to revoke a transaction status. The reason for this is that VISI does not allow the initiator or executor to send a message twice in succession.

In particular, the option to revoke a message has been a wish that has been among users for a long time. In addition, various stakeholders have long wanted to support the complete transaction pattern in VISI. After a first attempt to enable revocations [<https://github.com/nl-digigo/visi/issues/6>]

6.6 DEMO transaction pattern

DEMO considers an organisation as a system of social actors working together to achieve results (products and/or services) for the organisation. A human being is the only social actor we know. DEMO always considers human beings responsible for what happens in an organisation. Collaboration takes place in transactions involving two actors:

1. initiator, who wants something (the result).
2. executor, who accomplishes the result.

A transaction has three phases;

1. Order Phase, a conversation in which actors try to reach an agreement on the product to be produced.
2. Execution Phase, the product and/or service of the transaction being produced.
3. Result Phase, a conversation in which the actors try to reach an agreement on the produced product.

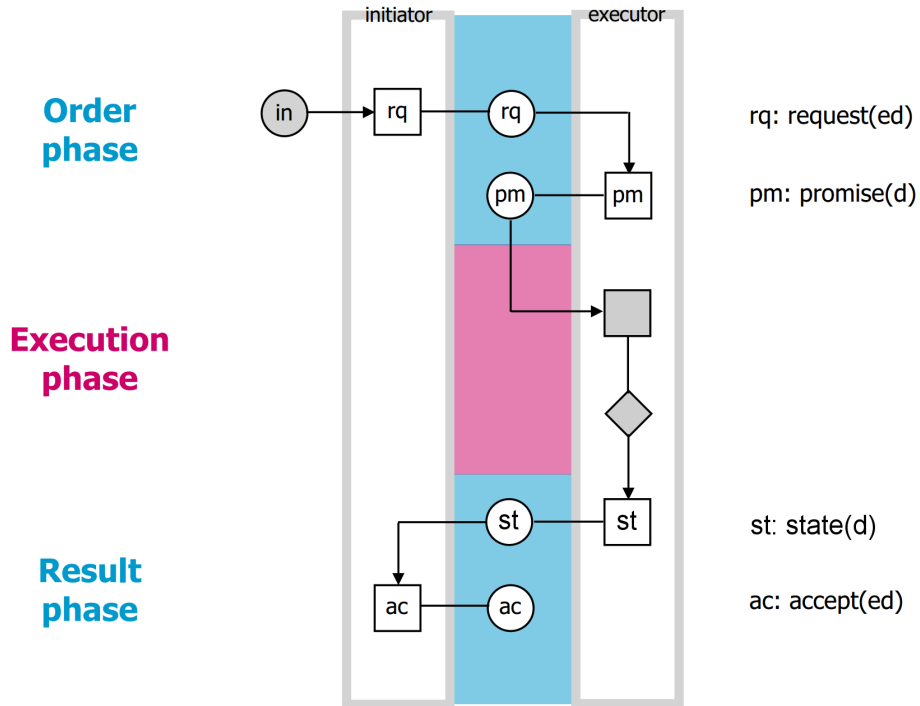


Figure 42:

Abbreviation	Action	State of Transaction
rq	Request	Requested
pm	Promise	Promised
st	State	Stated
ac	Accept	Accepted

Table 13:

The sequence of actions request, promise, state and accept leads to a successful transaction. This sequence is also called "happy flow".

After the action request or a state, the flow is extended with two discussions. The actions (events) are displayed by squares. The actions are: request, decline, quit, promise, state, reject, stop, accept.

An action creates a fact, which is displayed with a circle. The transaction has reached a certain state: requested, declined, quitted, promised, stated, rejected, stopped or accepted.

Abbreviation	Action	State of Transaction
rq	Request	Requested
pm	Promise	Promised
st	State	Stated
ac	Accept	Accepted
dc	Decline	Declined
qt	Quit	Quitted
rj	Reject	Rejected
sp	Stop	Stopped

Table 14:

The executor may not be able to execute the request. The executor declines the request. The transaction

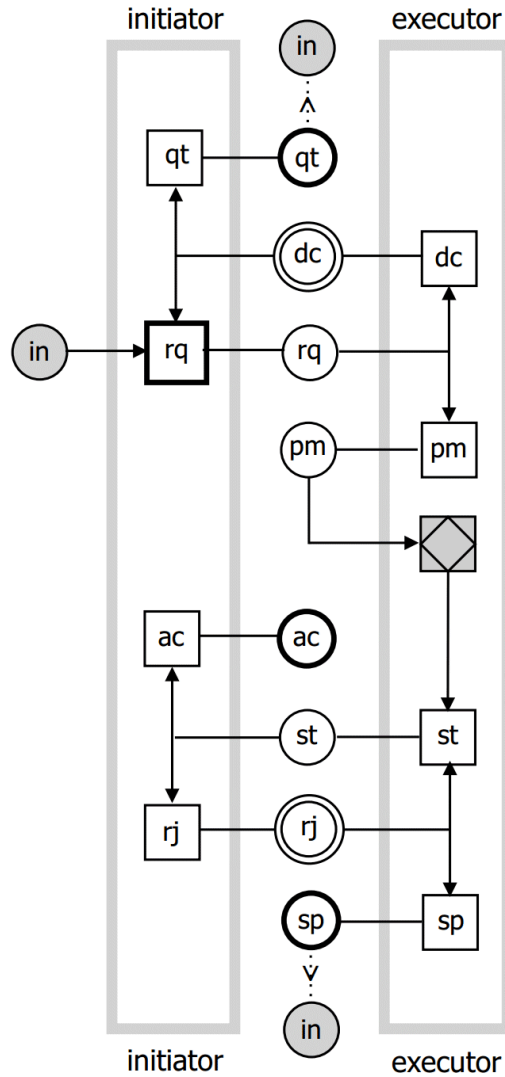


Figure 43:

is in a state of discussion in which the actors talk to each other to agree. (A discussion state is indicated by a white circle with a double black border.)

Usually, the actors agree, and this results in a (modified) request and a promise. When no agreement is reached, the initiator quits the transaction. The transaction has reached a final state, but failed.

Something comparable happens after a state by the executor. The initiator may not accept the product and/or service. The initiator rejects the product. The transaction is in a state of discussion in which the actors talk to each other to agree.

Usually, the actors agree, and this results in a (modified) state and an accept. When no agreement is reached, the executor stops the transaction. The transaction has reached a final state, but failed.

A complete transaction pattern arises when we also take into account the fact that actors can revoke their primary actions (request, promise, state, accept). The initiator can declare a revoke request and revoke accept. The executor can always declare a revoke promise and revoke state. A revoke must be followed by an allow or a refuse. An allow changes the state of the transaction. An allow of a revoke request changes the state of the transaction into "initiated". An allow of a revoke promise

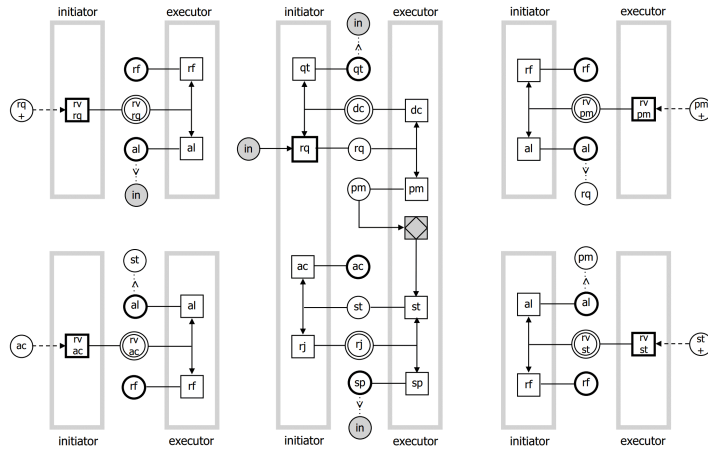


Figure 44:

changes the state of the transaction into "requested". An allow of a revoke state changes the state of the transaction into "promised". An allow of a revoke accept changes the state of the transaction into "stated".

A refuse doesn't change the state of the transaction.

Abbreviation	Action	State of Transaction
rq	Request	Requested
pm	Promise	Promised
st	State	Stated
ac	Accept	Accepted
dc	Decline	Declined
qt	Quit	Quitted
rj	Reject	Rejected
sp	Stop	Stopped
rv rq	Revoke Request	Revoked Request
rv pm	Revoke Promise	Revoked Promise
rv st	Revoke State	Revoked State
rv ac	Revoke Accept	Revoked Accept
al	Allow	Allowed
rf	Refuse	Refused

Table 15:

An example to illustrate.

The result was accepted by the initiator, and then the initiator detected an error. The action accept is revoked, because the result is not according to the request. The executor receives the revoke accept and can allow or refuse it. A negotiation may start between the actors about the error in the result. The actors can agree on a modified result. In that case, the action "state" is also revoked (and allowed). The executor creates a customised result and states it.

If the revoke is refused, the state of the transaction doesn't change (and is accepted).

Another example to illustrate.

The result has already been accepted, but the initiator wants something else. The action request is revoked, because a different result is requested. The executor receives the revoke request and can allow

or refuse it. A negotiation may start between the actors about a different result. The actors can agree on a different result. In that case, the action "request" is revoked (and allowed). The initiator sends a different request to the executor. The executor promises and states. The initiator accepts the different result, which is according to the different request.

If the revoke is refused, the state of the transaction doesn't change (and is accepted).

6.7 Definition of Framework

This chapter describes how a DEMO construction model can be modelled into a framework by using VISI Standard version 1.6.

6.7.1 DEMO or VISI framework?

From a framework, it should be recognisable if all transactions are according to VISI or the reference framework DEMO. By adding /DEMO to the namespace of VISI, a framework will be recognisable as a framework for DEMO usage. When there will be a new version of the VISI Standard - version 1.7 (or higher namespace) - the DEMO framework will be recognisable by the slash DEMO addition.

Example of the namespace in a framework of VISI Standard, version 1.6:
 ‘<visiXML _VISI_Systematics xmlns="http://www.visi.nl/schemas/20160331">‘

Example of the namespace in a framework for reference framework DEMO:
 ‘<visiXML _VISI_Systematics xmlns="http://www.visi.nl/schemas/20160331/DEMO">‘

Example of possible namespace in a framework of VISI Standard, version 1.7:
 ‘<visiXML _VISI_Systematics xmlns="http://www.visi.nl/schemas/20191231">‘

Example of the namespace in a framework for reference framework DEMO:
 ‘<visiXML _VISI_Systematics xmlns="http://www.visi.nl/schemas/20191231/DEMO">‘

6.7.2 DEMO transaction pattern in VISI: default 4 states

In paragraph [3.2](<https://github.com/nl-digigo/visi/wiki/3.2-DEMO-transaction-with-open-standard-VISI>) is described how the standard DEMO transaction pattern is described in a VISI framework. An important difference between the reference framework for DEMO and the frameworks for VISI is that each transaction for DEMO follows exactly the same pattern and therefore always contains the same order of messages and states.

6.7.3 DEMO revokes in VISI: separate transactions

The complete transaction pattern (including revokes) can also be described in the same way, but to simplify the modelling of frameworks, it has been decided to record revokes as separate transactions in a framework. This definition is described in paragraph [3.3](<https://github.com/nl-digigo/visi/wiki/3.3-Revokes-in-a-Transaction-Tree>).

6.7.4 Asynchronous & revokes

Paragraph [3.4]([https://github.com/nl-digigo/visi/wiki/3.4-DEMO-Crossing-Messages-\(and-order-of-events\)](https://github.com/nl-digigo/visi/wiki/3.4-DEMO-Crossing-Messages-(and-order-of-events))) explains how to deal with the asynchronicity of the VISI Standard and a result, crossing messages/revokes and the sequences of these events.

6.8 DEMO transaction with open standard VISI

6.8.1 Fixed pattern for a VISI transaction

To understand the restrictions that need to be applied to a VISI transaction to become a DEMO transaction, a short summary is given of the way a VISI transaction works.

A VISI transaction has a role which starts the transaction. This is called initiator. The initiator sends the first message of the transaction to another role. This role is called the executor of the transaction. A transaction has only two roles connected to it: initiator and executor. After receiving the message the executor is the owner of the transaction. The initiator cannot send a second message in the same transaction because the initiator is not the owner of the transaction. The executor is the owner and has to take action. The executor can transfer the ownership back to the initiator by sending a message to the initiator. So initiator or executor can never send a message twice in a row. In the VISI Standard, this behaviour is described by a property `initiatorToExecutor`, which can be set to true or false. The property belongs to `MessageInTransactionType` (MITT).

There is no limit to the number of messages within a transaction. A transaction can consist of one message. For example, the transaction "For Your Information". A transaction can consist of two messages. For example, "Question - Answer". Message "Question" needs to be sent from initiator to executor, and executor has to reply with message "Answer".

A transaction can consist of a dozen messages. For example, a transaction about "Contract Mutation". The pattern of messages has to go from initiator to executor to initiator to executor to initiator to etc.

As described in chapter [2](<https://github.com/nl-digigo/visi/wiki/2.-DEMO-transaction-pattern>), a DEMO transaction has a fixed/predefined pattern with a fixed number of messages and states. A special case is that two messages ("promise" and "state") need to be sent after each other, but by the same role - the executor. This behaviour differs from the VISI behaviour, but can be modelled by setting the property `initiatorToExecutor` for both messages ("promise" and "state") to false. So `MessageInTransactionType` of "promise" and "state" need to have the value false for property `initiatorToExecutor`.

In theory, this restriction to the design of frameworks is already possible with the first version of VISI, but this document applies to version 1.6. (and higher).

All VISI transactions must follow a fixed pattern to become a DEMO transaction. The messages are:

- Request;
- Decline;
- Quit;
- Promise;
- State;
- Accept;
- Reject;
- Stop.

6.8.1.1 Main transaction

The figure above shows there are two messages Request. These are the exact same messages. They are displayed as two messages, because differentiation is needed between a) the message Request that starts a transaction, and b) the message Request that follows after a message Decline. According to the open standard VISI, a message that starts a transaction is recognised as a message that has no previous message. Because there is a loop between message Request and message Decline, only one message Request would be insufficient, because no message that starts the transaction would be recognized.

1. message Request has no previous message.
2. message Decline has message Request (1.) and message Request (3.) as previous message.
3. message Request has message Decline as the previous message.

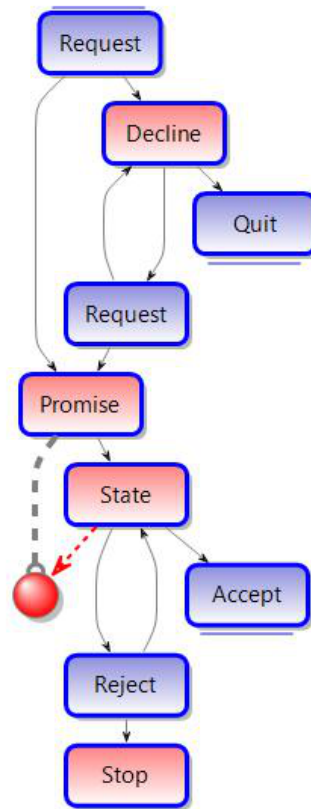


Figure 45:

4. message Quit has message Decline as previous message.
5. message Promise has message Request (1.) and message Request (3.) as previous message.
6. message State has message Promise as previous message.
7. etc.

To stay as close as possible to the current open standard VISI, this solution is chosen to best fit the functional requirement of explicit definition of the message that starts a transaction. An alternative solution would be to work with a message Initialise that would indicate the start of a transaction. This can be introduced in the following versions of this standard when the requirement to stay as close as possible to the existing standard is dropped.

6.8.1.2 Sub transactions

Also, (sub) transactions - started from a main transaction - use this default pattern, and have two messages Request to define the message that starts a transaction.

1. message Request has a) message Request/Promise/State/Accept from main transaction as previous message, and b) the property firstMessage set to true.
2. message Decline has message Request (1.) and message Request (3.) as previous message.
3. message Request has a) message Decline as previous message, and b) the property firstMessage set to false.
4. message Quit has message Decline as previous message.

5. message Promise has message Request (1.) and message Request (3.) as previous message.
6. message State has message Promise as previous message.
7. etc.

6.8.2 Mandatory transactionPhase and predefined values

According to the open standard VISI, the property transactionPhase of MessageInTransactionType is optional, but to determine the (DEMO) phase of a message, the use of the property transactionPhase of MessageInTransactionType is mandatory. TransactionPhaseTypes have predefined ids (TPT_DEMO_STATE_...) and descriptions (REQUESTED, PROMISED, STATED, ACCEPTED, QUITTED, DECLINED, REJECTED, STOPPED). All values for IDs and descriptions are in capital letters.

Every framework will have the following phases of a transaction:

```

1 <TransactionPhaseType id="TPT_DEMO_STATE_REQUESTED">
2   <description>REQUESTED</description>
3 </TransactionPhaseType>
4 <TransactionPhaseType id="TPT_DEMO_STATE_PROMISED">
5   <description>PROMISED</description>
6 </TransactionPhaseType>
7 <TransactionPhaseType id="TPT_DEMO_STATE_STATED">
8   <description>STATED</description>
9 </TransactionPhaseType>
10 <TransactionPhaseType id="TPT_DEMO_STATE_ACCEPTED">
11   <description>ACCEPTED</description>
12 </TransactionPhaseType>
13 <TransactionPhaseType id="TPT_DEMO_STATE_QUITTED">
14   <description>QUITTED</description>
15 </TransactionPhaseType>
16 <TransactionPhaseType id="TPT_DEMO_STATE_DECLINED">
17   <description>DECLINED</description>
18 </TransactionPhaseType>
19 <TransactionPhaseType id="TPT_DEMO_STATE_REJECTED">
20   <description>REJECTED</description>
21 </TransactionPhaseType>
22 <TransactionPhaseType id="TPT_DEMO_STATE_STOPPED">
23   <description>STOPPED</description>
24 </TransactionPhaseType>

```

6.8.3 Skip messages/transaction phases

To explicitly define if a message (and transaction phase) is applicable in a transaction or if a message needs to be skipped, functionality is introduced to flag a message as being 'HIDDEN'/'NOT ACTIVE'/'...'. For this flag, an existing property of a MessageType is used: state. Property state is a string, but the value of this string is a predefined one: PASSIVE. The value for state is in capital letters. Software that designs the framework will have to work according to this rule and visualise these messages differently. Software that executes the framework will have to skip/ignore these messages.

To minimise the size of frameworks, 8 general messages must be used:

```

1 <MessageType id="MT_NO_REQUEST">
2   <description>no request</description>
3   <state>PASSIVE</state>
4 </MessageType>
5 <MessageType id="MT_NO_PROMISE">
6   <description>no promise</description>
7   <state>PASSIVE</state>
8 </MessageType>
9 <MessageType id="MT_NO_STATE">
10  <description>no state</description>

```

```

11   <state>PASSIVE</state>
12 </MessageType>
13 <MessageType id="MT_NO_ACCEPT">
14   <description>no accept</description>
15   <state>PASSIVE</state>
16 </MessageType>
17 <MessageType id="MT_NO_QUIT">
18   <description>no quit</description>
19   <state>PASSIVE</state>
20 </MessageType>
21 <MessageType id="MT_NO_DECLINE">
22   <description>no decline</description>
23   <state>PASSIVE</state>
24 </MessageType>
25 <MessageType id="MT_NO_REJECT">
26   <description>no reject</description>
27   <state>PASSIVE</state>
28 </MessageType>
29 <MessageType id="MT_NO_STOP">
30   <description>no stop</description>
31   <state>PASSIVE</state>
32 </MessageType>

```

Transaction-specific MessageInTransactionTypes need to point to these default PASSIVE messages.

Example:

```

1  <MessageInTransactionType id="mit5349-promise">
2    <initiatorToExecutor>false</initiatorToExecutor>
3    <message>
4      <MessageTypeRef idref="MT_NO_PROMISE"/>
5    </message>
6    <previous>
7      <MessageInTransactionTypeRef idref="mit5348-request"/>
8      <MessageInTransactionTypeRef idref="mit5347-request"/>
9    </previous>
10   <transaction>
11     <TransactionTypeRef idref="T11_1_Aanleveren_Omgevingsmelding"/>
12   </transaction>
13 </MessageInTransactionType>

```

A transaction with all messages set to PASSIVE is the best way to start modelling a new transaction. The messages that need to be sent between initiator and executor are defined, and the pointer to a PASSIVE message can be replaced in the MessageInTransactionType.

For example, a message Promise:

```

1  <MessageInTransactionType id="mit5349-promise">
2    <initiatorToExecutor>false</initiatorToExecutor>
3    <message>
4      <MessageTypeRef idref="M11_1_Belofte_Aanleveren_Omgevingsmelding"/>
5    </message>
6    <previous>
7      <MessageInTransactionTypeRef idref="mit5348-request"/>
8      <MessageInTransactionTypeRef idref="mit5347-request"/>
9    </previous>
10   <transaction>
11     <TransactionTypeRef idref="T11_1_Aanleveren_Omgevingsmelding"/>
12   </transaction>
13 </MessageInTransactionType>

```

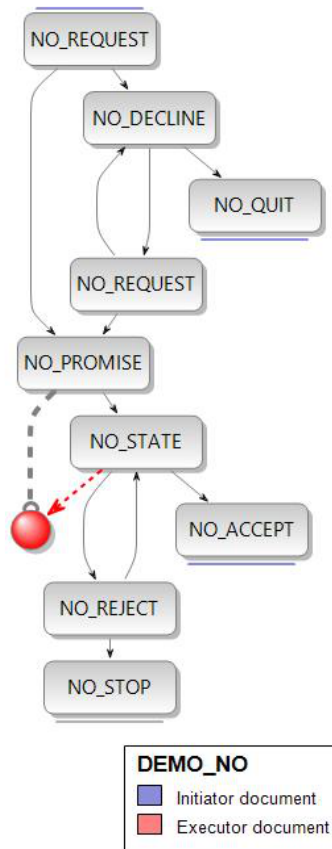


Figure 46:

6.8.4 Sub transactions

A sub-transaction can only be started from a main transaction when the phase of the main transaction is: REQUESTED, PROMISED, STATED or ACCEPTED.

Transferring from main to sub transaction and from sub to main transaction must be done according to the default transaction pattern phases. For example:

- From main transaction (phase=STATED) to sub transaction;
- Execution of sub transaction (phase=ACCEPTED) by sending messages Request, Promise, State and Accept;
- Return to main transaction (phase=STATED);
- Main transaction has two options to continue: message Accept or message Reject.

According to the open standard, VISI main and sub transactions need to be designed cascading. This means from main transaction T1 to sub transaction T66, to sub transaction T77, back to sub transaction T66, back to main transaction T1 (for short: T1>T66>T77>T66>T1). Skipping a sub transaction on the way back is NOT allowed (for short: NOT T1>T66>T77>T1). To model connecting DEMO transactions, cascading is not mandatory. For instance, because of a message Quit or a message Stop a sub-transaction can be terminated. DEMO transactions can be linked as a chain of transactions, and don't have to return to the main transaction. Software has to work according to the rule that after a message Quit or message Stop of a sub transaction, the flow returns to the main transaction, and needs to be continued in the main transaction at the point where the sub transaction was started. The exact position in the main transaction can be derived from the transaction phase, because there is a default

transaction pattern with default phases.

Mandatory sub transactions can be modelled a) with properties `sendBefore/sendAfter` of `MITTConditions`, or b) by simply continuing into a sub transaction by skipping message `Quit` and message `Stop` (see `PASSIVE`).

To model the execution order of sub transactions, `MITTConditions` (properties `sendBefore/sendAfter`) need to be used - in case of multiple sub transactions. Example: How to model requesting an "Advice" before requesting an "Approval"? Message `Request` and message `State` of sub transaction "Advise" have a `sendBefore` pointing to the first message `Request` of sub transaction "Approval". The result is that first the transaction "Advise" needs to be started/requested and stated, before the sub-transaction "Approval" can be started/requested.

6.8.5 Fact Bank per transaction

DEMO has a Fact Bank, but open standard VISI doesn't have such a central storage of all facts, which is accessible to all actors (initiators and executors) of all transactions. The reason is that not all actors work in the same organisations, and transactions are inside an organisation or between organisations. In other words, there is not one publicly available Fact Bank for all organisations involved in executing transactions.

Every instance of a transaction can be initiated by a different organisation (with different IT systems). Coordination and Production Facts between organisations are transferred digitally, and organisations cannot access each other's Fact Banks for Coordination and Production Facts of internal transactions of other organisations. For example, the internal Coordination and Production Facts of an insurance company are not accessible to the subcontractor hospitals, and vice versa, the internal Coordination and Production Facts of the hospitals are not accessible to the contractor insurance company.

According to the open standard VISI, every transaction contains the Coordination and Production Facts that are necessary for that particular transaction. Therefore, Coordination and Production Facts can only be transferred in a transaction by transferring messages and attachments.

Because dependent (internal) sub transactions are initiated by other organisations, facts from the main transaction need to be transferred to the dependent sub transaction if this particular sub transaction needs those Coordination and Production Facts.

6.8.5.1 Remark

During the creation of this documentation, it was decided to always include the complete transaction pattern in a framework. This means that skipped messages are explicitly modelled as implicit messages (see `PASSIVE`). These implicit messages always appear as `MessageInTransactionType` (MITT) in the frameworks. After all, when the MITTs are not included in the framework, an alternative has to be designed for capturing the following MITT properties:

- `transactionPhase`
- `openSecondaryTransactionsAllowed`

Future discussion can arise, and this discussion has also been conducted (by JL, JvG, GS), whether it is necessary to include MITTs in the framework. The properties `transactionPhase` and `openSecondaryTransactionsAllowed` could then be added to `MessageType`. This, however, requires an amendment to the open standard VISI (and we don't want to change it). For this reason, we have opted to continue working with MITTs (for the time being).

Also, the size of a framework is a point of attention. That is why only 8 general messages are allowed to define implicit messages (see `PASSIVE`).

6.9 Revokes in a Transaction Tree

The open standard VISI has the concept of replying to a received message like a ping pong game. In ping pong, only one player is allowed to hit the ball (depending on where the ball is on the table), and the ball is only allowed to be hit once by one player, and then the other player has to hit it, etc. In VISI, the initiator and executor send messages one by one. It is always clear which actor needs to reply to a received message.

Although communication is asynchronous, it is always clear which actor is the one to continue the transaction by replying to the received message.

By introducing the option to revoke a reply, the concept of replying on a received message like a ping pong game is gone.

To model the 4 revocation patterns of a DEMO transaction in every transaction will lead to much bigger and cluttered transactions, which are hard to design. A more efficient solution was chosen: the 4 revocation patterns of DEMO are modelled as separate transactions, which are valid for ALL transactions of a framework. Software for designing and executing transactions needs to follow this strict guideline.

By letting go of the concept of sending and receiving messages, revocations can also be modelled in a VISI framework. The complete transaction pattern that is realised with this makes the construction of frameworks, however, unclear and labour-intensive. In this trailer, it was therefore decided to model the 4 revocation patterns of DEMO as separate transactions and to provide software builders with a guideline on how to deal with these separate revoke transactions.

As a result, if revokes are present in a framework, then as a consequence a transaction can never be really closed, because the last message Accept with the status 'ACCEPTED' can still be revoked. (But this revoke can of course be denied.)

The 4 revocation transactions are "REVOKE REQUEST", "REVOKE PROMISE", "REVOKE STATE" and "REVOKE ACCEPT". These should be modelled in a framework as follows.

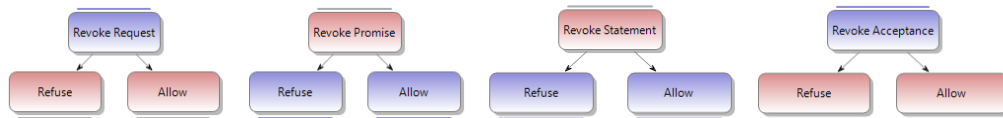


Figure 47:

The ids of the transactions are predefined:

```

1  <TransactionType id="TT_DEMO_Revoke_Pattern_Request">
2    <description>DEMO REVOKE PATTERN - REQUEST</description>
3    <initiator>
4      <RoleTypeRef idref="RT_Initiator"/>
5    </initiator>
6    <executor>
7      <RoleTypeRef idref="RT_Executor"/>
8    </executor>
9  </TransactionType>
10 <TransactionType id="TT_DEMO_Revoke_Pattern_Promise">
11   <description>DEMO REVOKE PATTERN - PROMISE</description>
12   <initiator>
13     <RoleTypeRef idref="RT_Initiator"/>
14   </initiator>
15   <executor>
16     <RoleTypeRef idref="RT_Executor"/>
17   </executor>
18 </TransactionType>
19 <TransactionType id="TT_DEMO_Revoke_Pattern_State">
20   <description>DEMO REVOKE PATTERN - STATE</description>
21   <initiator>

```

```

22     <RoleTypeRef idref="RT_Initiator"/>
23 </initiator>
24 <executor>
25     <RoleTypeRef idref="RT_Executor"/>
26 </executor>
27 </TransactionType>
28 <TransactionType id="TT_DEMO_Revoke_Pattern_Accept">
29     <description>DEMO REVOKE PATTERN – ACCEPT</description>
30     <initiator>
31         <RoleTypeRef idref="RT_Initiator"/>
32     </initiator>
33     <executor>
34         <RoleTypeRef idref="RT_Executor"/>
35     </executor>
36 </TransactionType>

```

Also, the IDs and description of the transaction phases are predefined:

```

1 <TransactionPhaseType id="TPT_DEMO_STATE_REVOKED_ACCEPT">
2     <description>REVOKED ACCEPT</description>
3 </TransactionPhaseType>
4 <TransactionPhaseType id="TPT_DEMO_STATE_ALLOWED_REVOKE_ACCEPT">
5     <description>ALLOWED REVOKE ACCEPT</description>
6 </TransactionPhaseType>
7 <TransactionPhaseType id="TPT_DEMO_STATE_REFUSED_REVOKE_ACCEPT">
8     <description>REFUSED REVOKE ACCEPT</description>
9 </TransactionPhaseType>
10 <TransactionPhaseType id="TPT_DEMO_STATE_REVOKED_PROMISE">
11     <description>REVOKED PROMISE</description>
12 </TransactionPhaseType>
13 <TransactionPhaseType id="TPT_DEMO_STATE_ALLOWED_REVOKE_PROMISE">
14     <description>ALLOWED REVOKE PROMISE</description>
15 </TransactionPhaseType>
16 <TransactionPhaseType id="TPT_DEMO_STATE_REFUSED_REVOKE_PROMISE">
17     <description>REFUSED REVOKE PROMISE</description>
18 </TransactionPhaseType>
19 <TransactionPhaseType id="TPT_DEMO_STATE_REVOKED_REQUEST">
20     <description>REVOKED REQUEST</description>
21 </TransactionPhaseType>
22 <TransactionPhaseType id="TPT_DEMO_STATE_ALLOWED_REVOKE_REQUEST">
23     <description>ALLOWED REVOKE REQUEST</description>
24 </TransactionPhaseType>
25 <TransactionPhaseType id="TPT_DEMO_STATE_REFUSED_REVOKE_REQUEST">
26     <description>REFUSED REVOKE REQUEST</description>
27 </TransactionPhaseType>
28 <TransactionPhaseType id="TPT_DEMO_STATE_REVOKED_STATE">
29     <description>REVOKED STATE</description>
30 </TransactionPhaseType>
31 <TransactionPhaseType id="TPT_DEMO_STATE_ALLOWED_REVOKE_STATE">
32     <description>ALLOWED REVOKE STATE</description>
33 </TransactionPhaseType>
34 <TransactionPhaseType id="TPT_DEMO_STATE_REFUSED_REVOKE_STATE">
35     <description>REFUSED REVOKE STATE</description>
36 </TransactionPhaseType>

```

6.9.1 Guideline for framework designers

The framework designer starts by downloading the [default framework with the DEMO transaction pattern]([https://github.com/nl-digigo/visi/blob/master/ISSUES%20docs/DEMO_voorbeeld_me_t_revoke_subtransacties_v2%20\(1\).xml](https://github.com/nl-digigo/visi/blob/master/ISSUES%20docs/DEMO_voorbeeld_me_t_revoke_subtransacties_v2%20(1).xml)).

([https://github.com/nl-digigo/visi/wiki/Richtlijn-voor-implementatie-van-het-compleet-DEMO-transactie-patroon-\(met-VISI-Standaard-versie-1.6\)](https://github.com/nl-digigo/visi/wiki/Richtlijn-voor-implementatie-van-het-compleet-DEMO-transactie-patroon-(met-VISI-Standaard-versie-1.6))).

In this basic framework for DEMO with open standard VISI, examples of the 4 revocation transactions are modelled in accordance with DEMO 3. These 4 revocation transactions must be present in the same way in every project-specific framework to unlock revocation options. The message Revoke ..., Allow, and Refuse always have the same message content (ComplexElements and SimpleElements). The IDs of the messages are: MT_Revoke_Accept, MT_Revoke_Promise, MT_Revoke_Request, MT_Revoke_State, MT_Allow, MT_Refuse. These IDs are reserved for the 4 revocation transactions, and only these IDs must unlock revoke functionality (when executing DEMO transactions). If the framework designer does not want to allow these revocation options, the 4 revocation transactions need to be excluded from the project-specific framework.

6.9.2 Software Guidelines

If one or more of the 4 revocations transactions are available in a framework, the software must offer these revocation options to the user in any transaction when it is valid according to the phase of the transaction. This means that a transaction REVOKE REQUEST can only be initiated from a sent message Request. The REVOKE REQUEST transaction is linked as a sub-transaction from the message where the REVOKE REQUEST transaction was called. For the link, the property initiatingTransactionMessageID is used. The message MT_Revoke_Request has this property.

So when a transaction REVOKE STATE is initiated from a STATE MITT T1, the STATE message becomes the parent of the transaction REVOKE STATE. This is recorded in the property initiatingTransactionMessageID of the REVOKE message by referring to the ID of the message from which the REVOKE message was initiated. Thus, in the above example, in the REVOKE message, as initiatingTransactionMessageID, the ID of the STATE message is registered.

In this case, the property openSecondaryTransactionsAllowed (false/true) cannot be used to enforce first completing the REVOKE transaction before the main transaction T1 continues. For this reason, software must act in such a way that the REVOKE transaction is processed before the main transaction T1 continues.

6.9.3 Revoke of main transaction and open sub transactions

Because REVOKE transactions are modelled as separate transactions, there are no 'connections' of message MT_Allow or message MT_Refuse back to transaction T1. The follow-up message in T1 that is allowed after an ALLOW or REFUSE of a REVOKE can be determined as follows.

Upon receiving a message MT_Allow, track back in the REVOKE transaction for the previous message to determine if it is an ALLOW of a revoke request, promise, state or accept.

There are now 4 possibilities:

1. revoke request -> allow, the main transaction cannot be continued because the transaction phase is QUITTED
2. revoke promise -> allow, the main transaction can continue with the messages Promise or Decline from transaction phase REQUESTED
3. revoke state -> allow, the main transaction can continue with the message State from transaction phase PROMISED
4. revoke accept -> allow, the main transaction can continue with the messages Accept of Reject with transaction phase STATED

Ad. 1. According to DEMO, a revoke of a Request results in a quit/termination of the transaction.

Upon receiving of a message MT_Refuse, there is one possibility: 5. revoke -> refuse, the main transaction continues where it was before the revoke transaction was initiated

6.9.4 Content of revoke messages

Because there are only 4 revocation transactions and 6 revocation messages, the content of the 6 revocation messages has the same properties for all transactions on which the revocation transactions are initiated.

The content of the 6 revocation messages is predefined but can best be kept simple and focused on executing revocation transactions.

For example:

```
1  <MessageType id="MT_Revoke_Request">
2    <description>... (REVOKE REQUEST)</description>
3    <complexElements>
4      <ComplexElementTypeRef idref="CET_Request"/>
5      <ComplexElementTypeRef idref="CET_Comments"/>
6      <ComplexElementTypeRef idref="CET_Response"/>
7    </complexElements>
8  </MessageType>
9  <ComplexElementType id="CET_Request">
10    <description>Request</description>
11    <simpleElements>
12      <SimpleElementTypeRef idref="SET_Request"/>
13    </simpleElements>
14  </ComplexElementType>
15  <SimpleElementType id="SET_Request">
16    <description>request</description>
17    <userDefinedType>
18      <UserDefinedTypeRef idref="UDT_StringMandatory"/>
19    </userDefinedType>
20  </SimpleElementType>
21  <ComplexElementType id="CET_Response">
22    <description>Response</description>
23    <simpleElements>
24      <SimpleElementTypeRef idref="SET_ResponseDate"/>
25    </simpleElements>
26  </ComplexElementType>
27  <SimpleElementType id="SET_ResponseDate">
28    <description>response date</description>
29    <userDefinedType>
30      <UserDefinedTypeRef idref="UDT_Date"/>
31    </userDefinedType>
32  </SimpleElementType>
```

6.9.5 Revokes & sub transactions

When a main transaction - that has already started one or more sub transactions - receives a request revoked then this revoke is handled as follows:

- If property `openSecondaryTransactionsAllowed = false` and sub transactions are open, then the main transaction is NOT allowed to be revoked.
- If property `openSecondaryTransactionsAllowed = true` and sub transactions are open, then the main transaction is allowed to be revoked. Sub transactions stay open, but can optionally be revoked by sending `Revoke Request(s)`.

6.9.6 Summary

- REVOKE messages have fixed messaging types (6 types: `MT_Revoke_Accept`, `MT_Revoke_Promise`, `MT_Revoke_Request`, `MT_Revoke_State`, `MT_Allow`, `MT_Refuse`)
- REVOKE transactions can only be initiated from the corresponding message (REVOKE REQUEST from a REQUEST, etc.)

- REVOKE transactions become sub-transactions of the transactions from which they are initiated. This is recorded with the property `initiatingTransactionMessageID`
- REVOKE transactions are processed first before the main transaction continues.
- REVOKE transactions - if present in the framework - always apply to all transactions.
- REVOKE of (sub) transactions are handled as described above.

6.10 DEMO Crossing Messages (and order of events)

Organisations communicating via the open standard VISI are located at different locations and in different networks. The exchange of information is not synchronised. Like email messages can cross each other on the internet. This chapter explicitly describes what rules apply if two DEMO messages cross each other.

VISI is ASYNCHRONOUS communication. In standard VISI communication, this is not an issue because there is no possibility for initiator or executor to send a second message before a reply is received. With the introduction of Revokes it will be possible to send two messages (a normal message and a revoke). Revoke has a higher priority than non revoke messages to be executed. So the revoke needs to be handled before any normal message in this transaction can be sent.

6.10.1 Crossing revoke and message of main transaction

It is possible that, for example, the executor sends a Revoke State at the exact same moment as the initiator sends an Accept. The priority when a Revoke crosses with a message of the main transaction is to first handle the Revoke.

In case the Revoke is replied with Refuse, the message of the main transaction determines the state of the transaction. In case the Revoke is replied with Allow, this determines the state of the transaction. The state becomes the state prior to the revoke.

For example:

Replace ??? with the correct data

- An Allow of Revoke Requests results in the state of transaction being ???.
- An Allow of Revoke Promise results in the state of transaction being Requested.
- An Allow of Revoke State results in the state of the transaction being Promised.
- An Allow of Revoke Accept results in the state of transaction being Stated.

In more detail:

- Organisation A sends an Accept.
- Organisation B sends a Revoke State;
- Organisation B receives the Accept, and sends a confirmation message;
- The main transaction of organisation A is not locked;
- The main transaction of organisation B is not locked;
- Organisation A receives the Revoke State and sends a confirmation message;
- The main transaction of organisation A is locked;
- The main transaction of organisation B is locked;

- First organisation A needs to reply to the Revoke Accept (with Allow or Refuse);
- Organisation B receives organisation A's response (Allow or Refuse) to the Revoke State and sends a confirmation message;
- The main transaction of organisation B is unlocked;
- Organisation A receives the confirmation message from organisation B;
- The main transaction of organisation A is unlocked;
- Organisation B has the reply (Allow or Refuse) to its Revoke State, and organisation B can now determine the state of the main transaction; In case of Refuse the state of the transaction is Accepted. And in the case of Allow, the state of the transaction is Promised.

6.10.2 Crossing revokes

It is possible that, for example, the executor sends a revoke at the exact same moment as the initiator sends a revoke. The priority when two Revokes cross each other is backwards in the transaction pattern. The order is: First Revoke Accept, then Revoke State, then Revoke Promise, then Revoke Request

The last Revoke that is replied with Allow determines the phase of the transaction. Normally, this will be a 'happy flow' transaction phase message, but in the case of two crossing revokes, both revokes need to be resolved before the regular message flow continues.

When a revoke is sent (and confirmed with a confirmation message), then that (side of the) transaction is frozen until it is replied to with an Allow or Refuse. When revokes cross each other (and are both confirmed), that means both sides of the transaction are frozen until both revokes are replied to (with allow or refuse).

When 2 Revokes cross each other, the following rules apply:

- before the main transaction continues, both revokes must be replied (with Allow or Refuse);
- if both revokes are refused, the state of the transaction remains the same as before the revokes;
- if one of the two revokes is allowed (and the other refused), the state of the transaction is equal to the state prior to the revoke (Allow of Revoke State result in state Promised);
- if both revokes are allowed, the state of the transaction is equal to the highest state reached by 1 of the revokes. (State order: 1. Request, 2. Promise, 3. State, 4. Accept).

Example:

- Both organisations send a revoke at the same time;
- Both organisations confirm the reception of the revokes;
- Each organisation handles the revoke they have received (by replying with allow or refuse);
- When both revokes are handled, replied, and the replies are confirmed, the state of the transaction is determined, and the main transaction will continue.

In more detail:

- Organisation A sends a Revoke Accept;
- Organisation B sends a Revoke State;
- Organisation B receives the Revoke Accept, and sends a confirmation message;

- The main transaction of organisation A is locked;
- Organisation A receives the Revoke State and sends a confirmation message;
- The main transaction of organisation B is locked;
- Organisation A replies the Revoke State (with Allow or Refuse);
- Organisation B replies the Revoke Accept (with Allow or Refuse);
- Organisation B receives organisation A's response (Allow or Refuse) to the Revoke State and sends a confirmation message;
- The main transaction of organisation A is still locked;
- The main transaction of organisation B is still locked;
- Organisation A receives organisation B's response (Allow or Refuse) to the Revoke Accept and sends a confirmation message;
- Organisation A receives the confirmation message from organisation B;
- Organisation B receives the confirmation message from organisation A;
- Organisation A has the reply (Allow or Refuse) to its Revoke Accept and a confirmation message on its reply to the revoke of organisation B, and organisation A can now determine the state of the main transaction;
- Based on the state of the main transaction organisation A can or cannot send a new message to organisation B;
- Organisation A can also send another Revoke message (for example, Revoke Request);
- Organisation B has the reply (Allow or Refuse) to its Revoke State and a confirmation message on its reply to the revoke of organisation A, and organisation B can now determine the state of the main transaction;
- Based on the state of the main transaction organisation B can or cannot send a new message to organisation A;
- Organisation B can also send another Revoke message (for example, Revoke Promise).

6.11 Getting started with DEMO Transaction Pattern

6.11.1 Prerequisite

Before starting to use the DEMO transaction pattern, gain knowledge about it by reading a [book](<http://www.springer.com/la/book/9783540291695>) or following a [course](<http://www.ee-institute.org/en/courses>).

6.11.2 Start with a VISI framework

Translating a VISI transaction to a DEMO transaction pattern is not straightforward and cannot be done automatically. The framework designer needs to have knowledge about the VISI transaction and interpret the names of VISI messages. As a first step, the VISI messages need to be tagged with the default actions from the DEMO transaction pattern (request, decline, quit, promise, state, reject, stop, accept). Also, the designer needs to search for the modelling in a VISI transaction of a revoke by means of a VISI message ("intrekking melding").

Second step is to determine which of the default actions of the DEMO transaction pattern are not available in the VISI messages. These actions are added to the framework as implicit actions (see chapter

[...](<https://github.com/nl-digigo/visi/wiki/3.-Definition-of-Framework>)).

The third step is to cut a VISI transaction up into one or more default DEMO transaction patterns.

Fourth step is to check if every action is only used once in a DEMO transaction pattern. For example, if there are two different message with are in a state, these messages need to be combined, or the transaction needs to be split into two transactions. Such messages can be two-state messages: 1) "positief advies", 2) "negatief advies". This can be combined into one message "advies". The message contains a simple element with the type of advice: "positief" or "negatief".

The appendix "[Bijlage A Translate VISI to DST method](<https://github.com/nl-digigo/visi/wiki/Bijlage-A---Translate-VISI-to-DST-method>)" expands on this method of translating VISI transactions to DEMO transactions. The appendix "[Bijlage B Translate VISI to DST example](<https://github.com/nl-digigo/visi/wiki/Bijlage-B---Translate-VISI-to-DST-example>)" gives an extensive example of how to translate one VISI transaction into several DEMO transaction patterns.

6.11.3 Start from scratch

The framework designer starts by downloading the [default framework with the DEMO transaction pattern] [https://github.com/nl-digigo/visi/blob/master/ISSUES%20docs/DEMO_voorbeeld_met_revoke_subtransacties_v2%20\(1\).xml](https://github.com/nl-digigo/visi/blob/master/ISSUES%20docs/DEMO_voorbeeld_met_revoke_subtransacties_v2%20(1).xml)).

6.12 Attachment A: Translate VISI to DST method

6.12.1 Method for converting a VISI framework to the DEMO standard transaction pattern

There are a number of guidelines you can follow to use the standard transaction pattern in VISI as described in the DEMO methodology. The most important points of attention are described below. Every message in an (existing) VISI transaction must be identified as 1 of the 14 different phases in the DST below to make the transaction compatible with the DST.

The happy flow in the DEMO standard transaction pattern (abbreviated as DST) consists of 4 consecutive steps or statuses. NB: We do not have the "Execute" phase in this application of the DST, as we only model and automate the communication.

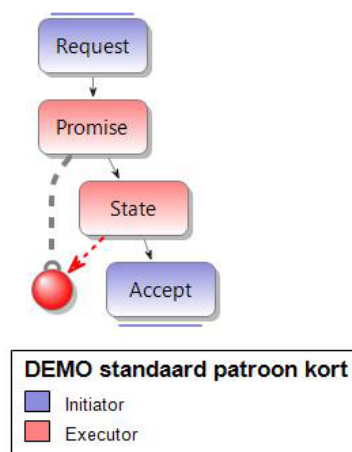


Figure 48:

In addition, there are 4 options to deviate from the happy flow.

And for every happy flow status, there is a Revoke phase, which must always be responded to with an Accept or Refuse. NOTE: In this application of the DST, the Allow and Refuse messages are always the same, regardless of which Revoke they are used for.

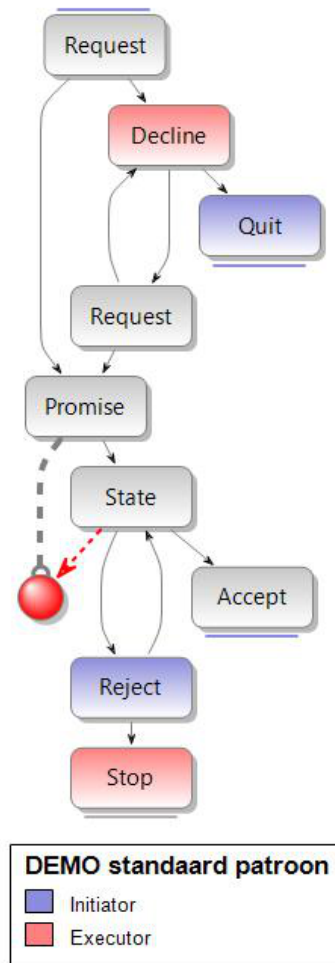


Figure 49:



Figure 50:

Because all phases are always available as options in the DST, the first step in converting a VISI framework to a DST-compatible framework is to create so-called “dummy messages”. Those are the 8 basic phases of the DST, so everything except the revokes. In the framework, you can set the “state” of these dummy messages to “passive”, thus indicating that these phases are implicit and should be skipped. It is useful to give them a recognisable name and ID, for example, “NO_REQUEST” and “NO_STOP”. These dummy messages will be used to fill in the missing phases in the DST. It is not necessary to create dummy messages for the revokes because they are always available as an option and will therefore not be specifically modelled.

After creating the 8 dummy messages, you can analyse the VISI transaction to determine how often the DST occurs. In a regular VISI transaction, it is possible to use an infinite number of messages, but the DST only has the 14 prescribed phases. This may result in a VISI transaction being divided into multiple DST transactions.

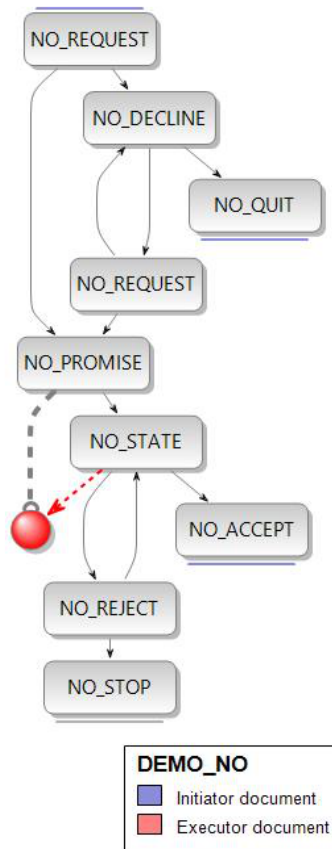


Figure 51:

Identifying the existing VISA messages to one of the phases from the DST is a matter of insight and can lead to discussion. For example, reporting a contract change can be seen as a State, but also as a Request. In the worked example, it was decided to model this as a Request. This choice has been made because this is the first message in the transaction and because you can argue that no (implicit) Request has yet been made.

NB: You could also argue that a contract change is a Revoke Request, but since building the contract is not included in the framework used, the original Request does not exist, and therefore you cannot start with a Revoke Request.

NB: When supplying previously (contractually) agreed documents, you can, for example, argue that a clear request has been made in advance (in the contract). If building a contract is not included in the example framework used, the first message of another VISA transaction, for example, the delivery of a document for acceptance, could be seen as a State.

NB: Since the argumentation regarding naming phases in processes can differ between framework builders, it is possible that the same process will be modelled in different ways. The standard cannot change this; at most, examples and templates can be provided as guidance.

When the messages are identified as phases in the DST, you can distinguish which phases belong together in the same DST transaction. For example, by changing the naming of the message to “T_01_Request_notification_of_a_change”, you can also make this visible in the framework. If there are missing phases, you can fill them with dummy messages. You always use the same messages for this.

It may happen that there are VISI messages that combine multiple DST phases in 1 message. In that case, split the message into multiple messages so that they can be modelled in their own DST transaction.



Figure 52:

Conversely, there may also be multiple VISI messages that concern the same DST phase. This often concerns a difference in the content of the message, but the result is the same. For example, whether a payment is made via cash or ATM. The result is the same, i.e. payment is made, but the information required may differ.

NB: Since we do not have a fact base in VISI and all information is in the message, a difference in the required information means that different messages have to be used - the cash message does not contain a "PIN code" field. You will then receive different messages for the same State. In addition, it may also happen that other messages or dependent transactions are initiated based on the chosen method. For example, if you opt for cash payment, change may be given back - which would result in an additional message (transaction?).

Subtransactions (dependent transactions) can be modelled with the DST in the same way as the main transaction. These will often consist of a single DST, often with missing phases. The entry and exit of the subtransaction will sometimes determine what options exist. For example, a judgment request that appears in the main transaction in a State message can be ignored (for example, giving a judgment of non-acceptance and still accepting it). Another example is a mandate request where, after the mandate (State) message that has not been given, only a Decline or Reject can be selected when returning to the main transaction.

Existing options in the system regarding mit conditions and whether or not you can continue with the main transaction if a sub-transaction is open will continue to exist.

A detailed practical example of this method can be found in [Appendix B Translate VISI to DST example](<https://github.com/nl-digigo/visi/wiki/Bijlage-B—Translate-VISI-to-DST-example>).

6.13 Attachment B Translate VISI to DST example

6.13.1 Example: Converting UAV2012 transaction changes to DEMO standard transaction pattern

Original VISI transactions:

TR_01 Submission: notification of a change - from OG_1 (blue, initiator) to ON_1 (red, executor)
 TR_02 Judgment request: quotation change - from OG_1 (blue, initiator) to OG_2 (red, executor)
 TR_03 Advice request: (quotation) change - from OG_2 (blue, initiator) to OG_3 (red, executor)

Identifying DEMO phases in the VISI transaction:

Notes on identifying DST phases in existing UAV2012 changes transactions:

- The main transaction consists of several DEMO transactions

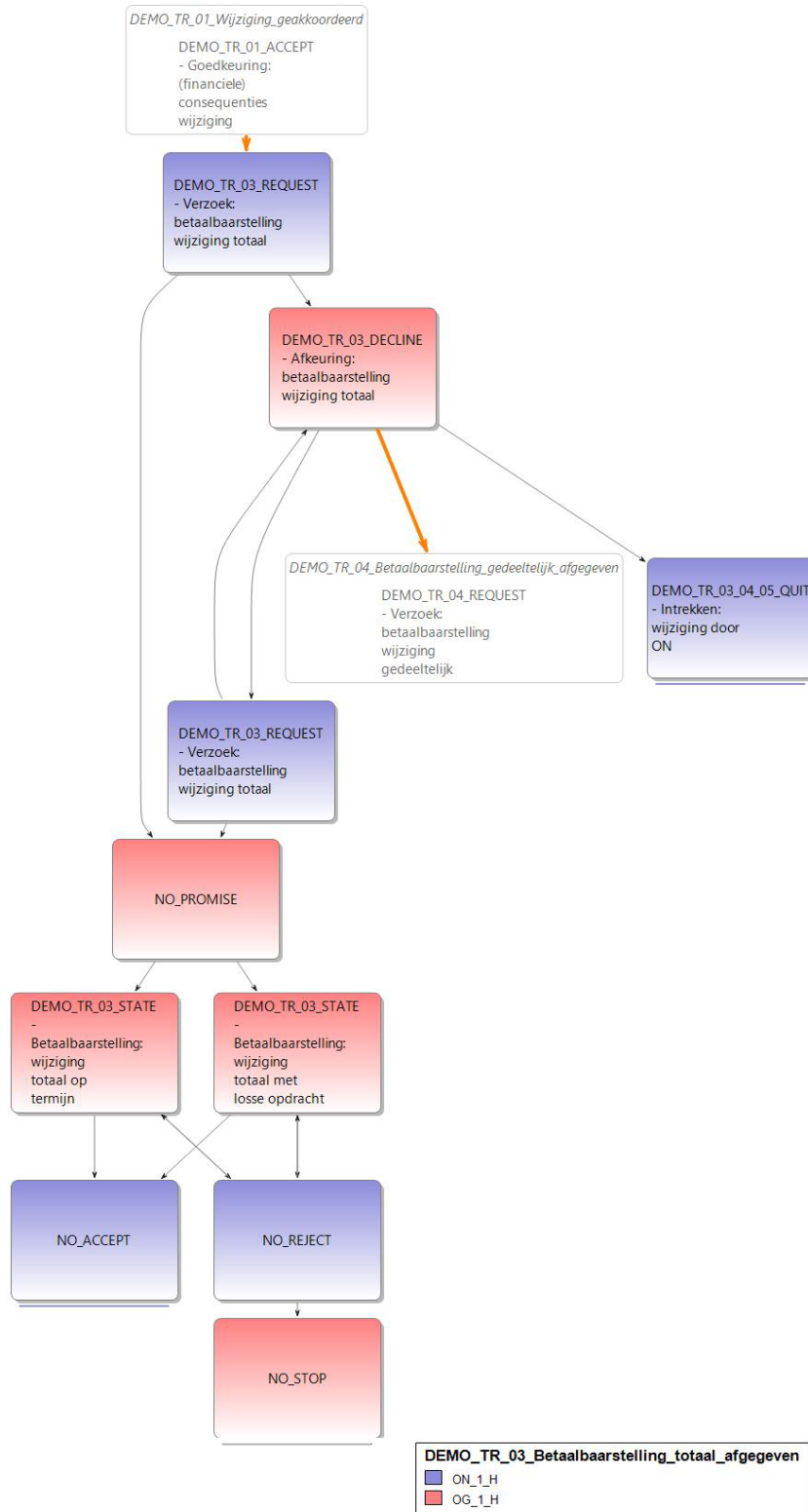


Figure 53:

- Subtransactions such as judgment requests or advice requests often consist of fewer phases in VISI frameworks (e.g. only request and state), in this example, no route is yet defined from the judgment

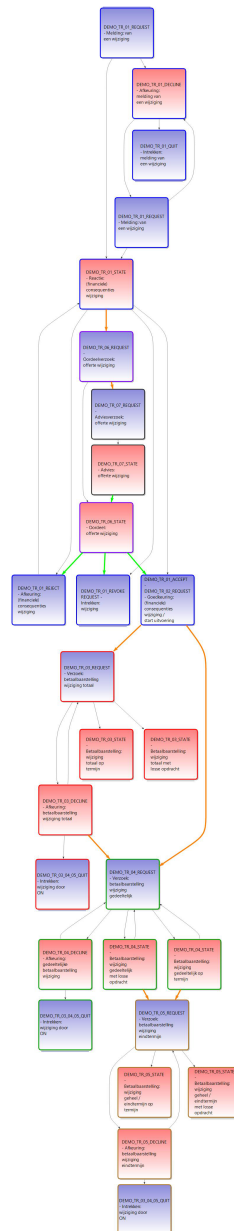


Figure 55:

in the main transaction. For example, with a mandate, you could see that the sub-transaction already indicates which message you will end up with in the main transaction - now you can ignore the judgment.

- Not all phases occur as a message (e.g. no promise, no stop in “DEMO_TR_01”)
- Some phases occur more often per transaction (e.g. “DEMO_TR_03_STATE”), because the content of the message may differ (payment per instalment or as a separate order)
- Payment can be made in 3 different ways: immediately, the total, partially and after the parts, with a final term. These are now 3 different transactions because the result is different, and also because otherwise you cannot make multiple requests with different content. But actually, these transactions are very similar. When do you split transactions? If the product/result is different. Does a different amount mean a different result? Does a different accounting method produce a different result? (in due course as a separate assignment)

- DEMO transaction 01 (DEMO_TR_01_Change_approved)



DEMO transaction 03a including passive (dummy) messages - split into a and b because there are different State options, where the content of the message may differ even though the result is the same.

DEMO transaction 03b including passive (dummy) messages - split into a and b because there are different State options, where the content of the message may differ even though the result is the same.

DEMO transaction 04 (DEMO TR 04 Payment partial issued)

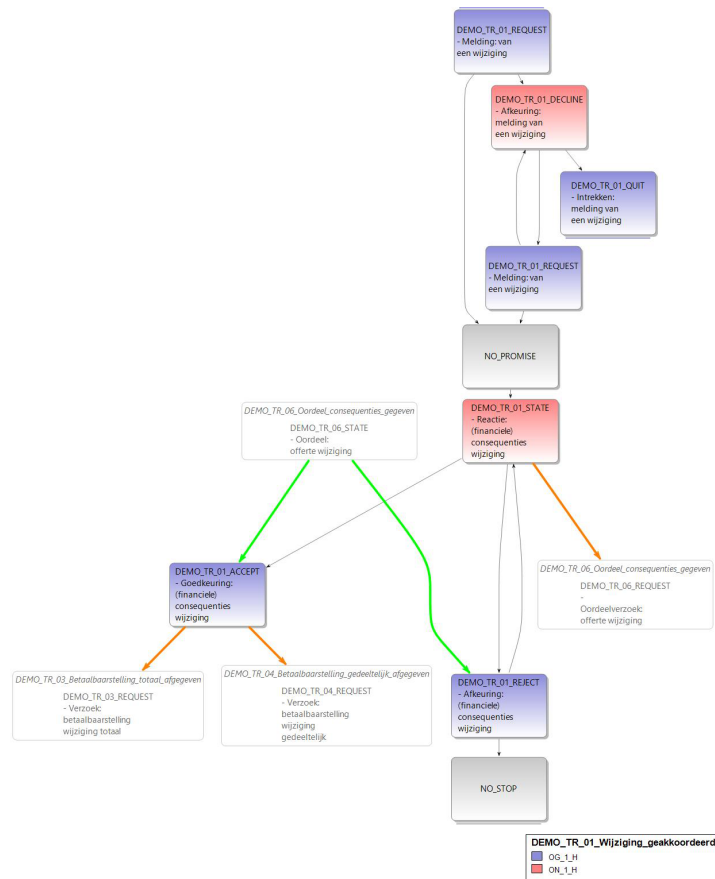


Figure 57:

DEMO transaction 04 including passive (dummy) messages

DEMO transaction 04a including passive (dummy) messages - split into a and b because there are different State options, where the content of the message may differ even though the result is the same.

DEMO_TR_04a_Payment_partial_in_term

DEMO transaction 04b including passive (dummy) messages - split into a and b because there are different State options, where the content of the message may differ even though the result is the same.

DEMO_TR_04b_Payment_partial_single_order

DEMO transaction 05 (DEMO_TR_05_Payment_final term_issued)

DEMO transaction 05 including passive (dummy) messages

DEMO transaction 05a including passive (dummy) messages - split into a and b because there are different State options, where the content of the message may differ even though the result is the same.

DEMO_TR_05a_Payment_final term_in_term

DEMO transaction 05b including passive (dummy) messages - split into a and b because there are different State options, where the content of the message may differ even though the result is the same.

DEMO_TR_05b_Payment_final term_single_assignment

DEMO transaction 06 (DEMO_TR_06_Judgment_consequences_given)

DEMO transaction 06 including passive (dummy) messages

DEMO transaction 07 (DEMO_TR_07_Advice_consequences_given)

DEMO transaction 07 (DEMO_TR_07_Advice_consequences_given)

Total scenario of all broken down DEMO-compatible transactions with dummy messages.

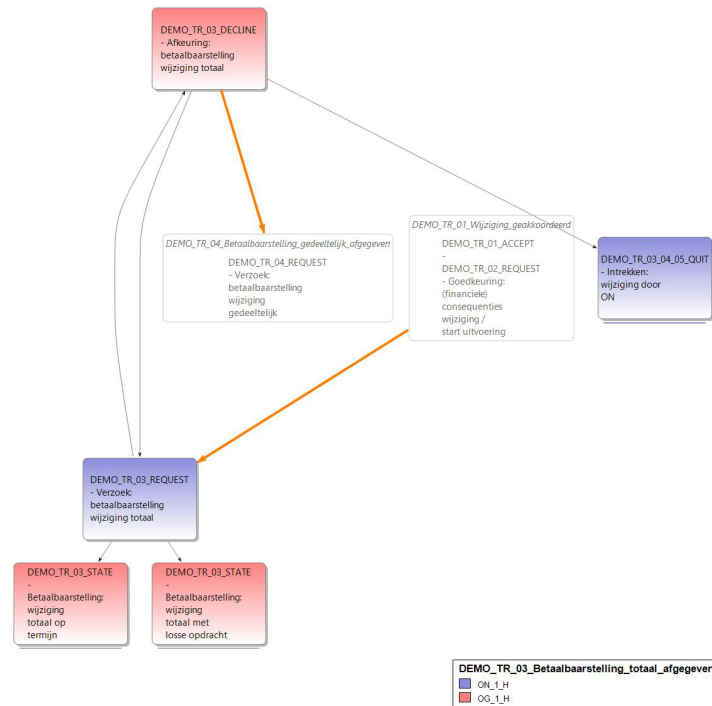


Figure 58:

Notes when splitting the VISI transaction into DEMO transactions:

- “DEMO_TR_02_Change_executed” is not in the scenario because otherwise a message would appear twice in the scenario (“DEMO_TR_01_ACCEPT - DEMO_TR_02_REQUEST - Approval: (financial) consequences of change/start of implementation”)
- The message “DEMO_TR_03_04_05_QUIT - Withdraw: change by ON” now appears 3 times to complete the DEMO transactions

Comments on making the split DEMO/VISI transactions DEMO compatible:

- Revoke “DEMO_TR_01_REVOKE REQUEST - Withdraw: change” has been removed from the transaction because revokes are automatically available
- “DEMO_TR_01_ACCEPT - DEMO_TR_02_REQUEST - Approval: (financial) consequences of change/start of implementation” has been renamed to “DEMO_TR_01_ACCEPT - Approval: (financial) consequences of change”
- In the model of “DEMO_TR_01”, you can no longer immediately go from request to state; there is now a promise in between - the software skips this message because it is set to passive (so the user can simply request and go to state)
- At “DEMO_TR_03_Betaalbaarstelling_totaal_issued” you now have 2 request messages, so that this corresponds to the standard transaction pattern in VISI - 1 request as a starting message, 1 request as a mutable message
- The problem of double states is more common, for example, if you want to pay at the pizzeria, you can pay with cash or with PIN - these are 2 different ways where different information is required from the payer. No information for cash, for PIN the payer needs a PIN code.
- By splitting the transactions, you have to set conditions on subtransactions that you can only start once. From the VISI transaction, some messages can only be sent once because this is included in the flow of the transaction, but you can start subtransactions as often as you want. So, for example, you must only be able to start a payment request for the total amount once.

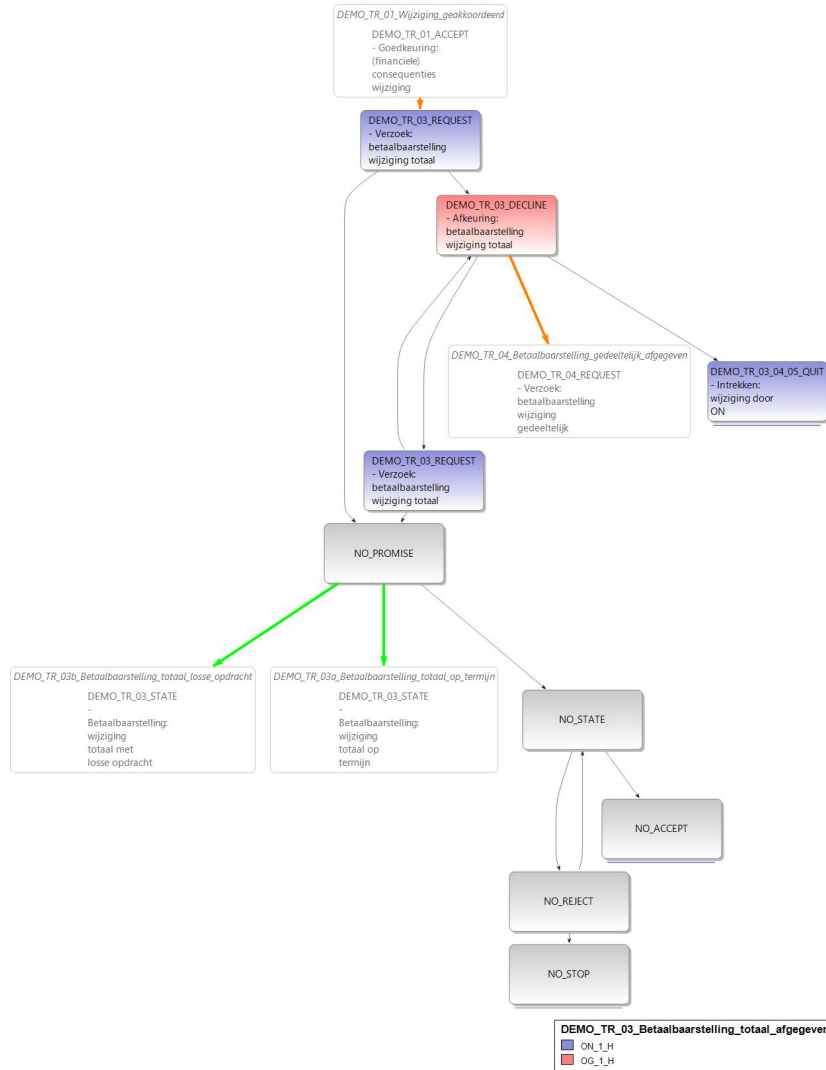


Figure 59:

6.14 DEMO

6.14.1 DEMO transaction pattern (#60) research

VISI originated from DEMO in the early stages. For the acceptance of implementation, some concepts of DEMO were lost. These DEMO concepts allow for easier and complete modelling of business processes. The compact DEMO models of the organisation's essence are the base for fully implementing the business processes with VISI frameworks. With version 1.8, we want to return to the possibility of including complete DEMO transactions. The option to include the DEMO transaction pattern has initiated the option to release the 'ping-pong' requirement of the VISI standard because, in DEMO, some steps in a transaction have message sequences from the same sender. To explain the concept, we have to introduce the terms message-from and message-to. A message sent from the initiator to the executor (with the attribute InitiatorToExecutor = true) is called a message-from. A message that is sent from the executor to the initiator (InitiatorToExecutor = false) is called message-to.

In VISI, a MITT type can have a previous Message. Another concept is the reverse term of the previous: next. Every previous pointer of message B to message A has an equivalent next pointer from message A to message B. Where the first message of a transaction has no previous pointers, the final message of a transaction has no next pointers. We assume that the 'next' value is calculated out of 'previous' values,

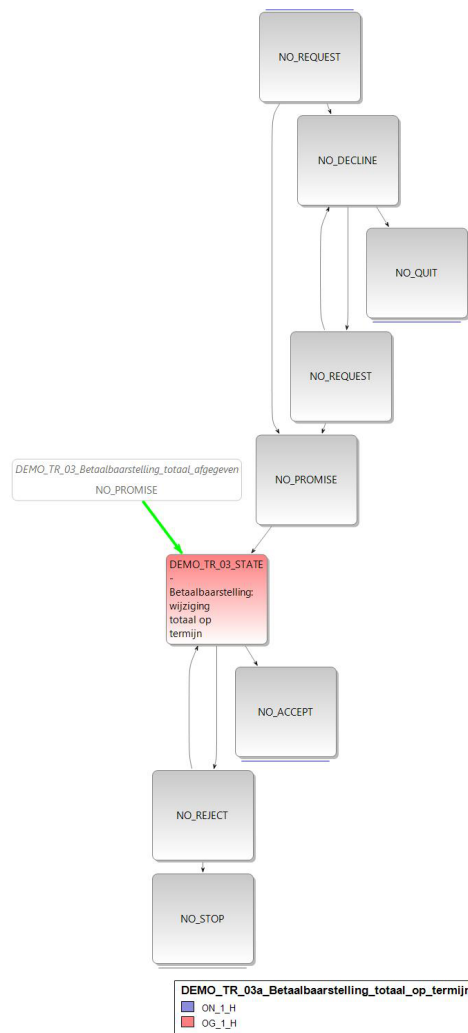


Figure 60:

and not stored in the framework.

We need these ‘message’ and ‘next’ concepts to clearly state the rules of the communication protocols 1.7 and 1.8.

For the 1.0-1.7 version protocol, the from and to messages were sent alternating between the initiator and executor, ‘ping-pong’. Currently, the messages within one transaction have to comply with the following rules.

- Every transaction starts with a ‘from’ message.
- Every from-message will have next pointers to to-messages when having a next pointer.*
- Every to-message, when having a next pointer, will have next pointers to from-messages.*
- *: with the exception that it is also possible that such a next-previous relation is not there within the transaction between two messages and that there is a route through a subtransaction.

< insert picture to show current communication >

For the 1.8 version protocol, the from and to messages can still be sent alternating, but an alternative is allowed.

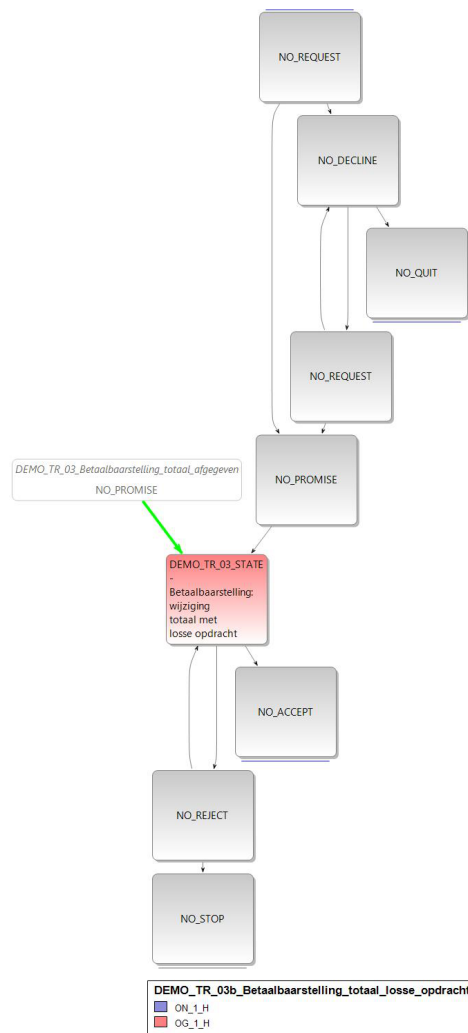


Figure 61:

- Every transaction starts with a ‘from’ message.
- All messages can contain a next message pointer. VISI has a clear overview of which party in a transaction is responsible at a certain step in the transaction. To keep facilitating this, all next messages of one message are to-messages or from-messages, not a mixed variant. So when a message is sent, it is always clear who needs to send the following message. See the left illustration of fig. 76.
- Next pointers (and, therefore, previous pointers) may point to the same message.

Frameworks with mixed initiator to executor settings, as next options from a message (the above "mixed variant"), need to be rejected by the VISI software.

As stated in the second item above, to retain the clear responsibility order, it is still only possible for a single side to send a message. While not revoking a message (see 6.14.2), the other side cannot send a message at the same time.

In handling the message, this has some consequences. The receiving party can not handle MITT types that have the same initiator to executor setting as the next-messages of that message. The message can only be shown to the receiver, but no reply is possible. Initiating a subtransaction (a next-message within another transaction type) can still be a possibility. When sending a message where the following message has the same initiator to executor setting, the message that has been handled should keep the responsibility for handling the transaction with the sender, because it needs to be handled again (see

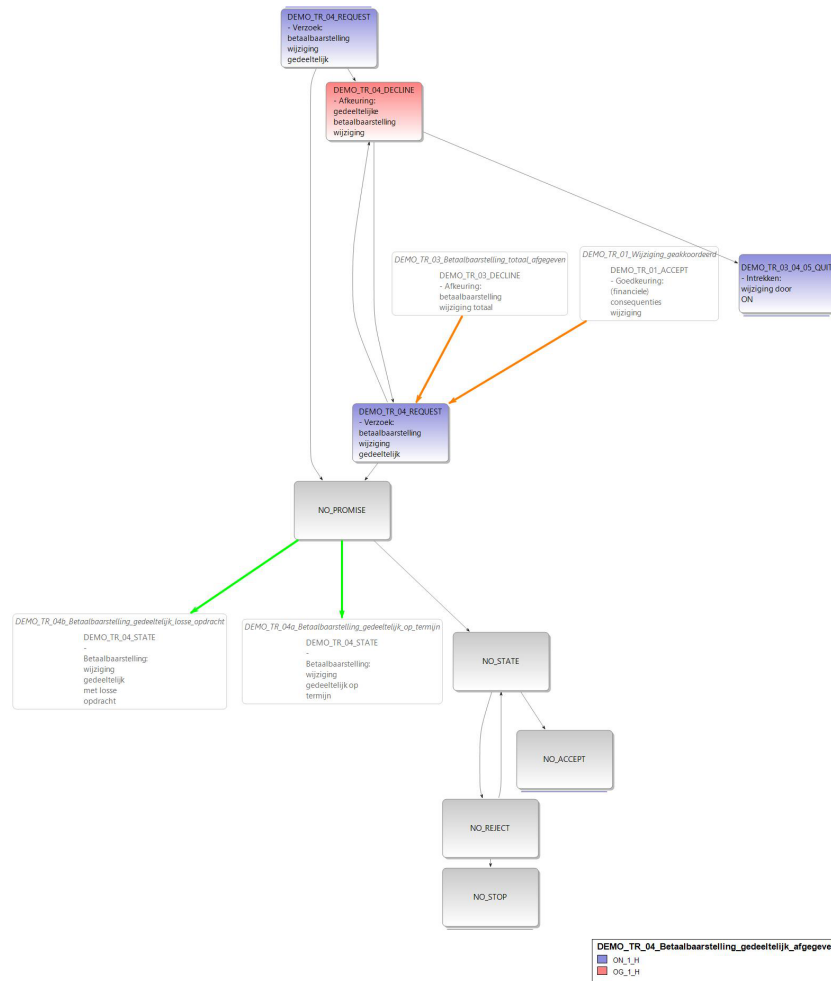


Figure 63:

follow-up message must have the transaction Phase ‘PhRevokeAllow’. A random follow-up message must have the transaction Phase ‘PhRevokeRefuse’. The three transaction phases, ‘PhRevokeRequest’, ‘PhRevokeAllow’, and ‘PhRevokeRefuse’, have to have hard-coded names that are present in every framework that needs to support Revoke functionality. If these phases are not present, the revoke option is not available.

The three minimum message types can be modelled as needed for that framework. For instance, it seems a minimum requirement to force the revoking party to write the reason for revoking .

Every Mitt(MessageInTransactionType) can have these extra attributes.

- <revokeAllowed> (default: true)
This is a flag to indicate whether the revoke option has to be enabled on this Mitt.
- <revokePattern> (default: reference to default revoke transaction)
If this option is not given, the revoke with ID TrRevoke is chosen, when present. When given, this alternative revoke transaction is offered instead of the default one.

Subtransactions that have been started from a message that is revoked can be revoked by the user themselves. There are no rules about Revoking or not-revoking Subtransactions of revoked messages, Subtransactions that started from a revoked message are running independently and cannot be used to answer the revoked message or the newly replaced message anymore.

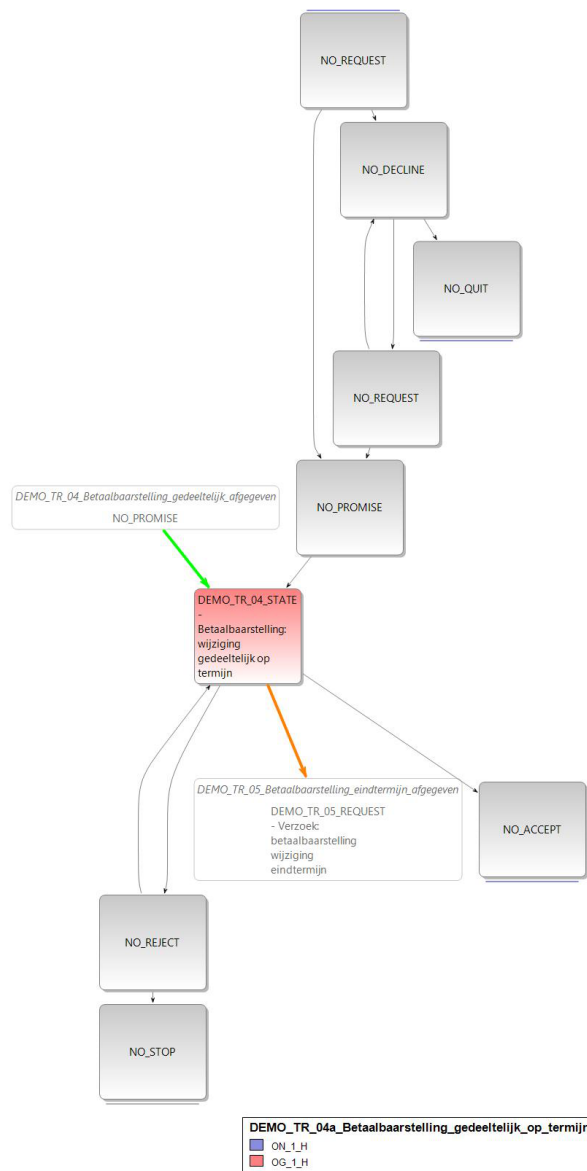


Figure 64:

When a first message is revoked, an empty transaction is available where no message is sent. The software must allow for a new first message or to do nothing, leaving an empty transaction with no further status.

A framework should be rejected by the VISI software in the following cases:

- Incomplete Revoke transactions. When a transaction has one of the three revoke transaction phases, the other two also must be in that transaction.
- A missing revoke pattern. When a revoke pattern is named in a mitt under revokePattern, that transaction must exist. (Such a missing reference will be found by validating the framework to _3.xsd)
- When the default revoke transaction TrRevoke does not have the Role RoRevoke as initiator and executor.
- When the default TrRevoke does not have the three revoke transaction phases

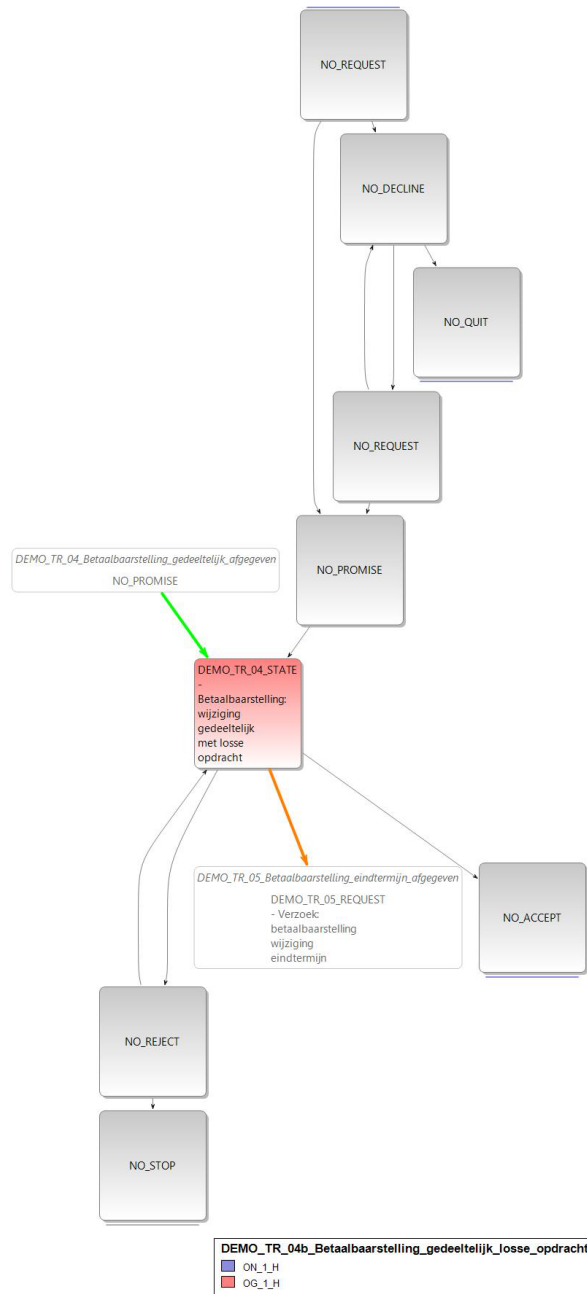


Figure 65:

Frameworks do not have to be rejected when:

- there are no Revoke transactions. Then, revoking is not possible within that VISA project
- only specific revoke patterns are in place, and TrRevoke is not present, then only the specified Mitts have the specified revoke patterns, and other Mitt's cannot be revoked

It is not allowed to create a PIR with the roletype RoRevoke, because every user can execute a revoke when available. As every VISA message instance needs to reference two Person in role instances, the revoke messages will be referenced to the PIR-instances of the initiator and executor of the revoked message. This is different to all other VISA messages, where the PIR is always related to the role types of the sent message itself.//

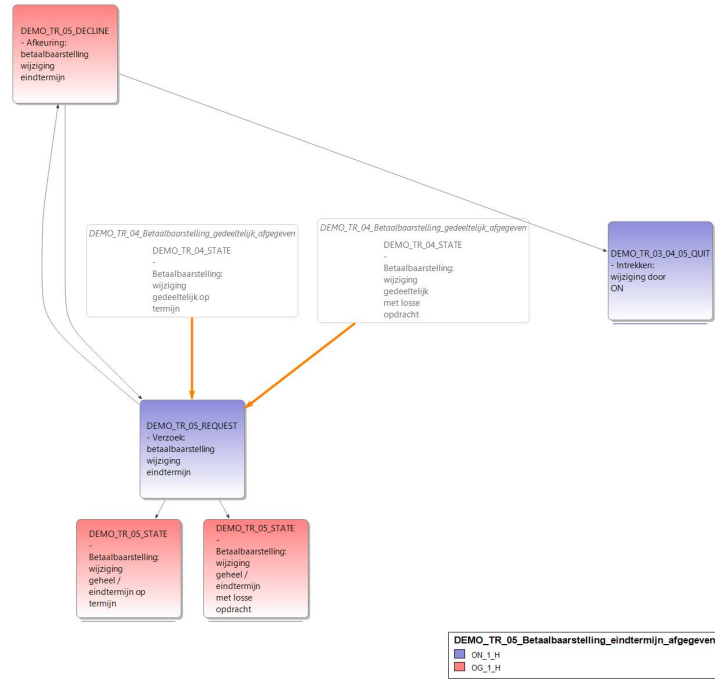


Figure 66:

As soon as a revoke transaction is initiated on a message, further replying in the transaction is forbidden. When the revoke and the next step arrive simultaneously on the different servers, the server that sent the revoke will not accept the sent message, so this message will only be available on the sending server, where this message has to be rendered inoperative, because the revoke request has to be handled. Only after closing the revoke transaction, the transaction itself can be resumed.

- When the revoke transaction is allowed, the transaction will be resumed from the message that was sent before the revoked message.
- When the revoke is refused, the transaction can be resumed as if there was no revoke, so with a reply to the revoke-requested message.

6.15 Data Model

This chapter describes the VISI data model (metamodel). The metamodel describes the concepts and relationships that make up VISI itself, i.e. concepts such as transaction, role, message, intention, etc.

We do not describe the concepts from concrete frameworks here because they are built with the help of VISI for any desired situation. Reference frameworks are described in the appendix. The metadata model is given as an entity model with a description.

fig. 83 shows the VISI data model. VISI has transaction types and roles. Each message (type) has exactly 1 intention. For each transaction type, all intentions associated with actions appear as shown in Table 3.

There is no entity role type, even though that would be obvious. The reason is that there are two role types, initiator and executor, and each transaction type always has an initiator and an executor. Therefore, the role types are modeled as relationships between role and transaction type.

There is no distinction between entity intent type resp. intent, because the concept is the same for message type as for message.

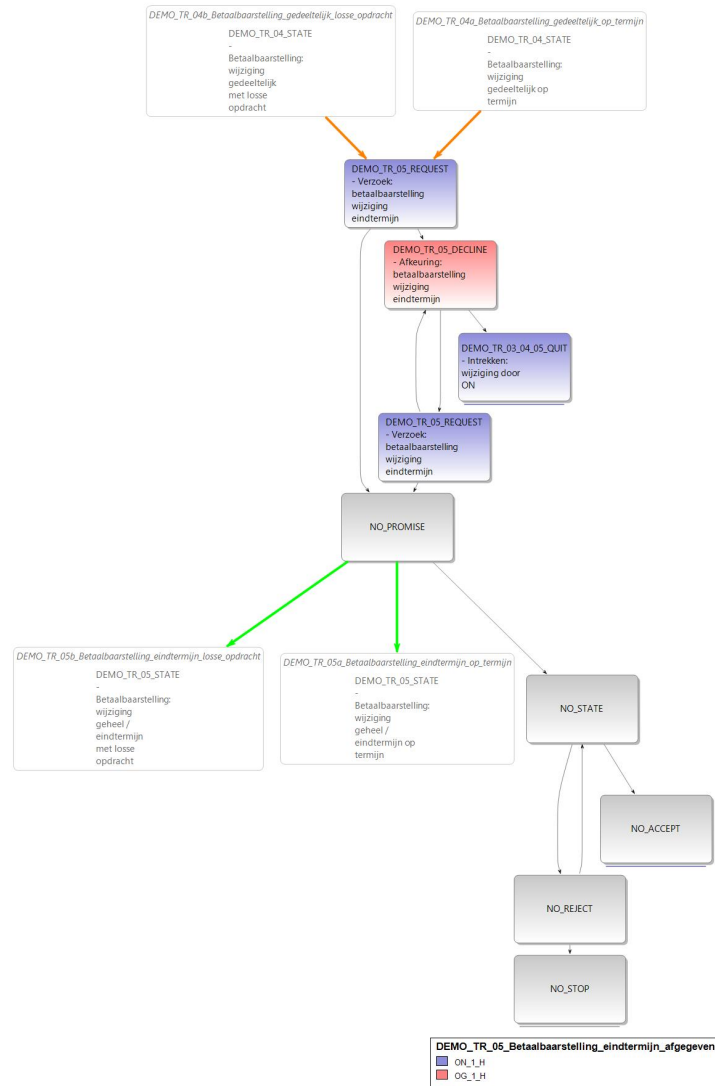


Figure 67:

A transaction (type) can have dependent transaction (type)s, this is shown with the relationship is (possibly) parent of.

When a framework is going to be applied, the people and the role(s) they will fulfill are defined. A person always belongs to an organisation. The transactions that will be executed are instances of one of the transaction types. For each instance of a transaction, a person who can have the role of initiator for the transaction type is appointed as the initiator and a person who can have the role of executor for the transaction type is appointed as the executor. The person with the role of initiator is the sender of messages associated with initiator actions from the complete transaction pattern and is the recipient of messages sent by the person with the executor role in this transaction. Every time the initiator or executor of a transaction sends a message, an instance of the message with that intent is created and sent.

6.15.1 Attributes

6.15.1.1 id

Unique code that provides a reference to this object instance within a given message. This reference can

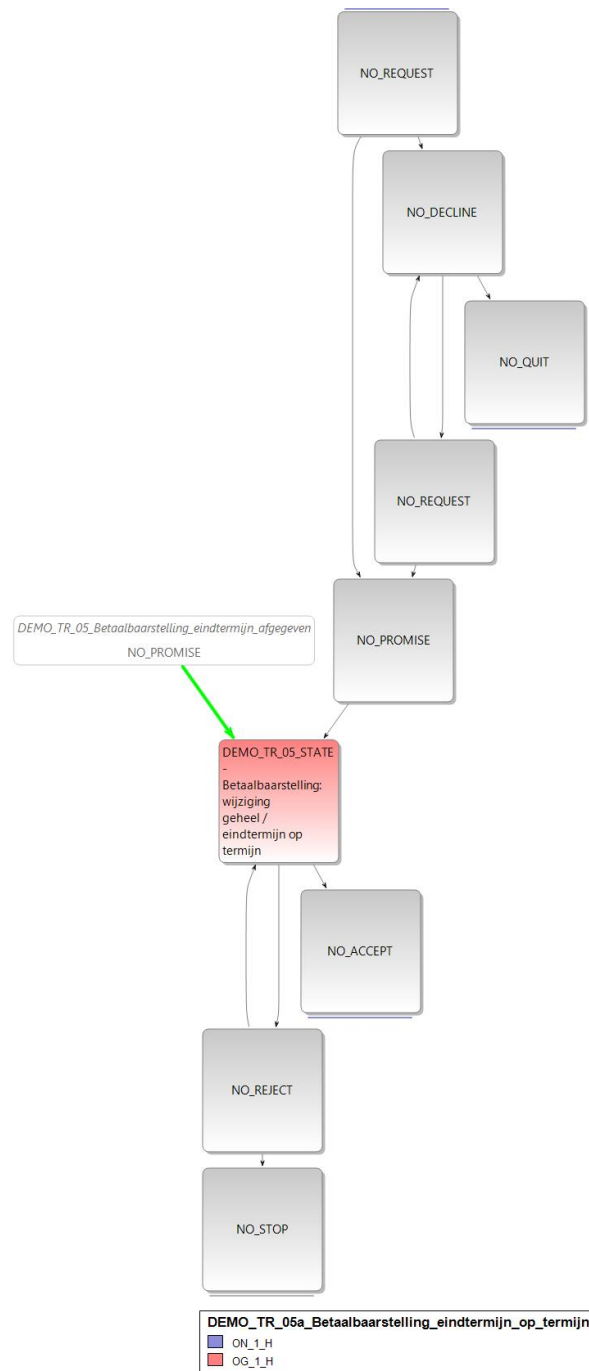


Figure 68:

be used again later.

Attribute 'id' is a specific definition in XML and is therefore subject to more restrictions than 'normal' attribute values. For example, an ID value cannot contain spaces and often no special characters. Also, an ID value cannot start with a number. See also [xml:id Version 1.0](<http://www.w3.org/TR/xml-id/>).

Example at message level: Piece of XML within a message (defines the organisation TNO of the OrganisationType Research Institution):

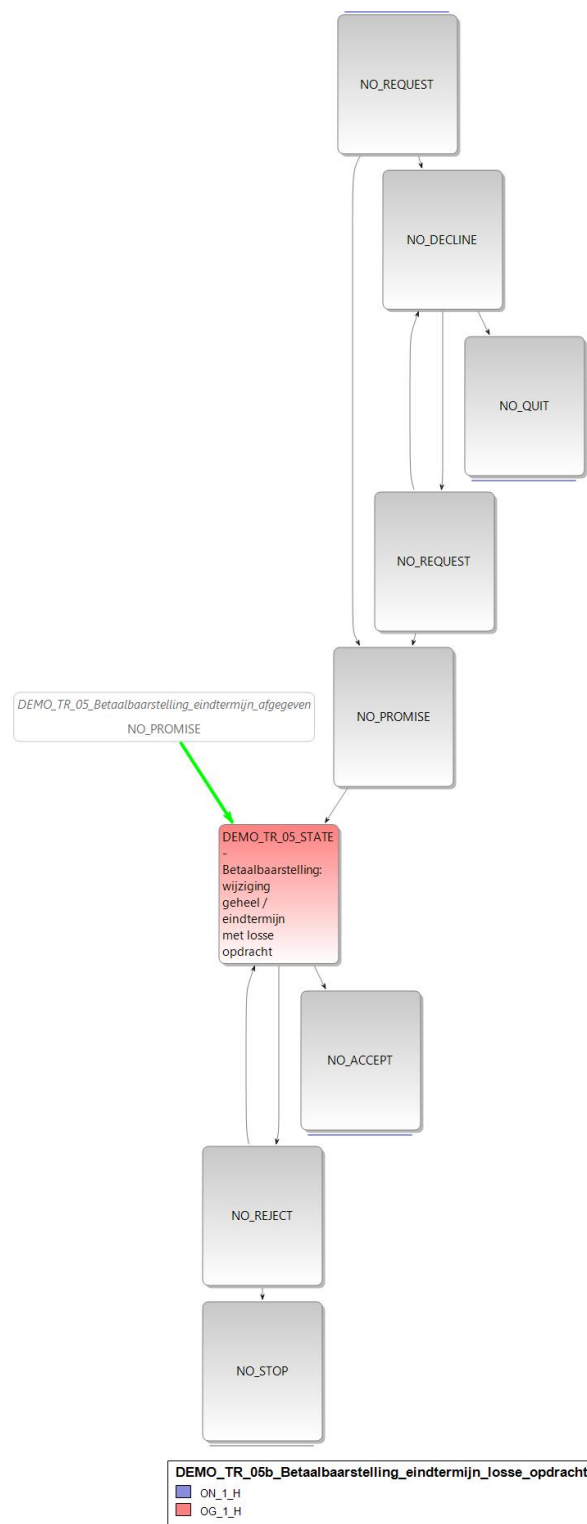


Figure 69:

```

1 <Onderzoeksinstelling id="TNO">
2   ...
3 </Onderzoeksinstelling>

```

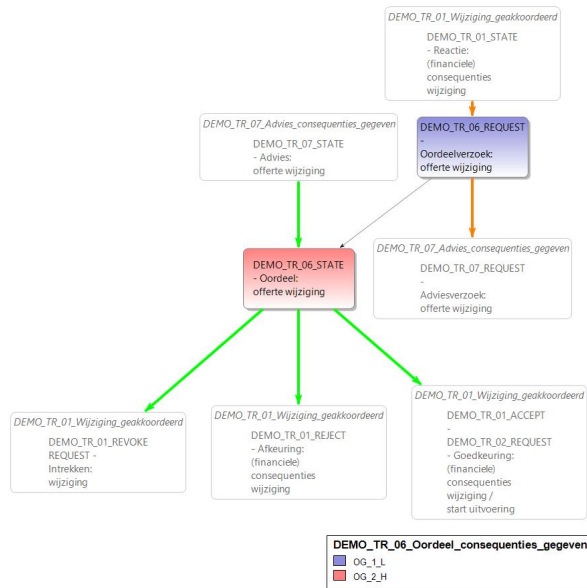


Figure 70:

The following piece of XML within this same message can now refer to TNO:

```
1 <OnderzoeksinstellingRef idref="TNO"/>
```

6.16 Systematics Part 2 Messages

6.16.1 Templates

6.16.1.1 AppendixGroup

Attributes: id [6.15.1.1]

Elements: state [6.16.2.25], dateLaMu [6.16.2.4], userLaMu [6.16.2.26]

References: group [6.16.3.4]

```
1 ENTITY AppendixGroup;
2   state : STRING;
3   dateLaMu : OPTIONAL DATETIME;
4   userLaMu : OPTIONAL STRING;
5   group : OPTIONAL GroupTemplate;
6 END_ENTITY;
```

The link table for the n:m relationship between attachments and groups.

Simple example at message level:

```
1 <AppendixGroup id="BijlageGroep_1">
2   <state>active</state>
3   <dateLaMu>2011-02-04T00:00:00Z</dateLaMu>
4   <userLaMu>bapa</userLaMu>
5   <group>
6     <StandardGroupType id="...">
7       ...
8     </StandardGroupType>
9   </group>
10 </AppendixGroup>
```

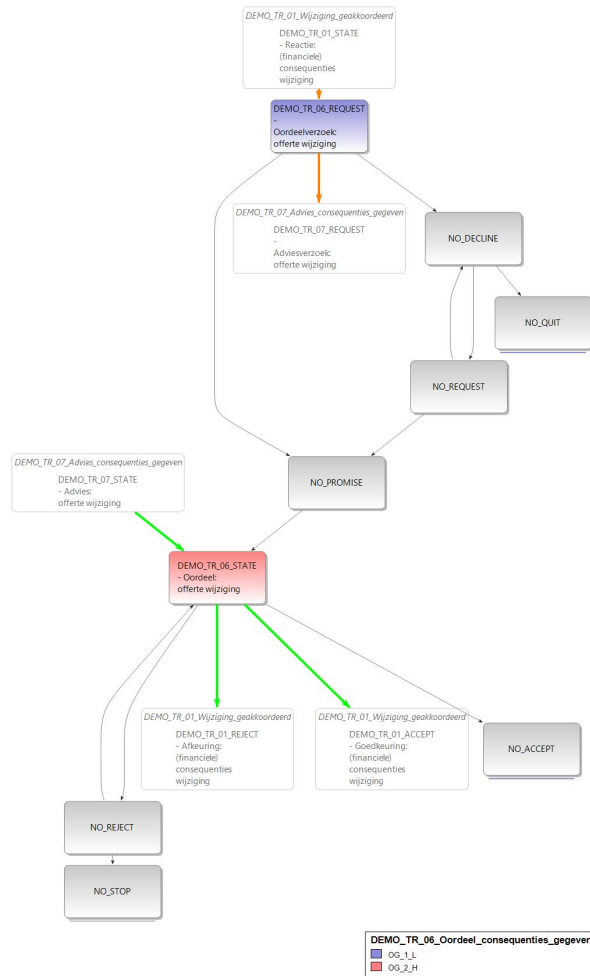


Figure 71:

Associated part of the framework:

```

1 <GroupType id="StandardGroupType">
2   <description>Standaard groep</description>
3   <startDate>2010-12-20T00:00:00Z</startDate>
4   <endDate>2011-12-31T00:00:00Z</endDate>
5   <state>active</state>
6   <dateLaMu>2010-12-20T00:00:00Z</dateLaMu>
7   <userLaMu>bapa</userLaMu>
8 </GroupType>

```

6.16.1.2 AppendixTemplate

Attributes: id [6.15.1.1]

Elements: name [6.16.2.20], fileLocation [6.16.2.13], fileType [6.16.2.14], fileVersion [6.16.2.15], documentIdentification [6.16.2.9], documentVersion [6.16.2.11], documentReference [6.16.2.10], objectCode [6.16.2.22], startDate [6.16.2.24], endDate [6.16.2.12], state [6.16.2.25], dateLaMu [6.16.2.4], userLaMu

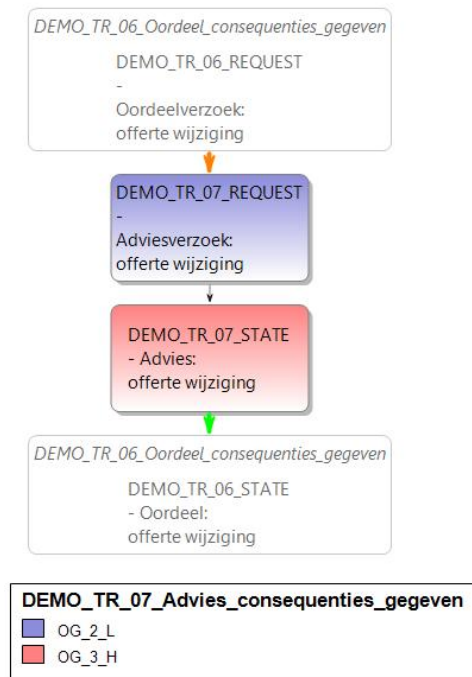


Figure 72:

[6.16.2.26], language [6.16.2.19]

References: message [6.16.1.3], appendixGroup [6.16.3.1], template [6.16.1.3]

```

1 ENTITY AppendixTemplate;
2     name : STRING;
3     fileLocation : STRING;
4     fileType : STRING;
5     fileVersion : STRING;
6     documentIdentification : OPTIONAL STRING;
7     documentVersion : OPTIONAL STRING;
8     documentReference : OPTIONAL STRING;
9     objectCode : OPTIONAL STRING;
10    startDate : OPTIONAL DATETIME;
11    endDate : OPTIONAL DATETIME;
12    state : OPTIONAL STRING;
13    dateLaMu : OPTIONAL DATETIME;
14    userLaMu : OPTIONAL STRING;
15    language : OPTIONAL STRING;
16    message : MessageTemplate;
17    appendixGroup : OPTIONAL AppendixGroup;
18    template : ComplexElementTemplate;
19 END_ENTITY;

```

The linked files are registered here.

Simple example at message level:

```

1 <Bijlage id="VoorbeeldDocument">
2     <name>Voorbeeld</name>
3     <fileLocation>\\srv-bouw\Public\project\docs\msword\</fileLocation>
4     <fileType>application/msword</fileType>
5     <fileVersion>2010</fileVersion>
6     <documentIdentification>345899</documentIdentification>
7     <documentVersion>1</documentVersion>
8     <documentReference>FG783990</documentReference>
9     <startDate>2011-02-04T00:00:00Z</startDate>

```

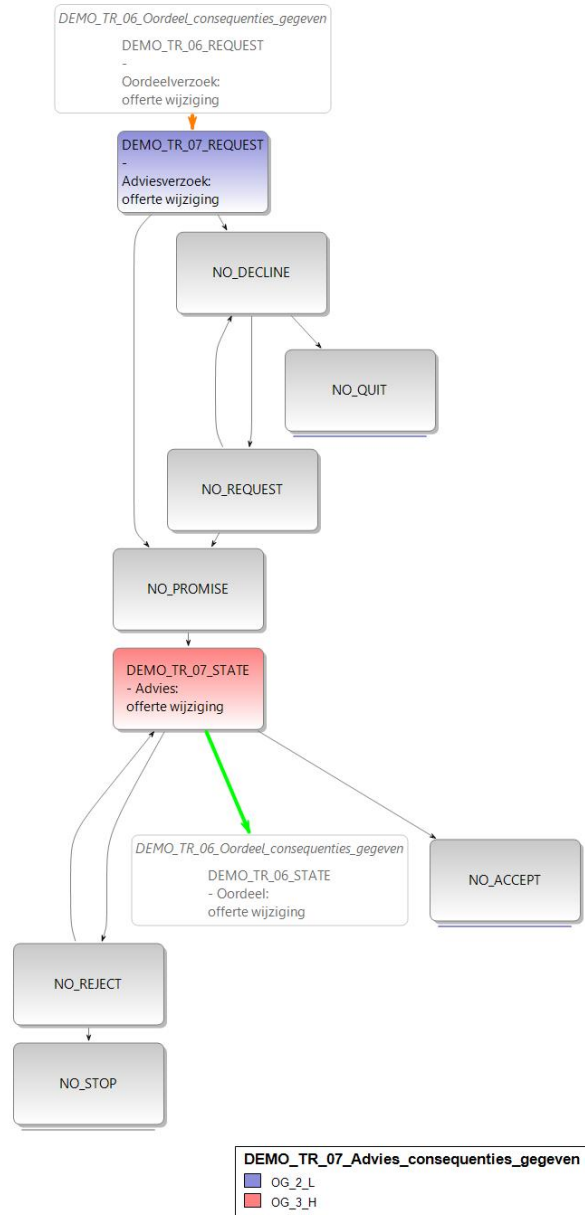


Figure 73:

```

10 <endDate>2011-12-31T00:00:00Z</endDate>
11 <state>active</state>
12 <dateLaMu>2011-02-04T00:00:00Z</dateLaMu>
13 <userLaMu>bapa</userLaMu>
14 <language>NL</language>
15 <appendixGroup>
16   <AppendixGroup id="...">
17     ...
18   </AppendixGroup>
19 </appendixGroup>
20 </Bijlage>

```

Associated part of the framework:

```

1 <AppendixType id="Bijlage">

```

```

2      <description>
3          Standaard bijlage definitie (geen zelf gedefinieerde velden)
4      </description>
5      <startDate>2011-02-04T00:00:00Z</startDate>

```

138

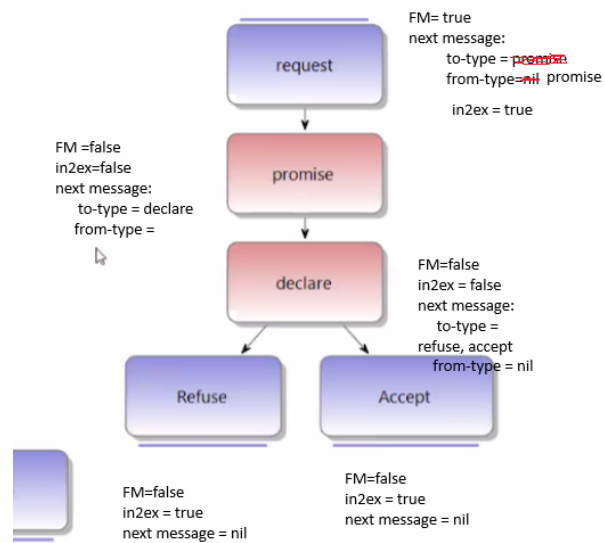


Figure 75: WIP. Next message example

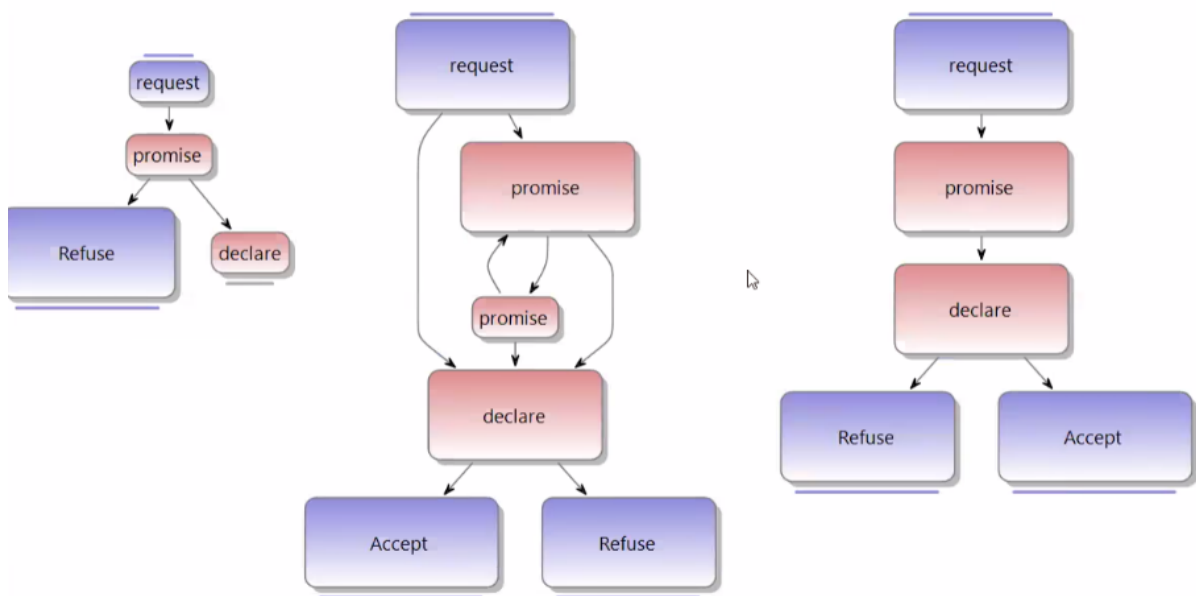


Figure 76: Possible patterns in VISI 1.8

```

6    <endDate>2011-12-31T00:00:00Z</endDate>
7    <state>active</state>
8    <dateLaMu>2011-02-04T00:00:00Z</dateLaMu>
9    <userLaMu>bapa</userLaMu>
10   <language>NL</language>
11  </AppendixType>

```

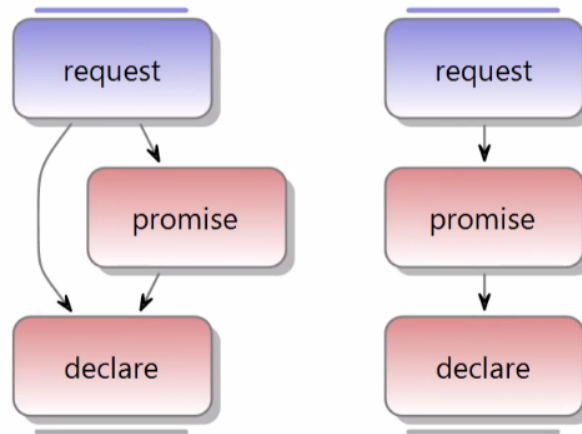


Figure 77: Optional and required promise

6.16.1.3 ComplexElementTemplate

Attributes: id [6.15.1.1]

```

1 ENTITY ComplexElementTemplate;
2     template : SimpleElementVirtual;
3 END_ENTITY;

```

6.16.1.4 GroupTemplate

Attributes: id [6.15.1.1]

Elements: name [6.16.2.20], description [6.16.2.8], creationDate [6.16.2.3], startDate [6.16.2.24], endDate [6.16.2.12], state [6.16.2.25], dateLaMu [6.16.2.4], userLaMu [6.16.2.26], versionNo [6.16.2.28]

References: transaction [6.16.3.12]

```

1 ENTITY GroupTemplate;
2     name : STRING;
3     description : STRING;
4     creationDate : DATETIME;
5     startDate : DATETIME;
6     endDate : DATETIME;
7     state : OPTIONAL STRING;
8     dateLaMu : OPTIONAL DATETIME;
9     userLaMu : OPTIONAL STRING;
10    versionNo : STRING;
11    transaction : TransactionTemplate;
12 END_ENTITY;

```

The group within which attachments of a message are placed for retrieving the documents.

Simple example at message level:

```

1 <StandardGroupType id="MenukaartAchtergronden">
2     <name>Menukaart Plaatjes </name>
3     <description>
4         Een aantal achtergronden ter verfraaiing van de menukaart
5     </description>
6     <creationDate>2011-02-04T00:00:00Z</creationDate>
7     <startDate>2011-02-04T00:00:00Z</startDate>
8     <endDate>2011-12-31T00:00:00Z</endDate>

```

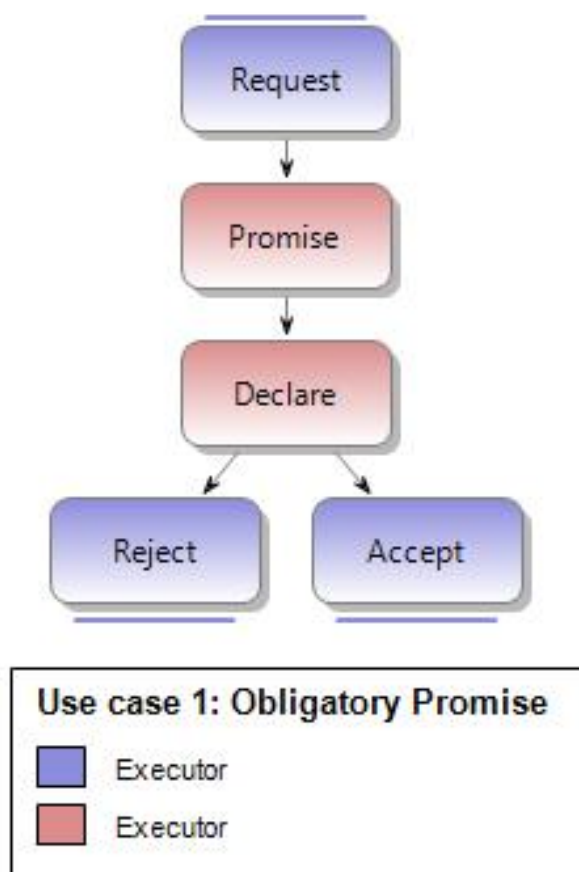



Figure 78: Caption

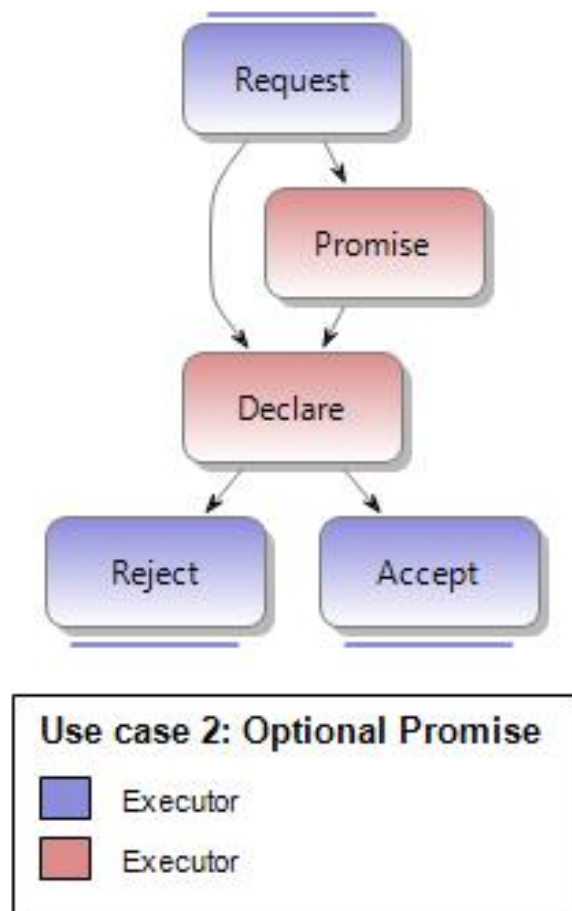


Figure 79: Caption

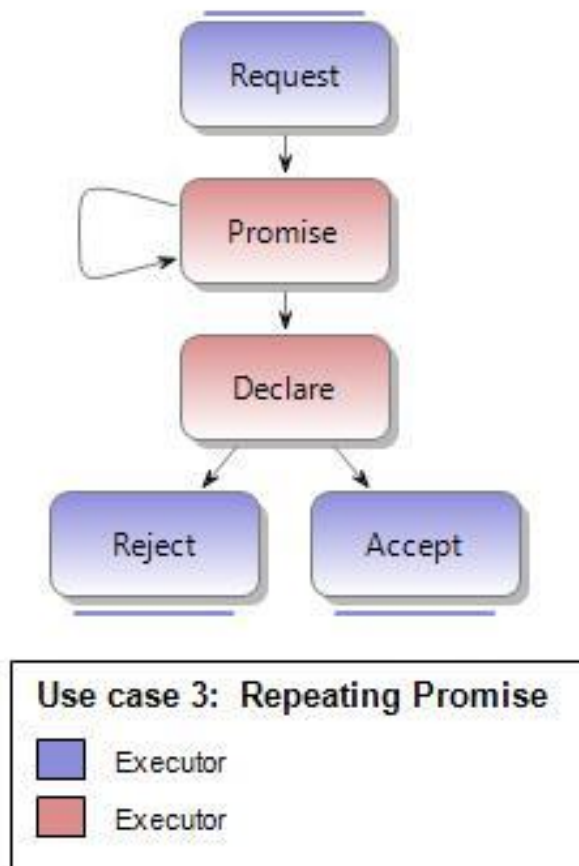


Figure 80: Caption

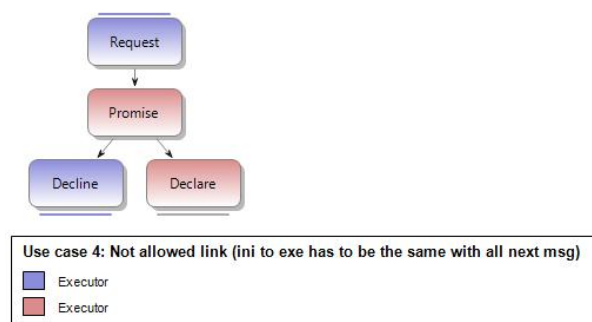


Figure 81: Caption

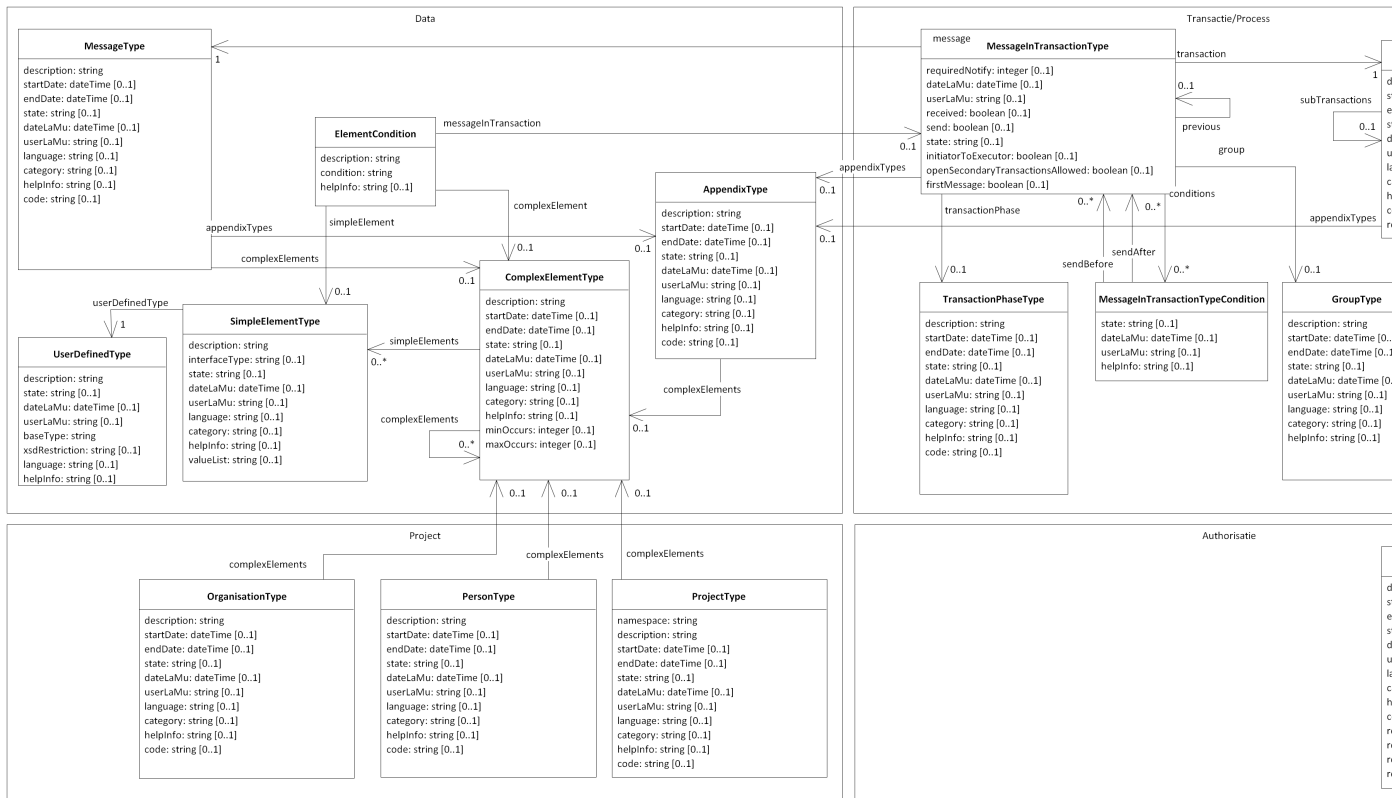


Figure 83: VISI Data model

6.16.1.5 MessageInTransactionTemplate

Attributes: id [6.15.1.1]

Elements: identification [6.16.2.16], dateSend [6.16.2.7], dateRead [6.16.2.6], state [6.16.2.25], dateLaMu [6.16.2.4], userLaMu [6.16.2.26], initiatorToExecutor [6.16.2.18]

```
1 ENTITY MessageInTransactionTemplate;  
2     identification : STRING;  
3     dateSend : OPTIONAL DATETIME;  
4     dateRead : OPTIONAL DATETIME;  
5     state : OPTIONAL STRING;  
6     dateLaMu : OPTIONAL DATETIME;  
7     userLaMu : OPTIONAL STRING;  
8 END_ENTITY;
```

This is the entity that makes it possible to include the actual MessageInTransactionType in the message. This means that the position in the workflow of a transaction can always be clearly identified.

6.16.1.6 MessageTemplate

Attributes: id [6.15.1.1]

Elements: identification [6.16.2.16], dateSend [[6.16.2.7], dateRead [[6.16.2.6], state [[6.16.2.25], dateLaMu [[6.16.2.4], userLaMu [[6.16.2.26], initiatingTransactionMessageID [[6.16.2.17], initiatorToExecutor [[6.16.2.18]

References: messageInTransaction [[6.16.3.7], transaction [[6.16.3.12], template [[6.16.1.3]

```
1 ENTITY MessageTemplate;  
2     identification : STRING;  
3     dateSend : DATETIME;  
4     dateRead : OPTIONAL DATETIME;  
5     state : OPTIONAL STRING;  
6     dateLaMu : OPTIONAL DATETIME;  
7     userLaMu : OPTIONAL STRING;  
8     initiatingTransactionMessageID : OPTIONAL STRING;  
9     initiatorToExecutor : BOOLEAN;  
10    messageInTransaction : MessageInTransactionTemplate;  
11    transaction : TransactionTemplate;  
12    template : ComplexElementTemplate;  
13 END_ENTITY;
```

An instance of the MessageType. This is the entity that holds the actual information exchange between OrganisationTemplate's [1.7] (organisations).

Simple example at message level:

```
1 <VerstrekkenVanMenukaartBericht id="a002">  
2     <identification>id a002</identification>  
3     <dateSend>2011-01-23T00:00:00Z</dateSend>  
4     <dateRead>2011-01-23T00:00:00Z</dateRead>  
5     <state>active</state>  
6     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>  
7     <userLaMu>bapa</userLaMu>  
8     <initiatingTransactionMessageID>  
9         a009  
10    </initiatingTransactionMessageID>  
11    <initiatorToExecutor>false</initiatorToExecutor>  
12    <messageInTransaction>
```

```

13     <BerichtInTransactie12Ref idref="BiT001"/>
14 </messageInTransaction>
15 <transaction>
16     <MenukaartVerkrijgenTransactie id="...">
17         ...
18     </MenukaartVerkrijgenTransactie>
19 </transaction>
20 <menukaart>
21     <Menukaart id="...">
22         ...
23     </Menukaart>
24     ...
25     <Menukaart id="...">
26         ...
27     </Menukaart>
28 </menukaart>
29 </VerstrekkenVanMenukaartBericht>

```

Associated part of the framework:

```

1 <TransactionType id="MenukaartVerkrijgenTransactie">
2     <description>De transactie om te komen tot het verkrijgen van de juiste menukaart</
    description>
3     <startDate>2011-01-23T00:00:00Z</startDate>
4     <endDate>2011-12-31T00:00:00Z</endDate>
5     <state>active</state>
6     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
7     <userLaMu>bapa</userLaMu>
8     <initiator>
9         <RoleTypeRef idref="Consument"/>
10    </initiator>
11    <executor>
12        <RoleTypeRef idref="Werknemer"/>
13    </executor>
14 </TransactionType>
15 <MessageType id="VerstrekkenVanMenukaartBericht">
16     <description>Bericht welke de menukaart bevat.</description>
17     <startDate>2011-01-23T00:00:00Z</startDate>
18     <endDate>2011-12-31T00:00:00Z</endDate>
19     <state>active</state>
20     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
21     <userLaMu>bapa</userLaMu>
22     <complexElements>
23         <ComplexElementTypeRef idref="Menukaart"/>
24     </complexElements>
25 </MessageType>
26 <ComplexElementType id="Menukaart">
27     <description>Kaart met aanwezige menu's</description>
28     <startDate>2011-01-23T00:00:00Z</startDate>
29     <endDate>2011-12-31T00:00:00Z</endDate>
30     <state>active</state>
31     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
32     <userLaMu>bapa</userLaMu>
33     <elements>
34         <SimpleElementTypeRef idref="MenukaartItems"/>
35     </elements>
36 </ComplexElementType>

```

6.16.1.7 OrganisationTemplate

Attributes: id [6.15.1.1]

Elements: name [6.16.2.20], abbreviation [6.16.2.1], state [6.16.2.25], dateLaMu [6.16.2.4], userLaMu [6.16.2.26]

References: contactPerson [6.16.3.2], template [6.16.1.3]

```

1 ENTITY OrganisationTemplate;
2     name : STRING;
3     abbreviation : STRING;
4     state : OPTIONAL STRING;
5     dateLaMu : OPTIONAL DATETIME;
6     userLaMu : OPTIONAL STRING;
7     contactPerson : PersonTemplate;
8     template : ComplexElementTemplate;
9 END_ENTITY;

```

The organisation that participates in the project by initiating or executing a TransactionTemplate [1.13] (transaction).

Simple example at message level:

```

1 <StandardOrganisationType id="TNO">
2     <name>
3         Nederlandse organisatie voor Toegepast Natuurwetenschappelijk Onderzoek
4     </name>
5     <abbreviation>TNO</abbreviation>
6     <state>active</state>
7     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
8     <userLaMu>bapa</userLaMu>
9     <contactPerson>
10         <StandardPersonType id="...">
11             ...
12         </StandardPersonType>
13     </contactPerson>
14 </StandardOrganisationType>

```

Associated part of the framework:

```

1 <PersonType id="StandardPersonType">
2     <description>Standaard persoons type</description>
3     <startDate>2011-01-23T00:00:00Z</startDate>
4     <endDate>2011-12-31T00:00:00Z</endDate>
5     <state>active</state>
6     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
7     <userLaMu>bapa</userLaMu>
8 </PersonType>
9 <OrganisationType id="StandardOrganisationType">
10     <description>Standaard organisation type</description>
11     <startDate>2011-01-23T00:00:00Z</startDate>
12     <endDate>2011-12-31T00:00:00Z</endDate>
13     <state>active</state>
14     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
15     <userLaMu>bapa</userLaMu>
16 </OrganisationType>

```

6.16.1.8 PersonInRole

Attributes: id [6.15.1.1]

Elements: state [6.16.2.25], dateLaMu [6.16.2.4], userLaMu [6.16.2.26]

References: successor [6.16.3.11], substituting [6.16.3.10], contactPerson [6.16.3.2], organisation [6.16.3.8], role [6.16.3.9]

```

1 ENTITY PersonInRole;
2     state : OPTIONAL STRING;
3     dateLaMu : OPTIONAL DATETIME;
4     userLaMu : OPTIONAL STRING;
5     successor : OPTIONAL PersonInRole;

```



```

6     substituting : OPTIONAL PersonInRole;
7     contactPerson : PersonTemplate;
8     organisation : OrganisationTemplate;
9     role : RoleTemplate;
10  END_ENTITY;

```

A person who fulfils a specific role for an organisation.

Simple example at message level:

```

1 <PersonInRole id="pir_A">
2   <state>active</state>
3   <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
4   <userLaMu>bapa</userLaMu>
5   <contactPerson>
6     <StandardPersonType id="...">
7       ...
8     </StandardPersonType>
9   </contactPerson>
10  <organisation>
11    <StandardOrganisationType id="...">
12      ...
13    </StandardOrganisationType>
14  </organisation>
15  <role>
16    <Consumer idref="...">
17      ...
18    </Consumer>
19  </role>
20 </PersonInRole>

```

Associated part of the framework:

```

1 <PersonType id="StandardPersonType">
2   <description>Standard person type</description>
3   <startDate>2011-01-23T00:00:00Z</startDate>
4   <endDate>2011-12-31T00:00:00Z</endDate>
5   <state>active</state>
6   <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
7   <userLaMu>bapa</userLaMu>
8 </PersonType>
9 <OrganisationType id="StandardOrganisationType">
10  <description>Standard organisation type</description>
11  <startDate>2011-01-23T00:00:00Z</startDate>
12  <endDate>2011-12-31T00:00:00Z</endDate>
13  <state>active</state>
14  <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
15  <userLaMu>bapa</userLaMu>
16 </OrganisationType>
17 <RoleType id="Consumer">
18  <description>Consuming person</description>
19  <startDate>2011-01-23T00:00:00Z</startDate>
20  <endDate>2011-12-31T00:00:00Z</endDate>
21  <state>active</state>
22  <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
23  <userLaMu>bapa</userLaMu>
24 </RoleType>

```

A person who fulfils a specific role for an organisation.

Example for a successor person X is the successor of person Y :

```

1 <PersonInRole id=pir_A">
2   <state>active</state>

```

```

3    <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
4    <userLaMu>bapa</userLaMu>
5    <successor>
6        <StandardPersonType id="pir_B">
7            ...
8        </StandardPersonType>
9    </successor>
10   <contactPerson>
11       <StandardPersonType id="pe_Y">
12           ...
13       </StandardPersonType>
14   </contactPerson>
15   <organisation>
16       <StandardOrganisationType id="or_A">
17           ...
18       </StandardOrganisationType>
19   </organisation>
20   <role>
21       <Consumer idref="ro_P">
22           ...
23       </Consumer>
24   </role>
25 </PersonInRole>
26 <PersonInRole id="pir_B">
27     <state>active</state>
28     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
29     <userLaMu>bapa</userLaMu>
30     <contactPerson>
31         <StandardPersonType id="pe_X">
32             ...
33         </StandardPersonType>
34     </contactPerson>
35     <organisation>
36         <StandardOrganisationType id="or_A">
37             ...
38         </StandardOrganisationType>
39     </organisation>
40     <role>
41         <Consumer idref="ro_P">
42             ...
43         </Consumer>
44     </role>
45 </PersonInRole>

```

A person who fulfils a specific role for an organisation.

Example for person Y is the substitute of person X :

```

1 <PersonInRole id="pir_A">
2     <state>active</state>
3     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
4     <userLaMu>bapa</userLaMu>
5     <substituting>
6         <StandardPersonType id="pir_B">
7             ...
8         </StandardPersonType>
9     </substituting>
10    <contactPerson>
11        <StandardPersonType id="pe_Y">
12            ...
13        </StandardPersonType>
14    </contactPerson>
15    <organisation>
16        <StandardOrganisationType id="or_A">
17            ...
18        </StandardOrganisationType>
19    </organisation>
20    <role>

```

```

21     <Consumer idref="ro_P">
22         ...
23     </Consumer>
24 </role>
25 </PersonInRole>
26 <PersonInRole id=pir_B">
27     <state>active</state>
28     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
29     <userLaMu>bapa</userLaMu>
30     <contactPerson>
31         <StandardPersonType id="pe_X">
32             ...
33         </StandardPersonType>
34     </contactPerson>
35     <organisation>
36         <StandardOrganisationType id="or_A">
37             ...
38         </StandardOrganisationType>
39     </organisation>
40 </role>
41     <Consumer idref="ro_P">
42         ...
43     </Consumer>
44 </role>
45 </PersonInRole>

```

Substitute and Successor can not end in a loop.

6.16.1.9 PersonTemplate

Attributes: id [6.15.1.1]

Elements: userName [6.16.2.27], name [6.16.2.20], state [6.16.2.25], dateLaMu [6.16.2.4], userLaMu [6.16.2.26]

References: template [6.16.1.3]

```

1 ENTITY PersonTemplate;
2     userName : STRING;
3     name : STRING;
4     state : OPTIONAL STRING;
5     dateLaMu : OPTIONAL DATETIME;
6     userLaMu : OPTIONAL STRING;
7     template : ComplexElementTemplate;
8 END_ENTITY;

```

The details of a person who participates in the project by fulfilling a certain role or being a contact person for a certain organisation.

Simple example at message level:

```

1 <StandardPersonType id="PBonsma">
2     <userName>bapa</userName>
3     <name>Peter Bonsma</name>
4     <state>active</state>
5     <dateLaMu>2011-02-04T00:00:00Z</dateLaMu>
6     <userLaMu>bapa</userLaMu>
7 </StandardPersonType>

```

Associated part of the framework:

```

1 <PersonType id="StandardPersonType">
2     <description>Standaard persoons type</description>
3     <startDate>2011-01-23T00:00:00Z</startDate>

```

```

4     <endDate>2011-12-31T00:00:00Z</endDate>
5     <state>active</state>
6     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
7     <userLaMu>bapa</userLaMu>
8 </PersonType>

```

6.16.1.10 ProjectTypeInstance

Attributes: id [6.15.1.1]

Elements: name [6.16.2.20], description [6.16.2.8], startDate [6.16.2.24], endDate [6.16.2.12], state [6.16.2.25], dateLaMu [6.16.2.4], userLaMu [6.16.2.26]

References: template [6.16.1.3]

```

1 ENTITY ProjectTypeInstance;
2     name : STRING;
3     description : STRING;
4     startDate : DATETIME;
5     endDate : DATETIME;
6     state : OPTIONAL STRING;
7     dateLaMu : OPTIONAL DATETIME;
8     userLaMu : OPTIONAL STRING;
9     template : ComplexElementTemplate;
10 END_ENTITY;

```

The project for which the communication has been created. The framework partly determines (through the XML field) what we can and must enter here.

Simple example at message level:

```

1 <StandardProjectType id="VISI">
2     <name>Het project VISI</name>
3     <description>Formalisering van VISI Systematiek</description>
4     <startDate>2011-02-04T00:00:00Z</startDate>
5     <endDate>2011-12-31T00:00:00Z</endDate>
6     <state>active</state>
7     <dateLaMu>2011-02-04T00:00:00Z</dateLaMu>
8     <userLaMu>bapa</userLaMu>
9 </StandardProjectType>

```

Associated part of the framework:

```

1 <ProjectType id="StandardProjectType">
2     <description>Standaard project type</description>
3     <startDate>2011-02-04T00:00:00Z</startDate>
4     <endDate>2011-12-31T00:00:00Z</endDate>
5     <state>active</state>
6     <dateLaMu>2011-02-04T00:00:00Z</dateLaMu>
7     <userLaMu>bapa</userLaMu>
8 </ProjectType>

```

6.16.1.11 RoleTemplate

Attributes: id [6.15.1.1]

Elements: name [6.16.2.20], description [6.16.2.8], state [6.16.2.25], dateLaMu [6.16.2.4], userLaMu [6.16.2.26], category [6.16.2.2]

```

1 ENTITY RoleTemplate;
2     name : STRING;
3     description : STRING;
4     state : OPTIONAL STRING;
5     dateLaMu : OPTIONAL DATETIME;
6     userLaMu : OPTIONAL STRING;
7     category : OPTIONAL STRING;
8 END_ENTITY;

```

The role that can be fulfilled by an organisation on behalf of a PersonTemplate [1.9] (person).

Simple example at message level:

```

1 <Consument id="Klant">
2     <name>Rol als klant</name>
3     <description>De rol als klant</description>
4     <state>active</state>
5     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
6     <userLaMu>bapa</userLaMu>
7 </Consument>

```

Associated part of the framework:

```

1 <RoleType id="Consument">
2     <description>Consumerend persoon</description>
3     <startDate>2011-01-23T00:00:00Z</startDate>
4     <endDate>2011-12-31T00:00:00Z</endDate>
5     <state>active</state>
6     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
7     <userLaMu>bapa</userLaMu>
8 </RoleType>

```

6.16.1.12 TransactionPhaseTemplate

Attributes: id [6.15.1.1]

Elements: name [6.16.2.20], description [6.16.2.8], dateReached [6.16.2.5], state [6.16.2.25], dateLaMu [6.16.2.4], userLaMu [6.16.2.26]

References: transaction [6.16.3.12]

```

1 ENTITY TransactionPhaseTemplate;
2     name : STRING;
3     description : STRING;
4     dateReached : DATETIME;
5     state : OPTIONAL STRING;
6     dateLaMu : OPTIONAL DATETIME;
7     userLaMu : OPTIONAL STRING;
8     transaction : TransactionTemplate;
9 END_ENTITY;

```

The phase in which a transaction is or has been. Simple example at message level:

```

1 <WachtenOpMenukaart id="tp003">
2     <name>...</name>
3     <description>Transaction Phase ...</description>
4     <dateReached>2011-02-04T00:00:00Z</dateReached>
5     <state>active</state>
6     <dateLaMu>2011-02-04T00:00:00Z</dateLaMu>
7     <userLaMu>bapa</userLaMu>
8     <transaction>

```

```

9      <MenukaartVerkrijgenTransactieRef idref="ta004"/>
10    </transaction>
11 </WachtenOpMenukaart>

```

Associated part of the framework:

```

1 <TransactionType id="MenukaartVerkrijgenTransactie">
2   <description>De transactie om te komen tot het verkrijgen van de juiste menukaart</
   description>
3   <startDate>2011-01-23T00:00:00Z</startDate>
4   <endDate>2011-12-31T00:00:00Z</endDate>
5   <state>active</state>
6   <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
7   <userLaMu>bapa</userLaMu>
8   <initiator>
9     <RoleTypeRef idref="Consument"/>
10  </initiator>
11  <executor>
12    <RoleTypeRef idref="Werknemer"/>
13  </executor>
14 </TransactionType>
15 <TransactionPhaseType id="WachtenOpMenukaart">
16   <description>Menukaart gevraagd maar nog niet gegeven</description>
17   <startDate>2011-01-23T00:00:00Z</startDate>
18   <endDate>2011-12-31T00:00:00Z</endDate>
19   <state>active</state>
20   <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
21   <userLaMu>bapa</userLaMu>
22 </TransactionPhaseType>

```

6.16.1.13 TransactionTemplate

Attributes: id [6.15.1.1]

Elements: number [6.16.2.21], name [6.16.2.20], description [6.16.2.8], startDate [6.16.2.24], endDate [6.16.2.12], state [6.16.2.25], dateLaMu [6.16.2.4], userLaMu [6.16.2.26], result [6.16.2.23]

References: initiator [6.16.3.5], executor [6.16.3.3]

```

1 ENTITY TransactionTemplate;
2   number : INTEGER;
3   name : STRING;
4   description : STRING;
5   startDate : DATETIME;
6   endDate : DATETIME;
7   state : OPTIONAL STRING;
8   dateLaMu : OPTIONAL DATETIME;
9   userLaMu : OPTIONAL STRING;
10  result : OPTIONAL STRING;
11  initiator : PersonInRole;
12  executor : PersonInRole;
13  project : ProjectTypeInstance;
14 END_ENTITY;

```

The transaction within which MessageTemplates [1.6] (messages) can be sent in order to perform a task within a project. Simple example at message level:

```

1 <MenukaartVerkrijgenTransactie id="DeTransactie">
2   <number>001</number>
3   <name>...</name>
4   <description>...</description>
5   <startDate>2011-01-23T00:00:00Z</startDate>
6   <endDate>2011-01-23T00:00:00Z</endDate>
7   <state>active</state>

```

```

8      <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
9      <userLaMu>bapa</userLaMu>
10     <initiator>
11         <PersonInRole id="...">
12             ...
13         </PersonInRole>
14     </initiator>
15     <executor>
16         <PersonInRole id="...">
17             ...
18         </PersonInRole>
19     </executor>
20 </MenukaartVerkrijgenTransactie>

```

Associated part of the framework:

```

1 <TransactionType id="MenukaartVerkrijgenTransactie">
2     <description>De transactie om te komen tot het verkrijgen van de juiste menukaart</
    description>
3     <startDate>2011-01-23T00:00:00Z</startDate>
4     <endDate>2011-12-31T00:00:00Z</endDate>
5     <state>active</state>
6     <dateLaMu>2011-01-23T00:00:00Z</dateLaMu>
7     <userLaMu>bapa</userLaMu>
8     <initiator>
9         <RoleTypeRef idref="Consument"/>
10    </initiator>
11    <executor>
12        <RoleTypeRef idref="Werknemer"/>
13    </executor>
14 </TransactionType>

```

6.16.2 Elements

6.16.2.1 abbreviation

```

1 abbreviation : STRING;

```

=Abbreviation of the organisation name. Is used, among other things, as a prefix for transaction identification.

6.16.2.2 category

```

1 category : OPTIONAL STRING;

```

Category to which this object instance belongs (this is a standard string field without restrictions).

Message level example:

```

1 <... id="...">
2     ...
3     <category>Omdat dit een standaard string veld is mag ik bijvoorbeeld deze tekst als
        categorie invullen</category>
4     ...
5 </...>

```

6.16.2.3 creationDate

1 creationDate : DATETIME;

Date of creation of specific object instance in this message.

Message level example:

```
1 <... id="...">
2   ...
3   <creationDate>2010-12-03T00:00:00Z</creationDate>
4   ...
5 </...>
```

6.16.2.4 dateLaMu

1 dateLaMu : OPTIONAL DATETIME;

Date of the creation of this object instance, or the last modification.

Message level example:

```
1 <... id="...">
2   ...
3   <dateLaMu>2010-12-03T00:00:00Z</dateLaMu>
4   ...
5 </...>
```

6.16.2.5 dateReached

1 dateReached : DATETIME;

Date ...

Message level example:

```
1 <... id="...">
2   ...
3   <dateReached>2008-02-04T00:00:00Z</dateReached>
4   ...
5 </...>
```

6.16.2.6 dateRead

1 dateRead : OPTIONAL DATETIME;

The date this message was read.

Message level example:

```
1 <... id="...">
2   ...
3   <dateRead>2010-12-03T00:00:00Z</dateRead>
4   ...
5 </...>
```

6.16.2.7 dateSend

```
1 dateSend : DATETIME; — MessageTemplate
2 dateSend : OPTIONAL DATETIME; — MessageInTransactionType
```

The date this message was sent.

Message level example:

```
1 <... id="...">
2   ...
3   <dateSend>2010-12-03T00:00:00Z</dateSend>
4   ...
5 </...>
```

6.16.2.8 description

```
1 description : STRING;
```

Description of this object instance.

Message level example:

```
1 <Projectuitvoerende id="TNO">
2   ...
3   <description>TNO is naast anderen projectuitvoerende van VISI</description>
4   ...
5 </Projectleverende>
```

6.16.2.9 documentIdentification

```
1 documentIdentification : OPTIONAL STRING;
```

Unique number or characteristic of a document or file by which the document is identifiable.

6.16.2.10 documentReference

```
1 documentReference : OPTIONAL STRING;
```

Reference that can be referenced as an attribute to identify a file or document.

6.16.2.11 documentVersion

```
1 documentVersion : OPTIONAL STRING;
```

Version of a document or file.

6.16.2.12 endDate

```
1 endDate : DATETIME;
2 endDate : OPTIONAL DATETIME; — AppendixTemplate
```

Validity end date of specific object instance in this message.

Message level example:

```
1 <... id="...">
2   ...
3   <endDate>2011-12-03</endDate>
4   ...
5 </...>
```

6.16.2.13 fileLocation

```
1 fileLocation : STRING;
```

Location of the file, preferably an internet location or shared server path.

Message level example:

```
1 <... id="...">
2   ...
3   <fileLocation>\\srv-path\Public\project-x\docs</fileLocation>
4   ...
5 </...>
```

6.16.2.14 fileType

```
1 fileType : STRING;
```

Type of the file, preferably the MIME (Multipurpose Internet Mail Extensions) type.

Message level example:

```
1 <... id="...">
2   ...
3   <fileType>text/plain</fileType>
4   ...
5 </...>
```

6.16.2.15 fileVersion

```
1 fileVersion : STRING;
```

Version of the file, an ascending integer number or the date as in the following example at message level:

```
1 <... id="...">
2   <fileVersion>20071202</fileVersion>
3 </...>
```

6.16.2.16 identification

```
1 identification : STRING;
```

Identification of this message, in itself, the id [2.1] is already an identification (at least for the computer), but for humans, a slightly more extensive identification may be useful, for example, at the message level:

```
1 <OpdrachtBevestiging id="X">
2   <identification>Dit bericht bevat de opdracht X ten behoeve vanonderdeel Y</
   identification>
3 </OpdrachtBevestiging>
```

6.16.2.17 initiatingTransactionMessageID

```
1 initiatingTransactionMessageID : OPTIONAL STRING;
```

Reference to the message associated with a secondary transaction that initiated the message.

6.16.2.18 initiatorToExecutor

```
1 initiatorToExecutor : BOOLEAN;
```

The direction of this specific message. Message level example:

```
1 <VoorbeeldBericht id="...">
2   ...
3   <initiatorToExecutor>false</initiatorToExecutor>
4   ...
5   <transaction>
6     <VoorbeeldTransactie id="...">
7       ...
8       <initiator>
9         <PersonInRole id="A">
10           ...
11         </PersonInRole>
12       </initiator>
13       <executor>
14         <PersonInRole id="B">
15           ...
16         </PersonInRole>
17       </executor>
```

```

18         </VoorbeeldTransactie>
19     </transaction>
20     ...
21 </VoorbeeldBericht>

```

in the example above, the Sample Message is sent from B (executor [6.16.3.3]) to A (initiator [6.16.3.5]).

6.16.2.19 language

```

1 language : OPTIONAL STRING;

```

The language of the attachment content.

Message level example:

```

1 <... id="...">
2     ...
3     <language>NL</language>
4     ...
5 </...>

```

6.16.2.20 name

```

1 name : STRING;

```

Naming.

6.16.2.21 number

```

1 number : INTEGER;

```

Transaction number.

6.16.2.22 objectCode

```

1 objectCode : OPTIONAL STRING;

```

Possibility to establish a relationship with an external index. This includes a work breakdown structure, work packages or specifications.

6.16.2.23 result

```

1 result : OPTIONAL STRING;

```

Result of this transaction, content is a string.

6.16.2.24 startDate

```
1 startDate : DATETIME;  
2 startDate : OPTIONAL DATETIME; — AppendixTemplate
```

Validity start date of specific object instance in this message.

Message level example:

```
1 <... id="...">  
2   ...  
3   <startDate>2010-12-03</startDate>  
4   ...  
5 </...>
```

6.16.2.25 state

```
1 state : OPTIONAL STRING;
```

Status in which this object instance is located.

Message level example:

```
1 <... id="...">  
2   ...  
3   <state>active</state>  
4   ...  
5 </...>
```

6.16.2.26 userLaMu

```
1 userLaMu : OPTIONAL STRING;
```

User who created or last modified this object instance (CAUTION: this is not a reference to a Person-TypeInstance instance).

Message level example:

```
1 <... id="...">  
2   ...  
3   <userLaMu>Peter Bonsma</userLaMu>  
4   ...  
5 </...>
```

6.16.2.27 userName

```
1 userName : STRING;
```

'Code name' of the user, such as a TRIGRAM.

Message level example:

```
1 <... id="...">
2   ...
3   <userName>BAP</userName>
4   ...
5 </...>
```

6.16.2.28 versionNo

```
1 versionNo : STRING;
```

Version number of this object instance within this project. When an adjustment is made to this object instance, this field also changes.

Message level example:

```
1 <... id="...">
2   ...
3   <versionNo>23</versionNo>
4   ...
5 </...>
```

6.16.3 References

6.16.3.1 appendixGroup

```
1 appendixGroup : OPTIONAL AppendixGroup;
```

A subgrouping to which the specific attachment belongs.

Message level example:

```
1 <Bijlage id="...">
2   ...
3   <appendixGroup>
4     <AppendixGroup id="...">
5       ...
6     </AppendixGroup>
7   </appendixGroup>
8   ...
9 </Bijlage>
```

6.16.3.2 contactPerson

```
1 contactPerson : PersonTemplate;
```

The person linked to a PersonInRole [1.8] object or linked to a specific organisation. Example at message level (with an organisation chosen as a basis):

```
1 <Organisatie id="...">
2   ...
3   <contactPerson>
4     <Persoon id="...">
5       ...
6     </Persoon>
7   </contactPerson>
8 </Organisatie>
```

So in the associated framework, there is an OrganisationType Organisation and a PersonType Person.

6.16.3.3 executor

```
1 executor : PersonInRole;
```

Which role filler is the executive.

6.16.3.4 group

```
1 group : OPTIONAL GroupTemplate;
```

The general group to which a set of attachments belongs.

Message level example:

```
1 <AppendixGroup id="...">
2   ...
3   <group>
4     <Groep id="...">
5       ...
6     </Groep>
7   </group>
8   ...
9 </AppendixGroup>
```

6.16.3.5 Initiator

```
1 initiator : PersonInRole;
```

Which role filler is the initiator.

6.16.3.6 message

```
1 message : MessageTemplate;
```

The message to which a specific attachment belongs.

Message level example:

```
1 <Bijlage id="...">
2   ...
3   <message>
4     <Bericht id="...">
5       ...
6     </Bericht>
7   </message>
8   ...
9 </Bijlage>
```

There is therefore an AppendixType Attachment and a MessageType Message in the associated framework.

6.16.3.7 messageInTransaction

```
1 messageInTransaction : MessageInTransactionTemplate;
```

Reference to the place the message occupies in the flow of the transaction.

6.16.3.8 organisation

```
1 organisation : OrganisationTemplate;
```

The organisation belonging to a PersonInRole [1.8] object.

Message level example:

```
1 <PersonInRole id="...">
2   ...
3   <organisation>
4     <Organisatie id="...">
5       ...
6     </Organisatie>
7   </organisation>
8   ...
9 </PersonInRole>
```

6.16.3.9 role

```
1 role : RoleTemplate;
```

Reference to a role that can be fulfilled by an organisation on behalf of a PersonTemplate (person).

6.16.3.10 substituting

```
1 substituting : OPTIONAL PersonInRole;
```

PersonInRole [1.8] on behalf of whom this PersonInRole can send messages.

6.16.3.11 successor

```
1 successor : OPTIONAL PersonInRole;
```

Successor to another person in a specific role.

6.16.3.12 transaction

```
1 transaction : TransactionTemplate;
```

The transaction to which a specific group, message or transaction phase belongs.

Example at message level (with a message selected as the basis):

```
1 <Bericht id="...">
2   ...
3   <transaction>
4     <Transactie id="...">
5       ...
6     </Transactie>
7   </transaction>
8   ...
9 </Bericht>
```

There is therefore a MessageType *Message* and a TransactionType *Transaction* in the associated framework.

7 Message Communication

In the original documentation, this chapter is called ‘SOAP’ because the original communication is over a SOAP connection. Due to the progress in protocols, we expect this to be subject to change and, therefore, we renamed the chapter. This document will only state the current version of the exchange protocol.

7.1 Introduction

This part of the document contains the guidelines for the implementation of VISI communication based on the SOAP protocol.

The aim of the guideline is to provide directions and boundaries for the implementation of VISI communication, which ensures that the information system concerned can technically exchange VISI messages with attachments to another information system that is set up in accordance with the guideline.

7.2 Goal

The goal of this document is to describe a strict and clear protocol that can establish communication of VISI messages between all VISI supporting software parties. This should be described in such a way that the protocol is implementable for all parties, requires no unusual implementation, no unusual hardware requirements, and is strict enough to allow communication. The persistence, access permissions, and storage means of the stored messages are outside the scope of this guideline.

7.3 Architecture

The architecture has been kept simple by supporting only one scenario. The scenario includes intercommunication between SOAP servers.

The purpose and background of this architecture:

- the architecture must be able to support communication between VISI servers in a project without the intervention of other servers
- communication must be able to be carried out securely
- messages must be able to be unambiguously communicated
- attachments must be able to be sent
- the sender must be able to find out whether a message has arrived correctly

Architecture:

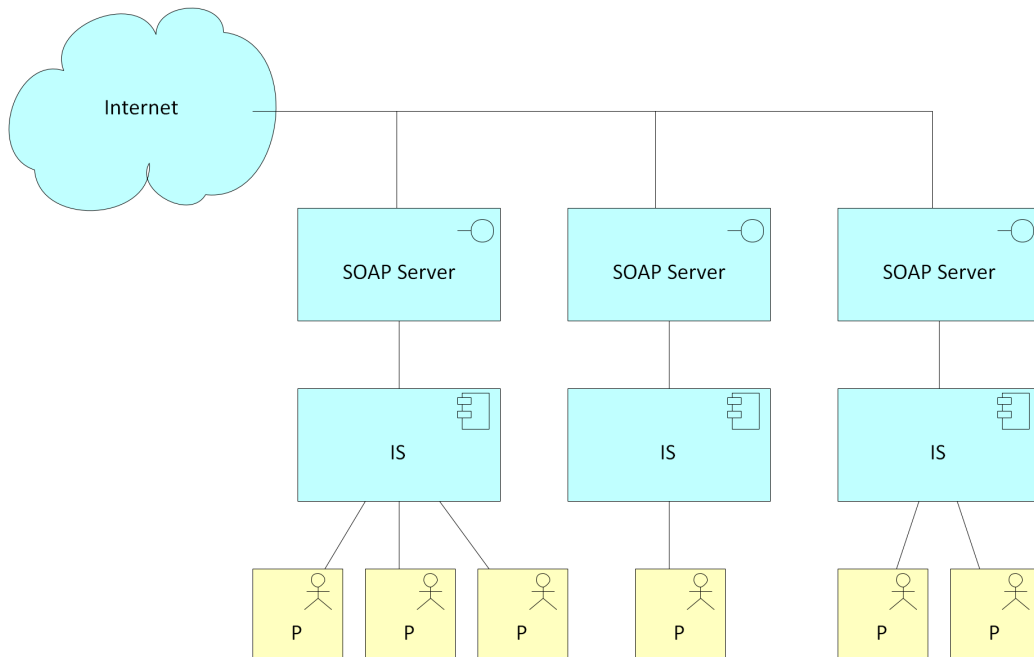


Figure 84: Architecture

Scenario

In this outlined scenario, the following objects can be found:

- The SOAP Server (**SOAP Server**), this is the server that handles the handling of VISI messages according to the path described in this document.
- The Information System (**IS**), this is the application that processes and generates VISI messages.
- The end user (**P** for person), this is a person who fulfils a certain role for a certain organisation.

7.4 Protocols

The MTOM (SOAP Message Transmission Optimisation Mechanism) protocol is chosen for the implementation. This solution is also based on a SOAP implementation of XOP (XML-binary Optimised Packaging). See the latest published version of MTOM and XOP at W3C⁶.

The VISI Standard enforces that communication between SOAP Servers takes place via https and SSL, so that the content of received messages cannot be intercepted. The use of HTTPS (TLS 1.0 or higher, with at least 128-bit encryption) for sending VISI messages and attachments is mandatory.

The maximum size of a VISI message (including attachments) is 10 GB. This is combined with "chunking" for further optimisation, a method of sending files in small chunks and then merging them back together. There is currently no limit on the amount of the appendix.

The protocol used can be found in the configuration. This message contains a simple element "SOAPProtocol" (language independent) under project information. The current capability is only "MTOM", so e.g. fig. 85.

```
1 <ProjectType id="Project_xyz">
2   ...
3   <description>Standard project</description>
4   ...
5   <complexElements>
6     ...
7     <ComplexElementTypeRef idref="OtherRandomComplexElement"/>
8     ...
9   </complexElements>
10 </ProjectType>
11 <ComplexElementType id="OtherRandomComplexElement">
12   <description>An other random complex element</description>
13   ...
14   <simpleElements>
15     ...
16     <SimpleElementTypeRef idref="SOAPProtocol"/>
17     ...
18   </simpleElements>
19 </ComplexElementType>
20 <SimpleElementType id="SOAPProtocol">
21   ...
22 </SimpleElementType>
```

Figure 85: MTOM Property Definition

And for the configuration fig. 86.

```
1 <Project_xyz id="Project-000">
2   ...
3   <otherRandomComplexElement>
4     ...
5     <OtherRandomComplexElementRef idref="ProjectConfiguration"/>
6     ...
7   </otherRandomComplexElement>
8 </Project_xyz>
9 <OtherRandomComplexElement id="ProjectConfiguration">
10   ...
11   <SOAPProtocol>MTOM</SOAPProtocol>
12   ...
13 </OtherRandomComplexElement>
```

Figure 86: MTOM Property Configuration

⁶<https://www.w3.org/TR/soap12-mtom/>

7.5 Message exchange

The message exchange (header, etc., server address) is set up on the basis of the precondition that all information about the present configuration, e.g. URL addresses of people in a certain role, etc., is contained in VISI messages according to the framework for the project to be executed. Of course, this precondition may not make the software dependent on a specific VISI framework.

In order to meet the above precondition, we will use the following convention within VISI.

- Some SimpleElementType objects that can be defined in the framework will be assigned a specific behaviour/interpretation based on their naming (attribute: id) across all frameworks.

We will assign the following SimpleElementTypes framework-crossing behaviour/definition:

SimpleElementType	Behavior
SOAPServerURL	Contains the URL of the SOAP server associated with this Person, Role or organisation
SOAPProtocol	Contains the SOAP protocol used

Table 16: Caption

7.5.1 Message exchange of fixed elements at framework level

Within this notation, we will propose the following use (note: this falls outside the new VISI concept):
At framework level:

- Each framework will contain the following bit of XML to map the SOAPServerURL to an organisation
fig. 87

```
1 <OrganisationType id="Organisatie">
2   ...
3   <description>Standard organisation</description>
4   ...
5   <complexElements>
6     ...
7     <ComplexElementTypeRef idref="RandomComplexElement"/>
8     ...
9   </complexElements>
10 </OrganisationType>
11 <ComplexElementType id="RandomComplexElement">
12   <description>A random complex element</description>
13   ...
14   <simpleElements>
15     ...
16     <SimpleElementTypeRef idref="SOAPServerURL"/>
17     ...
18   </simpleElements>
19 </ComplexElementType>
20 <SimpleElementType id="SOAPServerURL">
21   ...
22 </SimpleElementType>
```

Figure 87: SOAP Server URL Definition

Alternatives:

The choice made seems the most elegant; an alternative could be to assign the SOAPServerURL field to PersonInRole in the promotion step. However, this choice would make the VISI system dependent on SOAP and the conceptual choice to prescribe this communication.

7.5.2 Message exchange of fixed elements at message level

At message level:

We assume that a configuration is present for a project, as described in 4.2.

- This configuration defines for each organisation what its SOAPServerURL is shown in 88

```
1 <Organisatie id="Biereco">
2   <name>Biereco's Rebuildcompany BV</name>
3   ...
4   <randomComplexElement>
5     ...
6     <RandomComplexElementRef idref="BierecoInformation"/>
7     ...
8   </randomComplexElement>
9 </Organisatie>
10 <RandomComplexElement id="BierecoInformation">
11   ...
12   <SOAPServerURL>http://192.168.0.102/visi</SOAPServerURL>
13   ...
14 </RandomComplexElement>
```

Figure 88: SOAP Server URL Configuration

7.5.3 Consequences

The consequences of this approach are that the information system (IS) is able to find out for each message within a transaction which URL belongs to the sender and which URL belongs to the receiver. This data (URL sender, URL receiver) will have to be communicated with the SOAP server associated with this information system, of course, together with the VISI message itself. The communication between the information system and the associated SOAP server is outside the scope of this note and is up to the developer of the information system and the associated SOAP server itself.

7.5.4 Message exchange initialisation

To start a project, all parties will need to know who they are communicating with, which framework is being used, and have all other configuration information available. A configuration has been defined in order to always keep this information up-to-date and to prevent unauthorised parties from modifying this information. This configuration is located at a location specified by the software supplier (existing infrastructures then provide security and availability). More information about the configuration can be found in 5 and 10.

The initialisation of each information system starts with loading the configuration (i.e. in each individual information system). This configuration contains the link to the most recent framework (as every message points to a framework) and contains all relevant project information, including the roles, people and organisations that participate (incl. URL addresses).

7.5.5 Message exchange scenario

This description is suitable for all types of messages within transactions, both the first message within a transaction and responses to messages received within a transaction. It is important that messages within an organisation do not go through this path; this also means that these messages will not be placed on the central server.

In the examples:

- URL SOAP server sending party: http://192.168.0.102
- URL SOAP server receiving party: http://192.168.0.138

The order of message exchange:

1. The VISI message is created by the sending IS based on the received message (in case of a new transaction, the information is extracted from the configuration).

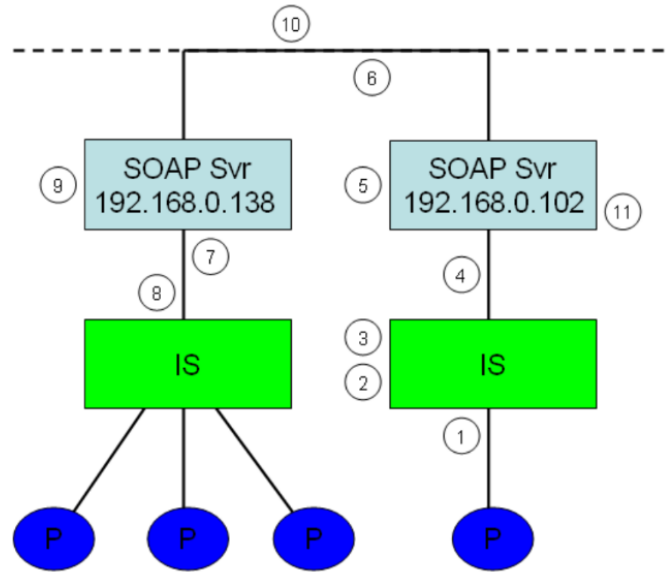


Figure 89: Message Exchange

2. The URL addresses of the sending and receiving IS are extracted from the formatted message by the sending IS.
3. The created VISI message + URL address of sending SOAP server + URL address of receiving SOAP server are communicated to the associated server (method of communication is up to the software parties themselves).
4. The SOAP server of the sending party builds a SOAP message as shown in fig. 90.

```

1 <SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
2   <SOAP-ENV:Header>
3     <ns2:UniqueID xmlns:ns2="http://www.visi.nl/schemas/soap/version-1.0">
4       <ns2:ID>m_a4fab2e7-251d-4bb3-9954-8733ebc088ee</ns2:ID>
5     </ns2:UniqueID>
6     <ns2:SOAPServerURL xmlns:ns2="http://www.visi.nl/schemas/soap/version-1.0">
7       <ns2:sender>https://192.168.0.102</ns2:sender>
8       <ns2:receiver>https://192.168.0.138</ns2:receiver>
9     </ns2:SOAPServerURL>
10  </SOAP-ENV:Header>
11  <SOAP-ENV:Body>
12    <ns2:parseMessage xmlns:ns2="http://www.visi.nl/schemas/soap/version-1.0">
13      <ns2:message>
14        <visiXML_MessageSchema ...>
15          </visiXML_MessageSchema>
16        </ns2:message>
17      </ns2:parseMessage>
18    </SOAP-ENV:Body>
19  </SOAP-ENV:Envelope>

```

Figure 90: Sending Message

note: the ID of each attachment is equal to the ID in the VISI message used to describe the metadata of an attachment. (<Data id="abc">)

5. The SOAP server of the sending party sends this message to the SOAP server of the receiving party.
6. The receiving party's SOAP server sends a standard SOAP exception error message to the sending party.
 - a. The SOAP server of the receiving party forwards the VISI message part of the message to the IS of the receiving party (method of communication is left to the software parties themselves).

7. The IS of the receiving party interprets the VISA message and, if approved, it sends this message + URL address of sending SOAP server + URL address of receiving SOAP server to its own SOAP server (i.e. of the receiving party). The method of communication is left to the software parties themselves.
8. If successful, the receiving party's SOAP server builds a SOAP response message as in fig. 91.

```

1 <SOAP-ENV:Envelope ...>
2   <SOAP-ENV:Header>
3     <SOAPServerURL ...>
4       <sender>http://192.168.0.102</sender>
5       <reciever>http://192.168.0.138</reciever>
6     </SOAPServerURL>
7     <UniqueID ...>
8       <ID>UniqueIDonMessageInitiatingSOAPServer_XYZ</ID>
9     </UniqueID>
10  </SOAP-ENV:Header>
11  <SOAP-ENV:Body>
12    <ERRORS>
13      <ERROR CODE="0"></ERROR>
14    </ERRORS>
15  </SOAP-ENV:Body>
16 </SOAP-ENV:Envelope>

```

Figure 91: Response No Error Message

In case of a single error fig. 92.

```

1 <SOAP-ENV:Envelope ...>
2   <SOAP-ENV:Header>
3     <SOAPServerURL ...>
4       <sender>http://192.168.0.102</sender>
5       <receiver>http://192.168.0.138</reciever>
6     </SOAPServerURL>
7     <UniqueID ...>
8       <ID>UniqueIDonMessageInitiatingSOAPServer_XYZ</ID>
9     </UniqueID>
10  </SOAP-ENV:Header>
11  <SOAP-ENV:Body>
12    <ERRORS>
13      <ERROR CODE="1"> Er is een fout opgetreden bij ...</ERROR>
14    </ERRORS>
15  </SOAP-ENV:Body>
16 </SOAP-ENV:Envelope>

```

Figure 92: Response Single Error Message

In case of multiple errors (e.g. with validation of the xsd) fig. 93.

In case of no error, the code is 0, in case of an unknown error, the code is 1; these error messages will have to be resolved by the programmers. All error messages with a higher code are errors that can be understood by the software.

9. The receiving party's SOAP server sends this response message to the sending party's SOAP server.
 - a. The sending party's SOAP server sends a standard SOAP exception error message to the receiving party as a standard response to this response message.
10. Response message back at square one, check whether the information corresponds to the message sent and if agreed, notify the IS of the receiving party that the message was sent and received as agreed.

Remark:

- The above process remains manageable if both parties use the same IS. In that case, the URL addresses of the receiving and the sending SOAP server are identical.
- No standard error codes higher than 1 were defined during the release of this protocol. As soon as such an error code has been recorded, it is clearly indicated what the error code is, a description of the error and an unambiguous description of how the software should respond.

```

1 <SOAP-ENV:Envelope ...>
2   <SOAP-ENV:Header>
3     <SOAPServerURL ...>
4       <sender>http://192.168.0.102</sender>
5       <receiver>http://192.168.0.138</reciever>
6     </SOAPServerURL>
7     <UniqueID ...>
8       <ID>UniqueIDonMessageInitiatingSOAPServer_XYZ</ID>
9     </UniqueID>
10  </SOAP-ENV:Header>
11  <SOAP-ENV:Body>
12    <ERRORS>
13      <ERROR CODE="1">Waarden van simpel element1 is niet volgens definitie</ERROR
14    >
15      <ERROR CODE="1">Waarden van simpel element2 is niet volgens definitie</ERROR
16    >
17    </ERRORS>
18  </SOAP-ENV:Body>
19 </SOAP-ENV:Envelope>

```

Figure 93: Response Multiple Error Message

- If there is no ('on time') response to a message, the same message will not be sent again. We continue to wait for the answer or find a solution outside of VISI.
- In step 6 and step 9, a message is always sent to the other party first.

7.6 Disclosure of the framework and update scenario

Frameworks are accessible to an information system via a URI. Every VISI-compatible message within a specific project refers to this framework (including the configuration).

The XML/XSD concept used within the VISI enforces that every message contains a reference to its schema via a URI. The VISI system then prescribes that the framework is stored on this URI (which always contains the extension XSD) with the extension xml.

Furthermore, VISI prescribes that once used frameworks be frozen forever. In case of changes in the framework, a new URI for this new framework will have to be devised. The applications are automatically notified by pointing the configuration to the XSD associated with this new framework.

With this, the use of VISI guarantees that all frameworks and all messages remain unambiguous in time despite adjustments to the frameworks, roles, persons or organisations or even the system.

When adjusting a framework, the following steps can be taken:

- Adjust the framework, for example, using a VISI framework editor.
- Distribute the proposed changes using the meta-framework and wait for agreement from all VISI partners involved in the project.
- Promote the framework and place both the framework and the promoted framework (with the same name, but different extension) in the desired directory.
- Adjust the configuration and distribute it to all VISI partners involved, and determine the effective date and time.

At the agreed time, the modified framework and the associated configuration will become active for new transactions. The applications will transfer existing transactions and new 'sub-transactions' through the old framework to let go. New transactions will be started via the new framework.

7.7 Disclosure of the configuration scenario

The exact content of a configuration is described elsewhere. Here is a description of what to do with updates.

There can be various reasons for updating a configuration, such as because people, organisations and/or

roles have been modified, more information is known about people, organisations, roles or the project, or because the framework has been modified.

In principle, the information of a configuration is volatile; it can be seen as a 'repository' of important project-specific information that is always up-to-date and available for the different information systems that play a role in this project. In contrast to frameworks, the information is therefore dynamic, where the location (URI) is static, i.e. each information system knows where to find the latest, most up-to-date information about the project (this URI is used to track a project). To start, see also 3.3.

In order to be able to find out the different versions of the configuration, VISI recommends performing an update as follows:

- Make a copy of the existing configuration. This message will have the same name with a serial number immediately after the name.
- Take care of the update of the project-specific message.
- Distribute the configuration to the VISI project partners and determine the effective date plus time.
- The new configuration is activated on the agreed date and time.

7.8 Attachments, Ref's en Id's

The attachments and elements in the header file are processed as described in 3. Use of the MTOM concept takes care of the handling of attachments and the VISI message itself. Also, when the body of a SOAP message is encrypted, this encryption will automatically be present on the attachments.

7.9 SOAP function calls

The chosen solution requires only two function calls.

parseMessage

(1 input variable of the type string, contains an XML file, see 3.6)

parseMessageConfirmation

(2 input variables of the type string, UniqueIDonMessageInitiatingSOAPServer_XYZ and the found errors errors, returns an XML file, see 3.6)

The content is the same in both cases, namely the VISI message preceded by the attachments. The form of the SOAP message should conform to the protocol chosen in the configuration.

'Meta' information about the different URL addresses and the unique ID for the sending SOAP server are included in the header of the SOAP message.

The method of implementation and use is described in 3.6.

7.10 Configuration

The configuration content has already been described in a previous document. The procedure for updating a configuration can be found in Chapter 5.

7.11 Optional implementation suggestions

Furthermore, on an infrastructure level, SOAP Servers can use the "whitelist" technique, which allows only authorised SOAP servers to send messages to a SOAP Server.

7.12 Visi werk sessie 16 mei 2023

7.12.1 leidraad 1.7

7.12.1.1 #72 Prio-801: Overdraagbaarheid lopende projecten of VISI archief bij switch van leverancier

wat is de huidige opbouw van een archief?

jeroen: archiveren en inlezen van archief deel maken van cretificering Jeroen en arne hebben een sleutel

gemaakt(gebruikt?) ter validatie van geslaagtheid van import/export. jeroen: aanbesteding amsterdam, dus alles open projecten die bij technia stonden moesten naar b&s. dus een export en import moeten er zijn. vergelijkingssleutel wordt gegenereerd en dan vergeleken na import wat b&s heeft geïmporteerd. wordt iets wat in 1.8 gaat komen.

b&s heeft software oplossing gemaakt om oude raamwerk te updaten naar nieuwe versie van raamwerk, maar dit is buiten de standaard om. dat loopt dan niet goed in je archief.

niek leest voor overdraging van lopende projecten getest, huidige archief formaat

kunnen inlezen moet een testcase worden in de validatie tests. huidige archief structuur gaan naar tests leverancier en opdrachtgever omwisselen en dan testen. jeroen heeft als het goed is een oplossing voor het char limit van 250 probleem, voor berichten en voor bijlages

structuur blijft gelijk voor 1.7 voor 1.8 misschien voor 1.7 wordt het nu getest

7.13 Message Signing

VISI software supplier is responsible for the management of their set of issued public keys. At the same time, each software supplier, upon request from the certification committee, must be able to produce a list of the issued certificates, which includes the following:

- public key
- public key friendly name; if available
- corresponding server where the certificate is supposed to be used
- corresponding validity start and end dates

Also, the software supplier must be able to demonstrate that the private key of a public key is in their possession.

Starting point for message exchange between the servers entails the following: all servers are known to each other, and no server access without IP whitelisting is possible.

This means that every software supplier knows which server pairs are communicating with each other. In case one, the client decides to host the VISI software in their private cloud and, as a result, requires a unique public key, it is then the responsibility of the software supplier to provide their client with such a public key.

In the event of the addition of the authentication layer in the future, the signing of the messages is not going to be affected. Only the ISO security level will change as a result. This, in turn, makes the central administration of the issued public keys unnecessary.

Given the fact that communicating server pairs are known in advance, if it becomes apparent that a server of a software supplier A is using the public key of the software supplier B, then it is most certainly true that said suppliers have exchanged their private keys.

It is worth mentioning that the Xades protocol and the Xades organisation should not be confused with each other. The desired VISI security level may not require such an organisation as Xades to check it. A message that is created to be sent will be signed to ensure that its contents will remain verifiable in the future. A signed message shows that the content has not been altered and is the original content from the moment of sending. This way, one ensures that the recipient did not tamper with the message. The signing will be sent along with the message in such a way that the signing is backwards compatible with non-signed messages. This is done by not including the signing in the parse message, but by appending it after the parse message, while still in the body. See section 7.13 for an example.

A signature can be enveloped or detached, whether it is included as an element of the file containing the signed data or a separate signature file is created, that refers to the data upon which it bears:

Four levels of baseline signatures have been defined by ETSI standards for the XAdES format. They are the:

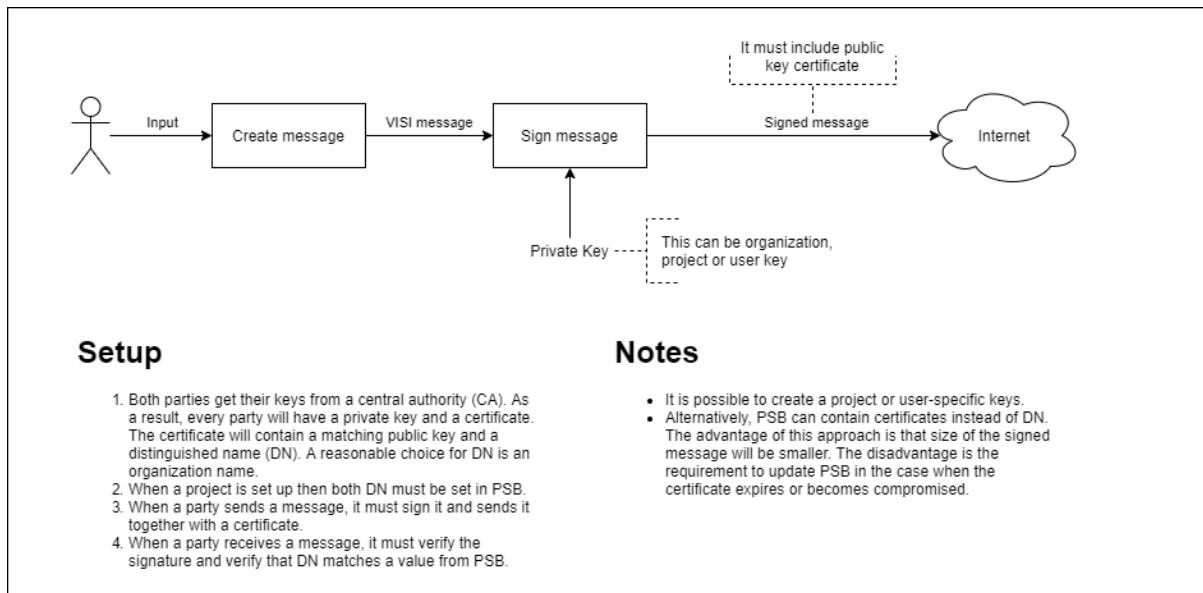


Figure 94: Signing Process

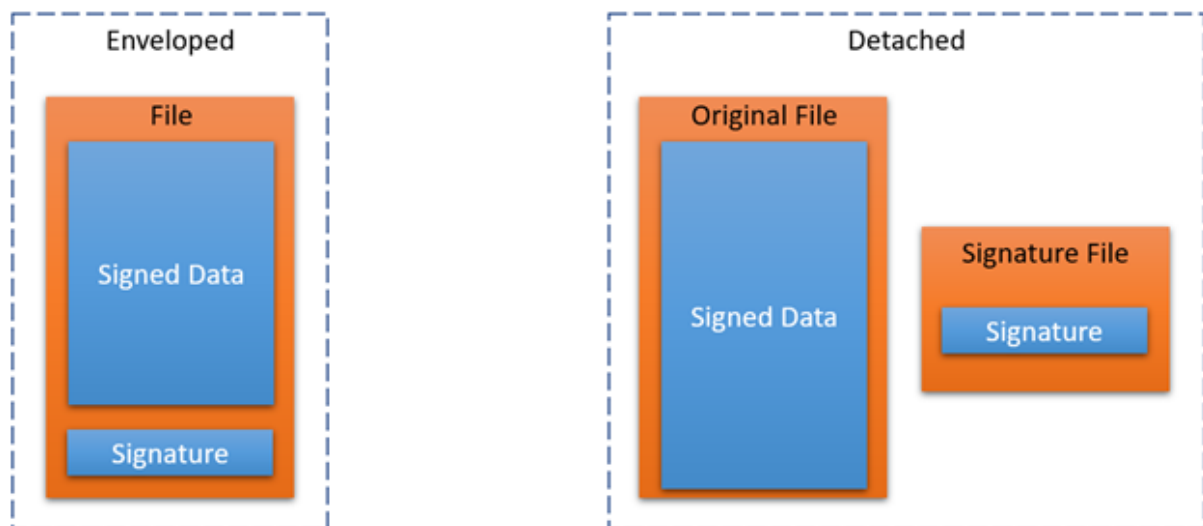


Figure 95: Enveloped and Detached

- *B-B level, which is the level of a Basic Signature, meaning that it is a signature that can be validated as long as the signing certificate is valid (not revoked or expired).*
- *B-T level, which is the level of a Signature with Time, meaning that it is a signature that proves that the signature existed at a given point in time. It is built from the previous level by adding a time stamp token on the signature as an unsigned property.*
- *B-LT level, which is the level of a Signature with Long-Term Validation Material, meaning that it is a signature that provides the long-term availability of the validation material by incorporating all the material or references to material required for validating the signature. It is built from the previous level by adding this material, that is: the complete certificate and revocation data on the signature and the time stamp(s) as unsigned properties.*

- *B-LTA level, which is the level of a Signature providing Long Term Availability and Integrity of Validation Material. It is built from the previous level by adding a time stamp token on the validation material as unsigned properties, thereby establishing evidence that the validation data existed at the indicated time. This level targets the long-term availability and integrity of validation material, and if appropriate measures are put in place (e.g. periodical timestamping), a signature at this level could still be validated long after the cryptographic algorithms used for its creation are no longer considered secure enough, or more simply, after the expiration of the validation data.*

For VISI, the XAdES-B-LT level is chosen, and the signing will be done using the enveloped form for the messages and the detached form for the appendices. Looking at fig. 95, the file can be interpreted as a VISI Soap message body. The parseMessage tags are first hashed into a digest value. Then other properties, such as the signing time, are hashed into a digest value. The properties and data are added underneath the parseMessage, and they have a child element with the digest value, and this whole SignedInfo section is hashed again in a digest value.

A signature is created over this last Digest value, and also added to the message. This means that the signed data is the parsed message, and the signature is added to the message body by XAdES. For an appendix, you need to look at the detached part of fig. 95. The original file is your appendix, and the signature file is the part added in the VISI message where the appendix is described. This means that your appendix contents are not included in the VISI message, as is the norm with VISI messages. This can be seen in section 7.13.

For version 1.7, the attachments will only be hashed using the same encryption method as the signing and the hash will be stored in the new template field fileChecksum.

```

1 <SOAP-ENV:Envelope ...>
2   <SOAP-ENV:Header>
3     <SOAPServerURL ...>
4       <sender>http://192.168.0.102</sender>
5       <receiver>http://192.168.0.138
6       </reciever>
7     </SOAPServerURL>
8     <UniqueID ...>
9       <ID>UniqueIDonMessageInitiatingSOAPServer_XYZ</ID>
10    </UniqueID>
11    <Appendices ...>
12      <count>2</count>
13    </Appendices>
14  </SOAP-ENV:Header>
15  <SOAP-ENV:Body>
16    <parseMessage ...>
17      <message>
18        <visiXML_MessageSchema ...>
19          ... // _VISI Message
20          <Data> // Appendix I
21            <Bijlage id="VoorbeeldDocument">
22              <name>Voorbeeld</name>
23              <fileLocation>\\srv-bouw\Public\project\docs\msword</fileLocation>
24              <fileType>application/msword</fileType>
25              <fileVersion>2010</fileVersion>
26 new—> <Checksum>00BD947603...2526F6223898</Checksum>
27              <documentIdentification>345899</documentIdentification>
28              <documentVersion>1</documentVersion>
29              <documentReference>FG783990</documentReference>
30              <startDate>2011-02-04T00:00:00Z</startDate>
31              <endDate>2011-12-31T00:00:00Z</endDate>
32              <state>active</state>
33              <dateLaMu>2011-02-04T00:00:00Z</dateLaMu>
34              <userLaMu>bapa</userLaMu>
35              <language>NL</language>
36              <appendixGroup>
37                <AppendixGroup id="...">
38                  ...
39                </AppendixGroup>

```

```

40         </appendixGroup>
41     </Bijlage>
42 </Data>
43 <Data ...>
44     ... // Appendix II
45 </Data>
46 </visiXML_MessageSchema>
47 new→ <Signature>
48 new→     ... // Signing message
49 new→ </Signature>
50 </message>
51 </parseMessage>
52 </SOAP-ENV:Body>
53 </SOAP-ENV:Envelope>

```

To verify a message, we first hash the parseMessage into a new digest value and compare it to the enclosed digest value of signedData. If this corresponds, we then proceed to extract the entire SignedInfo section, and we add the xmlns tag to the root. If we don't add the tag, we can never get a valid verification. The reason for this is that in the process of signing, Xades returns the section without a tag. We then hash this section. After hashing this entire section, we create a public key from the certificate in the message. Now we call an RSA verify with the hashed data, the signature of the message, the method (e.g. sha256) and the public key.

- Confirmation must be signed again. The encoding will be done using a minimum of SHA256. This can be enhanced to SHA384 or SHA512 in a later version, which should not impact the current implementation. The certificate that is used must be a trusted certificate from a trusted issuer, not a testing certificate.

The signing of the confirmation is not replicated in the current archives. To sign the confirmation, a body needs to be added. When the confirmation is not signed correctly, this creates an error that is not solvable in the current architecture.

```

1 Details: <errors id="signedData">
2   <error code="0"/>
3   <digestValue>ZAwOHs6JQ40HZQ/qBkHMyjL/T/prnX9l+Yvz9+EBgbc=</digestValue>
4 </errors>
5 <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
6   <SignedInfo>
7     <CanonicalizationMethod Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" />
8     <SignatureMethod Algorithm="http://www.w3.org/2001/04/xmldsig-more#rsa-sha256" />
9     <Reference URI="#signedData">
10       <Transforms>
11         <Transform Algorithm="http://www.w3.org/2000/09/xmldsig#enveloped-signature" />
12         <Transform Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" />
13       </Transforms>
14       <DigestMethod Algorithm="http://www.w3.org/2001/04/xmenc#sha256" />
15       <DigestValue>435yFioXVHyE/cMHxro/K9gqlx989zCXokEhaNQvTo4=</DigestValue>
16     </Reference>
17     <Reference URI="#SignedProperties" Type="http://uri.etsi.org/01903#SignedProperties"
18     >
19       <Transforms>
20         <Transform Algorithm="http://www.w3.org/2001/10/xml-exc-c14n#" />
21       </Transforms>
22       <DigestMethod Algorithm="http://www.w3.org/2001/04/xmenc#sha256" />
23       <DigestValue>0I6hfKzWDSla2IjpexFxfOGHDoHeZ+9Y90Doupk1jvY=</DigestValue>
24     </Reference>
25   </SignedInfo>
  <SignatureValue>AeNOFiN9C8AZkKOsYLWPnQwSCNY5+ix9oNEo5PcGaambkcYIkGQcF/YNsukvDXNiV+
  zSoe7xZXM1QRxsrCRzCqsf0Dp68pPO12m9lfqenpiZm1NNp5GzwmZMHq5QyfaV3JnjpxJVSZxqpaZRUwnjW7dzPjDH12P9dn26
  /RurOsLYSTqCAOp9DLyG/3OEA9rYK5oocK6GrUIZU/
  f9rQorILGngiJ4t6CTb6NBH4zAYouI0N5yabYDtfUymCp25pcZgJC9DB1bIIBr5Qh5ZeSciuaHNIIDhUps
  +06d/k2oeVqvzBvmNPjPanMI5MgDIU5cpPff3Gmskv3KQ3+
  LuZ7540NCRKHJNnguGsfZvISMikj40VfJ5BW6lntxRCv4ZUuwQN892VRMgaPTbKoU1Z1RzCSocmcG/
  aphzTpoRfEzAS87wLFUA/9j+ZggsACKH3Tlzcpsukx4fjDjwrZGNp29NI+9Zs0/
  gukdc2n18Qgl1clDLr8ila5nhPSF5dPypm1gNtOVyLh1znWx8UJfv/7MjtGHsEWO7ANJLKW4+
  yYm9mofYYR3VTqh60rMIT0FePu2rAiwUODP7zbISNA8oTa2+etik94BvJTPmN5jtSWE0yJ28SZN6KTHS/

```

```

26     FmOKv3cotQN5pGrsuqmJIjJdRntGNYU2H91I=</SignatureValue>
27 <KeyInfo>
28   <X509Data>
    <X509Certificate>MIIF+TCCA+
    GgAwIBAgIUO4F5qdx5SCVtSLkirjs4S1zvRjwwDQYJKoZIhvcNAQELBQAwwYsxGzA.JBgNVBAYTAk5MMRYwFAYDVQQIDA1Ob29yZC11
    /IiVJ4xy9t4N+GSXnxEewHN6aH0DBDG4WpMq6bFmzKvxhgu/gAn6cFPpU1Etf9Jjq5gw22JPQ/
    N3kECW72VT0kzznlU2rrQCngcIiTSQ2u0/YBqGGxfJnqSCYmuMn+7FzemawwJ/XHeR7DENnM.JngIqmyx/
    Kf9WuabtNcwP6gRU6j4M1zEWKcLjSITAmiW3rb97g7eRIdc/4/5aLWuQsw1EAT7Y3sySSzDBnabiez6Qrr/
    exK96pL4GEdbBna+Gw3YgmT56b9ddP7JCCpc0bf2rJmK7fuc3AJzznC3U4oPsfjPBh1WV+6X/
    er5cANwrqKw1q0p15s9AIZ+x/TEkPJSn23BCaoVxwp7o4dexus81dkJ24iauEuTpfuon6XbU2inRTfuvvmZ/
    Qbtzvv6hZcoBFNC+TMC1SyaLuM7QYzXfGmDcefMbN81+esgVQhaWMgLeC7dodsNIX4+
    nrYV5fv5Qiru6Ra0uCbxED3BcIcfQu3ZikyMkwuMDPaUBKp2uLvEAvNIHasAn3/
    ismM383rbFp3RX96iZqxVFXl2qaEJVPWT0pmaDgmOeSGc2D0zTGb+4
    dgjHtRUM6XeOea7Kjfp3UjCT2fD7vscUkAThMvVxjE3HEXeGcAc6Wk4JFu+89cbSoUWOO/0AFsc+yUV41FhV
    +
    X7AUqeCUqd9mqrhveMCAwEAaNTMFewHQYDVR0OBBYEFDiCagj7xaFOeDwpQh5vmPhrKXUKMB8GA1UdIwQYMBaAFDiCagj7xaFOe
    /wQFMAMBA8wDQYJKoZIhvcNAQELBQAwdggIBAI5FI8vNsQLHLZ8l8kyvk17dNlnRSu0ZwQS141MDJZw+
    YN8pw+MbNhvWf2z8IxiPsc9TKYXR+y21GVprz6s+
    v3Ku8ZFf3Kuc4iOjGa35LOnumwEwxPrjh1ABNwIXqJzAOakhi7MawSHFqiv9DOPjTHbEbmbXwbrCI1uO1CrYcYeVdBDarB6Bnyg
    /G9K5suDfCFYQjQcflHoN/
    hmHaif3yE29zXiSZr7oa5CP6K8r2Nfr1H3cJ4yQYXu7iqLFw13b5peSQM4TaRAviHjR3JZpwMuW1R8jkCDVUSY2byh0E
    +Y54illr5PErr7KNRXmE9xv21Aqr0/JQItkOApt2L/
    v2wtuu0ZfmTWOP6uw4Z0lkQYter7OyUKQvgZXjuzYu6jjMR4qknIbiftD1BXzmH0yZZvHX4zZ+
    KIq5IHrobqQppbQ4Fimz1D4HxJHS/FQQ5cBZAFbnKrv7TEGXcZPhpk2ncBtjwlv5rq+s53D5bA/
    rmIPFNtz4k9eOa0NLPSE0OcWdtBu9mWyZlWvUZPvdg5F0ejDQmtQUj/NDd2x5662Wp/
    A7awCEpqVW2OsEPvRaQaVGycG4avf2hAzNxRURWvmnv3nw0luzst9+2sqkxxx5Ohtj5NcgPJAIMu/
    asQOdoROA7lgH99G/Zf2xnG5r35ekXR0r3vF3vLILozM</X509Certificate>
29 </X509Data>
30 </KeyInfo>
31 <Object>
32   <xades:QualifyingProperties Target="#Signature" xmlns:xades141="http://uri.etsi.org
    /01903/v1.4.1#" xmlns:xades="http://uri.etsi.org/01903/v1.3.2#">
33     <xades:SignedProperties Id="#SignedProperties">
34       <xades:SignedSignatureProperties>
35         <xades:SigningTime>2025-10-22T12:22:10Z</xades:SigningTime>
36         <xades:SigningCertificate>
37           <xades:Cert>
38             <xades:CertDigest>
39               <DigestMethod Algorithm="http://www.w3.org/2001/04/xmldsig#sha512"/>
40               <DigestValue>WmSbUZ8V3wkSo+tQNWsfDKQlyBU=</DigestValue>
41             </xades:CertDigest>
42             <xades:IssuerSerial>
43               <X509IssuerName>E=info@bakkerspees.nl, CN=Bakker&Spees, O=Bakker&
    ;Spees, L=Amsterdam, S=Noord-Holland, C=NL</X509IssuerName>
44               <X509SerialNumber>339717849996594625234777628386423897186903213628</
    X509SerialNumber>
45             </xades:IssuerSerial>
46           </xades:Cert>
47         </xades:SigningCertificate>
48       </xades:SignedSignatureProperties>
49     </xades:SignedProperties>
50   </xades:QualifyingProperties>
51 </Object>
52 </Signature>

```

Listing 1: Signed confirmation

Distinguished name met department name in het PSB

For VISI 1.7, the test protocol has to be extended with a verification to check if the signing is (still) valid. Another extension to the test protocol is whether an already signed message gets rejected if it is altered after signing. There has to be an option to send a message that is wrongly signed to other SOAP servers, to verify that they correctly reject this. When a message is wrong, an error will have to be returned, detailing why the message is wrong. This can be an admin option, where either the content or the signature can be altered, using, for example, a search and replace. The signature is created over the contents of <parseMessage> ... </parseMessage>, including the tags themselves. For the appendix,

the signature or checksum is created over the entire appendix file.

Signing can be set as a configuration at the project level. An authorised user can turn on and off signing during the running of the project. Signing does not need to be on both sides of the VISI communication. Once a message is signed, the accepted message keeps the information on signing. When the signing is on, the signing is checked, and a wrong signature will result in the message being rejected. Archiving messages that are signed is constructed so that older software can read the archives. Signed archives are stored in the system and are archived again as signed messages.

Appendices will be hashed in the message that transported the appendix. Signing of the appendix will be done in a later version of VISI.

8 4. Implementation using an example

8.1 VISI version 1.6

8.1.1 4.1 Introduction

The example below forms the common thread in the explanation of the implementation of VISI. The example was deliberately chosen outside the scope of construction to reflect the broad applicability of the VISI system and to prevent possible application dependencies. The example is introduced here with a general description. The full details are included in Appendix 4.

8.1.2 4.2 Example speciality restaurant

Three well-known specialist chefs work in the speciality restaurant De Top Kok. The restaurant focuses on the discerning culinary consumer. Customers are interested in food and critical of the end product they are presented with. Every chef has his or her own speciality. Depending on the workload and knowledge, they are able to answer customer questions through the waiter. If the waiter thinks the chef's response is taking too long, the waiter looks at Wikipedia and comes up with an answer/advice himself. To prevent misunderstandings between the waiting staff and the kitchen, it has been agreed that the waiter places orders with chefs in a formal manner. The cook who first agrees to an order must execute the order. If all Cooks refuse the order, the customer will be informed by the waiter that the order is not possible in its current form. The customer must then resubmit the order to the waiter.

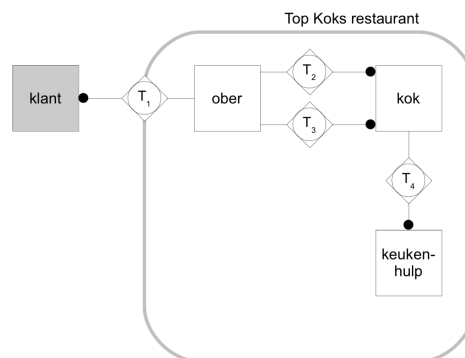


Figure 96:

In this example, the processing of an order is translated into VISI communication. The following roles and transactions are required to process the order.

Communication

Based on the above roles and transactions, communication can proceed as follows:

1	Ober $\rightarrow$ Klant	Wilt u de kaart zien?
2	Klant $\rightarrow$ Ober	Ja graag/Nee, nog niet (transactie eindigt dan).
3	Ober $\rightarrow$ Klant	Aanbieding menukaart met kaart als bijlage bij het bericht.

Role	Responsible for:
Customer	placing an order
Waiter	logistic processing of an order
Cook	realizing an order
Kitchen help	answering questions from the Cook

Table 17: Roles

Code	Name
T1	Order recording
T2	Ask Waiter to Cook
T3	Kok Assignment
T4	Ask a Chef for Kitchen Help

Table 18: Transactions

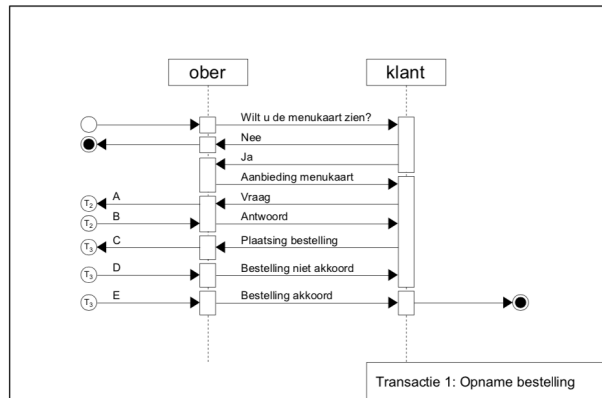


Figure 97: Transactieschema T1: Opname bestelling

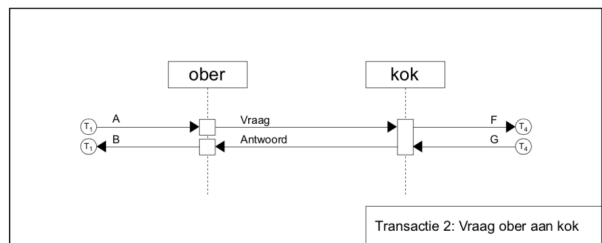


Figure 98: Transactieschema T2: Vraag ober aan kok

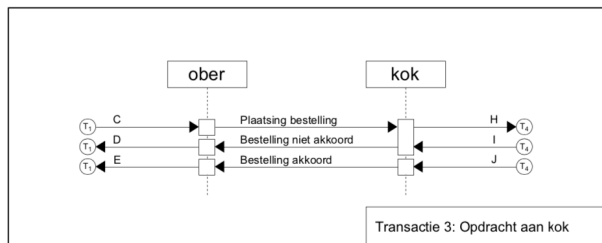


Figure 99: Transactieschema T3: Opdracht aan kok

⁴ Klant $\rightarrow$ Ober

Kan gerecht x ook in variant Y worden bereid?

⁵

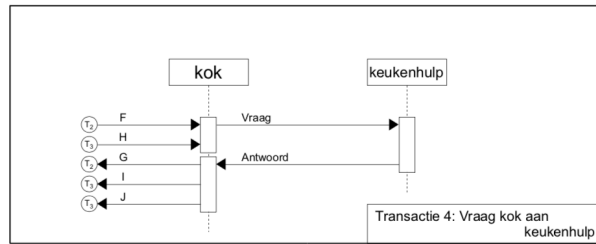


Figure 100: Transactieschema T4: Vraag kok aan keukenhulp

6		
7	Ober $\rightarrow$ Kok(s)	Vraag: Kan gerecht x ook in variant Y worden bereid?
8	Kok(s) $\rightarrow$ Ober bereidingswijze	Antwoord/Advies: Ja, maar ik adviseer deze
9		
10	Ober $\rightarrow$ Klant bereidingswijze.	Antwoord/Advies: Het kan maar we adviseren deze
11	Klant $\rightarrow$ Ober	Plaatsing bestelling.
12		
13	Ober $\rightarrow$ Kok	Plaatsing bestelling.
14	Kok $\rightarrow$ Keukenhulp	Is dit gerecht nog voorradig?
15	Keukenhulp $\rightarrow$ Kok	Ja/Nee
16	Kok $\rightarrow$ Ober	Akkoord/Niet Akkoord.
17		
18	Ober $\rightarrow$ Klant	Bevestiging bestelling/Bestelling niet mogelijk.

The following diagram shows all transactions and messages from the example in relation to each other. This diagram guides the test scenario for VISI-compatible software.

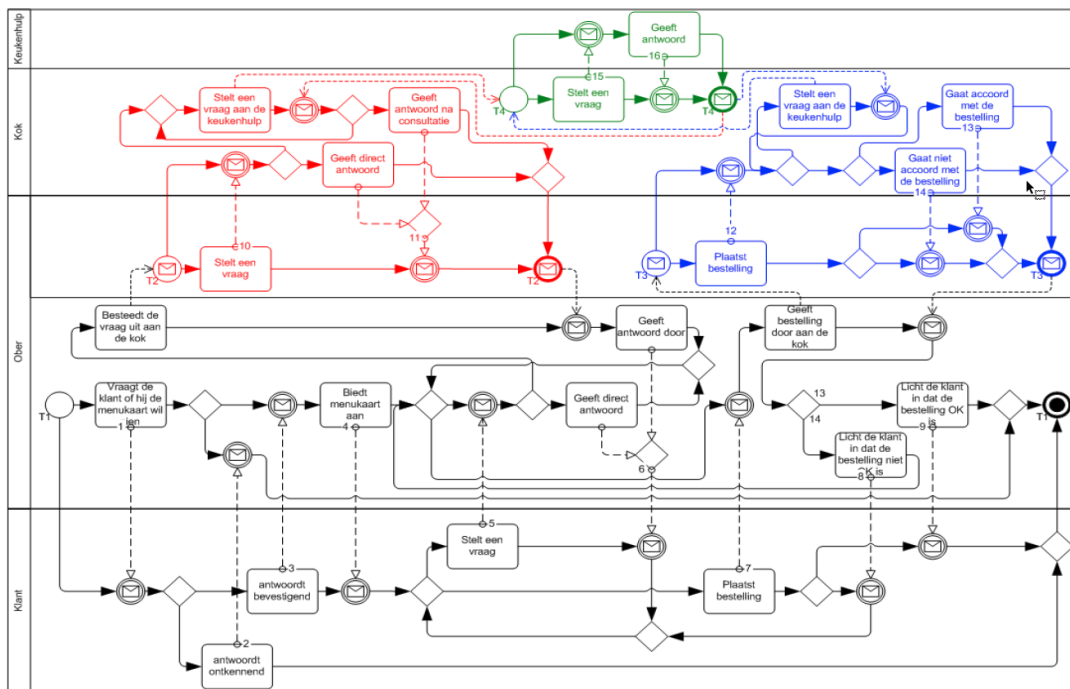


Figure 101: Complete restaurant workflow

8.2 Appendix 4 Detailed Implementation Example

8.2.1 Introduction

To properly understand the system, it is important to distinguish between:

1. the framework created (agreements regarding what communication is possible)
2. the project-specific message (the situation)
3. the messages (the communication itself)

This appendix is all about the development of a framework under the VISI system version 1.3.

A framework is made up of three important components:

- the transaction types
- the message types
- the simple elements

This applies to every VISI framework and therefore also to this framework. A digital version of the framework can be found in the file ' _7.xml'.

The system itself is laid out in two so-called Express schedules. These files have the extension .exp. The first file contains the express diagram of Systematics Part 1, in which the rules that specifically apply to VISI frameworks are laid down. A .xsd version that we can use when setting up a framework can be found in _3.xsd. The second file relates to Systematics Part 2. It contains the additional rules that are specifically important for VISI messages.

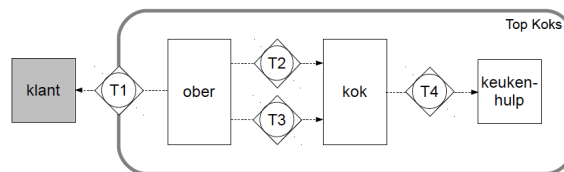


Figure 102:

8.2.2 Description of example (file “ 7.xml”)

8.2.2.1 Header, reference to the VISI System

The header of the framework refers to the VISI System in the form of an XML schema (.xsd extension), which forms the basic restriction of this and every other VISI-compliant framework.

```
1 <?xml version="1.0" encoding="UTF-8" standalone="no" ?> <visiXML_VISI_Systematics xmlns=
  "http://www.visi.nl/schemas/ 20160331">
```

8.2.2.2 ProjectType

The name of the project is recorded in the *ProjectType*. For example, if this concerns a framework to support land, road and hydraulic engineering projects based on administrative conditions such as the UAV or UAV-GC, this can be processed in the ID and description. In our case, it is a framework to support a number of operational processes in a restaurant.

```

1 <ProjectType id="ProjectType1">
2   <namespace>http://www.visi.nl/schemas/20160331/TestFramework</namespace>
3   <description>VISI Testscenario 's Raamwerk</description>
4   <startDate>2009-01-01T00:00:00</startDate>
5   <endDate>2010-01-01T00:00:00</endDate>
6   <state>active</state>
7   <dateLaMu>2009-01-28T00:00:00</dateLaMu>
8   <userLaMu>PWI</userLaMu>
9   <complexElements>
10    <ComplexElementTypeRef idref="CeSOAP"/>
11  </complexElements>
12 </ProjectType>

```

8.2.2.3 PersonType

In order to be able to define a transaction in the message later, we will now have to define a *PersonType*. A single *PersonType* is sufficient to create unlimited persons, but it is conceivable that there are different *PersonTypes*, each of which allows its own interpretation of specific personal data. Consider, for example, a categorisation into type: minor, adult and 65+. O.a. to different disciplines. In this simple example, we will limit ourselves to a *PersonType* and define it as simply as possible.

```

1 <PersonType id="StandardPersonType">
2   <description>Standard person type</description>
3   <startDate>2009-01-01T00:00:00</startDate>
4   <endDate>2010-01-01T00:00:00</endDate>
5   <state>active</state>
6   <dateLaMu>2009-01-28T00:00:00</dateLaMu>
7   <userLaMu>PWI</userLaMu>
8 </PersonType>

```

The id *StandardPersonType* will soon become a message-level object.

8.2.2.4 OrganisationType

To be able to define a transaction later, we will now have to define an *OrganisationType*. An *OrganisationType* is sufficient to create unlimited organisations, but it is conceivable that there are different *OrganisationTypes*, each of which allows its own interpretation of specific organisational data. Consider, for example, a private limited company, sole proprietorship or public law body. In this simple example, we will limit ourselves to an *OrganisationType* and define it as simply as possible.

```

1 <OrganisationType id="StandardOrganisationType">
2   <description>Standard organisation type</description>
3   <startDate>2009-01-01T00:00:00</startDate>
4   <endDate>2010-01-01T00:00:00</endDate>
5   <state>active</state>
6   <dateLaMu>2009-01-28T00:00:00</dateLaMu>
7   <userLaMu>PWI</userLaMu>
8   <complexElements>
9    <ComplexElementTypeRef idref="CeOrganisatie"/>
10  </complexElements>
11 </OrganisationType>

```

The id *StandardOrganisationType* will soon become a message-level object.

8.2.2.5 GroupType

In order to be able to define a transaction in the message, we will now have to define a *GroupType*. More complex examples will show that there is a lot of freedom within the system to define the message form. One *GroupType* is sufficient to create unlimited groups, but it is conceivable that there are different *GroupTypes*, each of which allows its own interpretation of specific group data. In this simple example, we will limit ourselves to a single *GroupType* and also define it as simply as possible.

```

1 <GroupType id="StandardGroupType">
2   <description>Standard Group Type</description>
3   <startDate>2009-01-01T00:00:00</startDate>
4   <endDate>2010-01-01T00:00:00</endDate>
5   <state>active</state>
6   <dateLaMu>2009-01-28T00:00:00</dateLaMu>
7   <userLaMu>PWI</userLaMu>
8 </GroupType>

```

The id *StandardGroupType* will soon become a message-level object.

8.2.2.6 RoleType

We will also have to define the roles that may appear in the transactions to be defined. In our case, we have four transactions involving four roles. In addition, the role of Cook is fulfilled by several people. The example contains four *RoleType*'s, namely:

1. Customer
2. Waiter
3. Cook
4. Kitchen helper

The roles are defined in the framework as follows.

```

1 <RoleType id="keukenhulp">
2   <description>Keukenhulp</description>
3   <startDate>2009-01-01T00:00:00</startDate>
4   <endDate>2010-01-01T00:00:00</endDate>
5   <state>active</state>
6   <dateLaMu>2009-02-02T00:00:00</dateLaMu>
7   <userLaMu>PWI</userLaMu>
8 </RoleType>
9
10 <RoleType id="klant">
11   <description>Klant</description>
12   <startDate>2009-01-01T00:00:00</startDate>
13   <endDate>2010-01-01T00:00:00</endDate>
14   <state>active</state>
15   <dateLaMu>2009-01-28T00:00:00</dateLaMu>
16   <userLaMu>PWI</userLaMu>
17 </RoleType>
18
19 <RoleType id="kok">
20   <description>Kok</description>
21   <startDate>2009-01-01T00:00:00</startDate>
22   <endDate>2010-01-01T00:00:00</endDate>
23   <state>active</state>
24   <dateLaMu>2009-01-31T00:00:00</dateLaMu>
25   <userLaMu>PWI</userLaMu>
26 </RoleType>
27
28 <RoleType id="ober">
29   <description>Ober</description>
30   <startDate>2009-01-01T00:00:00</startDate>
31   <endDate>2010-01-01T00:00:00</endDate>
32   <state>active</state>
33   <dateLaMu>2009-01-28T00:00:00</dateLaMu>
34   <userLaMu>PWI</userLaMu>
35 </RoleType>

```

We now have a role *customer*, a role *waiter*, a role *cook* and a role *kitchen assistant*.

8.2.2.7 TransactionPhaseType

The *TransactionPhaseType* is used to indicate in the *MessageInTransactionType* which phase a transaction is in. The following transaction states can currently be recognised and are used in most frameworks:

- Start
- Requested
- Promised/execution
- Change/hold
- Notification/ready
- Accepted/end

The *TransactionPhaseType*'s are defined as follows.

```
1 <TransactionPhaseType id="aanvaardEinde">
2   <description>Aanvaard/Einde</description>
3   <startDate>2008-05-04T00:00:00</startDate>
4   <endDate>2008-05-04T00:00:00</endDate>
5   <state>active</state>
6   <dateLaMu>2008-05-04T00:00:00</dateLaMu>
7   <userLaMu>MMA</userLaMu>
8 </TransactionPhaseType>
9
10 <TransactionPhaseType id="beloofdExecutie">
11   <description>Beloofd/Executie</description>
12   <startDate>2008-05-04T00:00:00</startDate>
13   <endDate>2008-05-04T00:00:00</endDate>
14   <state>active</state>
15   <dateLaMu>2008-05-04T00:00:00</dateLaMu>
16   <userLaMu>MMA</userLaMu>
17 </TransactionPhaseType>
18
19 <TransactionPhaseType id="meldingGereed">
20   <description>Melding Gereed</description>
21   <startDate>2008-05-04T00:00:00</startDate>
22   <endDate>2008-05-04T00:00:00</endDate>
23   <state>active</state>
24   <dateLaMu>2008-05-04T00:00:00</dateLaMu>
25   <userLaMu>MMA</userLaMu>
26 </TransactionPhaseType>
27
28 <TransactionPhaseType id="start">
29   <description>Start</description>
30   <startDate>2008-05-04T00:00:00</startDate>
31   <endDate>2008-05-04T00:00:00</endDate>
32   <state>active</state>
33   <dateLaMu>2008-05-04T00:00:00</dateLaMu>
34   <userLaMu>MMA</userLaMu>
35 </TransactionPhaseType>
36
37 <TransactionPhaseType id="verzocht">
38   <description>Verzocht</description>
39   <startDate>2008-05-04T00:00:00</startDate>
40   <endDate>2008-05-04T00:00:00</endDate>
41   <state>active</state>
42   <dateLaMu>2008-05-04T00:00:00</dateLaMu>
43   <userLaMu>MMA</userLaMu>
44 </TransactionPhaseType>
45
46 <TransactionPhaseType id="wijzigingHold">
47   <description>Wijziging/Hold</description>
48   <startDate>2008-05-04T00:00:00</startDate>
49   <endDate>2008-05-04T00:00:00</endDate>
50   <state>active</state>
51   <dateLaMu>2008-05-04T00:00:00</dateLaMu>
```

```
52 <userLaMu>MMA</userLaMu>
53 </TransactionPhaseType>
```

8.2.2.8 TransactionType

We will define four transactions based on the example. These transactions include the entire process from requesting the menu to confirmation that what has been ordered will be delivered. Within a transaction, we always indicate who starts it (the initiator) and who is being communicated with (the executor). A transaction, therefore, always takes place between two roles. Because more than two roles are involved in the process, it was decided to use three transactions, namely:

- T1 Order recording
- T2 Ask Waiter to Cook
- T3 Assignment Cook
- T4 Ask a Chef for Kitchen Help

The *TransactionType*'s are defined as follows.

```
1 <TransactionType id="t1_OpnameBestelling">
2   <description>T1 Opname van bestelling (ober aan klant)</description>
3   <startDate>2009-01-01T00:00:00</startDate>
4   <endDate>2010-01-01T00:00:00</endDate>
5   <state>active</state>
6   <dateLaMu>2011-10-26T00:00:00</dateLaMu>
7   <userLaMu>PWi</userLaMu>
8   <initiator>
9     <RoleTypeRef idref="ober"/>
10  </initiator>
11  <executor>
12    <RoleTypeRef idref="klant"/>
13  </executor>
14 </TransactionType>
15
16 <TransactionType id="t2_VraagOberAanKok">
17   <description>T2 Vraag (ober aan kok)</description>
18   <startDate>2009-01-01T00:00:00</startDate>
19   <endDate>2010-01-01T00:00:00</endDate>
20   <state>active</state>
21   <dateLaMu>2009-01-28T00:00:00</dateLaMu>
22   <userLaMu>PWi</userLaMu>
23   <initiator>
24     <RoleTypeRef idref="ober"/>
25   </initiator>
26   <executor>
27     <RoleTypeRef idref="kok"/>
28   </executor>
29 </TransactionType>
30
31 <TransactionType id="t3_OpdrachtKok">
32   <description>T3 Opdracht (ober aan kok)</description>
33   <startDate>2009-01-01T00:00:00</startDate>
34   <endDate>2010-01-01T00:00:00</endDate>
35   <state>active</state>
36   <dateLaMu>2009-01-31T00:00:00</dateLaMu>
37   <userLaMu>PWi</userLaMu>
38   <initiator>
39     <RoleTypeRef idref="ober"/>
40   </initiator>
41   <executor>
42     <RoleTypeRef idref="kok"/>
43   </executor>
44 </TransactionType>
45
46 <TransactionType id="t4_VraagKokAanKeukenhulp">
47   <description>T4 Vraag (kok aan keukenhulp)</description>
```

```

48     <startDate>2009-01-01T00:00:00 </startDate>
49     <endDate>2010-01-01T00:00:00 </endDate>
50     <state>active </state>
51     <dateLaMu>2011-10-26T00:00:00 </dateLaMu>
52     <userLaMu>PWi</userLaMu>
53     <initiator>
54         <RoleTypeRef idref="kok"/>
55     </initiator>
56     <executor>
57         <RoleTypeRef idref="keukenhulp"/>
58     </executor>
59     <appendixTypes>
60         <AppendixTypeRef idref="AppRapport"/>
61     </appendixTypes>
62 </TransactionType>

```

8.2.2.9 MessageType

The content and form of the content that the final messages will contain is defined within the *MessageType*. As can be seen in the transactions in the example, a significant number of messages are required. The messages and their format are defined below. In the same way as with *PersonType*, *OrganisationType*, *GroupType* and *AppendixType* (this is not used in this example), we can almost completely customise the form and content of the messages. Put our hand. In some messages, we have not come up with a self-defined form for the message; they are purely notifications.

```

1 <MessageType id="msgAanbiedingMenuKaart">
2     <description>Menukaart</description>
3     <startDate>2009-01-01T00:00:00 </startDate>
4     <endDate>2010-01-01T00:00:00 </endDate>
5     <state>active </state>
6     <dateLaMu>2009-01-29T00:00:00 </dateLaMu>
7     <userLaMu>PWi</userLaMu>
8     <appendixMandatory>true</appendixMandatory>
9     <complexElements>
10         <ComplexElementTypeRef idref="CeMenuKaart"/>
11         <ComplexElementTypeRef idref="CeDagMenu"/>
12         <ComplexElementTypeRef idref="CeDrankenKaart"/>
13         <ComplexElementTypeRef idref="CeMenuKaartType"/>
14     </complexElements>
15 </MessageType>
16
17 <MessageType id="msgAntwoord">
18     <description>Antwoord</description>
19     <startDate>2009-01-01T00:00:00 </startDate>
20     <endDate>2010-01-01T00:00:00 </endDate>
21     <state>active </state>
22     <dateLaMu>2011-10-26T00:00:00 </dateLaMu>
23     <userLaMu>PWi</userLaMu>
24     <complexElements>
25         <ComplexElementTypeRef idref="CeVraag"/>
26         <ComplexElementTypeRef idref="CeAntwoord"/>
27         <ComplexElementTypeRef idref="CeMenuKaart"/>
28     </complexElements>
29     <appendixTypes>
30         <AppendixTypeRef idref="AppTekening"/>
31     </appendixTypes>
32 </MessageType>
33
34 <MessageType id="msgAntwoordIsOvergenomen">
35     <description>Antwoord is overgenomen in antwoord aan ober</description>
36     <startDate>2009-01-01T00:00:00 </startDate>
37     <endDate>2010-01-01T00:00:00 </endDate>
38     <state>active </state>
39     <dateLaMu>2013-10-24T00:00:00 </dateLaMu>
40     <userLaMu>Arne</userLaMu>
41     <helpInfo>Dit bericht wordt alleen aangeboden als de kok een Antwoord naar de

```

```

42 ober gestuurd heeft.</helpInfo>
43     <complexContent>
44         <ComplexElementTypeRef idref="CeVraag"/>
45         <ComplexElementTypeRef idref="CeAntwoord"/>
46     </complexContent>
47 </MessageType>
48
49 <MessageType id="msgAntwoordWordtNietGebruikt">
50     <description>Antwoord wordt niet gebruikt</description>
51     <startDate>2009-01-01T00:00:00</startDate>
52     <endDate>2010-01-01T00:00:00</endDate>
53     <state>active</state>
54     <dateLaMu>2013-10-24T00:00:00</dateLaMu>
55     <userLaMu>Arne</userLaMu>
56     <helpInfo>Dit bericht wordt aangeboden zolang de kok nog niet gereageerd heeft
57 naar de ober. Hiermee sluit hij de transactie af en kan bijvoorbeeld een andere
58 keukenhulp inschakelen.</helpInfo>
59     <complexContent>
60         <ComplexElementTypeRef idref="CeVraag"/>
61         <ComplexElementTypeRef idref="CeAntwoord"/>
62     </complexContent>
63 </MessageType>
64
65 <MessageType id="msgBestellingAkkoord">
66     <description>Bestelling: akkoord</description>
67     <startDate>2009-01-01T00:00:00</startDate>
68     <endDate>2010-01-01T00:00:00</endDate>
69     <state>active</state>
70     <dateLaMu>2011-10-26T00:00:00</dateLaMu>
71     <userLaMu>PWi</userLaMu>
72     <complexContent>
73         <ComplexElementTypeRef idref="CeBestelling"/>
74     </complexContent>
75 </MessageType>
76
77 <MessageType id="msgBestellingIsAfgekeurdOpBasisVanAntwoord">
78     <description>Bestelling is afgekeurd op basis van antwoord</description>
79     <startDate>2009-01-01T00:00:00</startDate>
80     <endDate>2010-01-01T00:00:00</endDate>
81     <state>active</state>
82     <dateLaMu>2013-10-24T00:00:00</dateLaMu>
83     <userLaMu>Arne</userLaMu>
84     <helpInfo>Dit bericht wordt alleen aangeboden als de kok een Niet akkoord naar
85 de ober gestuurd heeft.</helpInfo>
86     <complexContent>
87         <ComplexElementTypeRef idref="CeVraag"/>
88         <ComplexElementTypeRef idref="CeAntwoord"/>
89     </complexContent>
90 </MessageType>
91
92 <MessageType id="msgBestellingIsGoedgekeurdOpBasisVanAntwoord">
93     <description>Bestelling is goedgekeurd op basis van antwoord</description>
94     <startDate>2009-01-01T00:00:00</startDate>
95     <endDate>2010-01-01T00:00:00</endDate>
96     <state>active</state>
97     <dateLaMu>2013-10-24T00:00:00</dateLaMu>
98     <userLaMu>Arne</userLaMu>
99     <helpInfo>Dit bericht wordt alleen aangeboden als de kok een Akkoord naar de
100 ober gestuurd heeft.</helpInfo>
101     <complexContent>
102         <ComplexElementTypeRef idref="CeVraag"/>
103         <ComplexElementTypeRef idref="CeAntwoord"/>
104     </complexContent>
105 </MessageType>
106
107 <MessageType id="msgBestellingNietAkkoord">
108     <description>Bestelling: niet akkoord</description>
109     <startDate>2009-01-01T00:00:00</startDate>
110     <endDate>2010-01-01T00:00:00</endDate>
111     <state>active</state>

```



```

112     <dateLaMu>2011-10-26T00:00:00 </dateLaMu>
113     <userLaMu>PWi</userLaMu>
114     <complexElements>
115         <ComplexElementTypeRef idref="CeBestelling"/>
116     </complexElements>
117 </MessageType>
118
119 <MessageType id="msgJa">
120     <description>Antwoord: Ja</description>
121     <startDate>2009-01-01T00:00:00 </startDate>
122     <endDate>2010-01-01T00:00:00 </endDate>
123     <state>active</state>
124     <dateLaMu>2009-03-16T00:00:00 </dateLaMu>
125     <userLaMu>PWi</userLaMu>
126     <complexElements>
127         <ComplexElementTypeRef idref="CeMenuKaartType"/>
128     </complexElements>
129 </MessageType>
130
131 <MessageType id="msgNee">
132     <description>Antwoord: Nee</description>
133     <startDate>2009-01-01T00:00:00 </startDate>
134     <endDate>2010-01-01T00:00:00 </endDate>
135     <state>active</state>
136     <dateLaMu>2009-01-29T00:00:00 </dateLaMu>
137     <userLaMu>PWi</userLaMu>
138 </MessageType>
139
140 <MessageType id="msgPlaatsingBestelling">
141     <description>Bestelling</description>
142     <startDate>2009-01-01T00:00:00 </startDate>
143     <endDate>2010-01-01T00:00:00 </endDate>
144     <state>active</state>
145     <dateLaMu>2009-01-31T00:00:00 </dateLaMu>
146     <userLaMu>PWi</userLaMu>
147     <appendixMandatory>false</appendixMandatory>
148     <complexElements>
149         <ComplexElementTypeRef idref="CeBestelling"/>
150     </complexElements>
151 </MessageType>
152
153 <MessageType id="msgVraag">
154     <description>Vraag</description>
155     <startDate>2009-01-01T00:00:00 </startDate>
156     <endDate>2010-01-01T00:00:00 </endDate>
157     <state>active</state>
158     <dateLaMu>2009-01-29T00:00:00 </dateLaMu>
159     <userLaMu>PWi</userLaMu>
160     <complexElements>
161         <ComplexElementTypeRef idref="CeVraag"/>
162         <ComplexElementTypeRef idref="CeMenuKaart"/>
163         <ComplexElementTypeRef idref="CeDagMenu"/>
164         <ComplexElementTypeRef idref="CeDrankenKaart"/>
165     </complexElements>
166 </MessageType>
167
168 <MessageType id="msgWiltuDeKaartZien">
169     <description>Wilt u de menukaart zien?</description>
170     <startDate>2009-01-01T00:00:00 </startDate>
171     <endDate>2010-01-01T00:00:00 </endDate>
172     <state>active</state>
173     <dateLaMu>2009-01-28T00:00:00 </dateLaMu>
174     <userLaMu>PWi</userLaMu>
175     <complexElements>
176         <ComplexElementTypeRef idref="CeReactieTermijn"/>
177         <ComplexElementTypeRef idref="CeBaseTypeTest"/>
178     </complexElements>
179 </MessageType>

```

8.2.2.10 MessageInTransactionType

This is a fairly complex object, which defines how the different MessageTypes are linked to the different TransactionTypes. It is possible to link a MessageType to multiple TransactionTypes and also vice versa: link the same MessageType to a TransactionType more than once.

If communication involves multiple transactions (and this communication must also remain traceable afterwards), then when drawing up the framework, account must be taken of the possibility that multiple instances of the second transaction occur during the transition from one transaction to another can be started. But in the event that the Waiter role places an order with several people in the Cook role with T2, we do not want to have to wait until all transactions T2 have been completed if we have already received a response from one of the people in the Cook role. that were initiated based on the msgQuestion message from T1. We even want to be able to continue with the primary transaction T1 without completing a secondary transaction. In this case, the OpenSecondaryTransactionsAllowed element should have no value. By including msgReply as MessageInTransaction6 with both MessageInTransaction5 and MessageInTransaction11 as previous, the primary transaction can continue as usual.

In the case where the Waiter initiates transaction T3 based on msgPlaceOrder from T1, we want at least one instance of T3 to be completed before sending the msgOrderAgree. To do this, we set the value true to OpenSecondaryTransactionsAllowed in MessageInTransactionType. With MessageInTransaction8 (Order not approved) it is a little more complicated. Assuming that the order has been placed with more than one chef, you could reason that if the first chef who responds does not agree, the waiter can wait to see whether one of the other chefs agrees. OpenSecondaryTransactionsAllowed = true also applies in that case.

Also note that for the first MessageInTransactionType T2 and T3 the previous MITT comes from T1. The previous field indicates which MITT should precede another MITT. This field is not linked to a specific transaction.

In the diagram, pay attention to the situation that occurs with MessageInTransaction6. Based on the Transaction Linking guideline that was implemented in version 1.2 of the system (see also the explanation in paragraph 1.2.1 of Appendix 5 to the Guidelines), MessageInTransaction6 can be sent even though MessageInTransaction10 has also been sent. The Waiter can therefore continue transaction T1 without completing transaction T2.

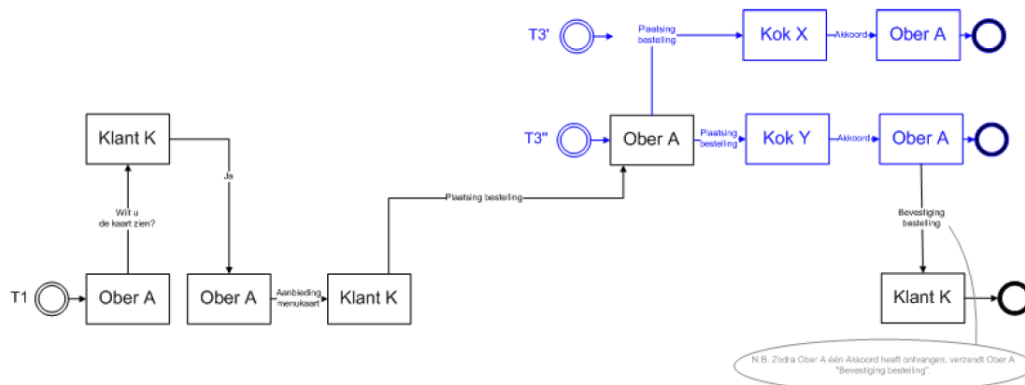


Figure 103:

```

1 <MessageInTransactionType id="BerichtInTransactie1">
2   <requiredNotify>0</requiredNotify>
3   <dateLaMu>2009-01-28T00:00:00</dateLaMu>
4   <userLaMu>PWI</userLaMu>
5   <received>true</received>
6   <send>true</send>
7   <state>active</state>
8   <initiatorToExecutor>true</initiatorToExecutor>
9   <openSecondaryTransactionsAllowed>false</openSecondaryTransactionsAllowed>
10  <firstMessage>true</firstMessage>
11  <message>
12    <MessageTypeRef idref="msgWiltuDeKaartZien"/>

```

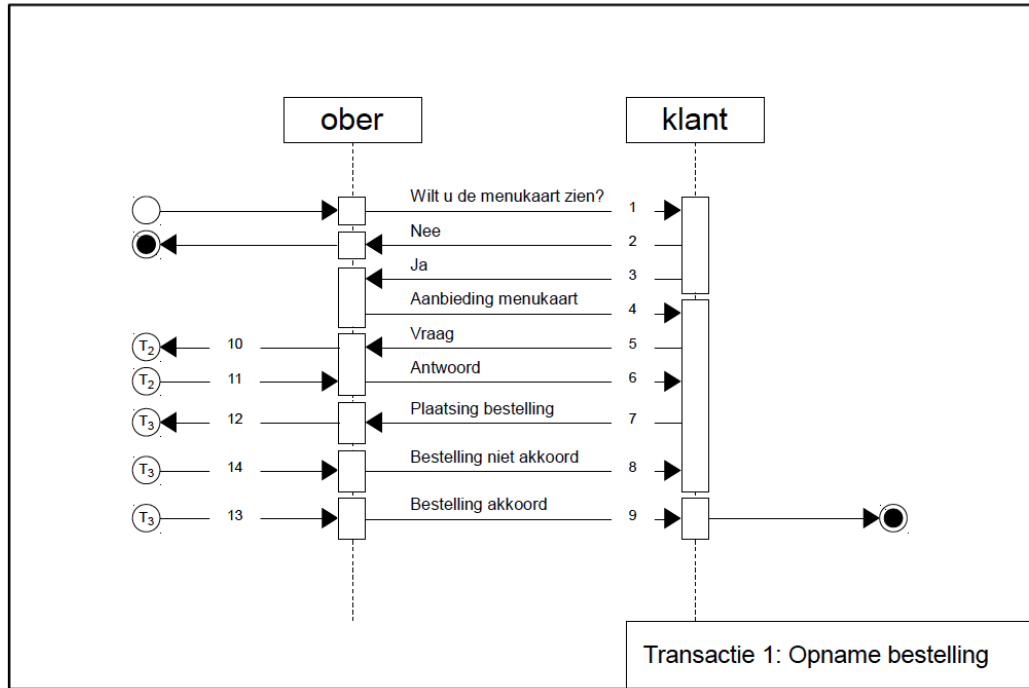


Figure 104:

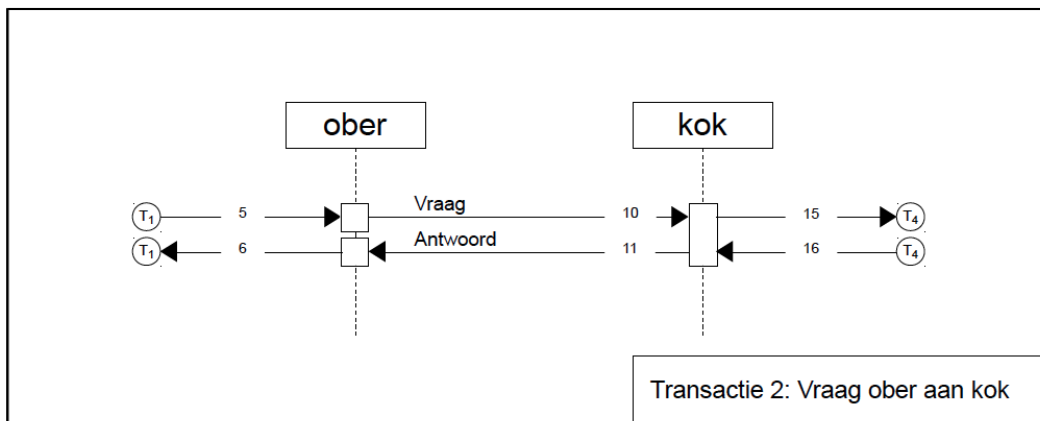


Figure 105:

```

13     </message>
14     <transaction>
15         <TransactionTypeRef idref="t1_OpnameBestelling"/>
16     </transaction>
17     <transactionPhase>
18         <TransactionPhaseTypeRef idref="start"/>
19     </transactionPhase>
20     <group>
21         <GroupTypeRef idref="StandardGroupType"/>
22     </group>
23 </MessageInTransactionType>
24
25 <MessageInTransactionType id="BerichtInTransactie10">
26     <requiredNotify>0</requiredNotify>
27     <dateLaMu>2009-01-31T00:00:00 </dateLaMu>

```

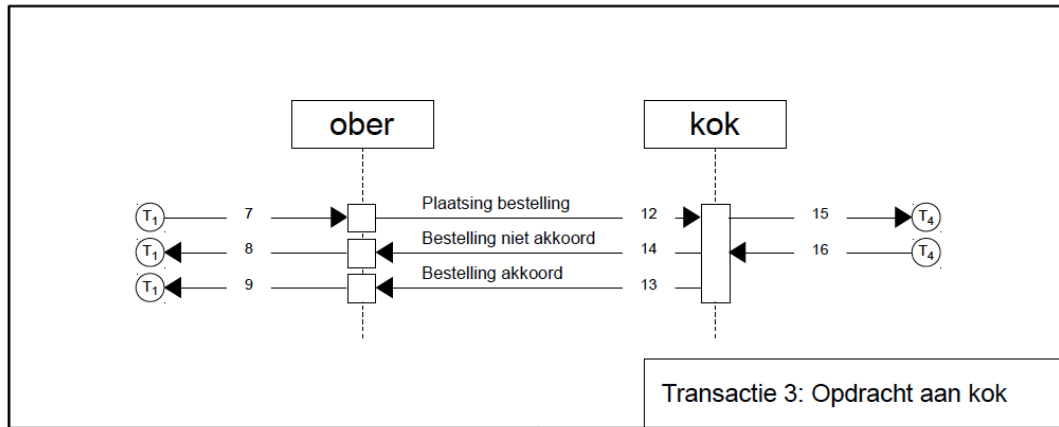


Figure 106:

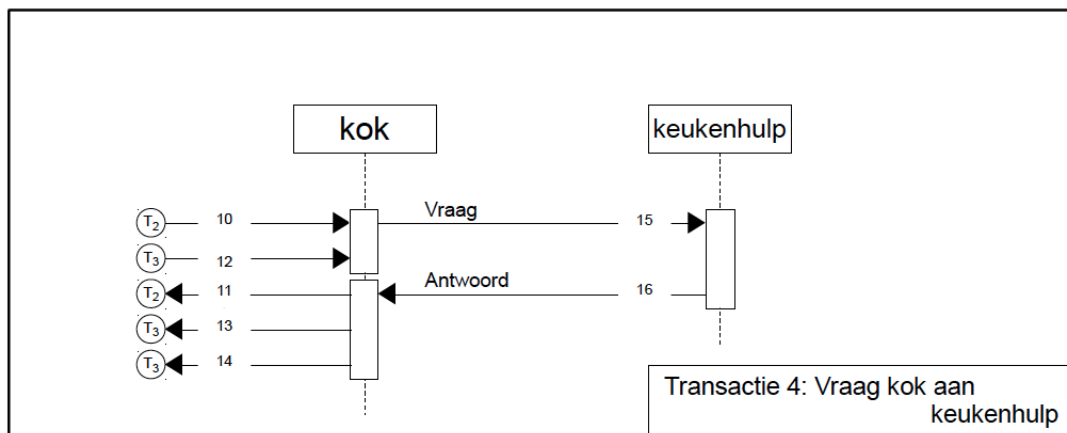


Figure 107:

```

28 <userLaMu>PWI</userLaMu>
29 <received>true</received>
30 <send>true</send>
31 <state>active</state>
32 <initiatorToExecutor>true</initiatorToExecutor>
33 <openSecondaryTransactionsAllowed>false</openSecondaryTransactionsAllowed>
34 <firstMessage>true</firstMessage>
35 <message>
36 <MessageTypeRef idref="msgVraag"/>
37 </message>
38 <previous>
39 <MessageInTransactionTypeRef idref="BerichtInTransactie5"/>
40 </previous>
41 <transaction>
42 <TransactionTypeRef idref="t2_VraagOberAanKok"/>
43 </transaction>
44 <transactionPhase>
45 <TransactionPhaseTypeRef idref="start"/>
46 </transactionPhase>
47 <group>
48 <GroupTypeRef idref="StandardGroupType"/>
49 </group>
50 </MessageInTransactionType>
51
52 <MessageInTransactionType id="BerichtInTransactie11">
53 <requiredNotify>0</requiredNotify>

```

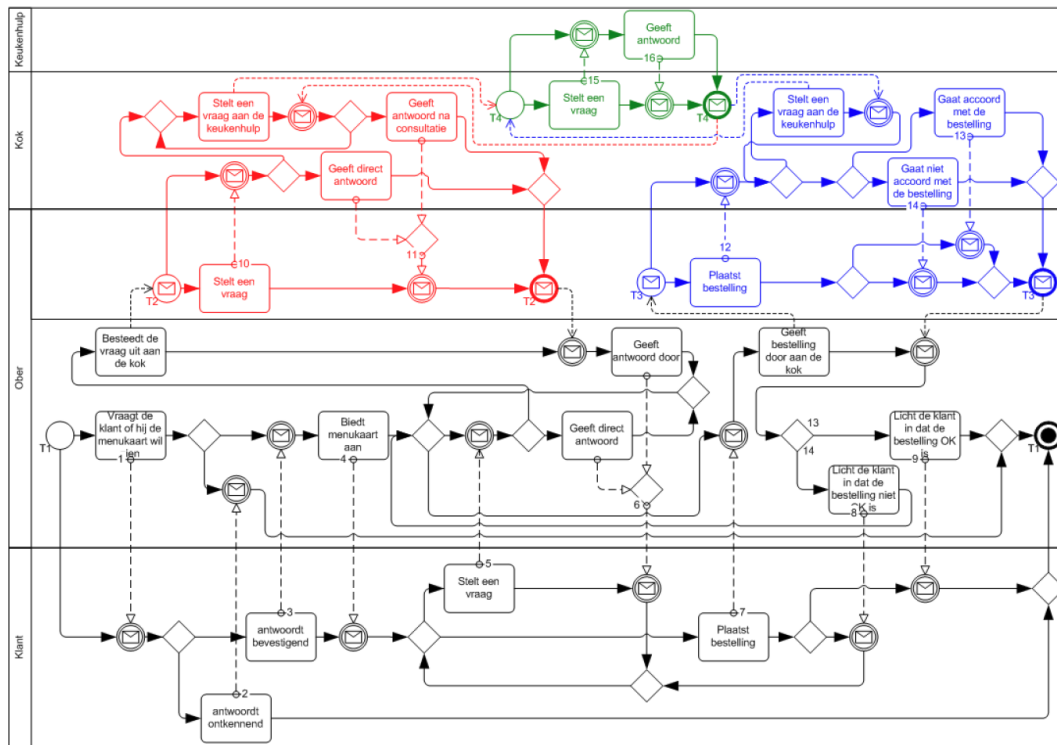


Figure 108:

```

54 <dateLaMu>2009-01-31T00:00:00 </dateLaMu>
55 <userLaMu>PWI</userLaMu>
56 <received>true</received>
57 <send>true</send>
58 <state>active</state>
59 <initiatorToExecutor>false</initiatorToExecutor>
60 <openSecondaryTransactionsAllowed>true</openSecondaryTransactionsAllowed>
61 <message>
62   <MessageTypeRef idref="msgAntwoord"/>
63 </message>
64 <previous>
65   <MessageInTransactionTypeRef idref="BerichtInTransactie10"/>
66   <MessageInTransactionTypeRef idref="BerichtInTransactie16"/>
67 </previous>
68 <transaction>
69   <TransactionTypeRef idref="t2_VraagOberAanKok"/>
70 </transaction>
71 <transactionPhase>
72   <TransactionPhaseTypeRef idref="aanvaardEinde"/>
73 </transactionPhase>
74 <group>
75   <GroupTypeRef idref="StandardGroupType"/>
76 </group>
77 </MessageInTransactionType>
78
79 <MessageInTransactionType id="BerichtInTransactie12">
80   <requiredNotify>0</requiredNotify>
81   <dateLaMu>2009-01-31T00:00:00 </dateLaMu>
82   <userLaMu>PWI</userLaMu>
83   <received>true</received>
84   <send>true</send>
85   <state>active</state>
86   <initiatorToExecutor>true</initiatorToExecutor>
87   <openSecondaryTransactionsAllowed>false</openSecondaryTransactionsAllowed>
88   <firstMessage>true</firstMessage>

```

```

89     <message>
90         <MessageTypeRef idref="msgPlaatsingBestelling"/>
91     </message>
92     <previous>
93         <MessageInTransactionTypeRef idref="BerichtInTransactie7"/>
94     </previous>
95     <transaction>
96         <TransactionTypeRef idref="t3_OpdrachtKok"/>
97     </transaction>
98     <transactionPhase>
99         <TransactionPhaseTypeRef idref="start"/>
100    </transactionPhase>
101    <group>
102        <GroupTypeRef idref="StandardGroupType"/>
103    </group>
104    <conditions>
105        <MessageInTransactionTypeConditionRef idref="MC_MaarEenBestellingToegestaan"/>
106    </conditions>
107 </MessageInTransactionType>
108
109 <MessageInTransactionType id="BerichtInTransactie13">
110     <requiredNotify>0</requiredNotify>
111     <dateLaMu>2009-01-31T00:00:00</dateLaMu>
112     <userLaMu>PWI</userLaMu>
113     <received>true</received>
114     <send>true</send>
115     <state>active</state>
116     <initiatorToExecutor>false</initiatorToExecutor>
117     <openSecondaryTransactionsAllowed>true</openSecondaryTransactionsAllowed>
118     <message>
119         <MessageTypeRef idref="msgBestellingAkkoord"/>
120     </message>
121     <previous>
122         <MessageInTransactionTypeRef idref="BerichtInTransactie12"/>
123         <MessageInTransactionTypeRef idref="BerichtInTransactie16"/>
124     </previous>
125     <transaction>
126         <TransactionTypeRef idref="t3_OpdrachtKok"/>
127     </transaction>
128     <transactionPhase>
129         <TransactionPhaseTypeRef idref="aanvaardEinde"/>
130     </transactionPhase>
131     <group>
132         <GroupTypeRef idref="StandardGroupType"/>
133     </group>
134 </MessageInTransactionType>
135
136 <MessageInTransactionType id="BerichtInTransactie14">
137     <requiredNotify>0</requiredNotify>
138     <dateLaMu>2009-01-31T00:00:00</dateLaMu>
139     <userLaMu>PWI</userLaMu>
140     <received>true</received>
141     <send>true</send>
142     <state>active</state>
143     <initiatorToExecutor>false</initiatorToExecutor>
144     <openSecondaryTransactionsAllowed>true</openSecondaryTransactionsAllowed>
145     <message>
146         <MessageTypeRef idref="msgBestellingNietAkkoord"/>
147     </message>
148     <previous>
149         <MessageInTransactionTypeRef idref="BerichtInTransactie12"/>
150         <MessageInTransactionTypeRef idref="BerichtInTransactie16"/>
151     </previous>
152     <transaction>
153         <TransactionTypeRef idref="t3_OpdrachtKok"/>
154     </transaction>
155     <transactionPhase>
156         <TransactionPhaseTypeRef idref="aanvaardEinde"/>
157     </transactionPhase>
158     <group>

```

```

159     <GroupTypeRef idref="StandardGroupType"/>
160 </group>
161 </MessageInTransactionType>
162
163 <MessageInTransactionType id="BerichtInTransactie15">
164     <requiredNotify>0</requiredNotify>
165     <dateLaMu>2009-02-02T00:00:00</dateLaMu>
166     <userLaMu>PWI</userLaMu>
167     <received>true</received>
168     <send>true</send>
169     <state>active</state>
170     <initiatorToExecutor>true</initiatorToExecutor>
171     <openSecondaryTransactionsAllowed>true</openSecondaryTransactionsAllowed>
172     <firstMessage>true</firstMessage>
173     <message>
174         <MessageTypeRef idref="msgVraag"/>
175     </message>
176     <previous>
177         <MessageInTransactionTypeRef idref="BerichtInTransactie10"/>
178         <MessageInTransactionTypeRef idref="BerichtInTransactie12"/>
179         <MessageInTransactionTypeRef idref="BerichtInTransactie16"/>
180     </previous>
181     <transaction>
182         <TransactionTypeRef idref="t4_VraagKokAanKeukenhulp"/>
183     </transaction>
184     <transactionPhase>
185         <TransactionPhaseTypeRef idref="start"/>
186     </transactionPhase>
187     <group>
188         <GroupTypeRef idref="StandardGroupType"/>
189     </group>
190     <conditions>
191         <MessageInTransactionTypeConditionRef idref="
192             MC_GeenVraagAanHulpAlsOberAntwoordHeeft"/>
193     </conditions>
194 </MessageInTransactionType>
195
196 <MessageInTransactionType id="BerichtInTransactie16">
197     <requiredNotify>0</requiredNotify>
198     <dateLaMu>2009-02-02T00:00:00</dateLaMu>
199     <userLaMu>PWI</userLaMu>
200     <received>true</received>
201     <send>true</send>
202     <state>active</state>
203     <initiatorToExecutor>false</initiatorToExecutor>
204     <openSecondaryTransactionsAllowed>false</openSecondaryTransactionsAllowed>
205     <message>
206         <MessageTypeRef idref="msgAntwoord"/>
207     </message>
208     <previous>
209         <MessageInTransactionTypeRef idref="BerichtInTransactie15"/>
210     </previous>
211     <transaction>
212         <TransactionTypeRef idref="t4_VraagKokAanKeukenhulp"/>
213     </transaction>
214     <transactionPhase>
215         <TransactionPhaseTypeRef idref="aanvaardEinde"/>
216     </transactionPhase>
217     <group>
218         <GroupTypeRef idref="StandardGroupType"/>
219     </group>
220 </MessageInTransactionType>
221
222 <MessageInTransactionType id="BerichtInTransactie17">
223     <requiredNotify>0</requiredNotify>
224     <received>false</received>
225     <send>false</send>
226     <state>active</state>
227     <initiatorToExecutor>true</initiatorToExecutor>
228     <message>

```

```

228     <MessageTypeRef idref="msgAntwoordIsOvergenomen"/>
229 </message>
230 <previous>
231     <MessageInTransactionTypeRef idref="BerichtInTransactie16"/>
232 </previous>
233 <transaction>
234     <TransactionTypeRef idref="t4_VraagKokAanKeukenhulp"/>
235 </transaction>
236 <group>
237     <GroupTypeRef idref="StandardGroupType"/>
238 </group>
239 <conditions>
240     <MessageInTransactionTypeConditionRef idref="MC_AntwoordIsVerstuurd"/>
241 </conditions>
242 </MessageInTransactionType>
243
244 <MessageInTransactionType id="BerichtInTransactie18">
245     <requiredNotify>0</requiredNotify>
246     <received>false</received>
247     <send>false</send>
248     <state>active</state>
249     <initiatorToExecutor>true</initiatorToExecutor>
250     <message>
251         <MessageTypeRef idref="msgAntwoordWordtNietGebruikt"/>
252     </message>
253     <previous>
254         <MessageInTransactionTypeRef idref="BerichtInTransactie16"/>
255     </previous>
256     <transaction>
257         <TransactionTypeRef idref="t4_VraagKokAanKeukenhulp"/>
258     </transaction>
259     <group>
260         <GroupTypeRef idref="StandardGroupType"/>
261     </group>
262     <conditions>
263         <MessageInTransactionTypeConditionRef idref="MC_AntwoordNietGebruikt"/>
264     </conditions>
265 </MessageInTransactionType>
266
267 <MessageInTransactionType id="BerichtInTransactie19">
268     <requiredNotify>0</requiredNotify>
269     <received>false</received>
270     <send>false</send>
271     <state>active</state>
272     <initiatorToExecutor>true</initiatorToExecutor>
273     <message>
274         <MessageTypeRef idref="msgBestellingIsAfgekeurdOpBasisVanAntwoord"/>
275     </message>
276     <previous>
277         <MessageInTransactionTypeRef idref="BerichtInTransactie16"/>
278     </previous>
279     <transaction>
280         <TransactionTypeRef idref="t4_VraagKokAanKeukenhulp"/>
281     </transaction>
282     <group>
283         <GroupTypeRef idref="StandardGroupType"/>
284     </group>
285     <conditions>
286         <MessageInTransactionTypeConditionRef idref="MC_AfkeuringIsVerstuurd"/>
287     </conditions>
288 </MessageInTransactionType>
289
290 <MessageInTransactionType id="BerichtInTransactie2">
291     <requiredNotify>0</requiredNotify>
292     <dateLaMu>2009-01-29T00:00:00</dateLaMu>
293     <userLaMu>PWI</userLaMu>
294     <received>true</received>
295     <send>true</send>
296     <state>active</state>
297     <initiatorToExecutor>false</initiatorToExecutor>

```



```

298     <openSecondaryTransactionsAllowed>false</openSecondaryTransactionsAllowed>
299     <message>
300         <MessageTypeRef idref="msgNee"/>
301     </message>
302     <previous>
303         <MessageInTransactionTypeRef idref="BerichtInTransactie1"/>
304     </previous>
305     <transaction>
306         <TransactionTypeRef idref="t1_OpnameBestelling"/>
307     </transaction>
308     <transactionPhase>
309         <TransactionPhaseTypeRef idref="aanvaardEinde"/>
310     </transactionPhase>
311     <group>
312         <GroupTypeRef idref="StandardGroupType"/>
313     </group>
314 </MessageInTransactionType>
315
316 <MessageInTransactionType id="BerichtInTransactie20">
317     <requiredNotify>0</requiredNotify>
318     <received>false</received>
319     <send>false</send>
320     <state>active</state>
321     <initiatorToExecutor>true</initiatorToExecutor>
322     <message>
323         <MessageTypeRef idref="msgBestellingIsGoedgekeurdOpBasisVanAntwoord"/>
324     </message>
325     <previous>
326         <MessageInTransactionTypeRef idref="BerichtInTransactie16"/>
327     </previous>
328     <transaction>
329         <TransactionTypeRef idref="t4_VraagKokAanKeukenhulp"/>
330     </transaction>
331     <group>
332         <GroupTypeRef idref="StandardGroupType"/>
333     </group>
334     <conditions>
335         <MessageInTransactionTypeConditionRef idref="MC_GoedkeuringIsVerstuurd"/>
336     </conditions>
337 </MessageInTransactionType>
338
339 <MessageInTransactionType id="BerichtInTransactie3">
340     <requiredNotify>0</requiredNotify>
341     <dateLaMu>2009-01-29T00:00:00</dateLaMu>
342     <userLaMu>PWI</userLaMu>
343     <received>true</received>
344     <send>true</send>
345     <state>active</state>
346     <initiatorToExecutor>false</initiatorToExecutor>
347     <openSecondaryTransactionsAllowed>false</openSecondaryTransactionsAllowed>
348     <firstMessage>false</firstMessage>
349     <message>
350         <MessageTypeRef idref="msgJa"/>
351     </message>
352     <previous>
353         <MessageInTransactionTypeRef idref="BerichtInTransactie1"/>
354     </previous>
355     <transaction>
356         <TransactionTypeRef idref="t1_OpnameBestelling"/>
357     </transaction>
358     <transactionPhase>
359         <TransactionPhaseTypeRef idref="verzocht"/>
360     </transactionPhase>
361     <group>
362         <GroupTypeRef idref="StandardGroupType"/>
363     </group>
364 </MessageInTransactionType>
365
366 <MessageInTransactionType id="BerichtInTransactie4">
367     <requiredNotify>0</requiredNotify>

```

```

368 <dateLaMu>2009-01-29T00:00:00 </dateLaMu>
369 <userLaMu>PWI</userLaMu>
370 <received>true</received>
371 <send>true</send>
372 <state>active</state>
373 <initiatorToExecutor>true</initiatorToExecutor>
374 <openSecondaryTransactionsAllowed>false</openSecondaryTransactionsAllowed>
375 <message>
376 <MessageTypeRef idref="msgAanbiedingMenuKaart"/>
377 </message>
378 <previous>
379 <MessageInTransactionTypeRef idref="BerichtInTransactie3"/>
380 </previous>
381 <transaction>
382 <TransactionTypeRef idref="t1_OpnameBestelling"/>
383 </transaction>
384 <transactionPhase>
385 <TransactionPhaseTypeRef idref="beloofdExecutie"/>
386 </transactionPhase>
387 <group>
388 <GroupTypeRef idref="StandardGroupType"/>
389 </group>
390 </MessageInTransactionType>
391
392 <MessageInTransactionType id="BerichtInTransactie5">
393 <requiredNotify>0</requiredNotify>
394 <dateLaMu>2009-01-29T00:00:00 </dateLaMu>
395 <userLaMu>PWI</userLaMu>
396 <received>true</received>
397 <send>true</send>
398 <state>active</state>
399 <initiatorToExecutor>false</initiatorToExecutor>
400 <openSecondaryTransactionsAllowed>false</openSecondaryTransactionsAllowed>
401 <message>
402 <MessageTypeRef idref="msgVraag"/>
403 </message>
404 <previous>
405 <MessageInTransactionTypeRef idref="BerichtInTransactie4"/>
406 <MessageInTransactionTypeRef idref="BerichtInTransactie6"/>
407 <MessageInTransactionTypeRef idref="BerichtInTransactie8"/>
408 </previous>
409 <transaction>
410 <TransactionTypeRef idref="t1_OpnameBestelling"/>
411 </transaction>
412 <transactionPhase>
413 <TransactionPhaseTypeRef idref="wijzigingHold"/>
414 </transactionPhase>
415 <group>
416 <GroupTypeRef idref="StandardGroupType"/>
417 </group>
418 </MessageInTransactionType>
419
420 <MessageInTransactionType id="BerichtInTransactie6">
421 <requiredNotify>0</requiredNotify>
422 <dateLaMu>2011-10-26T00:00:00 </dateLaMu>
423 <userLaMu>PWi</userLaMu>
424 <received>true</received>
425 <send>true</send>
426 <state>active</state>
427 <initiatorToExecutor>true</initiatorToExecutor>
428 <openSecondaryTransactionsAllowed>true</openSecondaryTransactionsAllowed>
429 <message>
430 <MessageTypeRef idref="msgAntwoord"/>
431 </message>
432 <previous>
433 <MessageInTransactionTypeRef idref="BerichtInTransactie5"/>
434 <MessageInTransactionTypeRef idref="BerichtInTransactie11"/>
435 </previous>
436 <transaction>
437 <TransactionTypeRef idref="t1_OpnameBestelling"/>

```

```

438     </transaction>
439     <transactionPhase>
440         <TransactionPhaseTypeRef idref="beloofdExecutie"/>
441     </transactionPhase>
442     <group>
443         <GroupTypeRef idref="StandardGroupType"/>
444     </group>
445     <appendixTypes>
446         <AppendixTypeRef idref="AppVideo"/>
447     </appendixTypes>
448 </MessageInTransactionType>
449
450 <MessageInTransactionType id="BerichtInTransactie7">
451     <requiredNotify>0</requiredNotify>
452     <dateLaMu>2009-01-31T00:00:00 </dateLaMu>
453     <userLaMu>PWI</userLaMu>
454     <received>true</received>
455     <send>true</send>
456     <state>active</state>
457     <initiatorToExecutor>false</initiatorToExecutor>
458     <openSecondaryTransactionsAllowed>false</openSecondaryTransactionsAllowed>
459     <message>
460         <MessageTypeRef idref="msgPlaatsingBestelling"/>
461     </message>
462     <previous>
463         <MessageInTransactionTypeRef idref="BerichtInTransactie4"/>
464         <MessageInTransactionTypeRef idref="BerichtInTransactie6"/>
465         <MessageInTransactionTypeRef idref="BerichtInTransactie8"/>
466     </previous>
467     <transaction>
468         <TransactionTypeRef idref="t1_OpnameBestelling"/>
469     </transaction>
470     <transactionPhase>
471         <TransactionPhaseTypeRef idref="meldingGereed"/>
472     </transactionPhase>
473     <group>
474         <GroupTypeRef idref="StandardGroupType"/>
475     </group>
476 </MessageInTransactionType>
477
478 <MessageInTransactionType id="BerichtInTransactie8">
479     <requiredNotify>0</requiredNotify>
480     <dateLaMu>2009-01-31T00:00:00 </dateLaMu>
481     <userLaMu>PWI</userLaMu>
482     <received>true</received>
483     <send>true</send>
484     <state>active</state>
485     <initiatorToExecutor>true</initiatorToExecutor>
486     <openSecondaryTransactionsAllowed>false</openSecondaryTransactionsAllowed>
487     <message>
488         <MessageTypeRef idref="msgBestellingNietAkkoord"/>
489     </message>
490     <previous>
491         <MessageInTransactionTypeRef idref="BerichtInTransactie14"/>
492     </previous>
493     <transaction>
494         <TransactionTypeRef idref="t1_OpnameBestelling"/>
495     </transaction>
496     <transactionPhase>
497         <TransactionPhaseTypeRef idref="beloofdExecutie"/>
498     </transactionPhase>
499     <group>
500         <GroupTypeRef idref="StandardGroupType"/>
501     </group>
502 </MessageInTransactionType>
503
504 <MessageInTransactionType id="BerichtInTransactie9">
505     <requiredNotify>0</requiredNotify>
506     <dateLaMu>2009-01-31T00:00:00 </dateLaMu>
507     <userLaMu>PWI</userLaMu>

```

```

508     <received>true</received>
509     <send>true</send>
510     <state>active</state>
511     <initiatorToExecutor>true</initiatorToExecutor>
512     <openSecondaryTransactionsAllowed>false</openSecondaryTransactionsAllowed>
513     <message>
514         <MessageTypeRef idref="msgBestellingAkkoord"/>
515     </message>
516     <previous>
517         <MessageInTransactionTypeRef idref="BerichtInTransactie13"/>
518     </previous>
519     <transaction>
520         <TransactionTypeRef idref="t1_OpnameBestelling"/>
521     </transaction>
522     <transactionPhase>
523         <TransactionPhaseTypeRef idref="aanvaardEinde"/>
524     </transactionPhase>
525     <group>
526         <GroupTypeRef idref="StandardGroupType"/>
527     </group>
528 </MessageInTransactionType>

```

8.2.2.11 MessageInTransactionTypeCondition

Further conditions may be imposed on the order in which messages are sent.

```

1 <MessageInTransactionTypeCondition id="MC_AfkeuringIsVerstuurd">
2     <sendAfter>
3         <MessageInTransactionTypeRef idref="BerichtInTransactie14"/>
4     </sendAfter>
5     <helpInfo>Melden dat een afkeuring van de bestelling is verstuurd op basis van
6 het antwoord mag alleen nadat die afkeuring verstuurd is.</helpInfo>
7 </MessageInTransactionTypeCondition>
8
9 <MessageInTransactionTypeCondition id="MC_AntwoordIsVerstuurd">
10     <sendAfter>
11         <MessageInTransactionTypeRef idref="BerichtInTransactie11"/>
12     </sendAfter>
13     <helpInfo>Antwoord is overgenomen mag alleen aangeboden worden nadat de kok een
14 antwoord terug aan de ober gegeven heeft.</helpInfo>
15 </MessageInTransactionTypeCondition>
16
17 <MessageInTransactionTypeCondition id="MC_AntwoordNietGebruikt">
18     <sendBefore>
19         <MessageInTransactionTypeRef idref="BerichtInTransactie14"/>
20         <MessageInTransactionTypeRef idref="BerichtInTransactie13"/>
21         <MessageInTransactionTypeRef idref="BerichtInTransactie11"/>
22     </sendBefore>
23     <helpInfo>Melden dat het antwoord niet gebruikt is, mag alleen maar gekozen
24 worden zolang er nog geen antwoord terug naar de ober gestuurd is.</helpInfo>
25 </MessageInTransactionTypeCondition>
26
27 <MessageInTransactionTypeCondition id="MC_GeenVraagAanHulpAlsOberAntwoordHeeft">
28     <sendBefore>
29         <MessageInTransactionTypeRef idref="BerichtInTransactie11"/>
30         <MessageInTransactionTypeRef idref="BerichtInTransactie13"/>
31         <MessageInTransactionTypeRef idref="BerichtInTransactie14"/>
32     </sendBefore>
33     <helpInfo>Testcase: Als de kok het antwoord op de bestelling of de vraag
34 gegeven heeft, mag er geen vraag meer naar de keukenhulp.</helpInfo>
35 </MessageInTransactionTypeCondition>
36
37 <MessageInTransactionTypeCondition id="MC_GoedkeuringIsVerstuurd">
38     <sendAfter>
39         <MessageInTransactionTypeRef idref="BerichtInTransactie13"/>
40     </sendAfter>
41     <helpInfo>Melden dat een goedkeuring van de bestelling is verstuurd op basis
42 van het antwoord mag alleen nadat die goedkeuring verstuurd is.</helpInfo>

```

```

43 </MessageInTransactionTypeCondition>
44
45 <MessageInTransactionTypeCondition id="MC_MaarEenBestellingToegestaan">
46   <sendBefore>
47     <MessageInTransactionTypeRef idref="BerichtInTransactie12"/>
48   </sendBefore>
49   <helpInfo>Testcase: als de klant een bestelling aan de ober geeft, mag er mmaar
50 1 bestelling naar de keuken gaan om dubbele bestellingen te voorkomen.</helpInfo>
51 </MessageInTransactionTypeCondition>

```

8.2.2.12 ComplexElementType

In the different messages, we want to enforce a specific XML form at the message level. Within this message, we have referenced a *ComplexElementType*. *ComplexElementTypes* are a grouping of any number of *SimpleElementTypes*. A *ComplexElementType* is a collection of elements (*SimpleElementType*'s). Each referenced element occurs exactly once. If we always want three of the same elements to be filled in, we can enforce this by including the reference to this *SimpleElementType* three times. In our example, we have included only one (but at least 1) of each *SimpleElementType* reference.

```

1 <ComplexElementType id="CeAntwoord">
2   <description>Antwoord</description>
3   <startDate>2009-01-01T00:00:00</startDate>
4   <endDate>2010-01-01T00:00:00</endDate>
5   <state>active</state>
6   <dateLaMu>2009-01-31T00:00:00</dateLaMu>
7   <userLaMu>PWI</userLaMu>
8   <simpleElements>
9     <SimpleElementTypeRef idref="Antwoord"/>
10  </simpleElements>
11 </ComplexElementType>
12
13 <ComplexElementType id="CeBaseTypeTest">
14   <description>Veldsoorten test</description>
15   <startDate>2013-10-25T00:00:00</startDate>
16   <endDate>2013-10-25T00:00:00</endDate>
17   <state>active</state>
18   <dateLaMu>2013-10-25T00:00:00</dateLaMu>
19   <userLaMu>Arne</userLaMu>
20   <simpleElements>
21     <SimpleElementTypeRef idref="SeBoolean"/>
22     <SimpleElementTypeRef idref="SeDate"/>
23     <SimpleElementTypeRef idref="SeDatetime"/>
24     <SimpleElementTypeRef idref="SeTime"/>
25     <SimpleElementTypeRef idref="SeDecimal"/>
26     <SimpleElementTypeRef idref="SeInteger"/>
27     <SimpleElementTypeRef idref="SeKeuzelijst"/>
28     <SimpleElementTypeRef idref="SeString"/>
29   </simpleElements>
30 </ComplexElementType>
31
32 <ComplexElementType id="CeBestelling">
33   <description>Bestelling</description>
34   <startDate>2009-01-01T00:00:00</startDate>
35   <endDate>2010-01-01T00:00:00</endDate>
36   <state>active</state>
37   <dateLaMu>2014-06-20T15:17:11</dateLaMu>
38   <userLaMu>Arne</userLaMu>
39   <helpInfo>de gerechten en drankjes worden in aparte tabellen ingevoerd, met per
40 regel de mogelijkheid een opmerking toe te voegen. Er is ook een totaalopmerking in
41 te voeren, als er iets bijzonders over bijvoorbeeld de manier van opdienen
42 ingevoerd moet worden.</helpInfo>
43   <complexElements>
44     <ComplexElementTypeRef idref="CeInhoudBestelling"/>
45     <ComplexElementTypeRef idref="CeInhoudDrankbestelling"/>
46   </complexElements>
47   <simpleElements>

```

```

48     <SimpleElementTypeRef idref="Opmerking"/>
49 </simpleElements>
50 </ComplexElementType>
51
52 <ComplexElementType id="CeDagMenu">
53     <description>Dagmenu</description>
54     <startDate>2009-01-01T00:00:00</startDate>
55     <endDate>2010-01-01T00:00:00</endDate>
56     <state>active</state>
57     <dateLaMu>2016-06-07T10:00:01</dateLaMu>
58     <userLaMu>Arne</userLaMu>
59     <complexElements>
60         <ComplexElementTypeRef idref="CeInhoudMenuKaart"/>
61     </complexElements>
62 </ComplexElementType>
63 <ComplexElementType id="CeDrankenKaart">
64     <description>Drankenkaart</description>
65     <startDate>2009-01-01T00:00:00</startDate>
66     <endDate>2010-01-01T00:00:00</endDate>
67     <state>active</state>
68     <dateLaMu>2016-06-07T10:00:21</dateLaMu>
69     <userLaMu>Arne</userLaMu>
70     <complexElements>
71         <ComplexElementTypeRef idref="CeInhoudMenuKaart"/>
72     </complexElements>
73 </ComplexElementType>
74
75 <ComplexElementType id="CeInhoudBestelling">
76     <description>Inhoud van de bestelling</description>
77     <startDate>2009-01-01T00:00:00</startDate>
78     <endDate>2010-01-01T00:00:00</endDate>
79     <state>active</state>
80     <dateLaMu>2009-01-31T00:00:00</dateLaMu>
81     <userLaMu>PWI</userLaMu>
82     <simpleElements>
83         <SimpleElementTypeRef idref="NaamGerecht"/>
84         <SimpleElementTypeRef idref="Opmerking"/>
85     </simpleElements>
86     <minOccurs>1</minOccurs>
87 </ComplexElementType>
88
89 <ComplexElementType id="CeInhoudDrankbestelling">
90     <description>Inhoud drankbestelling</description>
91     <startDate>2014-06-20T00:00:00</startDate>
92     <endDate>2014-06-20T00:00:00</endDate>
93     <state>active</state>
94     <dateLaMu>2014-06-20T15:11:28</dateLaMu>
95     <userLaMu>Arne</userLaMu>
96     <simpleElements>
97         <SimpleElementTypeRef idref="SeSoortDrank"/>
98         <SimpleElementTypeRef idref="SeHoeveelheid"/>
99         <SimpleElementTypeRef idref="Opmerking"/>
100     </simpleElements>
101     <minOccurs>1</minOccurs>
102     <maxOccurs>1</maxOccurs>
103 </ComplexElementType>
104
105 <ComplexElementType id="CeInhoudMenuKaart">
106     <description>Inhoud van de menukaart</description>
107     <startDate>2009-01-01T00:00:00</startDate>
108     <endDate>2010-01-01T00:00:00</endDate>
109     <state>active</state>
110     <dateLaMu>2009-01-31T00:00:00</dateLaMu>
111     <userLaMu>PWI</userLaMu>
112     <simpleElements>
113         <SimpleElementTypeRef idref="NaamMenukaartItem"/>
114         <SimpleElementTypeRef idref="Omschrijving"/>
115         <SimpleElementTypeRef idref="Prijs"/>
116     </simpleElements>
117     <maxOccurs>3</maxOccurs>

```

```

118 </ComplexElementType>
119
120 <ComplexElementType id="CeJaNee">
121   <description>Antwoord</description>
122   <startDate>2009-01-01T00:00:00</startDate>
123   <endDate>2010-01-01T00:00:00</endDate>
124   <state>active</state>
125   <dateLaMu>2009-01-29T00:00:00</dateLaMu>
126   <userLaMu>PWI</userLaMu>
127   <simpleElements>
128     <SimpleElementTypeRef idref="Wachttijd"/>
129     <SimpleElementTypeRef idref="JaNee"/>
130     <SimpleElementTypeRef idref="Opmerking"/>
131   </simpleElements>
132 </ComplexElementType>
133
134 <ComplexElementType id="CeMenuKaart">
135   <description>Menukaart</description>
136   <startDate>2009-01-01T00:00:00</startDate>
137   <endDate>2010-01-01T00:00:00</endDate>
138   <state>active</state>
139   <dateLaMu>2009-01-31T00:00:00</dateLaMu>
140   <userLaMu>PWI</userLaMu>
141   <complexElements>
142     <ComplexElementTypeRef idref="CeInhoudMenuKaart"/>
143   </complexElements>
144 </ComplexElementType>
145
146 <ComplexElementType id="CeMenuKaartType">
147   <description>MenukaartType</description>
148   <startDate>2009-01-01T00:00:00</startDate>
149   <endDate>2010-01-01T00:00:00</endDate>
150   <state>active</state>
151   <dateLaMu>2009-01-31T00:00:00</dateLaMu>
152   <userLaMu>PWI</userLaMu>
153   <simpleElements>
154     <SimpleElementTypeRef idref="MenuKaartType"/>
155   </simpleElements>
156 </ComplexElementType>
157
158 <ComplexElementType id="CeOrganisatie">
159   <description>Organisatie specs</description>
160   <startDate>2009-01-01T00:00:00</startDate>
161   <endDate>2010-01-01T00:00:00</endDate>
162   <state>active</state>
163   <dateLaMu>2011-10-26T00:00:00</dateLaMu>
164   <userLaMu>PWI</userLaMu>
165   <simpleElements>
166     <SimpleElementTypeRef idref="SOAPServerURL"/>
167   </simpleElements>
168 </ComplexElementType>
169
170 <ComplexElementType id="CeRapport">
171   <description>Rapport</description>
172   <startDate>2009-01-01T00:00:00</startDate>
173   <endDate>2010-01-01T00:00:00</endDate>
174   <state>active</state>
175   <dateLaMu>2009-02-03T00:00:00</dateLaMu>
176   <userLaMu>PWI</userLaMu>
177   <simpleElements>
178     <SimpleElementTypeRef idref="Auteur"/>
179     <SimpleElementTypeRef idref="Versie"/>
180   </simpleElements>
181 </ComplexElementType>
182
183 <ComplexElementType id="CeReactieTermijn">
184   <description>Reactietermijn</description>
185   <startDate>2009-01-01T00:00:00</startDate>
186   <endDate>2010-01-01T00:00:00</endDate>
187   <state>active</state>

```

```

188     <dateLaMu>2009-01-28T00:00:00 </dateLaMu>
189     <userLaMu>PWI</userLaMu>
190     <simpleElements>
191         <SimpleElementTypeRef idref="Wachtttijd"/>
192     </simpleElements>
193 </ComplexElementType>
194
195 <ComplexElementType id="CeSOAP">
196     <description>SOAP specificaties </description>
197     <startDate>2009-01-01T00:00:00 </startDate>
198     <endDate>2010-01-01T00:00:00 </endDate>
199     <state>active </state>
200     <dateLaMu>2011-10-26T00:00:00 </dateLaMu>
201     <userLaMu>PWI</userLaMu>
202     <simpleElements>
203         <SimpleElementTypeRef idref="SOAPProtocol"/>
204         <SimpleElementTypeRef idref="SOAPServerURL"/>
205         <SimpleElementTypeRef idref="SOAPCentralServerURL"/>
206     </simpleElements>
207 </ComplexElementType>
208
209 <ComplexElementType id="CeTekening">
210     <description>Tekening</description>
211     <startDate>2009-01-01T00:00:00 </startDate>
212     <endDate>2010-01-01T00:00:00 </endDate>
213     <state>active </state>
214     <dateLaMu>2009-02-03T00:00:00 </dateLaMu>
215     <userLaMu>PWI</userLaMu>
216     <simpleElements>
217         <SimpleElementTypeRef idref="AantalLagen"/>
218     </simpleElements>
219 </ComplexElementType>
220
221 <ComplexElementType id="CeVideo">
222     <description>Video</description>
223     <startDate>2009-01-01T00:00:00 </startDate>
224     <endDate>2010-01-01T00:00:00 </endDate>
225     <state>active </state>
226     <dateLaMu>2009-02-03T00:00:00 </dateLaMu>
227     <userLaMu>PWI</userLaMu>
228     <simpleElements>
229         <SimpleElementTypeRef idref="VideoFormaat"/>
230     </simpleElements>
231 </ComplexElementType>
232
233 <ComplexElementType id="CeVraag">
234     <description>Vraag</description>
235     <startDate>2009-01-01T00:00:00 </startDate>
236     <endDate>2010-01-01T00:00:00 </endDate>
237     <state>active </state>
238     <dateLaMu>2009-01-31T00:00:00 </dateLaMu>
239     <userLaMu>PWI</userLaMu>
240     <simpleElements>
241         <SimpleElementTypeRef idref="Vraag"/>
242     </simpleElements>
243 </ComplexElementType>

```

As can be seen, for example, with *CeMenuKaart*, it is possible to include a complex element in another complex element. This should be interpreted as a table where the simple elements from the linked complex element form the columns, and the values are included in the rows.

8.2.2.13 SimpleElementType

SimpleElementType is an element with a defined name, description and field formatting (for example, a string of maximum 9 characters or an integer). However, it can also contain a reference to a *ComplexElementType*, which means that at message level 0, an unlimited number of these *ComplexElementType* elements are present.

```

1 <SimpleElementType id="AantalLagen">
2   <description>Aantal lagen in de tekening</description>
3   <interfaceType></interfaceType>
4   <state>active</state>
5   <dateLaMu>2009-02-03T00:00:00</dateLaMu>
6   <userLaMu>PWI</userLaMu>
7   <userDefinedType>
8     <UserDefinedTypeRef idref="INTEGER"/>
9   </userDefinedType>
10 </SimpleElementType>
11
12 <SimpleElementType id="Antwoord">
13   <description>Antwoord</description>
14   <interfaceType></interfaceType>
15   <state>active</state>
16   <dateLaMu>2009-01-29T00:00:00</dateLaMu>
17   <userLaMu>PWI</userLaMu>
18   <userDefinedType>
19     <UserDefinedTypeRef idref="STRING"/>
20   </userDefinedType>
21 </SimpleElementType>
22
23 <SimpleElementType id="Auteur">
24   <description>Auteur</description>
25   <interfaceType></interfaceType>
26   <state>active</state>
27   <dateLaMu>2009-02-03T00:00:00</dateLaMu>
28   <userLaMu>PWI</userLaMu>
29   <userDefinedType>
30     <UserDefinedTypeRef idref="STRING"/>
31   </userDefinedType>
32 </SimpleElementType>
33
34 <SimpleElementType id="JaNee">
35   <description>Ja of Nee</description>
36   <interfaceType></interfaceType>
37   <state>active</state>
38   <dateLaMu>2009-01-29T00:00:00</dateLaMu>
39   <userLaMu>PWI</userLaMu>
40   <userDefinedType>
41     <UserDefinedTypeRef idref="BOOLEAN"/>
42   </userDefinedType>
43 </SimpleElementType>
44
45 <SimpleElementType id="MenuKaartType">
46   <description>Menukaarttype</description>
47   <interfaceType></interfaceType>
48   <state>active</state>
49   <dateLaMu>2009-02-02T00:00:00</dateLaMu>
50   <userLaMu>PWI</userLaMu>
51   <userDefinedType>
52     <UserDefinedTypeRef idref="STRING"/>
53   </userDefinedType>
54 </SimpleElementType>
55
56 <SimpleElementType id="NaamGerecht">
57   <description>Naam van het gerecht</description>
58   <interfaceType></interfaceType>
59   <state>active</state>
60   <dateLaMu>2009-01-29T00:00:00</dateLaMu>
61   <userLaMu>PWI</userLaMu>
62   <userDefinedType>
63     <UserDefinedTypeRef idref="STRING"/>
64   </userDefinedType>
65 </SimpleElementType>
66
67 <SimpleElementType id="NaamMenukaartItem">
68   <description>Naam van menukaartitem</description>
69   <interfaceType></interfaceType>

```

```

70     <state>active </state>
71     <dateLaMu>2009-01-29T00:00:00 </dateLaMu>
72     <userLaMu>PWI</userLaMu>
73     <userDefinedType>
74         <UserDefinedTypeRef idref="STRING"/>
75     </userDefinedType>
76 </SimpleElementType>
77
78 <SimpleElementType id="Omschrijving">
79     <description>Omschrijving</description>
80     <interfaceType></interfaceType>
81     <state>active</state>
82     <dateLaMu>2009-01-29T00:00:00 </dateLaMu>
83     <userLaMu>PWI</userLaMu>
84     <userDefinedType>
85         <UserDefinedTypeRef idref="STRING"/>
86     </userDefinedType>
87 </SimpleElementType>
88
89 <SimpleElementType id="Opmerking">
90     <description>Opmerking</description>
91     <interfaceType></interfaceType>
92     <state>active</state>
93     <dateLaMu>2009-01-31T00:00:00 </dateLaMu>
94     <userLaMu>PWI</userLaMu>
95     <userDefinedType>
96         <UserDefinedTypeRef idref="STRING"/>
97     </userDefinedType>
98 </SimpleElementType>
99
100 <SimpleElementType id="Prijs">
101     <description>Prijs</description>
102     <interfaceType></interfaceType>
103     <state>active</state>
104     <dateLaMu>2009-01-29T00:00:00 </dateLaMu>
105     <userLaMu>PWI</userLaMu>
106     <userDefinedType>
107         <UserDefinedTypeRef idref="STRING"/>
108     </userDefinedType>
109 </SimpleElementType>
110
111 <SimpleElementType id="SeBoolean">
112     <description>Boolean</description>
113     <interfaceType></interfaceType>
114     <state>active</state>
115     <dateLaMu>2013-10-25T00:00:00 </dateLaMu>
116     <userLaMu>Arne</userLaMu>
117     <userDefinedType>
118         <UserDefinedTypeRef idref="BOOLEAN"/>
119     </userDefinedType>
120 </SimpleElementType>
121
122 <SimpleElementType id="SeDate">
123     <description>Date/Datum</description>
124     <interfaceType></interfaceType>
125     <state>active</state>
126     <dateLaMu>2013-10-25T00:00:00 </dateLaMu>
127     <userLaMu>Arne</userLaMu>
128     <userDefinedType>
129         <UserDefinedTypeRef idref="DATE"/>
130     </userDefinedType>
131 </SimpleElementType>
132
133 <SimpleElementType id="SeDatetime">
134     <description>Datetime/Datum en tijd</description>
135     <interfaceType></interfaceType>
136     <state>active</state>
137     <dateLaMu>2013-10-25T00:00:00 </dateLaMu>
138     <userLaMu>Arne</userLaMu>
139     <helpInfo></helpInfo>

```

```

140     <userDefinedType>
141         <UserDefinedTypeRef idref="DATETIME"/>
142     </userDefinedType>
143 </SimpleElementType>
144
145 <SimpleElementType id="SeDecimal">
146     <description>Decimal/Decimaal</description>
147     <interfaceType></interfaceType>
148     <state>active</state>
149     <dateLaMu>2013-10-25T00:00:00</dateLaMu>
150     <userLaMu>Arne</userLaMu>
151     <helpInfo></helpInfo>
152     <userDefinedType>
153         <UserDefinedTypeRef idref="DECIMAL"/>
154     </userDefinedType>
155 </SimpleElementType>
156
157 <SimpleElementType id="SeHoeveelheid">
158     <description>Hoeveelheid</description>
159     <interfaceType></interfaceType>
160     <state>active</state>
161     <dateLaMu>2014-06-20T15:13:39</dateLaMu>
162     <userLaMu>Arne</userLaMu>
163     <helpInfo>De hoeveelheid drankjes die besteld zijn</helpInfo>
164     <userDefinedType>
165         <UserDefinedTypeRef idref="INTEGER"/>
166     </userDefinedType>
167 </SimpleElementType>
168
169 <SimpleElementType id="SeInteger">
170     <description>Integer/Geheel getal</description>
171     <interfaceType></interfaceType>
172     <state>active</state>
173     <dateLaMu>2013-10-25T00:00:00</dateLaMu>
174     <userLaMu>Arne</userLaMu>
175     <userDefinedType>
176         <UserDefinedTypeRef idref="INTEGER"/>
177     </userDefinedType>
178 </SimpleElementType>
179
180 <SimpleElementType id="SeKeuzelijst">
181     <description>Keuzelijst</description>
182     <interfaceType></interfaceType>
183     <state>active</state>
184     <dateLaMu>2013-10-25T00:00:00</dateLaMu>
185     <userLaMu>Arne</userLaMu>
186     <helpInfo></helpInfo>
187     <userDefinedType>
188         <UserDefinedTypeRef idref="Keuzelijst"/>
189     </userDefinedType>
190 </SimpleElementType>
191
192 <SimpleElementType id="SeSoortDrank">
193     <description>Soort drankje</description>
194     <interfaceType></interfaceType>
195     <state>active</state>
196     <dateLaMu>2014-06-20T15:12:24</dateLaMu>
197     <userLaMu>Arne</userLaMu>
198     <userDefinedType>
199         <UserDefinedTypeRef idref="STRING"/>
200     </userDefinedType>
201 </SimpleElementType>
202
203 <SimpleElementType id="SeString">
204     <description>String/Vrije tekst</description>
205     <interfaceType></interfaceType>
206     <state>active</state>
207     <dateLaMu>2013-10-25T00:00:00</dateLaMu>
208     <userLaMu>Arne</userLaMu>
209     <helpInfo></helpInfo>

```

```

210     <userDefinedType>
211         <UserDefinedTypeRef idref="STRING"/>
212     </userDefinedType>
213 </SimpleElementType>
214
215 <SimpleElementType id="SeTime">
216     <description>Time/Tijd</description>
217     <interfaceType></interfaceType>
218     <state>active</state>
219     <dateLaMu>2013-10-25T00:00:00</dateLaMu>
220     <userLaMu>Arne</userLaMu>
221     <helpInfo></helpInfo>
222     <userDefinedType>
223         <UserDefinedTypeRef idref="TIME"/>
224     </userDefinedType>
225 </SimpleElementType>
226
227 <SimpleElementType id="SOAPCentralServerURL">
228     <description>SOAP Central Server URL</description>
229     <interfaceType></interfaceType>
230     <state>active</state>
231     <dateLaMu>2009-03-16T00:00:00</dateLaMu>
232     <userLaMu>PWI</userLaMu>
233     <userDefinedType>
234         <UserDefinedTypeRef idref="STRING"/>
235     </userDefinedType>
236 </SimpleElementType>
237
238 <SimpleElementType id="SOAPProtocol">
239     <description>SOAP Protocol</description>
240     <interfaceType></interfaceType>
241     <state>active</state>
242     <dateLaMu>2009-03-16T00:00:00</dateLaMu>
243     <userLaMu>PWI</userLaMu>
244     <userDefinedType>
245         <UserDefinedTypeRef idref="STRING"/>
246     </userDefinedType>
247 </SimpleElementType>
248
249 <SimpleElementType id="SOAPServerURL">
250     <description>SOAP Server URL</description>
251     <interfaceType></interfaceType>
252     <state>active</state>
253     <dateLaMu>2009-03-16T00:00:00</dateLaMu>
254     <userLaMu>PWI</userLaMu>
255     <userDefinedType>
256         <UserDefinedTypeRef idref="STRING"/>
257     </userDefinedType>
258 </SimpleElementType>
259
260 <SimpleElementType id="Versie">
261     <description>Versienummer</description>
262     <interfaceType></interfaceType>
263     <state>active</state>
264     <dateLaMu>2009-02-03T00:00:00</dateLaMu>
265     <userLaMu>PWI</userLaMu>
266     <userDefinedType>
267         <UserDefinedTypeRef idref="INTEGER"/>
268     </userDefinedType>
269 </SimpleElementType>
270
271 <SimpleElementType id="VideoFormaat">
272     <description>Videoformaat</description>
273     <interfaceType></interfaceType>
274     <state>active</state>
275     <dateLaMu>2009-02-03T00:00:00</dateLaMu>
276     <userLaMu>PWI</userLaMu>
277     <userDefinedType>
278         <UserDefinedTypeRef idref="STRING"/>
279     </userDefinedType>

```

```

280 </SimpleElementType>
281
282 <SimpleElementType id="Vraag">
283   <description>Vraag</description>
284   <interfaceType></interfaceType>
285   <state>active</state>
286   <dateLaMu>2009-01-29T00:00:00</dateLaMu>
287   <userLaMu>PWI</userLaMu>
288   <userDefinedType>
289     <UserDefinedTypeRef idref="STRING"/>
290   </userDefinedType>
291 </SimpleElementType>
292
293 <SimpleElementType id="Wachttijd">
294   <description>Wachttijd (in seconden)</description>
295   <interfaceType></interfaceType>
296   <state>active</state>
297   <dateLaMu>2009-02-02T00:00:00</dateLaMu>
298   <userLaMu>PWI</userLaMu>
299   <userDefinedType>
300     <UserDefinedTypeRef idref="INTEGER"/>
301   </userDefinedType>
302 </SimpleElementType>

```

8.2.2.14 ElementCondition

With the *ElementCondition*, it is possible to indicate exceptions to the standard inheritance of simple elements (as defined in the guideline for reusing and blocking data elements) in consecutive messages. If the example indicates that an order is not possible, it is not desirable for the values in the next message to be fixed. For this, we create a *ElementCondition* where we state that for a certain *MessageInTransactionType*, the value of the *SimpleElements* has the condition *FREE* (editable). In our example, we do this at *MessageInTransaction9*. In this MITT, an order is placed for the first time (no value available yet), or after it has been announced that an order is not possible (values are available from the previous message, but must be editable).

```

1 <ElementCondition id="ElementCondition1">
2   <description>De bestelling moet gewijzigd kunnen worden na een negatieve
3   reactie van de kok.</description>
4   <condition>FREE</condition>
5   <messageInTransaction>
6     <MessageInTransactionTypeRef idref="BerichtInTransactie7"/>
7   </messageInTransaction>
8 </ElementCondition>
9
10 <ElementCondition id="ElementCondition2">
11   <description>Na een antwoord volgt mogelijk een nieuwe vraag.</description>
12   <condition>EMPTY</condition>
13   <simpleElement>
14     <SimpleElementTypeRef idref="Vraag"/>
15   </simpleElement>
16   <messageInTransaction>
17     <MessageInTransactionTypeRef idref="BerichtInTransactie5"/>
18   </messageInTransaction>
19 </ElementCondition>
20
21 <ElementCondition id="ElementCondition3">
22   <description>Wachttijd mag nooit worden overschreven.</description>
23   <condition>FIXED</condition>
24   <simpleElement>
25     <SimpleElementTypeRef idref="Wachttijd"/>
26   </simpleElement>
27 </ElementCondition>
28
29 <ElementCondition id="ElementCondition1vanScenario10">
30   <description>Free op Prijs SE in Child CE Inhoud van de menukaart</description>
31   <condition>FREE</condition>

```

```

32     <helpInfo/>
33     <complexElements>
34         <ComplexElementTypeRef idref="CeInhoudMenuKaart"/>
35     </complexElements>
36     <simpleElement>
37         <SimpleElementTypeRef idref="Prijs"/>
38     </simpleElement>
39 </ElementCondition>
40
41 <ElementCondition id="ElementCondition2vanScenario10">
42     <description>Free op Omschrijving SE in Parent CE Menukaart</description>
43     <condition>FREE</condition>
44     <helpInfo/>
45     <complexElements>
46         <ComplexElementTypeRef idref="CeMenuKaart"/>
47     </complexElements>
48     <simpleElement>
49         <SimpleElementTypeRef idref="Omschrijving"/>
50     </simpleElement>
51 </ElementCondition>
52
53 <ElementCondition id="ElementCondition3vanScenario10">
54     <description>Empty op Parent CE Dagmenu</description>
55     <condition>EMPTY</condition>
56     <helpInfo/>
57     <complexElements>
58         <ComplexElementTypeRef idref="CeDagMenu"/>
59     </complexElements>
60 </ElementCondition>
61
62 <ElementCondition id="ElementCondition4vanScenario10">
63     <description>Free op MITT, Parent CE, Child CE, en SE</description>
64     <condition>FREE</condition>
65     <helpInfo/>
66     <complexElements>
67         <ComplexElementTypeRef idref="CeDagMenu"/>
68         <ComplexElementTypeRef idref="CeInhoudMenuKaart"/>
69     </complexElements>
70     <simpleElement>
71         <SimpleElementTypeRef idref="NaamMenukaartItem"/>
72     </simpleElement>
73     <messageInTransaction>
74         <MessageInTransactionTypeRef idref="BerichtInTransactie5"/>
75     </messageInTransaction>
76 </ElementCondition>
77
78 <ElementCondition id="ElementCondition5vanScenario10">
79     <description>Free op Parent en Child CE</description>
80     <condition>FREE</condition>
81     <complexElements>
82         <ComplexElementTypeRef idref="CeDrankenKaart"/>
83         <ComplexElementTypeRef idref="CeInhoudMenuKaart"/>
84     </complexElements>
85 </ElementCondition>
86
87 <ElementCondition id="ElementCondition6vanScenario10">
88     <description>Fixed op parent en child CE en op SE prijs</description>
89     <condition>FIXED</condition>
90     <helpInfo/>
91     <complexElements>
92         <ComplexElementTypeRef idref="CeDrankenKaart"/>
93         <ComplexElementTypeRef idref="CeInhoudMenuKaart"/>
94     </complexElements>
95     <simpleElement>
96         <SimpleElementTypeRef idref="Prijs"/>
97     </simpleElement>
98 </ElementCondition>
99
100 <ElementCondition id="ElementCondition7vanScenario10">
101     <description>Foute element condition: Inhoud van de menukaart is child CE en

```

```

102 menukaart is parent CE. Deze EC heeft inhoud als parant gedefinieerd en menukaart
103 als child. Deze situatie bestaat niet in dit raamwerk.</description>
104 <condition>FREE</condition>
105 <helpInfo/>
106 <complexElements>
107     <ComplexElementTypeRef idref="CeInhoudMenuKaart"/>
108     <ComplexElementTypeRef idref="CeMenuKaart"/>
109 </complexElements>
110 <messageInTransaction>
111     <MessageInTransactionTypeRef idref="BerichtInTransactie6"/>
112 </messageInTransaction>
113 </ElementCondition>

```

8.2.2.15 UserDefinedType

UserDefinedType is used to indicate what the user can enter for a certain element, for example, a string or an integer. The restrictions are also determined, for example, a string of a maximum of 9 characters or an integer smaller than 1000. For the restrictions, all restrictions allowed by XML are possible.

```

1 <UserDefinedType id="BOOLEAN">
2     <description>BOOLEAN</description>
3     <state>active</state>
4     <dateLaMu>2009-01-29T00:00:00</dateLaMu>
5     <userLaMu>PWI</userLaMu>
6     <baseType>BOOLEAN</baseType>
7 </UserDefinedType>
8
9 <UserDefinedType id="DATE">
10     <description>DATE</description>
11     <state>active</state>
12     <dateLaMu>2013-10-25T00:00:00</dateLaMu>
13     <userLaMu>Arne</userLaMu>
14     <baseType>DATE</baseType>
15 </UserDefinedType>
16
17 <UserDefinedType id="DATETIME">
18     <description>DATETIME</description>
19     <state>active</state>
20     <dateLaMu>2013-10-25T00:00:00</dateLaMu>
21     <userLaMu>Arne</userLaMu>
22     <baseType>DATETIME</baseType>
23 </UserDefinedType>
24
25 <UserDefinedType id="DECIMAL">
26     <description>DECIMAL</description>
27     <state>active</state>
28     <dateLaMu>2013-10-25T00:00:00</dateLaMu>
29     <userLaMu>Arne</userLaMu>
30     <baseType>DECIMAL</baseType>
31 </UserDefinedType>
32
33 <UserDefinedType id="INTEGER">
34     <description>INTEGER</description>
35     <state>active</state>
36     <dateLaMu>2009-01-29T00:00:00</dateLaMu>
37     <userLaMu>PWI</userLaMu>
38     <baseType>INTEGER</baseType>
39 </UserDefinedType>
40
41 <UserDefinedType id="Keuzelijst">
42     <description>Keuzelijst</description>
43     <state>active</state>
44     <dateLaMu>2013-10-25T00:00:00</dateLaMu>
45     <userLaMu>Arne</userLaMu>
46     <baseType>STRING</baseType>
47     <xsdRestriction>\&lt;xs:enumeration value="Keuze 1"/&gt;\&lt;xs:enumeration value="
    Keuze 2"/&gt;\&lt;xs:enumeration value="Keuze 3"/&gt;</xsdRestriction>

```

```

48 </UserDefinedType>
49
50 <UserDefinedType id="STRING">
51   <description>STRING</description>
52   <state>active</state>
53   <dateLaMu>2009-01-29T00:00:00</dateLaMu>
54   <userLaMu>PWI</userLaMu>
55   <baseType>STRING</baseType>
56 </UserDefinedType>
57
58 <UserDefinedType id="TIME">
59   <description>TIME</description>
60   <state>active</state>
61   <dateLaMu>2013-10-25T00:00:00</dateLaMu>
62   <userLaMu>Arne</userLaMu>
63   <baseType>TIME</baseType>
64 </UserDefinedType>

```

8.2.3 Elaboration of messages

This section focuses on drawing up the VISI messages. The VISI framework from section 1.1 describes the messages that can be used in a project. The VISI messages will have to comply with a message scheme based on the framework. The message schedule can be generated automatically using a program called 'Promoter'. The Promoter uses a VISI framework and the VISI system Part 1 and Part 2 as input and produces a schedule (XSD) in which all possible messages, including accessories, are recorded. The diagram in the following figure shows the role of the Promoter in the creation of VISI messages.

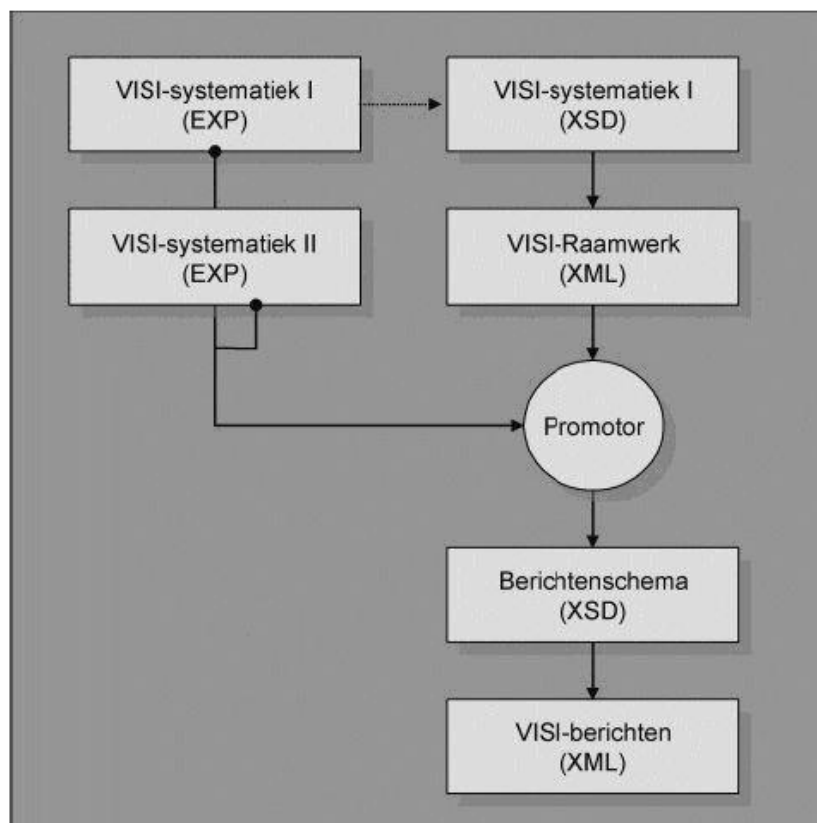


Figure 109:

The Promoter is made freely available in the form of a Dynamically Linked Library (DLL), which is a

library with functions that can be used by multiple applications. This makes it possible to integrate the Promoter into other software. The files and additional documentation can be downloaded from the VISI website www.crow.nl/visi.

By promoting the framework, the message schedules are recorded in an xsd file (10.xsd). The messages we will create in this section must comply with these schedules. First, a project-specific message is drawn up. A scenario with sixteen messages is then worked through.

8.2.3.1 The project specific message

The project-specific message places the framework in the context of a project. In the project-specific message, people are linked to roles, and the specific name is given to the project. In addition, the SOAP protocol used is indicated and the IP address of the SOAP central server; see also Appendix 8 (SOAP protocol) to the Guidelines.

In version 1.2 of the system, the option has also been added to indicate a successor for someone who leaves during the project. In practice, personnel changes appear to occur regularly in projects. During a personnel change, transactions remain open. By specifying a so-called successor, this person takes over the transactions of his/her predecessor.

The following individuals will be associated with the roles below:

Kees de Vries → Customer
Piet Jansen → Waiter
Dirk Zwart → Cook
Rob Taal → Cook
Jan Toet → Cook
Pietje Puk → Kitchen helper

To get an idea of how a *successor* should be indicated in the project-specific message, two versions of the project-specific message have been drawn up. In the first project-specific message (project_specific_message_to_and_with there is no mention of a successor. In the second version, it is, and the commented-out element can be activated.

Projectspecifiek bericht 1: “project_specifiek_bericht_tot_en_met_bericht_6.xml”.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework"
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4     .visi.nl/schemas/20160331/TestFramework10.xsd">
5   <ProjectType1 id="Project1">
6     <name>Project Top Koks</name>
7     <description>Project voor het opnemen van een bestelling</description>
8     <startDate>2008-05-04T00:00:00.0Z</startDate>
9     <endDate>2009-05-04T00:00:00.0Z</endDate>
10    <ceSOAP id="ProjectGegevens">
11      <sOAPProtocol>MTOM</sOAPProtocol>
12      <sOAPServerURL>https://192.168.0.1/visi.wsdl</sOAPServerURL>
13      <sOAPCentralServerURL>https://www.crow.nl/testcases/case001/visi.wsdl</
14      sOAPCentralServerURL>
15    </ceSOAP>
16  </ProjectType1>
17  <StandardOrganisationType id="consument">
18    <name>Consument</name>
19    <abbreviation>CSMT</abbreviation>
20    <contactPerson>
21      <StandardPersonTypeRef idref="KeesDeVries" />
22    </contactPerson>
23    <ceOrganisatie>
24      <CeOrganisatie id="OrganisatieGegevensConsument">
```

```

25         <sOAPServerURL>https://192.168.0.102/specifiek_project/visi.wsd</
sOAPServerURL>
26         </CeOrganisatie>
27     </ceOrganisatie>
28 </StandardOrganisationType>
29 <StandardOrganisationType id="restaurant">
30     <name>Restaurant</name>
31     <abbreviation>RSTR</abbreviation>
32     <contactPerson>
33         <StandardPersonTypeRef idref="PietJansen" />
34     </contactPerson>
35     <ceOrganisatie>
36         <CeOrganisatie id="OrganisatieGegevensRestaurant">
37             <sOAPServerURL>https://192.168.0.105/specifiek_project/visi.wsd</
sOAPServerURL>
38             </CeOrganisatie>
39         </ceOrganisatie>
40     </StandardOrganisationType>
41 <StandardPersonType id="KeesDeVries">
42     <userName>kvd</userName>
43     <name>Kees de Vries</name>
44 </StandardPersonType>
45 <StandardPersonType id="PietJansen">
46     <userName>pj</userName>
47     <name>Piet Jansen</name>
48 </StandardPersonType>
49 <StandardPersonType id="DirkZwart">
50     <userName>dz</userName>
51     <name>Dirk Zwart</name>
52 </StandardPersonType>
53 <StandardPersonType id="RobTaal">
54     <userName>rt</userName>
55     <name>Rob Taal</name>
56 </StandardPersonType>
57 <klant id="klantRol">
58     <name>Klant Rol</name>
59     <description>Diegene die de klant is</description>
60 </klant>
61 <ober id="oberRol">
62     <name>Ober Rol</name>
63     <description>Diegene die serveert</description>
64 </ober>
65 <kok id="kokRol">
66     <name>Kok Rol</name>
67     <description>Diegene die kookt</description>
68 </kok>
69 <PersonInRole id="PiR001">
70     <contactPerson>
71         <StandardPersonTypeRef idref="KeesDeVries" />
72     </contactPerson>
73     <organisation>
74         <StandardOrganisationTypeRef idref="consument" />
75     </organisation>
76     <role>
77         <klantRef idref="klantRol" />
78     </role>
79 </PersonInRole>
80 <PersonInRole id="PiR002">
81     <contactPerson>
82         <StandardPersonTypeRef idref="PietJansen" />
83     </contactPerson>
84     <organisation>
85         <StandardOrganisationTypeRef idref="restaurant" />
86     </organisation>
87     <role>
88         <klantRef idref="oberRol" />
89     </role>
90 </PersonInRole>
91 <PersonInRole id="PiR003">
92     <contactPerson>

```

```

93         <StandardPersonTypeRef idref="DirkZwart" />
94     </contactPerson>
95     <organisation>
96         <StandardOrganisationTypeRef idref="restaurant" />
97     </organisation>
98     <role>
99         <klantRef idref="kokRol" />
100     </role>
101 </PersonInRole>
102 <PersonInRole id="PiR004">
103     <substituting>
104         <PersonInRoleRef idref="PiR003" />
105     </substituting>
106     <contactPerson>
107         <StandardPersonTypeRef idref="RobTaal" />
108     </contactPerson>
109     <organisation>
110         <StandardOrganisationTypeRef idref="restaurant" />
111     </organisation>
112     <role>
113         <klantRef idref="kokRol" />
114     </role>
115 </PersonInRole>
116 </visiXML_MessageSchema>

```

Projectspecifiek bericht 2: "project_specifiek_bericht_vanaf_bericht_7.xml".

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework"
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4     .visi.nl/schemas/20160331/TestFramework10.xsd">
5     <ProjectType1 id="Project1">
6         <name>Project Top Koks</name>
7         <description>Project voor het opnemen van een bestelling</description>
8         <startDate>2008-05-04T00:00:00.0Z</startDate>
9         <endDate>2009-05-04T00:00:00.0Z</endDate>
10        <ceSOAP>
11            <CeSOAP id="ProjectGegevens">
12                <sOAPPProtocol>MITOM</sOAPPProtocol>
13                <sOAPServerURL>https://192.168.0.1/visi.wsdl</sOAPServerURL>
14                <sOAPCentralServerURL>https://www.crow.nl/testcases/case001/visi.wsdl</
15                sOAPCentralServerURL>
16            </CeSOAP>
17        </ceSOAP>
18    </ProjectType1>
19    <StandardOrganisationType id="consument">
20        <name>Consument</name>
21        <abbreviation>CSMT</abbreviation>
22        <contactPerson>
23            <StandardPersonTypeRef idref="KeesDeVries" />
24        </contactPerson>
25        <ceOrganisatie>
26            <CeOrganisatie id="OrganisatieGegevensConsument">
27                <sOAPServerURL>https://192.168.0.102/specifiek_project/visi.wsdl</
28                sOAPServerURL>
29            </CeOrganisatie>
30        </ceOrganisatie>
31    </StandardOrganisationType>
32    <StandardOrganisationType id="restaurant">
33        <name>Restaurant</name>
34        <abbreviation>RSTR</abbreviation>
35        <contactPerson>
36            <StandardPersonTypeRef idref="PietJansen" />
37        </contactPerson>
38        <ceOrganisatie>
39            <CeOrganisatie id="OrganisatieGegevensRestaurant">
40                <sOAPServerURL>https://192.168.0.105/specifiek_project/visi.wsdl</
41                sOAPServerURL>
42            </CeOrganisatie>

```

```

39     </ceOrganisatie>
40 </StandardOrganisationType>
41 <StandardPersonType id="KeesDeVries">
42     <userName>kvd</userName>
43     <name>Kees de Vries</name>
44 </StandardPersonType>
45 <StandardPersonType id="PietJansen">
46     <userName>pj</userName>
47     <name>Piet Jansen</name>
48 </StandardPersonType>
49 <StandardPersonType id="DirkZwart">
50     <userName>dz</userName>
51     <name>Dirk Zwart</name>
52 </StandardPersonType>
53 <StandardPersonType id="RobTaal">
54     <userName>rt</userName>
55     <name>Rob Taal</name>
56 </StandardPersonType>
57 <StandardPersonType id="JanToet">
58     <userName>jt</userName>
59     <name>Jan Toet</name>
60 </StandardPersonType>
61 <klant id="klantRol">
62     <name>Klant Rol</name>
63     <description>Diegene die de klant is</description>
64 </klant>
65 <ober id="oberRol">
66     <name>Ober Rol</name>
67     <description>Diegene die serveert</description>
68 </ober>
69 <kok id="kokRol">
70     <name>Kok Rol</name>
71     <description>Diegene die kookt</description>
72 </kok>
73 <PersonInRole id="PiR001">
74     <contactPerson>
75         <StandardPersonTypeRef idref="KeesDeVries" />
76     </contactPerson>
77     <organisation>
78         <StandardOrganisationTypeRef idref="consument" />
79     </organisation>
80     <role>
81         <klantRef idref="klantRol" />
82     </role>
83 </PersonInRole>
84 <PersonInRole id="PiR002">
85     <contactPerson>
86         <StandardPersonTypeRef idref="PietJansen" />
87     </contactPerson>
88     <organisation>
89         <StandardOrganisationTypeRef idref="restaurant" />
90     </organisation>
91     <role>
92         <klantRef idref="oberRol" />
93     </role>
94 </PersonInRole>
95 <PersonInRole id="PiR003">
96     <contactPerson>
97         <StandardPersonTypeRef idref="DirkZwart" />
98     </contactPerson>
99     <organisation>
100         <StandardOrganisationTypeRef idref="restaurant" />
101     </organisation>
102     <role>
103         <klantRef idref="kokRol" />
104     </role>
105 </PersonInRole>
106 <PersonInRole id="PiR004">
107     <successor>
108         <PersonInRoleRef idref="PiR005" />

```

```

109     </successor>
110     <contactPerson>
111         <StandardPersonTypeRef idref="RobTaal" />
112     </contactPerson>
113     <organisation>
114         <StandardOrganisationTypeRef idref="restaurant" />
115     </organisation>
116     <role>
117         <klantRef idref="kokRol" />
118     </role>
119 </PersonInRole>
120 <PersonInRole id="PiR005">
121     <contactPerson>
122         <StandardPersonTypeRef idref="JanToet" />
123     </contactPerson>
124     <organisation>
125
126         <StandardOrganisationTypeRef idref="restaurant" />
127     </organisation>
128     <role>
129         <klantRef idref="kokRol" />
130     </role>
131 </PersonInRole>
132 </visiXML_MessageSchema>

```

8.2.3.2 Compose messages

In this section, a scenario is worked through step by step, and the associated messages are created. The scenario discusses how to deal with messages that previously have an MITT from another transaction. Matters such as successor and element condition are also discussed. It may be useful to keep the diagram from Chapter 1 (Figure 1) in mind.

8.2.3.3 2.2.1. The first message ("eerste_bericht.xml")

The framework stipulates that the first message concerns the question from the waiter role to the customer role, whether the customer wants to see the menu. This is a very simple message that does not contain any complex elements yet. In the first message, an additional explanation is given for each part that makes up the message.

Header, reference to the message schema (promoted framework).

The message header refers to the message schema, which forms the basic restriction of this and every other message according to the defined framework. The receiving software will now have to retrieve the associated framework based on this header or check whether this framework is already present in its cache.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:
  schemaLocation="http://www.visi.nl/schemas/ 20160331 /TestFramework 10.xsd" xmlns="
  http://www.visi.nl/schemas/ 20160331 /TestFramework">

```

The message

We then define the first message within the transaction. Within the framework, multiple messages are defined for the relevant transaction. The messages are linked to transaction *T1Recording* order by means of MITT type. The message that starts with is *msgWiltuDeKaartZien*. This message has no previous message via the MITT.

```

1 <msgWiltuDeKaartZien id="bericht001">
2     <identification />
3     <dateSend>2008-05-04T00:00:00.0Z</dateSend>

```

```

4      <initiatorToExecutor>true</initiatorToExecutor>
5      <messageInTransaction>
6          <BerichtInTransactie1Ref idref="BiT001" />
7      </messageInTransaction>
8      <transaction>
9          <t1_OpnameBestellingRef idref="transactie001" />
10     </transaction>
11     <ceReactieTermijn>
12         <CeReactieTermijnRef idref="ce001a" />
13     </ceReactieTermijn>
14     <ceBaseTypeTest>
15         <CeBaseTypeTestRef idref="ce001b" />
16     </ceBaseTypeTest>
17 </msgWiltuDeKaartZien>

```

MITT

When system 1.2 was introduced, it was already agreed to include the associated MessageInTransaction-Type in the message. This decision arises from the problems surrounding the possibility of including the same message several times within a transaction at different places in the flow of the transaction. This made it possible to imagine cases where the same message within a transaction has other follow-up messages, depending on the position in the flow. To prevent this, it is recorded.

```

1 <BerichtInTransactie1 id="BiT001">
2     <identification />
3 </BerichtInTransactie1>

```

Transaction

Then we define the transaction that is used. Within the transaction, we see standard fields, 'who created it and when' and the option for a more extensive description and naming. But we also see the initiator and executor, in our case Piet Jansen (*PiR002*), in the role of waiter, asks whether Kees de Vries (*PiR001*), in the role of customer, wants to see the menu.

```

1 <t1_OpnameBestelling id="transactie001">
2     <number>1</number>
3     <name />
4     <description>Onderwerp van de transactie</description>
5     <startDate>2008-05-04T00:00:00.0Z</startDate>
6     <endDate>2009-05-04T00:00:00.0Z</endDate>
7     <initiator>
8         <PersonInRoleRef idref="PiR002" />
9     </initiator>
10    <executor>
11        <PersonInRoleRef idref="PiR001" />
12    </executor>
13    <project>
14        <ProjectType1Ref idref="Project1" />
15    </project>
16 </t1_OpnameBestelling>

```

Project

A *ProjectType* is indicated at framework level. The project-specific context is provided in the project-specific message. The project data is processed in the message as follows.

```

1 <ProjectType1 id="Project1">
2     <name>Project Top Koks</name>
3     <description>Project voor het opnemen van een bestelling</description>
4     <startDate>2008-05-04T00:00:00.0Z</startDate>
5     <endDate>2009-05-04T00:00:00.0Z</endDate>

```

```

6      <ceSOAP>
7          <CeSOAP id="ProjectGegevens">
8              <sOAPProtocol>MIOM</sOAPProtocol>
9              <sOAPServerURL>https://192.168.0.1/visi.wSDL</sOAPServerURL>
10             <sOAPCentralServerURL>https://www.crow.nl/testcases/case001/visi.wSDL</
              sOAPCentralServerURL>
11         </CeSOAP>
12     </ceSOAP>
13 </ProjectType1>

```

PersonInRole

Both the initiator and executor, already mentioned in the transaction, are specific persons belonging to a specific organisation who behave according to a specific role. The persons, organisations and roles are defined separately. However, to get a good initiator and executor definition, the collection object *PersonInRole* exists (this object exists for every message schema regardless of the framework).

```

1 <PersonInRole id="PiR001">
2     <contactPerson>
3         <StandardPersonTypeRef idref="KeesDeVries" />
4     </contactPerson>
5     <organisation>
6         <StandardOrganisationTypeRef idref="consument" />
7     </organisation>
8     <role>
9         <klantRef idref="klantRol" />
10    </role>
11 </PersonInRole>
12 <PersonInRole id="PiR002">
13     <contactPerson>
14         <StandardPersonTypeRef idref="PietJansen" />
15     </contactPerson>
16     <organisation>
17         <StandardOrganisationTypeRef idref="restaurant" />
18     </organisation>
19     <role>
20         <klantRef idref="oberRol" />
21     </role>
22 </PersonInRole>

```

Instances of roles At the framework level, we have already created the Waiter role. Here we define an instance of this role, namely the *waiterRole*. We need this authority to ask if the customer wants to see the menu.

```

1 <ober id="oberRol">
2     <name>Ober Rol</name>
3     <description>Diegene die serveert</description>
4 </ober>

```

We do the same for the Customer role (the customerRole).

```

1 <klant id="klantRol">
2     <name>Klant Rol</name>
3     <description>Diegene die de klant is</description>
4 </klant>

```

DefaultOrganisationType

At the framework level, we have only created an OrganisationType, so it is simple to choose according to which object we want to specify the participating organisations. In this section, the URL of the SOAP server is also given to the project using a complex element.

```

1 <StandardOrganisationType id="consument">
2   <name>Consument</name>
3   <abbreviation>CSMT</abbreviation>
4   <contactPerson>
5     <StandardPersonTypeRef idref="KeesDeVries" />
6   </contactPerson>
7   <ceOrganisatie>
8     <CeOrganisatie id="OrganisatieGegevensConsument">
9       <sOAPServerURL>https://192.168.0.102/specifiek_project/visi.wsd</
sOAPServerURL>
10     </CeOrganisatie>
11   </ceOrganisatie>
12 </StandardOrganisationType>
13 <StandardOrganisationType id="restaurant">
14   <name>Restaurant</name>
15   <abbreviation>RSTR</abbreviation>
16   <contactPerson>
17     <StandardPersonTypeRef idref="PietJansen" />
18   </contactPerson>
19   <ceOrganisatie>
20     <CeOrganisatie id="OrganisatieGegevensRestaurant">
21       <sOAPServerURL>https://192.168.0.105/specifiek_project/visi.wsd</
sOAPServerURL>
22     </CeOrganisatie>
23   </ceOrganisatie>
24 </StandardOrganisationType>

```

Note: A single organisation is also possible if both roles are fulfilled within the same organisation.

DefaultPersonType

At the framework level, we have created only one PersonType. So it is easy to choose according to which object we want to specify the participating persons.

```

1 <StandardPersonType id="KeesDeVries">
2   <userName>kvd</userName>
3   <name>Kees de Vries</name>
4 </StandardPersonType>
5 <StandardPersonType id="PietJansen">
6   <userName>pj</userName>
7   <name>Piet Jansen</name>
8 </StandardPersonType>

```

Note: Only one person is also possible if both roles are fulfilled by one person (this seems unnatural, but can be advantageous for administrative reasons).

The complete first message now looks like this:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:
  schemaLocation="http://www.visi.nl/schemas/ 20160331 /TestFramework 10.xsd"
3   xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework">
4   <msgWiltuDeKaartZien id="bericht001">
5     <identification />
6     <dateSend>2008-05-04T00:00:00.0Z</dateSend>
7     <initiatorToExecutor>true</initiatorToExecutor>
8     <messageInTransaction>
9       <BerichtInTransactie1Ref idref="BiT001" />
10    </messageInTransaction>
11    <transaction>
12      <t1_OpnameBestellingRef idref="transactie001" />
13    </transaction>
14    <ceReactieTermijn>
15      <CeReactieTermijnRef idref="ce001a"/>

```

```

16         </ceReactieTermijn>
17         <ceBaseTypeTest>
18             <CeBaseTypeTestRef idref="ce001b"/>
19         </ceBaseTypeTest>
20     </msgWiltuDeKaartZien>
21     <CeReactieTermijn id="ce001a">
22         <wachttijd>40</wachttijd>
23     </CeReactieTermijn>
24     <CeBaseTypeTest id="ce001b">
25         <seBoolean>true</seBoolean>
26         <seDate>2008-05-04</seDate>
27         <seDatetime>2008-05-04T00:00:00.0Z</seDatetime>
28         <seTime>00:00:00.0Z</seTime>
29         <seDecimal>100.00</seDecimal>
30         <seInteger>123</seInteger>
31         <seKeuzelijst>Keuze 1</seKeuzelijst>
32         <seString>een string</seString>
33     </CeBaseTypeTest>
34     <BerichtInTransactie1 id="BiT001">
35         <identification />
36     </BerichtInTransactie1>
37     <t1_OpnameBestelling id="transactie001">
38         <number>1</number>
39         <name />
40         <description>Onderwerp van de transactie</description>
41         <startDate>2008-05-04T00:00:00.0Z</startDate>
42         <endDate>2009-05-04T00:00:00.0Z</endDate>
43         <initiator>
44             <PersonInRoleRef idref="PiR002" />
45         </initiator>
46         <executor>
47             <PersonInRoleRef idref="PiR001" />
48         </executor>
49         <project>
50             <ProjectType1Ref idref="Project1" />
51         </project>
52     </t1_OpnameBestelling>
53     <ProjectType1 id="Project1">
54         <name>Project Top Koks</name>
55         <description>Project voor het opnemen van een bestelling</description>
56         <startDate>2008-05-04T00:00:00.0Z</startDate>
57         <endDate>2009-05-04T00:00:00.0Z</endDate>
58         <ceSOAP>
59             <CeSOAP id="ProjectGegevens">
60                 <sOAPProtocol>MTOM</sOAPProtocol>
61                 <sOAPServerURL>https://192.168.0.1/visi.wSDL</sOAPServerURL>
62                 <sOAPCentralServerURL>https://www.crow.nl/testcases/case001/visi.wSDL</
sOAPCentralServerURL>
63             </CeSOAP>
64         </ceSOAP>
65     </ProjectType1>
66     <StandardOrganisationType id="consument">
67         <name>Consument</name>
68         <abbreviation>CSMT</abbreviation>
69         <contactPerson>
70             <StandardPersonTypeRef idref="KeesDeVries" />
71         </contactPerson>
72         <ceOrganisatie>
73             <CeOrganisatie id="OrganisatieGegevensConsument">
74                 <sOAPServerURL>https://192.168.0.102/specifiek_project/visi.wSDL</
sOAPServerURL>
75             </CeOrganisatie>
76         </ceOrganisatie>
77     </StandardOrganisationType>
78     <StandardOrganisationType id="restaurant">
79         <name>Restaurant</name>
80         <abbreviation>RSTR</abbreviation>
81         <contactPerson>
82             <StandardPersonTypeRef idref="PietJansen" />
83         </contactPerson>

```

```

84     <ceOrganisatie>
85         <CeOrganisatie id="OrganisatieGegevensRestaurant">
86             <sOAPServerURL>https://192.168.0.105/specifiek_project/visi.wSDL</
sOAPServerURL>
87         </CeOrganisatie>
88     </ceOrganisatie>
89 </StandardOrganisationType>
90 <StandardPersonType id="KeesDeVries">
91     <userName>kvd</userName>
92     <name>Kees de Vries</name>
93 </StandardPersonType>
94 <StandardPersonType id="PietJansen">
95     <userName>pj</userName>
96     <name>Piet Jansen</name>
97 </StandardPersonType>
98 <klant id="klantRol">
99     <name>Klant Rol</name>
100     <description>Diegene die de klant is</description>
101 </klant>
102 <ober id="oberRol">
103     <name>Ober Rol</name>
104     <description>Diegene die serveert</description>
105 </ober>
106 <PersonInRole id="PiR001">
107     <contactPerson>
108         <StandardPersonTypeRef idref="KeesDeVries" />
109     </contactPerson>
110     <organisation>
111         <StandardOrganisationTypeRef idref="consument" />
112     </organisation>
113     <role>
114         <klantRef idref="klantRol" />
115     </role>
116 </PersonInRole>
117 <PersonInRole id="PiR002">
118     <contactPerson>
119         <StandardPersonTypeRef idref="PietJansen" />
120     </contactPerson>
121     <organisation>
122         <StandardOrganisationTypeRef idref="restaurant" />
123     </organisation>
124     <role>
125         <klantRef idref="oberRol" />
126     </role>
127 </PersonInRole>
128 </visiXML_MessageSchema>

```

8.2.3.4 The second message (“tweede_bericht.xml”)

After the waiter asks with the first message whether the customer wants to see the card, the customer indicates with the second message (msgYes) that he indeed wants to see the card. Since the message is almost identical to the first message, here is the entire message.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
    .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
4     <msgJa id="bericht002">
5         <identification />
6         <dateSend>2008-05-04T00:00:00.0Z</dateSend>
7         <initiatorToExecutor>false</initiatorToExecutor>
8         <messageInTransaction>
9             <BerichtInTransactie3Ref idref="BiT001" />
10        </messageInTransaction>
11        <transaction>
12            <t1_OpnameBestellingRef idref="transactie001" />
13        </transaction>

```

```

14     <ceMenuKaartType>
15         <CeMenuKaartType id="menuKaartType001">
16             <menuKaartType>Dinerkaart </menuKaartType>
17         </CeMenuKaartType>
18     </ceMenuKaartType>
19 </msgJa>
20 <BerichtInTransactie3 id="BiT001">
21     <identification />
22 </BerichtInTransactie3>
23 <t1_OpnameBestelling id="transactie001">
24     <number>1</number>
25     <name />
26     <description />
27     <startDate>2008-05-04T00:00:00.0Z</startDate>
28     <endDate>2009-05-04T00:00:00.0Z</endDate>
29     <initiator>
30         <PersonInRoleRef idref="PiR002" />
31     </initiator>
32     <executor>
33         <PersonInRoleRef idref="PiR001" />
34     </executor>
35     <project>
36         <ProjectType1Ref idref="Project1" />
37     </project>
38 </t1_OpnameBestelling>
39 <ProjectType1 id="Project1">
40     <name>Project Top Koks</name>
41     <description>Project voor het opnemen van een bestelling
42     </description>
43     <startDate>2008-05-04T00:00:00.0Z</startDate>
44     <endDate>2009-05-04T00:00:00.0Z</endDate>
45     <ceSOAP>
46         <CeSOAP id="ProjectGegevens">
47             <sOAPProtocol>MIOM</sOAPProtocol>
48             <sOAPServerURL>http://192.168.0.1/visi.wSDL</sOAPServerURL>
49             <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wSDL</
SOAPCentralServerURL>
50         </CeSOAP>
51     </ceSOAP>
52 </ProjectType1>
53 <StandardOrganisationType id="consument">
54     <name>Consument</name>
55     <abbreviation>CSMT</abbreviation>
56     <contactPerson>
57         <StandardPersonTypeRef idref="KeesDeVries" />
58     </contactPerson>
59     <ceOrganisatie>
60         <CeOrganisatie id="OrganisatieGegevensConsument">
61             <sOAPServerURL>http://192.168.0.102/specifiek_project/visi.wSDL</
SOAPServerURL>
62         </CeOrganisatie>
63     </ceOrganisatie>
64 </StandardOrganisationType>
65 <StandardOrganisationType id="restaurant">
66     <name>Restaurant</name>
67     <abbreviation>RSTR</abbreviation>
68     <contactPerson>
69         <StandardPersonTypeRef idref="PietJansen" />
70     </contactPerson>
71     <ceOrganisatie>
72         <CeOrganisatie id="OrganisatieGegevensRestaurant">
73             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wSDL</
SOAPServerURL>
74         </CeOrganisatie>
75     </ceOrganisatie>
76 </StandardOrganisationType>
77 <StandardPersonType id="KeesDeVries">
78     <userName>kvd</userName>
79     <name>Kees de Vries</name>
80 </StandardPersonType>

```

```

81 <StandardPersonType id="PietJansen">
82   <userName>pj</userName>
83   <name>Piet Jansen</name>
84 </StandardPersonType>
85 <klant id="klantRol">
86   <name>Klant Rol</name>
87   <description>Diegene die de klant is</description>
88 </klant>
89 <ober id="oberRol">
90   <name>Ober Rol</name>
91   <description>Diegene die serveert</description>
92 </ober>
93 <PersonInRole id="PiR001">
94   <contactPerson>
95     <StandardPersonTypeRef idref="KeesDeVries" />
96   </contactPerson>
97   <organisation>
98     <StandardOrganisationTypeRef idref="consument" />
99   </organisation>
100   <role>
101     <klantRef idref="klantRol" />
102   </role>
103 </PersonInRole>
104 <PersonInRole id="PiR002">
105   <contactPerson>
106     <StandardPersonTypeRef idref="PietJansen" />
107   </contactPerson>
108   <organisation>
109     <StandardOrganisationTypeRef idref="restaurant" />
110   </organisation>
111   <role>
112     <klantRef idref="oberRol" />
113   </role>
114 </PersonInRole>
115 </visiXML_MessageSchema>

```

8.2.3.5 The third message (“derde_bericht.xml”)

The waiter received a positive response from the customer in the second message. He will now offer the menu to the customer with the third message.

To do this, the message contains the complex element *CeMenuKaart*. This complex element is not directly related to simple elements, but to another complex element *CeContentMenuKaart*. It has been agreed that a complex element within a complex element is interpreted as a table.

The message will therefore look like this:

```

1 <msgAanbiedingMenuKaart id="bericht003">
2   <identification />
3   <dateSend>2008-05-04T00:00:00.0Z</dateSend>
4   <initiatorToExecutor>true</initiatorToExecutor>
5   <messageInTransaction>
6     <BerichtInTransactie4Ref idref="BiT001" />
7   </messageInTransaction>
8   <transaction>
9     <t1_OpnameBestellingRef idref="transactie001" />
10  </transaction>
11  <ceMenuKaart>
12    <CeMenuKaart id="menuKaart001">
13      <ceInhoudMenuKaart>
14        <CeInhoudMenuKaart id="inhoudMenuKaart001">
15          <naamMenukaartItem>Carpatio</naamMenukaartItem>
16          <omschrijving>Dunne lapjes vlees + garnering</omschrijving>
17          <prijs>25.00</prijs>
18        </CeInhoudMenuKaart>
19        <CeInhoudMenuKaart id="inhoudMenuKaart002">

```

```

20         <naamMenukaartItem>Biefstuk</naamMenukaartItem>
21         <omschrijving>Biefstuk + gepofte aardappelen en salade</omschrijving>
22     >
23         <prijs >35.00</prijs>
24     </CeInhoudMenuKaart>
25     <CeInhoudMenuKaart id="inhoudMenuKaart003">
26         <naamMenukaartItem>Vruchten</naamMenukaartItem>
27         <omschrijving>Verse vruchten</omschrijving>
28         <prijs >12.50</prijs>
29     </CeInhoudMenuKaart>
30 </ceInhoudMenuKaart>
31 </CeMenuKaart>
32 </ceMenuKaart>
33 <ceMenuKaartType>
34     <CeMenuKaartType id="menuKaartType001">
35         <menuKaartType>Dinerkaart</menuKaartType>
36     </CeMenuKaartType>
37 </ceMenuKaartType>
</msgAanbiedingMenuKaart>

```

The entire message looks like this.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4   .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
5   <msgAanbiedingMenuKaart id="bericht003">
6       <identification />
7       <dateSend>2008-05-04T00:00:00Z</dateSend>
8       <initiatorToExecutor>true</initiatorToExecutor>
9       <messageInTransaction>
10         <BerichtInTransactie4Ref idref="BiT001" />
11       </messageInTransaction>
12       <transaction>
13         <t1_OpnameBestellingRef idref="transactie001" />
14       </transaction>
15       <ceMenuKaart>
16         <CeMenuKaart id="menuKaart001">
17             <ceInhoudMenuKaart>
18                 <CeInhoudMenuKaart id="inhoudMenuKaart001">
19                     <naamMenukaartItem>Carpatio</naamMenukaartItem>
20                     <omschrijving>Dunne lapjes vlees + garneering</omschrijving>
21                     <prijs >25.00</prijs>
22                 </CeInhoudMenuKaart>
23                 <CeInhoudMenuKaart id="inhoudMenuKaart002">
24                     <naamMenukaartItem>Biefstuk</naamMenukaartItem>
25                     <omschrijving>Biefstuk + gepofte aardappelen en garneering</
26 omschrijving>
27                     <prijs >35.00</prijs>
28                 </CeInhoudMenuKaart>
29             </ceInhoudMenuKaart>
30         </CeMenuKaart>
31     </ceMenuKaart>
32     <ceMenuKaartType>
33         <CeMenuKaartType id="menuKaartType001">
34             <menuKaartType>Dinerkaart</menuKaartType>
35         </CeMenuKaartType>
36     </ceMenuKaartType>
37 </msgAanbiedingMenuKaart>
38 <BerichtInTransactie4 id="BiT001">
39     <identification />
40 </BerichtInTransactie4>
41 <t1_OpnameBestelling id="transactie001">
42     <number>1</number>
43

```

```

46     <name />
47     <description />
48     <startDate>2008-05-04T00:00:00.0Z</startDate>
49     <endDate>2009-05-04T00:00:00.0Z</endDate>
50     <initiator>
51         <PersonInRoleRef idref="PiR002" />
52     </initiator>
53     <executor>
54         <PersonInRoleRef idref="PiR001" />
55     </executor>
56     <project>
57         <ProjectType1Ref idref="Project1" />
58     </project>
59 </t1_OpnameBestelling>
60 <ProjectType1 id="Project1">
61     <name>Project Top Koks</name>
62     <description>Project voor het opnemen van een bestelling
63 </description>
64     <startDate>2008-05-04T00:00:00.0Z</startDate>
65     <endDate>2009-05-04T00:00:00.0Z</endDate>
66     <ceSOAP>
67         <CeSOAP id="ProjectGegevens">
68             <sOAPProtocol>MIOM</sOAPProtocol>
69             <sOAPServerURL>http://192.168.0.1/visi.wSDL</sOAPServerURL>
70             <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wSDL</
71 sOAPCentralServerURL>
72         </CeSOAP>
73     </ceSOAP>
74 </ProjectType1>
75 <StandardOrganisationType id="consument">
76     <name>Consument</name>
77     <abbreviation>CSMT</abbreviation>
78     <contactPerson>
79         <StandardPersonTypeRef idref="KeesDeVries" />
80     </contactPerson>
81     <ceOrganisatie>
82         <CeOrganisatie id="OrganisatieGegevensConsument">
83             <sOAPServerURL>http://192.168.0.102/specifiek_project/visi.wSDL</
84 sOAPServerURL>
85         </CeOrganisatie>
86     </ceOrganisatie>
87 </StandardOrganisationType>
88 <StandardOrganisationType id="restaurant">
89     <name>Restaurant</name>
90     <abbreviation>RSTR</abbreviation>
91     <contactPerson>
92         <StandardPersonTypeRef idref="PietJansen" />
93     </contactPerson>
94     <ceOrganisatie>
95         <CeOrganisatie id="OrganisatieGegevensRestaurant">
96             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wSDL</
97 sOAPServerURL>
98         </CeOrganisatie>
99     </ceOrganisatie>
100 </StandardOrganisationType>
101 <StandardPersonType id="KeesDeVries">
102     <userName>kvd</userName>
103     <name>Kees de Vries</name>
104 </StandardPersonType>
105 <StandardPersonType id="PietJansen">
106     <userName>pj</userName>
107     <name>Piet Jansen</name>
108 </StandardPersonType>
109 <klant id="klantRol">
110     <name>Klant Rol</name>
111     <description>Diegene die de klant is</description>
112 </klant>
113 <ober id="oberRol">
114     <name>Ober Rol</name>
115     <description>Diegene die serveert</description>

```

```

113     </ober>
114     <PersonInRole id="PiR001">
115         <contactPerson>
116             <StandardPersonTypeRef idref="KeesDeVries" />
117         </contactPerson>
118         <organisation>
119             <StandardOrganisationTypeRef idref="consument" />
120         </organisation>
121         <role>
122             <klantRef idref="klantRol" />
123         </role>
124     </PersonInRole>
125     <PersonInRole id="PiR002">
126         <contactPerson>
127             <StandardPersonTypeRef idref="PietJansen" />
128         </contactPerson>
129         <organisation>
130             <StandardOrganisationTypeRef idref="restaurant" />
131         </organisation>
132         <role>
133             <klantRef idref="oberRol" />
134         </role>
135     </PersonInRole>
136 </visiXML_MessageSchema>

```

8.2.3.6 The fourth message (“vierde_bericht.xml”)

After the customer has received the menu from the waiter, the customer wants to ask a question. The customer in question loves mayonnaise and would like to have it with the jacket potatoes. The question is, of course, whether this is possible in a well-regarded culinary restaurant. To request this, the customer sends the fourth message to the waiter. The message *msgQuestion* contains a complex element containing a simple element with which the question can be asked.

The message looks like this:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4     .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
5   <msgVraag id="bericht004">
6     <identification />
7     <dateSend>2008-05-04T00:00:00.0Z</dateSend>
8     <initiatorToExecutor>false</initiatorToExecutor>
9     <messageInTransaction>
10       <BerichtInTransactie5Ref idref="BiT001" />
11     </messageInTransaction>
12     <transaction>
13       <t1_OpnameBestellingRef idref="transactie001" />
14     </transaction>
15     <ceVraag>
16       <CeVraag id="vraag001">
17         <vraag>Kan er ook mayonaise bij de gepofte aardappelen geserveerd worden
18       </vraag>
19     </CeVraag>
20   </msgVraag>
21   <BerichtInTransactie15 id="BiT001">
22     <identification />
23   </BerichtInTransactie15>
24   <t1_OpnameBestelling id="transactie001">
25     <number>1</number>
26     <name />
27     <description />
28     <startDate>2008-05-04T00:00:00.0Z</startDate>
29     <endDate>2009-05-04T00:00:00.0Z</endDate>
30     <initiator>

```

```

30         <PersonInRoleRef idref="PiR002" />
31     </initiator>
32     <executor>
33         <PersonInRoleRef idref="PiR001" />
34     </executor>
35     <project>
36         <ProjectType1Ref idref="Project1" />
37     </project>
38 </t1_OpnameBestelling>
39 <ProjectType1 id="Project1">
40     <name>Project Top Koks</name>
41     <description>Project voor het opnemen van een bestelling </description>
42     <startDate>2008-05-04T00:00:00.0Z</startDate>
43     <endDate>2009-05-04T00:00:00.0Z</endDate>
44     <ceSOAP>
45         <CeSOAP id="ProjectGegevens">
46             <sOAPProtocol>MIOM</sOAPProtocol>
47             <sOAPServerURL>http://192.168.0.1/visi.wsd</sOAPServerURL>
48             <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wsd</
sOAPCentralServerURL>
49         </CeSOAP>
50     </ceSOAP>
51 </ProjectType1>
52 <StandardOrganisationType id="consument">
53     <name>Consument</name>
54     <abbreviation>CSMT</abbreviation>
55     <contactPerson>
56         <StandardPersonTypeRef idref="KeesDeVries" />
57     </contactPerson>
58     <ceOrganisatie>
59         <CeOrganisatie id="OrganisatieGegevensConsument">
60             <sOAPServerURL>http://192.168.0.102/specifiek_project/visi.wsd</
sOAPServerURL>
61         </CeOrganisatie>
62     </ceOrganisatie>
63 </StandardOrganisationType>
64 <StandardOrganisationType id="restaurant">
65     <name>Restaurant</name>
66     <abbreviation>RSTR</abbreviation>
67     <contactPerson>
68         <StandardPersonTypeRef idref="PietJansen" />
69     </contactPerson>
70     <ceOrganisatie>
71         <CeOrganisatie id="OrganisatieGegevensRestaurant">
72             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wsd</
sOAPServerURL>
73         </CeOrganisatie>
74     </ceOrganisatie>
75 </StandardOrganisationType>
76 <StandardPersonType id="KeesDeVries">
77     <userName>kvd</userName>
78     <name>Kees de Vries</name>
79 </StandardPersonType>
80 <StandardPersonType id="PietJansen">
81     <userName>pj</userName>
82     <name>Piet Jansen</name>
83 </StandardPersonType>
84 <klant id="klantRol">
85     <name>Klant Rol</name>
86     <description>Diegene die de klant is</description>
87 </klant>
88 <ober id="oberRol">
89     <name>Ober Rol</name>
90     <description>Diegene die serveert</description>
91 </ober>
92 <PersonInRole id="PiR001">
93     <contactPerson>
94         <StandardPersonTypeRef idref="KeesDeVries" />
95     </contactPerson>
96     <organisation>

```



```

97         <StandardOrganisationTypeRef idref="consument" />
98     </organisation>
99     <role>
100         <klantRef idref="klantRol" />
101     </role>
102 </PersonInRole>
103 <PersonInRole id="PiR002">
104     <contactPerson>
105         <StandardPersonTypeRef idref="PietJansen" />
106     </contactPerson>
107     <organisation>
108         <StandardOrganisationTypeRef idref="restaurant" />
109     </organisation>
110     <role>
111         <klantRef idref="oberRol" />
112     </role>
113 </PersonInRole>
114 </visiXML_MessageSchema>

```

8.2.3.7 The fifth and sixth messages (“vijfde_bericht.xml” and “sixth_message.xml”)

The waiter has received a question from the customer, but has no answer himself. The waiter will now have to consult the cook. He will do this by sending the *msgVraag* from transaction *T2_VraagOberAanKok*. As previous in the MITT, this message has the message *msgQuestion* from *T1_Recording_order*. Since the same complex element and simple element appear in this message, the values from these elements, as described in the guideline ‘Reuse and blocking data element’ (see also the explanation in Appendix 5, section 1.2.2), must be adopted.

Because the waiter wants an answer to the customer’s question as quickly as possible, the waiter decides to ask two chefs the same question at the same time. This creates two instances of the message and the transaction. Please note that the guidelines for copying and blocking data elements again apply to the messages. The fifth and sixth messages are shown below.

The fifth message

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4     .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
5     <msgVraag id="bericht005">
6         <identification />
7         <dateSend>2008-05-04T00:00:00.0Z</dateSend>
8         <initiatingTransactionMessageID>bericht004</initiatingTransactionMessageID>
9         <initiatorToExecutor>true</initiatorToExecutor>
10        <messageInTransaction>
11            <BerichtInTransactie11Ref idref="BiT001" />
12        </messageInTransaction>
13        <transaction>
14            <t2_VraagOberAanKokRef idref="transactie002" />
15        </transaction>
16        <ceVraag>
17            <CeVraag id="vraag002">
18                <vraag>Kan er ook mayonaise bij de gepofte aardappelen geserveerd worden
19            </vraag>
20        </CeVraag>
21    </msgVraag>
22    <BerichtInTransactie11 id="BiT001">
23        <identification />
24    </BerichtInTransactie11>
25    <t2_VraagOberAanKok id="transactie002">
26        <number>2</number>
27        <name />
28        <description />
29        <startDate>2008-05-04T00:00:00.0Z</startDate>

```

```

29     <endDate>2009-05-04T00:00:00.0Z</endDate>
30     <initiator>
31         <PersonInRoleRef idref="PiR002" />
32     </initiator>
33     <executor>
34         <PersonInRoleRef idref="PiR003" />
35     </executor>
36     <project>
37         <ProjectType1Ref idref="Project1" />
38     </project>
39 </t2_VraagOberAanKok>
40 <ProjectType1 id="Project1">
41     <name>Project Top Koks</name>
42     <description>Project voor het opnemen van een bestelling </description>
43     <startDate>2008-05-04T00:00:00.0Z</startDate>
44     <endDate>2009-05-04T00:00:00.0Z</endDate>
45     <ceSOAP>
46         <CeSOAP id="ProjectGegevens">
47             <sOAPProtocol>MTOM</sOAPProtocol>
48             <sOAPServerURL>http://192.168.0.1/visi.wsdl</sOAPServerURL>
49             <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wsdl</
sOAPCentralServerURL>
50         </CeSOAP>
51     </ceSOAP>
52 </ProjectType1>
53 <StandardOrganisationType id="restaurant">
54     <name>Restaurant</name>
55     <abbreviation>RSTR</abbreviation>
56     <contactPerson>
57         <StandardPersonTypeRef idref="PietJansen" />
58     </contactPerson>
59     <ceOrganisatie>
60         <CeOrganisatie id="OrganisatieGegevensRestaurant">
61             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wsdl</
sOAPServerURL>
62         </CeOrganisatie>
63     </ceOrganisatie>
64 </StandardOrganisationType>
65 <StandardPersonType id="PietJansen">
66     <userName>pj</userName>
67     <name>Piet Jansen</name>
68 </StandardPersonType>
69 <StandardPersonType id="DirkZwart">
70     <userName>dz</userName>
71     <name>Dirk Zwart</name>
72 </StandardPersonType>
73 <ober id="oberRol">
74     <name>Ober Rol</name>
75     <description>Diegene die serveert</description>
76 </ober>
77 <kok id="kokRol">
78     <name>Kok Rol</name>
79     <description>Diegene die kookt</description>
80 </kok>
81 <PersonInRole id="PiR002">
82     <contactPerson>
83         <StandardPersonTypeRef idref="PietJansen" />
84     </contactPerson>
85     <organisation>
86         <StandardOrganisationTypeRef idref="restaurant" />
87     </organisation>
88     <role>
89         <klantRef idref="oberRol" />
90     </role>
91 </PersonInRole>
92 <PersonInRole id="PiR003">
93     <contactPerson>
94         <StandardPersonTypeRef idref="DirkZwart" />
95     </contactPerson>
96     <organisation>

```

```

97         <StandardOrganisationTypeRef idref="restaurant" />
98     </organisation>
99     <role>
100         <klantRef idref="kokRol" />
101     </role>
102 </PersonInRole>
103 </visiXML_MessageSchema>

```

The sixth message

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4     .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
5   <msgVraag id="bericht006">
6     <identification />
7     <dateSend>2008-05-04T00:00:00.0Z</dateSend>
8     <initiatingTransactionMessageID>bericht004</initiatingTransactionMessageID>
9     <initiatorToExecutor>true</initiatorToExecutor>
10    <messageInTransaction>
11      <BerichtInTransactie11Ref idref="BiT001" />
12    </messageInTransaction>
13    <transaction>
14      <t2_VraagOberAanKokRef idref="transactie003" />
15    </transaction>
16    <ceVraag>
17      <CeVraag id="vraag003">
18        <vraag>Kan er ook mayonaise bij de gepofte aardappelen geserveerd worden
19      </vraag>
20    </CeVraag>
21  </msgVraag>
22  <BerichtInTransactie11 id="BiT001">
23    <identification />
24  </BerichtInTransactie11>
25  <t2_VraagOberAanKok id="transactie003">
26    <number>3</number>
27    <name />
28    <description />
29    <startDate>2008-05-04T00:00:00.0Z</startDate>
30    <endDate>2009-05-04T00:00:00.0Z</endDate>
31    <initiator>
32      <PersonInRoleRef idref="PiR002" />
33    </initiator>
34    <executor>
35      <PersonInRoleRef idref="PiR004" />
36    </executor>
37    <project>
38      <ProjectType1Ref idref="Project1" />
39    </project>
40  </t2_VraagOberAanKok>
41  <ProjectType1 id="Project1">
42    <name>Project Top Koks</name>
43    <description>Project voor het opnemen van een bestelling</description>
44    <startDate>2008-05-04T00:00:00.0Z</startDate>
45    <endDate>2009-05-04T00:00:00.0Z</endDate>
46    <ceSOAP>
47      <CeSOAP id="ProjectGegevens">
48        <sOAPProtocol>MTOM</sOAPProtocol>
49        <sOAPServerURL>http://192.168.0.1/visi.wsdl</sOAPServerURL>
50        <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wsdl</
51        sOAPCentralServerURL>
52      </CeSOAP>
53    </ProjectType1>
54  </StandardOrganisationType id="restaurant">
55    <name>Restaurant</name>
56    <abbreviation>RSTR</abbreviation>
57    <contactPerson>

```

```

57         <StandardPersonTypeRef idref="PietJansen" />
58     </contactPerson>
59     <ceOrganisatie>
60         <CeOrganisatie id="OrganisatieGegevensRestaurant">
61             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wSDL</
sOAPServerURL>
62         </CeOrganisatie>
63     </ceOrganisatie>
64 </StandardOrganisationType>
65 <StandardPersonType id="PietJansen">
66     <userName>pj</userName>
67     <name>Piet Jansen</name>
68 </StandardPersonType>
69 <StandardPersonType id="RobTaal">
70     <userName>rt</userName>
71     <name>Rob Taal</name>
72 </StandardPersonType>
73 <ober id="oberRol">
74     <name>Ober Rol</name>
75     <description>Diegene die serveert</description>
76 </ober>
77 <kok id="kokRol">
78     <name>Kok Rol</name>
79     <description>Diegene die kookt</description>
80 </kok>
81 <PersonInRole id="PiR002">
82     <contactPerson>
83         <StandardPersonTypeRef idref="PietJansen" />
84     </contactPerson>
85     <organisation>
86         <StandardOrganisationTypeRef idref="restaurant" />
87     </organisation>
88     <role>
89         <klantRef idref="oberRol" />
90     </role>
91 </PersonInRole>
92 <PersonInRole id="PiR004">
93     <contactPerson>
94         <StandardPersonTypeRef idref="RobTaal" />
95     </contactPerson>
96     <organisation>
97         <StandardOrganisationTypeRef idref="restaurant" />
98     </organisation>
99     <role>
100         <klantRef idref="kokRol" />
101     </role>
102 </PersonInRole>
103 </visiXML_MessageSchema>

```

8.2.3.8 The seventh message (“zevende_bericht.xml”)

The person in the role of cook to whom the fifth message was sent is Dirk Zwart. Dirk Zwart is a very ambitious chef with star allure. Because he sets the bar quite high for himself, he constantly balances on the edge of being stressed. The question regarding mayonnaise is the straw that breaks the camel’s back. Dirk Zwart throws his oven glove on the floor and runs off, never to return.

In order not to make customers wait too long for their orders and questions, a new chef will be hired immediately and appointed as his successor. It concerns Jan Toet. The project-specific message is modified for this with a successor (see file: 12_project_specific_message_from_message_7.xml).

As Dirk Zwart’s successor, Jan Toet immediately decides to give a positive answer to the customer’s question that he received through the waiter. Please note that the guideline for inheriting and blocking data elements applies to this message again.

The message looks like this:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4   .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
5   <msgAntwoord id="bericht007">
6     <identification />
7     <dateSend>2008-05-04T00:00:00.0Z</dateSend>
8     <initiatingTransactionMessageID>bericht004</initiatingTransactionMessageID>
9     <initiatorToExecutor>false</initiatorToExecutor>
10    <messageInTransaction>
11      <BerichtInTransactie12Ref idref="BiT001" />
12    </messageInTransaction>
13    <transaction>
14      <t2_VraagOberAanKokRef idref="transactie003" />
15    </transaction>
16    <ceVraag>
17      <CeVraag id="vraag004">
18        <vraag>Kan er ook mayonaise bij de gepofte aardappelen geserveerd worden
19      </vraag>
20    </CeVraag>
21    <ceAntwoord>
22      <CeAntwoord id="antwoord001">
23        <antwoord>Dat is prima</antwoord>
24      </CeAntwoord>
25    </ceAntwoord>
26  </msgAntwoord>
27  <BerichtInTransactie12 id="BiT001">
28    <identification />
29  </BerichtInTransactie12>
30  <t2_VraagOberAanKok id="transactie003">
31    <number>3</number>
32    <name />
33    <description />
34    <startDate>2008-05-04T00:00:00.0Z</startDate>
35    <endDate>2009-05-04T00:00:00.0Z</endDate>
36    <initiator>
37      <PersonInRoleRef idref="PiR002" />
38    </initiator>
39    <executor>
40      <PersonInRoleRef idref="PiR004" />
41    </executor>
42    <project>
43      <ProjectType1Ref idref="Project1" />
44    </project>
45  </t2_VraagOberAanKok>
46  <ProjectType1 id="Project1">
47    <name>Project Top Koks</name>
48    <description>Project voor het opnemen van een bestelling </description>
49    <startDate>2008-05-04T00:00:00.0Z</startDate>
50    <endDate>2009-05-04T00:00:00.0Z</endDate>
51    <ceSOAP>
52      <CeSOAP id="ProjectGegevens">
53        <sOAPProtocol>MITOM</sOAPProtocol>
54        <sOAPServerURL>http://192.168.0.1/visi.wsdl</sOAPServerURL>
55        <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wsdl</
56      </CeSOAP>
57    </ceSOAP>
58  </ProjectType1>
59  <StandardOrganisationType id="restaurant">
60    <name>Restaurant</name>
61    <abbreviation>RSTR</abbreviation>
62    <contactPerson>
63      <StandardPersonTypeRef idref="PietJansen" />
64    </contactPerson>
65    <ceOrganisatie>
66      <CeOrganisatie id="OrganisatieGegevensRestaurant">
```

```

66         <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wsdl</
sOAPServerURL>
67         </CeOrganisatie>
68     </ceOrganisatie>
69 </StandardOrganisationType>
70 <StandardPersonType id="PietJansen">
71     <userName>pj</userName>
72     <name>Piet Jansen</name>
73 </StandardPersonType>
74 <StandardPersonType id="RobTaal">
75     <userName>rt</userName>
76     <name>Rob Taal</name>
77 </StandardPersonType>
78 <StandardPersonType id="JanToet">
79     <userName>jt</userName>
80     <name>Jan Toet</name>
81 </StandardPersonType>
82 <ober id="oberRol">
83     <name>Ober Rol</name>
84     <description>Diegene die serveert</description>
85 </ober>
86 <kok id="kokRol">
87     <name>Kok Rol</name>
88     <description>Diegene die kookt</description>
89 </kok>
90 <PersonInRole id="PiR002">
91     <contactPerson>
92         <StandardPersonTypeRef idref="PietJansen" />
93     </contactPerson>
94     <organisation>
95         <StandardOrganisationTypeRef idref="restaurant" />
96     </organisation>
97     <role>
98         <klantRef idref="oberRol" />
99     </role>
100 </PersonInRole>
101 <PersonInRole id="PiR004">
102     <successor>
103         <PersonInRoleRef idref="PiR005" />
104     </successor>
105     <contactPerson>
106         <StandardPersonTypeRef idref="RobTaal" />
107     </contactPerson>
108     <organisation>
109         <StandardOrganisationTypeRef idref="restaurant" />
110     </organisation>
111     <role>
112         <klantRef idref="kokRol" />
113     </role>
114 </PersonInRole>
115 <PersonInRole id="PiR005">
116     <contactPerson>
117         <StandardPersonTypeRef idref="JanToet" />
118     </contactPerson>
119     <organisation>
120         <StandardOrganisationTypeRef idref="restaurant" />
121     </organisation>
122     <role>
123         <klantRef idref="kokRol" />
124     </role>
125 </PersonInRole>
126 </visiXML_MessageSchema>

```

8.2.3.9 The eighth message (“achtste_bericht.xml”)

The waiter received a response from a chef via the seventh message. Based on this message, the waiter wants to send the answer to the customer. To do this, the waiter must use *msgAnswer* from the primary

transaction *T1_RecordingOrder*. The previous one in the MITT in this case is msgAnswer from transaction *T2_VraagOberAanKok*. The guideline for copying and blocking data elements also applies here.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
   .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
4   <msgAntwoord id="bericht008">
5     <identification />
6     <dateSend>2008-05-04T00:00:00.0Z</dateSend>
7     <initiatorToExecutor>false</initiatorToExecutor>
8     <messageInTransaction>
9       <BerichtInTransactie12Ref idref="BiT001" />
10    </messageInTransaction>
11    <transaction>
12      <t1_OpnameBestellingRef idref="transactie001" />
13    </transaction>
14    <ceVraag>
15      <CeVraag id="vraag005">
16        <vraag>Kan er ook mayonaise bij de gepofte aardappelen geserveerd worden
17      </vraag>
18    </CeVraag>
19  </ceVraag>
20  <ceAntwoord>
21    <CeAntwoord id="antwoord002">
22      <antwoord>Dat is prima</antwoord>
23    </CeAntwoord>
24  </ceAntwoord>
25 </msgAntwoord>
26 <BerichtInTransactie12 id="BiT001">
27   <identification />
28 </BerichtInTransactie12>
29 <t1_OpnameBestelling id="transactie001">
30   <number>1</number>
31   <name />
32   <description />
33   <startDate>2008-05-04T00:00:00.0Z</startDate>
34   <endDate>2009-05-04T00:00:00.0Z</endDate>
35   <initiator>
36     <PersonInRoleRef idref="PiR002" />
37   </initiator>
38   <executor>
39     <PersonInRoleRef idref="PiR001" />
40   </executor>
41   <project>
42     <ProjectType1Ref idref="Project1" />
43   </project>
44 </t1_OpnameBestelling>
45 <ProjectType1 id="Project1">
46   <name>Project Top Koks</name>
47   <description>Project voor het opnemen van een bestelling </description>
48   <startDate>2008-05-04T00:00:00.0Z</startDate>
49   <endDate>2009-05-04T00:00:00.0Z</endDate>
50   <ceSOAP>
51     <CeSOAP id="ProjectGegevens">
52       <sOAPPProtocol>MIOM</sOAPPProtocol>
53       <sOAPServerURL>http://192.168.0.1/visi.wsdl</sOAPServerURL>
54       <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wsdl</
55       sOAPCentralServerURL>
56     </CeSOAP>
57   </ProjectType1>
58 <StandardOrganisationType id="consument">
59   <name>Consument</name>
60   <abbreviation>CSMT</abbreviation>
61   <contactPerson>
62     <StandardPersonTypeRef idref="KeesDeVries" />
63   </contactPerson>
64   <ceOrganisatie>
65     <CeOrganisatie id="OrganisatieGegevensConsument">

```

```

65         <sOAPServerURL>http://192.168.0.102/specifiek_project/visi.wsdl</
sOAPServerURL>
66         </CeOrganisatie>
67     </ceOrganisatie>
68 </StandardOrganisationType>
69 <StandardOrganisationType id="restaurant">
70     <name>Restaurant</name>
71     <abbreviation>RSTR</abbreviation>
72     <contactPerson>
73         <StandardPersonTypeRef idref="PietJansen" />
74     </contactPerson>
75     <ceOrganisatie>
76         <CeOrganisatie id="OrganisatieGegevensRestaurant">
77             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wsdl</
sOAPServerURL>
78             </CeOrganisatie>
79         </ceOrganisatie>
80     </StandardOrganisationType>
81 <StandardPersonType id="KeesDeVries">
82     <userName>kvd</userName>
83     <name>Kees de Vries</name>
84 </StandardPersonType>
85 <StandardPersonType id="PietJansen">
86     <userName>pj</userName>
87     <name>Piet Jansen</name>
88 </StandardPersonType>
89 <klant id="klantRol">
90     <name>Klant Rol</name>
91     <description>Diegene die de klant is</description>
92 </klant>
93 <ober id="oberRol">
94     <name>Ober Rol</name>
95     <description>Diegene die serveert</description>
96 </ober>
97 <PersonInRole id="PiR001">
98     <contactPerson>
99         <StandardPersonTypeRef idref="KeesDeVries" />
100     </contactPerson>
101     <organisation>
102         <StandardOrganisationTypeRef idref="consument" />
103     </organisation>
104     <role>
105         <klantRef idref="klantRol" />
106     </role>
107 </PersonInRole>
108 <PersonInRole id="PiR002">
109     <contactPerson>
110         <StandardPersonTypeRef idref="PietJansen" />
111     </contactPerson>
112     <organisation>
113         <StandardOrganisationTypeRef idref="restaurant" />
114     </organisation>
115     <role>
116         <klantRef idref="oberRol" />
117     </role>
118 </PersonInRole>
119 </visiXML_MessageSchema>

```

8.2.3.10 The ninth message (“negende_bericht.xml”)

Now that the customer has answered the question he asked the waiter, the customer can place an order. The customer does this with the message *msgPlaceOrder*. In this message, there is a complex element within a complex element.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
    .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">

```



```

4      <msgPlaatsingBestelling id="bericht009">
5          <identification />
6          <dateSend>2008-05-04T00:00:00.0Z</dateSend>
7          <initiatorToExecutor>true</initiatorToExecutor>
8          <messageInTransaction>
9              <BerichtInTransactie7Ref idref="BiT001" />
10         </messageInTransaction>
11         <transaction>
12             <t1_OpnameBestellingRef idref="transactie001" />
13         </transaction>
14         <ceBestelling>
15             <CeBestelling id="bestelling001">
16                 <ceInhoudBestelling>
17                     <CeInhoudBestelling id="inhoudBestelling001">
18                         <naamGerecht>Biefstuk</naamGerecht>
19                         <opmerking>met mayonaise</opmerking>
20                     </CeInhoudBestelling>
21                 </ceInhoudBestelling>
22             </CeBestelling>
23         </ceBestelling>
24     </msgPlaatsingBestelling>
25     <BerichtInTransactie7 id="BiT001">
26         <identification />
27     </BerichtInTransactie7>
28     <t1_OpnameBestelling id="transactie001">
29         <number>1</number>
30         <name />
31         <description />
32         <startDate>2008-05-04T00:00:00.0Z</startDate>
33         <endDate>2009-05-04T00:00:00.0Z</endDate>
34         <initiator>
35             <PersonInRoleRef idref="PiR002" />
36         </initiator>
37         <executor>
38             <PersonInRoleRef idref="PiR001" />
39         </executor>
40         <project>
41             <ProjectType1Ref idref="Project1" />
42         </project>
43     </t1_OpnameBestelling>
44     <ProjectType1 id="Project1">
45         <name>Project Top Koks</name>
46         <description>Project voor het opnemen van een bestelling </description>
47         <startDate>2008-05-04T00:00:00.0Z</startDate>
48         <endDate>2009-05-04T00:00:00.0Z</endDate>
49         <ceSOAP>
50             <CeSOAP id="ProjectGegevens">
51                 <sOAPProtocol>MIOM</sOAPProtocol>
52                 <sOAPServerURL>http://192.168.0.1/visi.wSDL</sOAPServerURL>
53                 <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wSDL</
SOAPCentralServerURL>
54             </CeSOAP>
55         </ceSOAP>
56     </ProjectType1>
57     <StandardOrganisationType id="consument">
58         <name>Consument</name>
59         <abbreviation>CSMT</abbreviation>
60         <contactPerson>
61             <StandardPersonTypeRef idref="KeesDeVries" />
62         </contactPerson>
63         <ceOrganisatie>
64             <CeOrganisatie id="OrganisatieGegevensConsument">
65                 <sOAPServerURL>http://192.168.0.102/specifiek_project/visi.wSDL</
SOAPServerURL>
66             </CeOrganisatie>
67         </ceOrganisatie>
68     </StandardOrganisationType>
69     <StandardOrganisationType id="restaurant">
70         <name>Restaurant</name>
71         <abbreviation>RSTR</abbreviation>

```

```

72     <contactPerson>
73         <StandardPersonTypeRef idref="PietJansen" />
74     </contactPerson>
75     <ceOrganisatie>
76         <CeOrganisatie id="OrganisatieGegevensRestaurant">
77             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wsdl</
sOAPServerURL>
78         </CeOrganisatie>
79     </ceOrganisatie>
80 </StandardOrganisationType>
81 <StandardPersonType id="KeesDeVries">
82     <userName>kvd</userName>
83     <name>Kees de Vries</name>
84 </StandardPersonType>
85 <StandardPersonType id="PietJansen">
86     <userName>pj</userName>
87     <name>Piet Jansen</name>
88 </StandardPersonType>
89 <klant id="klantRol">
90     <name>Klant Rol</name>
91     <description>Diegene die de klant is</description>
92 </klant>
93 <ober id="oberRol">
94     <name>Ober Rol</name>
95     <description>Diegene die serveert</description>
96 </ober>
97 <PersonInRole id="PiR001">
98     <contactPerson>
99         <StandardPersonTypeRef idref="KeesDeVries" />
100     </contactPerson>
101     <organisation>
102         <StandardOrganisationTypeRef idref="consument" />
103     </organisation>
104     <role>
105         <klantRef idref="klantRol" />
106     </role>
107 </PersonInRole>
108 <PersonInRole id="PiR002">
109     <contactPerson>
110         <StandardPersonTypeRef idref="PietJansen" />
111     </contactPerson>
112     <organisation>
113         <StandardOrganisationTypeRef idref="restaurant" />
114     </organisation>
115     <role>
116         <klantRef idref="oberRol" />
117     </role>
118 </PersonInRole>
119 </visiXML_MessageSchema>

```

8.2.3.11 The tenth message ("tiende_bericht.xml")

The waiter has now received the order from the customer and will send it with the message *msgPlacingOrder* from transaction *T3_OpdrachtKok* (this message has in the MITT as previous *msgPlacingOrder* from transaction *T1_Recording_order*). The directive on the acquisition and blocking of data elements is again in force in this situation.

The message looks like this:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
. visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
4     <msgPlaatsingBestelling id="bericht010">
5         <identification />
6         <dateSend>2008-05-04T00:00:00.0Z</dateSend>

```

```

7      <initiatingTransactionMessageID>bericht009</initiatingTransactionMessageID>
8      <initiatorToExecutor>true</initiatorToExecutor>
9      <messageInTransaction>
10         <BerichtInTransactie13Ref idref="BiT001" />
11      </messageInTransaction>
12      <transaction>
13         <t3_OpdrachtKokRef idref="transactie004" />
14      </transaction>
15      <ceBestelling>
16         <CeBestelling id="bestelling002">
17            <ceInhoudBestelling>
18               <CeInhoudBestelling id="inhoudBestelling002">
19                  <naamGerecht>Biefstuk</naamGerecht>
20                  <opmerking>met mayonaise</opmerking>
21               </CeInhoudBestelling>
22            </ceInhoudBestelling>
23         </CeBestelling>
24      </ceBestelling>
25 </msgPlaatsingBestelling>
26 <BerichtInTransactie13 id="BiT001">
27    <identification />
28 </BerichtInTransactie13>
29 <t3_OpdrachtKok id="transactie004">
30    <number>4</number>
31    <name />
32    <description />
33    <startDate>2008-05-04T00:00:00.0Z</startDate>
34    <endDate>2009-05-04T00:00:00.0Z</endDate>
35    <initiator>
36       <PersonInRoleRef idref="PiR002" />
37    </initiator>
38    <executor>
39       <PersonInRoleRef idref="PiR005" />
40    </executor>
41    <project>
42       <ProjectType1Ref idref="Project1" />
43    </project>
44 </t3_OpdrachtKok>
45 <ProjectType1 id="Project1">
46    <name>Project Top Koks</name>
47    <description>Project voor het opnemen van een bestelling </description>
48    <startDate>2008-05-04T00:00:00.0Z</startDate>
49    <endDate>2009-05-04T00:00:00.0Z</endDate>
50    <ceSOAP>
51       <CeSOAP id="ProjectGegevens">
52          <sOAPProtocol>MIOM</sOAPProtocol>
53          <sOAPServerURL>http://192.168.0.1/visi.wSDL</sOAPServerURL>
54          <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wSDL</
sOAPCentralServerURL>
55       </CeSOAP>
56    </ceSOAP>
57 </ProjectType1>
58 <StandardOrganisationType id="restaurant">
59    <name>Restaurant</name>
60    <abbreviation>RSTR</abbreviation>
61    <contactPerson>
62       <StandardPersonTypeRef idref="PietJansen" />
63    </contactPerson>
64    <ceOrganisatie>
65       <CeOrganisatie id="OrganisatieGegevensRestaurant">
66          <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wSDL</
sOAPServerURL>
67       </CeOrganisatie>
68    </ceOrganisatie>
69 </StandardOrganisationType>
70 <StandardPersonType id="PietJansen">
71    <userName>pj</userName>
72    <name>Piet Jansen</name>
73 </StandardPersonType>
74 <StandardPersonType id="JanToet">

```

```

75     <userName>jt </userName>
76     <name>Jan Toet</name>
77 </StandardPersonType>
78 <ober id="oberRol">
79     <name>Ober Rol</name>
80     <description>Diegene die serveert</description>
81 </ober>
82 <kok id="kokRol">
83     <name>Kok Rol</name>
84     <description>Diegene die kookt</description>
85 </kok>
86 <PersonInRole id="PiR002">
87     <contactPerson>
88         <StandardPersonTypeRef idref="PietJansen" />
89     </contactPerson>
90     <organisation>
91         <StandardOrganisationTypeRef idref="restaurant" />
92     </organisation>
93     <role>
94         <klantRef idref="oberRol" />
95     </role>
96 </PersonInRole>
97 <PersonInRole id="PiR005">
98     <contactPerson>
99         <StandardPersonTypeRef idref="JanToet" />
100    </contactPerson>
101    <organisation>
102        <StandardOrganisationTypeRef idref="restaurant" />
103    </organisation>
104    <role>
105        <klantRef idref="kokRol" />
106    </role>
107 </PersonInRole>
108 </visiXML_MessageSchema>

```

8.2.3.12 The eleventh message ("elfde_bericht.xml")

The cook receives the order and starts looking for the mayonnaise. Unfortunately, the jar has already expired two years ago. As a result, he cannot accept the order in this form. He sends the message *msgOrderDisagree* to the waiter. The directive for the acquisition and blocking of data elements is again in force in this situation.

The message looks like this:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4     .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
5 <msgBestellingNietAkkoord id="bericht011">
6     <identification />
7     <dateSend>2008-05-04T00:00:00.0Z</dateSend>
8     <initiatingTransactionMessageID>bericht009</initiatingTransactionMessageID>
9     <initiatorToExecutor>false</initiatorToExecutor>
10    <messageInTransaction>
11        <BerichtInTransactie15Ref idref="BiT001" />
12    </messageInTransaction>
13    <transaction>
14        <t3_OpdrachtKokRef idref="transactie004" />
15    </transaction>
16    <ceBestelling>
17        <CeBestelling id="bestelling003">
18            <ceInhoudBestelling>
19                <CeInhoudBestelling id="inhoudBestelling003">
20                    <naamGerecht>Biefstuk</naamGerecht>
21                    <opmerking>met mayonaise</opmerking>

```

```

22         </ceInhoudBestelling>
23     </CeBestelling>
24 </ceBestelling>
25 </msgBestellingNietAkkoord>
26 <BerichtInTransactie15 id="BiT001">
27     <identification />
28 </BerichtInTransactie15>
29 <t3_OpdrachtKok id="transactie004">
30     <number>4</number>
31     <name />
32     <description />
33     <startDate>2008-05-04T00:00:00.0Z</startDate>
34     <endDate>2009-05-04T00:00:00.0Z</endDate>
35     <initiator>
36         <PersonInRoleRef idref="PiR002" />
37     </initiator>
38     <executor>
39         <PersonInRoleRef idref="PiR005" />
40     </executor>
41     <project>
42         <ProjectType1Ref idref="Project1" />
43     </project>
44 </t3_OpdrachtKok>
45 <ProjectType1 id="Project1">
46     <name>Project Top Koks</name>
47     <description>Project voor het opnemen van een bestelling </description>
48     <startDate>2008-05-04T00:00:00.0Z</startDate>
49     <endDate>2009-05-04T00:00:00.0Z</endDate>
50     <ceSOAP>
51         <CeSOAP id="ProjectGegevens">
52             <sOAPProtocol>MIOM</sOAPProtocol>
53             <sOAPServerURL>http://192.168.0.1/visi.wSDL</sOAPServerURL>
54             <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wSDL</
sOAPCentralServerURL>
55         </CeSOAP>
56     </ceSOAP>
57 </ProjectType1>
58 <StandardOrganisationType id="restaurant">
59     <name>Restaurant</name>
60     <abbreviation>RSTR</abbreviation>
61     <contactPerson>
62         <StandardPersonTypeRef idref="PietJansen" />
63     </contactPerson>
64     <ceOrganisatie>
65         <CeOrganisatie id="OrganisatieGegevensRestaurant">
66             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wSDL</
sOAPServerURL>
67         </CeOrganisatie>
68     </ceOrganisatie>
69 </StandardOrganisationType>
70 <StandardPersonType id="PietJansen">
71     <userName>pj</userName>
72     <name>Piet Jansen</name>
73 </StandardPersonType>
74 <StandardPersonType id="JanToet">
75     <userName>jt</userName>
76     <name>Jan Toet</name>
77 </StandardPersonType>
78 <ober id="oberRol">
79     <name>Ober Rol</name>
80     <description>Diegene die serveert</description>
81 </ober>
82 <kok id="kokRol">
83     <name>Kok Rol</name>
84     <description>Diegene die kookt</description>
85 </kok>
86 <PersonInRole id="PiR002">
87     <contactPerson>
88         <StandardPersonTypeRef idref="PietJansen" />
89     </contactPerson>

```

```

90     <organisation>
91         <StandardOrganisationTypeRef idref="restaurant" />
92     </organisation>
93     <role>
94         <klantRef idref="oberRol" />
95     </role>
96 </PersonInRole>
97 <PersonInRole id="PiR005">
98     <contactPerson>
99         <StandardPersonTypeRef idref="JanToet" />
100     </contactPerson>
101     <organisation>
102         <StandardOrganisationTypeRef idref="restaurant" />
103     </organisation>
104     <role>
105         <klantRef idref="kokRol" />
106     </role>
107 </PersonInRole>
108 </visiXML_MessageSchema>

```

8.2.3.13 The twelfth message ("twaalfde_bericht.xml")

If the waiter receives the message from the cook indicating that the cook does not agree with the order, the waiter can do nothing other than send a message to the customer indicating that the order is not possible. To do this, he uses the message *msgOrderNotAgreed* from *T1_OpnameOrder* (this message has the previous *msgOrderNotAgreed* from transaction *T3_OpdrachtKok* in the MITT). The directive on the acquisition and blocking of data elements is again in force in this situation.

The message looks like this:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4     .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
5     <msgBestellingNietAkkoord id="bericht012">
6         <identification />
7         <dateSend>2008-05-04T00:00:00.0Z</dateSend>
8         <initiatorToExecutor>false</initiatorToExecutor>
9         <messageInTransaction>
10             <BerichtInTransactie8Ref idref="BiT001" />
11         </messageInTransaction>
12         <transaction>
13             <t1_OpnameBestellingRef idref="transactie001" />
14         </transaction>
15         <ceBestelling>
16             <CeBestelling id="bestelling004">
17                 <ceInhoudBestelling>
18                     <CeInhoudBestelling id="inhoudBestelling004">
19                         <naamGerecht>Biefstuk</naamGerecht>
20                         <opmerking>met mayonaise</opmerking>
21                     </CeInhoudBestelling>
22                 </ceInhoudBestelling>
23             </CeBestelling>
24         </ceBestelling>
25     </msgBestellingNietAkkoord>
26     <BerichtInTransactie8 id="BiT001">
27         <identification />
28     </BerichtInTransactie8>
29     <t1_OpnameBestelling id="transactie001">
30         <number>1</number>
31         <name />
32         <description />
33         <startDate>2008-05-04T00:00:00.0Z</startDate>
34         <endDate>2009-05-04T00:00:00.0Z</endDate>
35         <initiator>
36             <PersonInRoleRef idref="PiR002" />

```

```

36     </initiator>
37     <executor>
38         <PersonInRoleRef idref="PiR001" />
39     </executor>
40     <project>
41         <ProjectType1Ref idref="Project1" />
42     </project>
43 </t1_OpnameBestelling>
44 <ProjectType1 id="Project1">
45     <name>Project Top Koks</name>
46     <description>Project voor het opnemen van een bestelling </description>
47     <startDate>2008-05-04T00:00:00.0Z</startDate>
48     <endDate>2009-05-04T00:00:00.0Z</endDate>
49     <ceSOAP>
50         <CeSOAP id="ProjectGegevens">
51             <sOAPProtocol>MTOM</sOAPProtocol>
52             <sOAPServerURL>http://192.168.0.1/visi.wsd</sOAPServerURL>
53             <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wsd</
sOAPCentralServerURL>
54         </CeSOAP>
55     </ceSOAP>
56 </ProjectType1>
57 <StandardOrganisationType id="consument">
58     <name>Consument</name>
59     <abbreviation>CSMT</abbreviation>
60     <contactPerson>
61         <StandardPersonTypeRef idref="KeesDeVries" />
62     </contactPerson>
63     <ceOrganisatie>
64         <CeOrganisatie id="OrganisatieGegevensConsument">
65             <sOAPServerURL>http://192.168.0.102/specifiek_project/visi.wsd</
sOAPServerURL>
66         </CeOrganisatie>
67     </ceOrganisatie>
68 </StandardOrganisationType>
69 <StandardOrganisationType id="restaurant">
70     <name>Restaurant</name>
71     <abbreviation>RSTR</abbreviation>
72     <contactPerson>
73         <StandardPersonTypeRef idref="PietJansen" />
74     </contactPerson>
75     <ceOrganisatie>
76         <CeOrganisatie id="OrganisatieGegevensRestaurant">
77             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wsd</
sOAPServerURL>
78         </CeOrganisatie>
79     </ceOrganisatie>
80 </StandardOrganisationType>
81 <StandardPersonType id="KeesDeVries">
82     <userName>kvd</userName>
83     <name>Kees de Vries</name>
84 </StandardPersonType>
85 <StandardPersonType id="PietJansen">
86     <userName>pj</userName>
87     <name>Piet Jansen</name>
88 </StandardPersonType>
89 <klant id="klantRol">
90     <name>Klant Rol</name>
91     <description>Diegene die de klant is</description>
92 </klant>
93 <ober id="oberRol">
94     <name>Ober Rol</name>
95     <description>Diegene die serveert</description>
96 </ober>
97 <PersonInRole id="PiR001">
98     <contactPerson>
99         <StandardPersonTypeRef idref="KeesDeVries" />
100     </contactPerson>
101     <organisation>
102         <StandardOrganisationTypeRef idref="consument" />

```

```

103     </organisation>
104     <role>
105         <klantRef idref="klantRol" />
106     </role>
107 </PersonInRole>
108 <PersonInRole id="PiR002">
109     <contactPerson>
110         <StandardPersonTypeRef idref="PietJansen" />
111     </contactPerson>
112     <organisation>
113         <StandardOrganisationTypeRef idref="restaurant" />
114     </organisation>
115     <role>
116         <klantRef idref="oberRol" />
117     </role>
118 </PersonInRole>
119 </visiXML_MessageSchema>

```

8.2.3.14 The thirteenth message (“dertiende_bericht.xml”)

The customer is obviously not happy when he receives the message from the waiter that the order is not possible. He considers walking out of the restaurant angrily, but then takes a different tack. Instead of the steak, he only orders a Carpaccio. For this, he uses the message *msgPlaceOrder*. Because the message requires the order to be modified, an exception is made to the guideline for the inheritance and blocking of data elements. The element condition *ElementCondition1* is included in the framework for this purpose. This condition indicates that in this MITT *MessageInTransaction9* the condition is FREE. This means that orders can be adjusted. The customer does this in this message.

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4   .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
5   <msgPlaatsingBestelling id="bericht013">
6       <identification />
7       <dateSend>2008-05-04T00:00:00.0Z</dateSend>
8       <initiatorToExecutor>true</initiatorToExecutor>
9       <messageInTransaction>
10         <BerichtInTransactie9Ref idref="BiT001" />
11       </messageInTransaction>
12       <transaction>
13         <t1_OpnameBestellingRef idref="transactie001" />
14       </transaction>
15       <ceBestelling>
16         <CeBestelling id="bestelling005">
17             <ceInhoudBestelling>
18                 <CeInhoudBestelling id="inhoudBestelling005">
19                     <naamGerecht>Carpaccio</naamGerecht>
20                     <opmerking>Ik hoop dat jullie dit wel hebben</opmerking>
21                 </CeInhoudBestelling>
22             </ceInhoudBestelling>
23         </CeBestelling>
24       </ceBestelling>
25   </msgPlaatsingBestelling>
26   <BerichtInTransactie9 id="BiT001">
27       <identification />
28   </BerichtInTransactie9>
29   <t1_OpnameBestelling id="transactie001">
30       <number>1</number>
31       <name />
32       <description />
33       <startDate>2008-05-04T00:00:00.0Z</startDate>
34       <endDate>2009-05-04T00:00:00.0Z</endDate>
35       <initiator>
36         <PersonInRoleRef idref="PiR002" />
37       </initiator>
38       <executor>

```



```

38         <PersonInRoleRef idref="PiR001" />
39     </executor>
40 </project>
41     <ProjectType1Ref idref="Project1" />
42 </project>
43 </t1_OpnameBestelling>
44 <ProjectType1 id="Project1">
45     <name>Project Top Koks</name>
46     <description>Project voor het opnemen van een bestelling </description>
47     <startDate>2008-05-04T00:00:00.0Z</startDate>
48     <endDate>2009-05-04T00:00:00.0Z</endDate>
49     <ceSOAP>
50         <CeSOAP id="ProjectGegevens">
51             <sOAPProtocol>MTOM</sOAPProtocol>
52             <sOAPServerURL>http://192.168.0.1/visi.wSDL</sOAPServerURL>
53             <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wSDL</
sOAPCentralServerURL>
54         </CeSOAP>
55     </ceSOAP>
56 </ProjectType1>
57 <StandardOrganisationType id="consument">
58     <name>Consument</name>
59     <abbreviation>CSMT</abbreviation>
60     <contactPerson>
61         <StandardPersonTypeRef idref="KeesDeVries" />
62     </contactPerson>
63     <ceOrganisatie>
64         <CeOrganisatie id="OrganisatieGegevensConsument">
65             <sOAPServerURL>http://192.168.0.102/specifiek_project/visi.wSDL</
sOAPServerURL>
66         </CeOrganisatie>
67     </ceOrganisatie>
68 </StandardOrganisationType>
69 <StandardOrganisationType id="restaurant">
70     <name>Restaurant</name>
71     <abbreviation>RSTR</abbreviation>
72     <contactPerson>
73         <StandardPersonTypeRef idref="PietJansen" />
74     </contactPerson>
75     <ceOrganisatie>
76         <CeOrganisatie id="OrganisatieGegevensRestaurant">
77             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wSDL</
sOAPServerURL>
78         </CeOrganisatie>
79     </ceOrganisatie>
80 </StandardOrganisationType>
81 <StandardPersonType id="KeesDeVries">
82     <userName>kvd</userName>
83     <name>Kees de Vries</name>
84 </StandardPersonType>
85 <StandardPersonType id="PietJansen">
86     <userName>pj</userName>
87     <name>Piet Jansen</name>
88 </StandardPersonType>
89 <klant id="klantRol">
90     <name>Klant Rol</name>
91     <description>Diegene die de klant is</description>
92 </klant>
93 <ober id="oberRol">
94     <name>Ober Rol</name>
95     <description>Diegene die serveert</description>
96 </ober>
97 <PersonInRole id="PiR001">
98     <contactPerson>
99         <StandardPersonTypeRef idref="KeesDeVries" />
100     </contactPerson>
101     <organisation>
102         <StandardOrganisationTypeRef idref="consument" />
103     </organisation>
104     <role>

```

```

105         <klantRef idref="klantRol" />
106     </role>
107 </PersonInRole>
108 <PersonInRole id="PiR002">
109     <contactPerson>
110         <StandardPersonTypeRef idref="PietJansen" />
111     </contactPerson>
112     <organisation>
113         <StandardOrganisationTypeRef idref="restaurant" />
114     </organisation>
115     <role>
116         <klantRef idref="oberRol" />
117     </role>
118 </PersonInRole>
119 </visiXML_MessageSchema>

```

The waiter has now received the order again from the customer and will send it with the message *msgPlacingOrder* from transaction *T3_OpdrachtKok* (this message has in the MITT as previous *msgPlacingOrder* from transaction *textitT1_Recording order*). The directive on the acquisition and blocking of data elements is again in force in this situation.

The message looks like this:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4     .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
5     <msgPlaatsingBestelling id="bericht014">
6         <identification />
7         <dateSend>2008-05-04T00:00:00.0Z</dateSend>
8         <initiatingTransactionMessageID>bericht013</initiatingTransactionMessageID>
9         <initiatorToExecutor>true</initiatorToExecutor>
10        <messageInTransaction>
11            <BerichtInTransactie13Ref idref="BiT001" />
12        </messageInTransaction>
13        <transaction>
14            <t1_OpnameBestellingRef idref="transactie005" />
15        </transaction>
16        <ceBestelling>
17            <CeBestelling id="bestelling006">
18                <ceInhoudBestelling>
19                    <CeInhoudBestelling id="inhoudBestelling006">
20                        <naamGerecht>Carpatio</naamGerecht>
21                        <opmerking>Ik hoop dat jullie dit wel hebben</opmerking>
22                    </CeInhoudBestelling>
23                </ceInhoudBestelling>
24            </CeBestelling>
25        </ceBestelling>
26    </msgPlaatsingBestelling>
27    <BerichtInTransactie13 id="BiT001">
28        <identification />
29    </BerichtInTransactie13>
30    <t3_OpdrachtKok id="transactie005">
31        <number>5</number>
32        <name />
33        <description />
34        <startDate>2008-05-04T00:00:00.0Z</startDate>
35        <endDate>2009-05-04T00:00:00.0Z</endDate>
36        <initiator>
37            <PersonInRoleRef idref="PiR002" />
38        </initiator>
39        <executor>
40            <PersonInRoleRef idref="PiR005" />
41        </executor>
42        <project>
43            <ProjectType1Ref idref="Project1" />
44        </project>

```

```

44 </t3_OpdrachtKok>
45 <ProjectType1 id="Project1">
46   <name>Project Top Koks</name>
47   <description>Project voor het opnemen van een bestelling </description>
48   <startDate>2008-05-04T00:00:00.0Z</startDate>
49   <endDate>2009-05-04T00:00:00.0Z</endDate>
50   <ceSOAP>
51     <CeSOAP id="ProjectGegevens">
52       <sOAPProtocol>MTOM</sOAPProtocol>
53       <sOAPServerURL>http://192.168.0.1/visi.wSDL</sOAPServerURL>
54       <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wSDL</
sOAPCentralServerURL>
55     </CeSOAP>
56   </ceSOAP>
57 </ProjectType1>
58 <StandardOrganisationType id="restaurant">
59   <name>Restaurant</name>
60   <abbreviation>RSTR</abbreviation>
61   <contactPerson>
62     <StandardPersonTypeRef idref="PietJansen" />
63   </contactPerson>
64   <ceOrganisatie>
65     <CeOrganisatie id="OrganisatieGegevensRestaurant">
66       <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wSDL</
sOAPServerURL>
67     </CeOrganisatie>
68   </ceOrganisatie>
69 </StandardOrganisationType>
70 <StandardPersonType id="PietJansen">
71   <userName>pj</userName>
72   <name>Piet Jansen</name>
73 </StandardPersonType>
74 <StandardPersonType id="JanToet">
75   <userName>jt</userName>
76   <name>Jan Toet</name>
77 </StandardPersonType>
78 <ober id="oberRol">
79   <name>Ober Rol</name>
80   <description>Diegene die serveert</description>
81 </ober>
82 <kok id="kokRol">
83   <name>Kok Rol</name>
84   <description>Diegene die kookt</description>
85 </kok>
86 <PersonInRole id="PiR002">
87   <contactPerson>
88     <StandardPersonTypeRef idref="PietJansen" />
89   </contactPerson>
90   <organisation>
91     <StandardOrganisationTypeRef idref="restaurant" />
92   </organisation>
93   <role>
94     <klantRef idref="oberRol" />
95   </role>
96 </PersonInRole>
97 <PersonInRole id="PiR005">
98   <contactPerson>
99     <StandardPersonTypeRef idref="JanToet" />
100   </contactPerson>
101   <organisation>
102     <StandardOrganisationTypeRef idref="restaurant" />
103   </organisation>
104   <role>
105     <klantRef idref="kokRol" />
106   </role>
107 </PersonInRole>
108 </visiXML_MessageSchema>

```

8.2.3.15 The fifteenth message ("vijftiende_bericht.xml")

The cook receives the adjusted order. Carpaccio is never a problem, so he can accept the order. He sends the message *msgOrderAccept* to the waiter. The directive on the acquisition and blocking of data elements is again in force in this situation.

The message looks like this:

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
4     .visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
5   <msgBestellingAkkoord id="bericht015">
6     <identification />
7     <dateSend>2008-05-04T00:00:00.0Z</dateSend>
8     <initiatingTransactionMessageID>bericht013</initiatingTransactionMessageID>
9     <initiatorToExecutor>false</initiatorToExecutor>
10    <messageInTransaction>
11      <BerichtInTransactie14Ref idref="BiT001" />
12    </messageInTransaction>
13    <transaction>
14      <t1_OpnameBestellingRef idref="transactie005" />
15    </transaction>
16    <ceBestelling>
17      <CeBestelling id="bestelling007">
18        <ceInhoudBestelling>
19          <CeInhoudBestelling id="inhoudBestelling007">
20            <naamGerecht>Carpaccio</naamGerecht>
21            <opmerking>Ik hoop dat jullie dit wel hebben</opmerking>
22          </CeInhoudBestelling>
23        </ceInhoudBestelling>
24      </CeBestelling>
25    </ceBestelling>
26  </msgBestellingAkkoord>
27  <BerichtInTransactie14 id="BiT001">
28    <identification />
29  </BerichtInTransactie14>
30  <t3_OpdrachtKok id="transactie005">
31    <number>5</number>
32    <name />
33    <description />
34    <startDate>2008-05-04T00:00:00.0Z</startDate>
35    <endDate>2009-05-04T00:00:00.0Z</endDate>
36    <initiator>
37      <PersonInRoleRef idref="PiR002" />
38    </initiator>
39    <executor>
40      <PersonInRoleRef idref="PiR005" />
41    </executor>
42    <project>
43      <ProjectType1Ref idref="Project1" />
44    </project>
45  </t3_OpdrachtKok>
46  <ProjectType1 id="Project1">
47    <name>Project Top Koks</name>
48    <description>Project voor het opnemen van een bestelling</description>
49    <startDate>2008-05-04T00:00:00.0Z</startDate>
50    <endDate>2009-05-04T00:00:00.0Z</endDate>
51    <ceSOAP>
52      <CeSOAP id="ProjectGegevens">
53        <sOAPProtocol>MTOM</sOAPProtocol>
54        <sOAPServerURL>http://192.168.0.1/visi.wsdl</sOAPServerURL>
55        <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wsdl</
56        sOAPCentralServerURL>
57      </CeSOAP>
58    </ProjectType1>
59  <StandardOrganisationType id="restaurant">
60    <name>Restaurant</name>
61    <abbreviation>RSTR</abbreviation>
```

```

61     <contactPerson>
62         <StandardPersonTypeRef idref="PietJansen" />
63     </contactPerson>
64     <ceOrganisatie>
65         <CeOrganisatie id="OrganisatieGegevensRestaurant">
66             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wsdl</
sOAPServerURL>
67         </CeOrganisatie>
68     </ceOrganisatie>
69 </StandardOrganisationType>
70 <StandardPersonType id="PietJansen">
71     <userName>pj</userName>
72     <name>Piet Jansen</name>
73 </StandardPersonType>
74 <StandardPersonType id="JanToet">
75     <userName>jt</userName>
76     <name>Jan Toet</name>
77 </StandardPersonType>
78 <ober id="oberRol">
79     <name>Ober Rol</name>
80     <description>Diegene die serveert</description>
81 </ober>
82 <kok id="kokRol">
83     <name>Kok Rol</name>
84     <description>Diegene die kookt</description>
85 </kok>
86 <PersonInRole id="PiR002">
87     <contactPerson>
88         <StandardPersonTypeRef idref="PietJansen" />
89     </contactPerson>
90     <organisation>
91         <StandardOrganisationTypeRef idref="restaurant" />
92     </organisation>
93     <role>
94         <klantRef idref="oberRol" />
95     </role>
96 </PersonInRole>
97 <PersonInRole id="PiR005">
98     <contactPerson>
99         <StandardPersonTypeRef idref="JanToet" />
100     </contactPerson>
101     <organisation>
102         <StandardOrganisationTypeRef idref="restaurant" />
103     </organisation>
104     <role>
105         <klantRef idref="kokRol" />
106     </role>
107 </PersonInRole>
108 </visiXML_MessageSchema>

```

8.2.3.16 The sixteenth message ("zestiende_bericht.xml")

When the waiter receives the message from the chef that he agrees with the order, the waiter can send the message to the customer to confirm the order. For this, he uses the message *msgBestellingAccoord* from *T1_OpnameBestelling* (this message has the previous *msgBestellingAccoord* from transaction *T3_OpdrachtKok* in the MITT). The directive on the acquisition and blocking of data elements is again in force in this situation. The message is also the last message from the primary transaction *T1_RecordingOrder*. The transaction has now been completed.

The message looks like this:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <visiXML_MessageSchema xmlns="http://www.visi.nl/schemas/ 20160331 /TestFramework "
3     xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www
. visi.nl/schemas/ 20160331 /TestFramework 10.xsd ">
4     <msgBestellingAkkoord id="bericht016">

```

```

5      <identification />
6      <dateSend>2008-05-04T00:00:00.0Z</dateSend>
7      <initiatorToExecutor>false</initiatorToExecutor>
8      <messageInTransaction>
9          <BerichtInTransactie10Ref idref="BiT001" />
10     </messageInTransaction>
11     <transaction>
12         <t1_OpnameBestellingRef idref="transactie001" />
13     </transaction>
14     <ceBestelling>
15         <CeBestelling id="bestelling008">
16             <ceInhoudBestelling>
17                 <CeInhoudBestelling id="inhoudBestelling008">
18                     <naamGerecht>Carpatio</naamGerecht>
19                     <opmerking>Ik hoop dat jullie dit wel hebben</opmerking>
20                 </CeInhoudBestelling>
21             </ceInhoudBestelling>
22         </CeBestelling>
23     </ceBestelling>
24 </msgBestellingAkkoord>
25 <BerichtInTransactie10 id="BiT001">
26     <identification />
27 </BerichtInTransactie10>
28 <t1_OpnameBestelling id="transactie001">
29     <number>1</number>
30     <name />
31     <description />
32     <startDate>2008-05-04T00:00:00.0Z</startDate>
33     <endDate>2009-05-04T00:00:00.0Z</endDate>
34     <initiator>
35         <PersonInRoleRef idref="PiR002" />
36     </initiator>
37     <executor>
38         <PersonInRoleRef idref="PiR001" />
39     </executor>
40     <project>
41         <ProjectType1Ref idref="Project1" />
42     </project>
43 </t1_OpnameBestelling>
44 <ProjectType1 id="Project1">
45     <name>Project Top Koks</name>
46     <description>Project voor het opnemen van een bestelling </description>
47     <startDate>2008-05-04T00:00:00.0Z</startDate>
48     <endDate>2009-05-04T00:00:00.0Z</endDate>
49     <ceSOAP>
50         <CeSOAP id="ProjectGegevens">
51             <sOAPProtocol>MTOM</sOAPProtocol>
52             <sOAPServerURL>http://192.168.0.1/visi.wSDL</sOAPServerURL>
53             <sOAPCentralServerURL>http://www.crow.nl/testcases/case001/visi.wSDL</
sOAPCentralServerURL>
54         </CeSOAP>
55     </ceSOAP>
56 </ProjectType1>
57 <StandardOrganisationType id="consument">
58     <name>Consument</name>
59     <abbreviation>CSMT</abbreviation>
60     <contactPerson>
61         <StandardPersonTypeRef idref="KeesDeVries" />
62     </contactPerson>
63     <ceOrganisatie>
64         <CeOrganisatie id="OrganisatieGegevensConsument">
65             <sOAPServerURL>http://192.168.0.102/specifiek_project/visi.wSDL</
sOAPServerURL>
66         </CeOrganisatie>
67     </ceOrganisatie>
68 </StandardOrganisationType>
69 <StandardOrganisationType id="restaurant">
70     <name>Restaurant</name>
71     <abbreviation>RSTR</abbreviation>
72     <contactPerson>

```

```

73         <StandardPersonTypeRef idref="PietJansen" />
74     </contactPerson>
75     <ceOrganisatie>
76         <CeOrganisatie id="OrganisatieGegevensRestaurant">
77             <sOAPServerURL>http://192.168.0.105/specifiek_project/visi.wsdl</
sOAPServerURL>
78         </CeOrganisatie>
79     </ceOrganisatie>
80 </StandardOrganisationType>
81 <StandardPersonType id="KeesDeVries">
82     <userName>kvd</userName>
83     <name>Kees de Vries</name>
84 </StandardPersonType>
85 <StandardPersonType id="PietJansen">
86     <userName>pj</userName>
87     <name>Piet Jansen</name>
88 </StandardPersonType>
89 <klant id="klantRol">
90     <name>Klant Rol</name>
91     <description>Diegene die de klant is</description>
92 </klant>
93 <ober id="oberRol">
94     <name>Ober Rol</name>
95     <description>Diegene die serveert</description>
96 </ober>
97 <PersonInRole id="PiR001">
98     <contactPerson>
99         <StandardPersonTypeRef idref="KeesDeVries" />
100     </contactPerson>
101     <organisation>
102         <StandardOrganisationTypeRef idref="consument" />
103     </organisation>
104     <role>
105         <klantRef idref="klantRol" />
106     </role>
107 </PersonInRole>
108 <PersonInRole id="PiR002">
109     <contactPerson>
110         <StandardPersonTypeRef idref="PietJansen" />
111     </contactPerson>
112     <organisation>
113         <StandardOrganisationTypeRef idref="restaurant" />
114     </organisation>
115     <role>
116         <klantRef idref="oberRol" />
117     </role>
118 </PersonInRole>
119 </visiXML_MessageSchema>

```

8.3 Procedure for applying for a VISI quality mark

Version 1.02, dd 22 March 2012

8.3.1 Application for quality mark

- The supplier who wishes to obtain the VISI quality mark must submit a written request to CROW via the application form on the VISI website. This allows all applicants to submit the application in the same transparent manner.
- On the application form, the supplier declares by placing a ‘check mark’ next to the statement that he:
 - is aware of the requirements and guidelines regarding the VISI system;
 - agrees with the procedure for applying for the VISI quality mark;

- agrees with the contents of the VISI quality mark agreement. The supplier also states on the application form:
 - * its official company name with full (visiting) address;
 - * title, initials, surname and position of the representative who will sign the VISI quality mark agreement on behalf of the supplier;
 - * the email address of this contact person;
 - * the full name and surname of the software product for which the supplier wishes to obtain the VISI quality mark;
 - * the version number of this software product;
 - * the website where the product information can be found (if available).

Signing the quality mark agreement and issuing the certificate takes place after the test has been successfully completed and the procedure has been completed correctly.

- The application form can only be submitted if it has been completed in full and if the supplier agrees with this procedure, with the requirements and guidelines, and with the content of the quality mark agreement.
- This procedure, the MODEL of the ‘VISI quality mark agreement for VISI-compatible software’ as well as the requirements and guidelines that the software product must meet to obtain the VISI quality mark, can be freely downloaded from the VISI website.
- The application form is submitted to the responsible CROW project manager via the VISI website. The supplier will receive a cc of this message. The CROW project manager checks whether the application form has been completed in full and with correct text.
- If the submitted application form has been completed incorrectly or incompletely, CROW will inform the supplier of this as soon as possible, but no later than within 7 working days of receipt.
- If the submitted application form is completed correctly and completely, CROW will issue an order to the testing body (TNO) to carry out the test within 7 working days of receipt. CROW will inform the supplier of this order.

8.3.2 Test

- The TNO person who carries out the test will make an appointment with the supplier within 5 working days of receiving the order, about the date on which the test will be carried out. The date will be determined by mutual agreement.
- The supplier must arrange for its own account on the day on which the test is carried out:
 - that two separate information systems are ready with the software product to be tested, which can communicate with each other with their own SOAP server, so that the handling of VISI messages can be tested based on the required SOAP protocol. Both systems must be set up based on the version of the VISI system to be tested, with the standard test framework and project-specific messages for the test.
 - that a workplace has been set up with access to both information systems. This workplace must be ready for use before the start of the first part of the day on which the test is performed. Everything must be ready for use at the start of the day on which the test is performed.
 - that an employee is available with sufficient technical background of VISI and of both systems to be used, to answer questions, and to support the TNO person who carries out the test in the actions that need to be taken to demonstrate that the software to be tested meets the requirements. This employee must be immediately available (on call) throughout the day.

- If TNO finds shortcomings during the test that are of such a nature that this will not lead to a VISI quality mark, and if these shortcomings are not immediately resolved in accordance with the requirements set for the VISI quality mark, then TNO will carry out the test breakdown. TNO will immediately inform CROW of the reason in writing. In turn, CROW will confirm to the supplier in writing within 3 working days that the test has been cancelled and why.
- If TNO has determined that the supplier's software product meets the stated requirements, TNO will communicate its findings in writing to CROW within 5 working days after the test in an advisory report.
- CROW assesses whether the procedure has been completed correctly and, if so and if TNO's advice is positive, it will issue the VISI quality mark within 10 working days. The VISI quality mark guarantees that the tested software product has been tested with positive results for the 'VISI compatibility' property.
- If, in the opinion of CROW, the procedure has *not* been completed correctly, CROW will inform the supplier of this in writing within 3 working days of the discovery.
- If a test has been cancelled or has ended with a negative result, or if the procedure has not been completed correctly, and the supplier still wants to obtain the VISI quality mark, the supplier will have to submit a new application, and the entire procedure will have to be repeated again. run through.

8.3.3 VISI quality mark and VISI certificate

- For the provision of the VISI quality mark, CROW and the supplier will both sign the '*VISI quality mark agreement for VISI-compatible software*'.
The appendix A to the quality mark agreement, which must also be mutually signed, specifies which version of the VISI system and which software product it concerns (with version number). The VISI quality mark and the VISI mark are described in Appendix B to the agreement.
- With the VISI quality mark agreement and the associated appendices, the supplier also receives the '*VISI certificate*' from CROW, with which the supplier can demonstrate, at the request of customers, that CROW has issued the VISI quality mark for the software product in question.
- CROW and the supplier will make further agreements regarding the date and location of signing, and any publicity activities associated with signing the VISI quality mark agreement.

8.3.4 Time expenditure and costs

- The time spent by the supplier and TNO (excluding any travel time) is estimated as follows:
 - Set up testing environment (approx. 0.5 day for supplier).
 - Carrying out the test (approx. 1 day for both TNO and supplier).
 - Report of test (approx. 0.5 day for TNO).

The costs of this are borne by the supplier. CROW will pass on TNO's bill (actual time spent plus any travel costs), plus 10% of CROW's office costs, to the supplier.

- If a test has been aborted, or has ended with a negative result, or if the procedure has not been completed correctly, the time spent by the supplier and TNO up to that point will be borne by the supplier. CROW will then pass on TNO's bill (actual time spent plus any travel costs), plus 10% CROW's office costs, to the supplier.

8.4 VISI Quality mark Test

8.4.1 VISI Software conformance test

- Date:
- Supplier:
- Address:
- Contact:
- VISI Systematic: 1.6 yet to describe: min-max rows, authentication soap
- Software product:
- Version code:
- Test executed by: Peter Willems
- Inspection Institute: TNO

A software product that supports VISI communication and meets the conformance requirements can generate messages for, and interpret messages from, any other software product that supports VISI communication and meets the conformance requirements, based on any VISI framework offered. From systematics version 1.2, matters that influence the behaviour of the software are also recorded in the VISI framework. Since the behaviour on these points must be in accordance with the VISI framework, it is tested whether the software interprets these points correctly.

8.4.2 Test Environment

- Two separate environments that can communicate with each other using their own SOAP server (so that the handling of VISI messages can be tested based on the agreed SOAP protocol)
- The individual systems must be set up based on the VISI system 1.3 with the standard test framework and project-specific message for testing.
- A workplace with access to the information system
- An employee who has sufficient technical background in VISI and the system itself to answer questions and support the TNO person performing the assessment with the actions that need to be taken to demonstrate certain requirements.

8.4.3 Test

- http/https
 - Verifying addresses on HTTPS
 - A project-specific message containing an (http) SOAP server address should be rejected.
- 10.xsd validation
 - Outgoing VISI messages must comply with the 10.xsd definition of the current framework of the project in question.
 - Checking a sent message against the 10.xsd validation in an XML editor.
 - Have the supplier demonstrate that outgoing messages are compared with 10.xsd and not sent if they do not match
 - Incoming messages should not be accepted by the system if they do not match the 10.xsd definition of the current framework of the project in question.

- Have the supplier demonstrate that incoming messages are compared with 10.xsd and not accepted if they do not match.
 - In addition, in the event of an invalid message, the system must send the correct error code in the response message from the soap server.
- Soapserver
 - The SOAP server must function according to the current version of the SOAP guideline.
 - Have the supplier demonstrate that the response messages are handled correctly.
- Succession and authorisation
 - If a successor is indicated for a person in the project-specific message, all communication should be transferred to the successor of that person. Persons can also be authorised to act on behalf of the role filler.
 - Initiate a transaction based on a specific person in a role. Then, a successor is defined in the project-specific message for this person. The communication should now be able to be handled by the successor.
 - In the project-specific message, a person is authorised to act on behalf of the role filler. The software provides the opportunity for this.
- Attachment capacity: An attachment of up to 120 Mb must be able to be sent with the VISI message.
- Appendix Types: Software product only shows the appendix types that can actually be selected.
- Required: Subject, A message cannot be sent without the subject field being filled in.
- Transaction number: Each transaction is given a unique identification.
- Meta-Framework: The meta-framework transaction is being executed correctly.
- Framework namespace: The software product is capable of having more than one framework operational at the same time.
The software product ensures that no different frameworks with the same namespace exist in a project.
- element conditions: If more than one element condition applies in a given situation, the precedence table must be used correctly.

8.4.4 Scenario's

A number of scenarios have been developed from the VISI test framework. A number of scenarios of increasing complexity will be tested.

- Scenario 2a:
 - Attempts to modify messages that are not permitted by the framework must be rejected by the software.
 - Checking whether the order is correctly transferred from the customer to the waiter and from the waiter to the chef.
 - Check that no messages can be sent that conflict with the structure of the framework.
- Scenario 4c:
 - Correct operation of framework structures.
 - ElementCondition is processed correctly on message 6
- Scenario 5b:
 - Correct functioning of message flow (open-secondary-transaction attribute values).
 - Supplier demonstrates that the course of the message traffic can be traced based on the individual messages (previous attribute values).

8.4.5 VISI 1.4 features

- Security VISI communication: A project-specific message containing a (http) SOAP server address should be rejected.
- Test for DateTime, Boolean, etc. Test if the following fields work as follows (Test Scenario 1):
 - Boolean: This must contain a checkbox or 0 - 1 or true-false or true-false (required)
 - Datetime/Date and Time: Date and time must be filled in (required)
 - Time: A time must be entered (required)
 - Decimal/Decimal: A number, with or without decimal values, must be entered (required)
 - Integer: An integer must be entered (required)
 - Option List: One of the values "Option 1", "Option 2" or "Option 3" must be entered (required)
 - String / Free text: Any characters can be entered (not required)
- Archiving and reactivating projects
 - Archive a VISI-project.
 - Import the archived VISI-project into a different VISI-environment.
 - Compare the old project with the imported one.
 - The content of both projects, including frameworks, project-specific messages, transactions, attachments, etc. must be exactly the same.
 - If necessary, after adjusting the soap address, communication with the project should be possible.
- Order of messages to be sent: The waiter sends the order to the cook. Check that the waiter is not allowed to start a new order subtransaction.
- Testing the functionality for initiating a project with the meta framework.
 - Scenario A.1: Project Initiator offers Project Executor a proposal for a new VISI project with a new framework and project-specific message as attachments. Project Executor responds to the proposal with acceptance of the new VISI project. Project Initiator confirms the project initiation to the Project Executor. Project Executor reports back to Project Initiator that the project has been set up.
 - Scenario A.2: Project Initiator offers Project Executor a proposal for a new VISI project with a new framework and project-specific message as attachments. Project Executor responds to the proposal with acceptance of the new VISI project. Project Initiator confirms the project initiation to the Project Executor. Project Executor reports back to Project Initiator that the project has been set up.
 - Scenario B: Change Initiator offers VISI Communication Manager a proposal to change an existing VISI project with an amended framework and an amended project-specific message as attachments. VISI Communication Manager responds to the proposal with approval. Change Initiator confirms the proposed change with a proposal of an effective date and time to the VISI Communicator. VISI Communicator responds to the proposal with approval. Change Initiator confirms the effective date and time.

8.4.6 VISI 1.5 features

minrow - maxrow

8.4.7 VISI 1.6 features

Attachment size: An attachment of up to 10GB must be able to be sent with the VISI message.

8.4.8 Additional requirements

- A: Ensuring the authenticity of exchanged VISI messages and associated files.
 - A1
 - * A VISI message with any associated file(s) must be stored in the VISI product in such a way that the message and associated file(s) can be retrieved in relation to each other at all times. This relationship may never be broken.
 - * Analyse the functioning of the search functions provided. Can the files of messages be shown in the search result? And can the associated message be obtained in a search result of files? The relationship between the message and the associated file must not be lost during the test.
 - A2
 - * It shall not be possible to modify or delete originally sent and/or received VISI messages and associated files in the VISI product.
 - * Analyse the user interface for features that appear to offer a way to modify and/or delete a message and/or linked file. Then test the effect of the features found.
 - A3
 - * The VISI product must check whether a user is authorised to access the system. Access must be obtained by means that the person in question can keep under his/her sole control.
 - * Make several attempts to gain unauthorised access to the system.
 - A4
 - * Only the persons specified in the project-specific message are allowed to perform VISI communication in the VISI product according to the VISI roles assigned to them.
 - * Test whether messages are visible that are not intended for the role/person in question.
- B: Presenting the communication structure defined in a VISI framework in a recognisable manner.
 - B1
 - * The software may only show a user those transactions for which the user is responsible based on their role(s). However, this requirement does not apply to a so-called 'viewing function'. This is a facility for the administrator of the VISI product that offers the possibility to make all message traffic visible (without the possibility to influence the course of the message traffic).
 - * Analyse the user interface to see if this is indeed the case
 - B2
 - * After selecting a VISI message, a user should be able to see which files are linked to the message.
 - * Analyse the user interface to see if this link is presented in a visually distinctive way after selecting a message.
 - B3
 - * Once a message has been selected, the following data should be available:
 - The transaction type with associated 'description'
 - The naming of the message type
 - The initiator
 - The executor
 - The date the message was sent
 - * Check the user interface for the retrievability of the mentioned information fields.
 - B4
 - * It should be possible to display transactions in relation to each other in the manner defined in the underlying VISI framework and where the user is involved as a person-in-role (initiator or executor).

- * If there is a transaction of which the current transaction is a subtransaction, then that information must be shown.
 If a subtransaction can be started from the current node in the framework, then that information must be shown.
 If one or more subtransactions have been started from the current node, then that information must be shown.
 Is it shown that the current transaction can (not) be continued if there are still subtransactions that have not yet been completed?
- C: The ability to reproduce VISI communications after completion of the project.
 - C1
 - * In order to perform an audit trail, messages must be reproducible in the order in which they were sent or received, within the context of a VISI user's own organisation and the system in use.
 - * Does the VISI product provide a way to track and analyse all communications after the project has ended?
 - C2
 - * Linked files must be exportable in the format in which they were sent.
 - * For some file types, compare the original file with the exported file. For example, by means of a checksum (MD5 or SHA-1 hash).

8.5 5. VISI Quality mark

To ensure that a specific version of the system has been implemented correctly, CROW can award the VISI quality mark to software that meets the requirements and guidelines set by the VISI standard. The inspection itself is carried out by an independent inspection institute on behalf of CROW.

The VISI quality mark guarantees that the tested software product has been tested with a positive result for the property VISI compatibility. The VISI quality mark refers to the version number of the VISI standard to which it applies, and to the version number of the software in question. It is the responsibility of the software supplier to have a current VISI quality mark in his possession.

The VISI quality mark is awarded according to a fixed procedure, which is briefly described here. The complete procedure, the model of the 'Quality mark agreement for software working according to the VISI system' as well as the requirements and guidelines that the software product must meet in order to obtain the VISI quality mark, can be downloaded freely from the VISI website (www.crow.nl/visi).

The supplier who wants to obtain the VISI quality mark must submit a written request to CROW via the application form on the VISI website. This allows all applicants to submit the application in a uniform manner. With the application form, the supplier indicates, among other things, which software product and version it concerns, that he has taken note of the requirements and guidelines, that he agrees with the procedure and that he agrees with the content of the VISI Quality Mark Agreement. The signing of the quality mark agreement itself and the awarding of the certificate take place after the test has been successfully completed and the procedure has been followed correctly.

The test itself is carried out on behalf of CROW by an independent testing body (currently, this is TNO). The date on which the test is carried out is determined in mutual consultation with the software supplier. The supplier then makes all the necessary hardware and software available. If the testing body finds shortcomings that are of such a nature that this will not lead to a VISI quality mark, and if these shortcomings are not immediately resolved in accordance with the requirements set for the VISI quality mark, the test will be terminated and no quality mark will be awarded. If the supplier still wants to obtain the VISI quality mark, he will have to submit a new application, and the entire procedure will have to be repeated.

For the provision of the VISI quality mark, CROW and the supplier will both sign the ‘Quality mark agreement for software working according to the VISI system’. Appendix A to the quality mark agreement, which must also be signed by both parties, will specify which version of the VISI system and which software product (with version number) it concerns. Appendix B to the quality mark agreement describes the VISI mark. With the Quality Mark Agreement, the supplier will also receive the ‘VISI certificate’ from CROW for the most recently tested version, with which the supplier can demonstrate, at the request of customers, that CROW has issued the VISI quality mark for the software product in question.

The time spent by the supplier and TNO is estimated at 3 to 4 half days, for a test that is completed in one go. The costs of the test are for the account of the supplier.

HS 8

9 Testprotocol

The scenarios below are generated from the scenario visualizer based on the test framework interaction_framework_visi1_8_in1_8format_v_3h.xml.

9.1 Test framework VISI 1.8 - Scenario overview

The test protocol contains the following scenarios for message exchange:

- Login into the environment
- User has a Role
- Scenario 001a Send message and reply
- Scenario 001b Same Init-Exec
- Scenario 001c Send message with response date
- Scenario 002 UDT - XSD Restrictions No Accents Required Input
- Scenario 002 UDT - XSD Restrictions No Accents Optional Input
- Scenario 002 Appendix Mandatory
- Scenario 002 Appendix Type Drawing
- Scenario 002 Table required rows
- Scenario 002 Table optional rows
- Scenario 003c UTF-8
- Scenario 004a / 004b Serie
- Scenario 005a / 005b serie
- Scenario 005c / 005d - serie loop
- Scenario 006a / 006b / 006c serie
- Scenario 007a Element Conditions
- Scenario 007a / 007b Element Conditions table
- Scenario 008a Promise obligatory
- Scenario 008b Promise optional
- Scenario 008c Repeated promise
- Scenario 10 Sendafter AND-Conditie
- Scenario 11a Sequence number - change order A-B-Bi-A
- Scenario 11b Sequence number - change order C-B-A-Ai
- Scenario 11c Sequence number - change order D-B
- Scenario 11d Sequence number - change order B-A
- Scenario 11e Sequence number - information document A
- Scenario 11f Sequence number - information document B
- Scenario 11g Sequence number - Approval document A
- Scenario 11h Sequence number - Approval document B
- Scenario 11i Sequence number - Progress payment claim
- Scenario 009a Revoke Use Case
- Scenario 12 Signing



Figure 110: TC71

9.2 TransactionTypes in TestFramework

The test protocol contains the following TransactionTypes with corresponding Initiator (I) and Executor (E):

Test Case: TC69 - Login into the environment

****Pre-conditions:****

The following is required: The software is available. Project is created/imported. VISI framework is imported, PSB is imported

****Test Steps:****

- Log in to the application with username and password

****Expected Result:****

The application visually confirms the login

Test Case: TC70 - User has a Role

****Pre-conditions:****

TC1 Log in to the environment

****Test Steps:****

- Show that the logged-in user has a role from the available roles in PSB

****Expected Result:****

Ariana has Role A and Organisation A person role

Transaction 001a Send message and reply (TC71)

Test Case: TC71 - Scenario 001a Send message and reply

****Pre-conditions:****

TC1 Login into the environment TC2 User has a Role

****Test Steps:****

- Transaction 001a Send message and reply
- As Ariana sends a message "FirstMessage Sending a message" to Benjamin from TransactionType Transaction 001a Send message and reply. No additional checks.

- As Benjamin replies to the message by sending "SecondMessage Responding to a message" to Ariana. No additional checks.

****Expected Result:****

Able to send a message to Benjamin. Benjamin receives a message.

Test Case: TC72 - Scenario 001b Same Init-Exec

****Pre-conditions:****

****Test Steps:****

- Transaction 001b Same Init-Exec
- As Chloe initiates a message "FirstMessage NoRequiredContent" and selects user Chloe. Check that this is possible.
- As Chloe sent a message with no required content.
- As receiver of message "FirstMessage NoRequiredContent", check that the message is received and cannot be responded to.

****Expected Result:**** Chloe sends a message.

Test Case: TC73 - Scenario 001c Send message with response date

****Pre-conditions:****

****Test Steps:****

- Transaction 001c Send message with response date
- As Ariana selects the message "FirstMessage Sending a message with response date" and gives a date input to the field "Response date". A date should be selected/set.
- As Ariana sent a message "FirstMessage Sending a message with response date" to Benjamin
- After sending the message "FirstMessage Sending a message with response date", check as Ariana that the application shows the corresponding interface to a response date, depending on the software vendor.
- As Benjamin, check that the application shows the corresponding interface to a response date, depending on the software vendor.
- As Benjamin replies to the message by sending "SecondMessage Responding to a message" to Ariana. There are no checks in this step. This message only exists to ensure that "FirstMessage Sending a message with response date" is not the last message in this transaction type.

****Expected Result:****

The only requirement is that the software application can recognise "true" and interpret it as a reaction date. There are no requirements on the interface or functionalities for a reaction date.

Transaction 002 Multiple scenarios for basic functions (TC74 & TC75 & TC76 & TC77 & TC78 & TC79 & TC80)

Test Case: TC74 - Scenario 002 UDT - XSD Restrictions No Accents Required Input

****Pre-conditions:****

****Test Steps:****

- Transaction 002 Multiple scenarios for basic functions
- As Ariana drafts the message "FirstMessage XSD Restrictions No Accents Required Input" to Benjamin and checks if the following UDT restrictions/formats are displayed correctly:

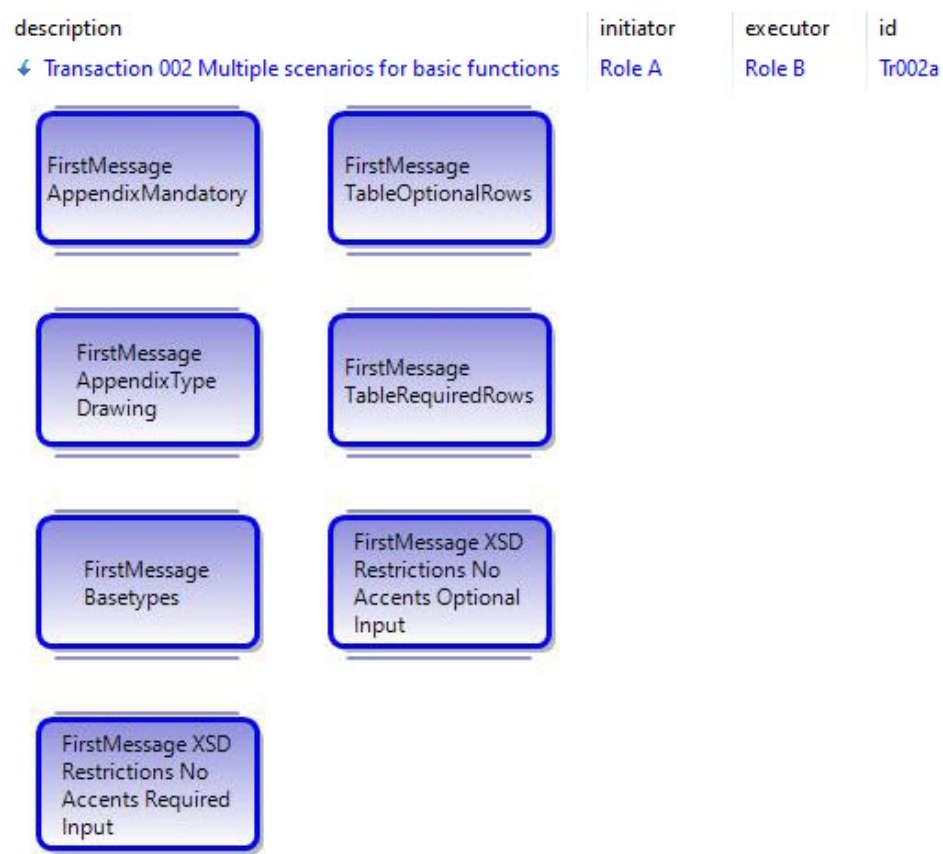


Figure 111: TC74 & TC75 & TC76 & TC77 & TC78 & TC79 & TC80

- Decimal Required: required input only consists of numeric data, and decimals are included only (up to) two characters.
- Enumeration Value: Value is filled automatically from picklist - input "Value filled automatically"
- Length Meters Required: Length value can only be numeric, and value input is required
- Mileage Required: Mileage value can only be numeric and with 3 decimals, and value input is required. Check if the message can be sent when leaving this field empty
- Picklist Street Names: shows all enumeration values in the picklist. Check if precisely one value is selected
- String Euro Required: Euro value can only be numeric. Check that letters aren't permitted. Check if displayed format is ##.###,## or #.###,## or ###,##
- String Required: String value can accept any keyboard characters, including special characters, spaces, and line breaks
- String Required Min 4 Max 8: String value must contain at least 4 characters and cannot exceed 8 characters
- Week Number Required: Required input. Week number can only contain numeric data. Format of week number 00 up until 53. Check if the message can be sent with week number 55. This should not be allowed. Show that the message can NOT be sent if this field is left empty.
- Year Number Required: Year number can only contain numeric data up to 4 digits and must start with "20". Check if the message can NOT be sent if this field is left empty
-

****Expected Result:****

UDT restrictions/formats are displayed correctly

Test Case: TC75 - Scenario 002 UDT - XSD Restrictions No Accents Optional Input

****Pre-conditions:****

****Test Steps:****

- Transaction 002 Multiple scenarios for basic functions
- As Ariana drafts the message "FirstMessage XSD Restrictions No Accents Optional Input" to Benjamin and checks if the following UDT restrictions/formats are displayed correctly:
- Length Meters Optional: value can only be numeric, value input is not required
- Mileage Optional: value can only be numeric, value input is not required. Check if you can send a message when leaving this field empty
- Week Number Optional: can only contain numeric data. Format of week number 00 up until 53. Check if you can send a message when leaving this field empty
- Year Number Optional: can only contain numeric data up to 4 digits and must start with "20". Check if the message can be sent if this field is left empty

****Expected Result:****

UDT restrictions/formats are displayed correctly

Test Case: TC76 - Scenario 002 Appendix Mandatory

****Pre-conditions:****

****Test Steps:****

- Transaction 002 Multiple scenarios for basic functions
- As Ariana makes a draft version of the message "FirstMessage AppendixMandatory" and tries to send the message. Check if the message can be sent without adding an appendix. This should not be allowed.
- If the message can not be sent without an appendix, add a test document as an appendix type "AppendixType" and send the message to Benjamin

****Expected Result:****

The message cannot be sent without the appendix

Test Case: TC77 - Scenario 002 Appendix Type Drawing

****Pre-conditions:****

****Test Steps:****

- Transaction 002 Multiple scenarios for basic functions
- As Ariana makes a draft version of the message "FirstMessage AppendixType Drawing" and adds an appendix type 'Drawing'.
- The added appendix should now have a metadata field called "Number of layers in drawing". Check that this field is offered in the message. The value of the field is integer-optional. Input value "1" or another number.
- Show that you cannot add an appendix with appendix type "AppendixType"
- Send a message from the previous step and check, as Ariana, that the application shows the value of "Amount of layers in drawing" in the metadata of the appendix of the sent message.

****Expected Result:****

Appendix with metadata field called "Number of layers in drawing". The value of the field is integer-optional.

Test Case: TC78 - Scenario 002 Table required rows ****Pre-conditions:****

****Test Steps:****

- Transaction 002 Multiple scenarios for basic functions
- As Ariana drafts a message, "FirstMessage TableRequiredRows" and tries to send the message to Benjamin without entering a row in the table content. This should not be allowed. It is allowed to have a row without data input in the 3 fields.
- As Ariana enters 4 rows of table content and checks if the message can be sent to Benjamin. This should not be allowed.
- As Ariana, make sure you have 1, 2 or 3 rows of table content and send the message to Benjamin.

****Expected Result:****

Messages can only be sent with 1, 2 or 3 rows in the table. It's not required to have input for the fields Column 1-Column 2-Column 3. This testcase is about the number of rows.

Test Case: TC79 - Scenario 002 Table optional rows

****Pre-conditions:****

****Test Steps:****

- Transaction 002 Multiple scenarios for basic functions

- As Ariana sent a message "TableOptionalRows" without inputting data into the table to Benjamin. You should be allowed to send a message without the table content.

****Expected Result:****

The message can be sent without table content.

Test Case: TC80 - Scenario 002 Base types

****Pre-conditions:****

****Test Steps:****

- Transaction 002 Multiple scenarios for basic functions
- As Ariana draft message "BaseTypes" and check the following types of User-defined types:
 - Boolean: type of input either or
 - Date: input value has format DD/MM/YYYY. Order might depend on country/format.
 - Date Time: input value has format DD/MM/YYYY - HH/MM(/SS). Order might depend on country/format.
 - Time: input value has format HH/MM(/SS)
 - Decimal: input value only consists of numeric data, and decimals are included only (up to) two characters
 - Integer: input value only consists of numeric data, and input can be a positive value only
 - String field accepts any keyboard characters, including special characters, spaces, and line breaks
- As Ariana sends a message "FirstMessage Basetypes" to Benjamin
- As Benjamin receives the message "FirstMessage Basetypes" and checks if the input is displayed as expected

****Expected Result:****

UDT basetypes are displayed correctly

Test Case: TC81 - Scenario 003a SingleEscape **Pre-conditions:**

****Test Steps:****

- Transaction 003a " SingleEscape &
- As Ariana sends a message "FirstMessage "SingleEscape" Accents" to Benjamin and inputs a string of signs containing a special character using the characters “ & < > ’
- As Benjamin, check if the special characters are displayed correctly in the software. Input value of textfield AND TransactionType description AND MessageType description AND SimpleElement-Type description
- As Benjamin replies to message "FirstMessage "SingleEscape" Accents" to Ariana with message "SecondMessage 'SingleEscape' Accents" and shows that characters “ & < > ’ are displayed as “ & < > ’

****Expected Result:**** Characters “ & < > ’ are shown correctly

Test Case: TC82 - Scenario 003b DoubleEscape **Pre-conditions:**

****Test Steps:****

- Transaction 003b DoubleEscape
- As Ariana sends a message "FirstMessage DoubleEscape Accents" to Benjamin and chooses option "ampersand: &" from the list in field "Enumeration Accents"



Figure 112: TC84

- As Benjamin show that the special character & is displayed as & in the received message
- As Benjamin replies with the message "SecondMessage DoubleEscape Accents" to Ariana and shows that the character & is displayed correctly, and that the message can be sent

****Expected Result:**** Characters “ & < > ’ are shown correctly

Test Case: TC83 - Scenario 003c UTF-8 ****Pre-conditions:****

****Test Steps:****

- Transaction 003c UTF-8
- As Ariana send message "FirstMessage UTF-8 Ўçäïöü®ΨΩ" to Benjamin with UTF-8 characters (Ўçäïöü®ΨΩ) in a description and with manual input of UTF-8 characters in string-field "UTF-8 characters Ўçäïöü®ΨΩ"
- As Benjamin show that the special characters Ўçäïöü®ΨΩ are displayed correctly in the received message

****Expected Result:****

Test Case: TC84 - Scenario 004a / 004b Serie ****Pre-conditions:****

****Test Steps:****

- Transaction 004a Serie
- As Ariana starts transaction Tr004a by sending message "Serie - Order" to Benjamin
- As Benjamin shows that you cannot send a message "Serie - Confirmation" as a response to a message "Serie - Order" from Ariana
- As Benjamin starts transaction Tr004b by sending a message "Serie - Order" to Chloe
- As Chloe replies to the message by sending "Serie - Confirmation" to Benjamin
- As Benjamin sent a message "Serie - Confirmation" to Ariana

****Expected Result:**** Tr004a final message can only be sent as a response to the final message of Tr004b

Test Case: TC85 - Scenario 005a / 005b serie ****Pre-conditions:****

****Test Steps:****

- Transaction 005a serie
- As Ariana starts transaction Tr005a by sending message "Serie - Order" to Benjamin
- As Benjamin starts transaction Tr005b by drafting a message "Serie - Order" and selecting Chloe AND Chris as recipients.

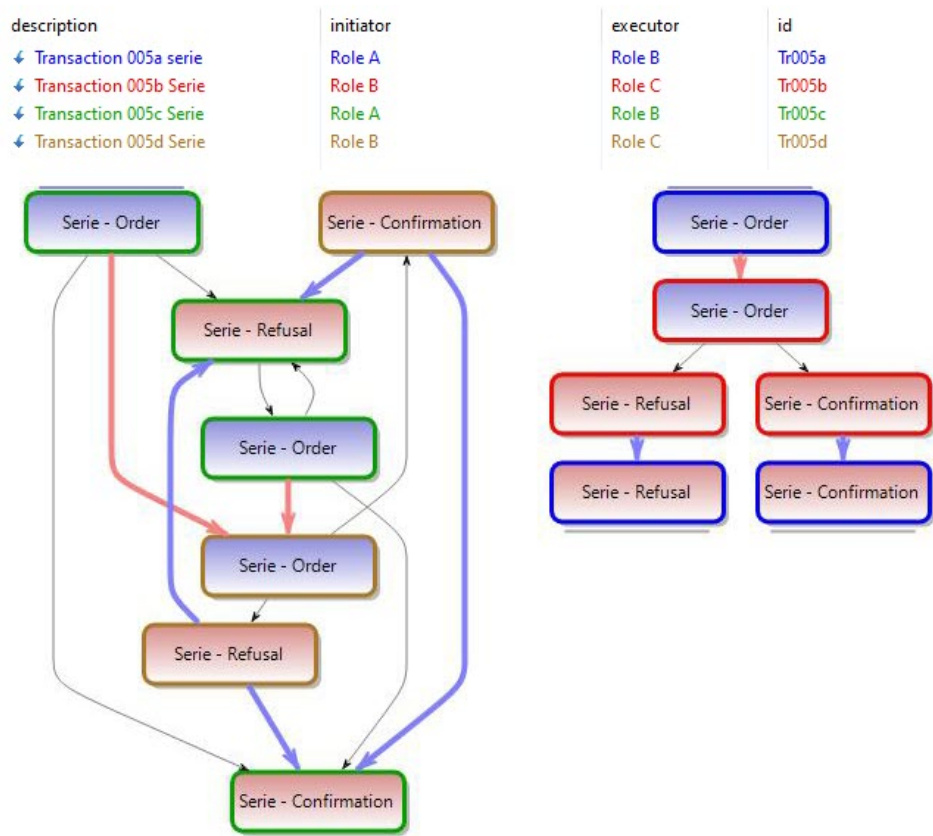


Figure 113: TC85 & TC86

- Send the message to two recipients.
- As Chris replied to the message by sending "Serie - Confirmation" to Benjamin
- As Benjamin replies to the message from Chris and sends the message "Serie - Confirmation" in the main transaction Transaction 005a serie to Ariana
- As Chloe sends a message "Serie - Refusal" to Benjamin
- Show that Benjamin received the message "Serie - Refusal" from Chloe, but cannot send a message to Ariana in the main transaction Transaction 005a serie again.

****Expected Result:**** Tr005a, only one answer can be sent to Ariana, even if you have two answers from related transaction Tr500b

Test Case: TC86 - Scenario 005c / 005d - serie loop ****Pre-conditions:****

****Test Steps:****

- Transaction 005c Serie
- As Ariana starts transaction Tr005c by sending message "Serie - Order" to Benjamin
- As Benjamin starts transaction Tr005d by sending a message "Serie - Order" to Chloe. The input value of string field 1 can be ignored.
- As Chloe replies to the message by sending "Serie- Refusal" to Benjamin. Message "Serie - Confirmation" can be ignored. The input value of string field 1 can be ignored.
- As Benjamin replies to the main transaction by sending or "Serie - Refusal" to Ariana
- As Ariana (re) starts transaction Tr005c by sending message "Serie - Order" to Benjamin
- As Benjamin (re)start transaction Tr005d by sending message "Serie - Order" to Chloe
- As Chloe replies to the message by sending "Serie- Confirmation" to Benjamin
- As Benjamin uses the response of Chloe and returns to the main transaction by sending or "Serie Confirmation" to Ariana.
- As Ariana, check that the transaction is closed after receiving the message "Serie - Confirmation" from Benjamin. No more messages can be sent.

****Expected Result:**** Ariana restarts the initiated transaction Tr005c after Refusal

Test Case: TC87 - Scenario 006a / 006b / 006c serie ****Pre-conditions:****

****Test Steps:****

- Transaction 006a Serie
- As Ariana sent a message "Serie - Order" to Benjamin
- As Benjamin replies to the message by sending the message "Serie - Question" to Ariana
- As Ariana starts (sub)transaction Tr006b by sending message "Serie - Question" to Chloe
- As Chloe shows that you can't reply to the message "Serie - Question" from Ariana
- As Chloe starts (sub)transaction Tr006c by sending message "Serie - Question" to Doris
- As Doris replies to Chloe by sending a message "Serie - Answer" to Chloe
- As Chloe uses the response of Doris to reply to Ariana by sending a message "Serie - Answer"
- As Ariana uses the response from Chloe to reply to the message "Serie - Question" by sending "Serie - Answer" to Benjamin

- As Benjamin closed the main transaction Tr006a by sending either "Serie - Refusal" or "Serie Confirmation" to Ariana. Both messages are end messages after which the transaction should be closed (no more messages can be sent).
- As Ariana shows, no more messages can be sent.

****Expected Result:**** Three layers where layers A-C and C-D after A-B are obligatory. A-B / B-A // A-C /// C-D /// D-C /// C-A // A-B / B-A.

Test Case: TC88 - Scenario 007a Element Conditions ****Pre-conditions:****

****Test Steps:****

- Transaction 007a Element Conditions
- We will use the FirstMessage four times in this testcase. If you want, you can send the message four times and use them for the four different responses used in this testcase
- As Ariana drafted a message, "FirstMessage Element Conditions" to Benjamin. Any input in the four text fields is mandatory. Send the message with input in the four text fields.
- As Benjamin replies with the message "SecondMessage Element Conditions Default" and checks if the string value in the field "EC GLOBAL DEFAULT" is fixed. The input cannot be altered. No ElementCondition = default fixed.
- As Ariana drafted a message, "FirstMessage Element Conditions" to Benjamin. Any input in the four text fields is mandatory. Send the message with input in the four text fields.
- As Benjamin replies with the message "SecondMessage Element Conditions Fixed" and checks if the string value in fields "EC GLOBAL DEFAULT" and "EC GLOBAL FIXED" is fixed. The input cannot be altered.
- Ec04 "EC Global Fixed SE EC GLOBAL FIXED" is activated on SE "EC GLOBAL FIXED" Ec08 "EC Global Default - MITT level SE Fixed" is activated on SE "EC GLOBAL DEFAULT"
- As Ariana drafted a message, "FirstMessage Element Conditions" to Benjamin. Any input in the four text fields is mandatory. Send the message with input in the four text fields.
- As Benjamin replies with the message "SecondMessage Element Conditions Free" and shows that string values for the available SE's are the same as the prior message and that the input can be altered.
- Ec05 "EC Global Free SE EC GLOBAL free" is activated on SE "EC GLOBAL FREE" Ec07 "EC Global Default - MITT level SE Free" is activated on SE "EC GLOBAL DEFAULT"
- As Ariana drafted a message, "FirstMessage Element Conditions" to Benjamin. Any input in the four text fields is mandatory. Send the message with input in the four text fields.
- As Benjamin replies with the message "SecondMessage Element Conditions Empty" and shows that string values for the available SE's are empty and that new input is allowed.
- Ec03 "EC Global Empty SE EC GLOBAL EMPTY" is activated on SE "EC GLOBAL FREE" Ec06 "EC Global Default - MITT level SE Empty" is activated on SE "EC GLOBAL DEFAULT"

****Expected Result:**** Input fields are empty, free or fixed as instructed in the testcase.

Test Case: TC89 - Scenario 007a / 007b Element Conditions table ****Pre-conditions:****

****Test Steps:****

- Transaction 007a Element Conditions
- As Ariana drafted the message "FirstMessage Element Conditions Table" to Benjamin. Create two rows of input in the table "Element conditions all SE - Parent-CE/Child-CE". Any input in the two text fields is mandatory. Note: minimum rows are not mandatory

- Send the message with input in the two text fields of the columns of the table.
- As Benjamin starts transaction Tr007b by drafting message "FirstMessage Element Conditions Table" to Chloe. Show that the table content string values for the available SE's are the same as the prior message and that the input to both fields cannot be altered. Show that you cannot remove or add any rows to the table. Send a message.
- Ec11 "EC Global Empty Parent CE - MITT level Child CE Fixed" is activated on CE "Element conditions all SE - Child-CE".
- As Chloe replies to Benjamin by drafting a message "SecondMessage Element Conditions Table". Show that the table content string values for the available SE's are the same as the prior message and that the input to both fields can be altered. Add another row to the table and give input to both fields. Send message
- Ec12 "EC Global Empty Parent CE - MITT level Child CE Free" is activated on CE "Element conditions all SE - Child-CE".
- As Benjamin uses the response from Chloe to reply to Ariana by drafting a message "SecondMessage Element Conditions Table". Show that the table content string values for the available SE's are the same as the prior message. Input field SE "EC GLOBAL DEFAULT" can be altered. Input field SE "EC GLOBAL FIXED" cannot be altered. Change input in existing rows of field SE EC GLOBAL DEFAULT. Show that you cannot add another row. Input is required in both fields, but you are not allowed to make changes to the field EC GLOBAL FIXED anymore, even in a new row. Send a message with only the table input from Chloe.
- Ec13 "EC Global Empty Parent CE - MITT level Child CE Free - MITT level SE EC GLOBAL DEFAULT Fixed" is activated on SE "EC GLOBAL DEFAULT" Ec14 "EC Global Empty Parent CE
- MITT level Child CE Free" is activated on CE "Element conditions all SE - Child-CE".

****Expected Result:**** Input fields in the table are empty, free or fixed as instructed in the testcase.

Test Case: TC90 - Scenario 008a Promise obligatory ****Pre-conditions:****

****Test Steps:****

- Transaction 008a Promise: Obligatory Promise
- As Ariana sent a message "Request" to Benjamin. No input required.
- As Benjamin replies to Ariana. Show that the message "Declare" cannot be sent. Send a message "Promise" to Ariana.
- As Benjamin sends a message "Declare" to Ariana
- As Ariana sends a message "Reject" or "Accept" to Benjamin

****Expected Result:**** Message Promise is required before message Declare. Promise and Declare come from one user in one role.

Test Case: TC91 - Scenario 008b Promise optional ****Pre-conditions:****

****Test Steps:****

- Transaction 008b Promise: Optional Promise
- As Ariana sent a message "Request" to Benjamin. No input required.
- As Benjamin shows, the message "Promise" is available as a reply. Send the message "Declare" to Ariana.
- As Ariana sends a message "Reject" or "Accept" to Benjamin

****Expected Result:**** Message Promise is optional before message Declare. Promise and Declare come from one user in one role.

Test Case: TC92 - Scenario 008c Repeated promise ****Pre-conditions:****

****Test Steps:****

- Transaction 008c Promise: Repeating Promise
- As Ariana sent a message "Request" to Benjamin. No input required.
- As Benjamin sent a message "Promise" to Ariana.
- As Benjamin shows, the messages "Promise" and "Declare" are both available to send to Ariana. Send another message "Promise" to Ariana.
- As Benjamin shows, the message "Promise" is available again. Send a message "Declare" to Ariana instead.
- As Ariana sends a message "Reject" or "Accept" to Benjamin

****Expected Result:**** Message Promise can be sent more than once before message Declare. Promise and Declare come from one user in one role.

Test Case: TC93 - Scenario 10 Sendafter AND-Condition ****Pre-conditions:****

****Test Steps:****

- Transaction 10 Sendafter AND-Condition Main Transaction
- As Ariana sends a message "Sendafter AND-Condition Start Message" to Benjamin
- As Benjamin drafted a message using the received message "Sendafter AND-Condition Start Message" from Ariana. Show that messages "Sendafter AND-Condition SecondSubMessage" and "Sendafter AND-Condition ThirdSubMessage" and "Sendafter AND-Condition Final Message" cannot be selected. Send message "Sendafter AND-Condition FirstSubMessage" to Chloe.
- As Benjamin drafted a message using the received message "Sendafter AND-Condition Start Message" from Ariana. Show that message "Sendafter AND-Condition FirstSubMessage" cannot be selected. Send message "Sendafter AND-Condition ThirdSubMessage" to Doris.
- As Benjamin drafted a message using the received message "Sendafter AND-Condition Start Message" from Ariana. Show that messages "Sendafter AND-Condition SecondSubMessage" and "Sendafter AND-Condition ThirdSubMessage" can be selected. Send message "Sendafter AND-Condition SecondSubMessage" to Doris.
- As Benjamin drafted a message using the received message "Sendafter AND-Condition Start Message" from Ariana. Show that messages "Sendafter AND-Condition SecondSubMessage" and "Sendafter AND-Condition Final Message" can be selected. Show that message "Sendafter AND-Condition ThirdSubMessage" cannot be selected. Send message "Sendafter AND-Condition Final Message" to Ariana.

****Expected Result:**** Transaction 10 Sendafter AND-Condition Main Transaction: limited order of messages because of send before and send after MITT conditions.

Test Case: TC94 - Scenario 11a Sequence number - change order A-B-Bi-A ****Pre-conditions:****

****Test Steps:****

- Transaction 11 Sequence Change order test organisation A
- As Ariana drafts the message "Sequence Change order test message" to Benjamin
- Show that the input value of the field "Change order sequence number" is "20000" or higher (depending on how many transactions have been initiated) and cannot be adjusted.
- Send message to Benjamin

- As Benjamin drafted a message "Sequence Change order test message" to Bruno using the received message "Sequence Change order test message" from Ariana
- Show that the input value of the field "Change order sequence number" has the same value as the prior message and cannot be adjusted. Send a message to Bruno.
- As Bruno drafted the response message "Sequence Change order test message" to Benjamin
- Show that the input value of the field "Change order sequence number" has the same value as the prior message and cannot be adjusted. Send a message to Benjamin
- As Benjamin drafted a response message, "Sequence Change order test message" to Ariana
- Show that the input value of the field "Change order sequence number" has the same value as the prior message and cannot be adjusted. Send a message to Ariana.

****Expected Result:**** Sequence number is displayed as instructed

Test Case: TC95 - Scenario 11b Sequence number - change order C-B-A-Ai ****Pre-conditions:****

****Test Steps:****

- Transaction 11 Sequence Change order test organisation C (prepares for B)
- As Chloe drafts a message, "Sequence Change order test message" to Benjamin
- Show that the input value of the field "Change order sequence number" is "60000" or higher (depending on how many transactions have been initiated) and cannot be adjusted.
- Send message to Benjamin
- As Benjamin drafted a message "Sequence Change order test message" to Ariana using the received message from Chloe
- Show that the input value of the field "Change order sequence number" has the same value as the prior message and cannot be adjusted. Send a message to Ariana.
- As Benjamin's role draft response message "Sequence Change order test message" to Chloe
- Show that the input value of the field "Change order sequence number" has the same value as the prior message and cannot be adjusted. Send a message to Chloe
- As Ariana drafts a message "Sequence Change order test message" to Alexander, using the received message from Benjamin
- Show that the input value of the field "Change order sequence number" has the same value as the prior message and cannot be adjusted. Send a message to Alexander.
- As Ariana drafts a response message, "Sequence Change order test message" to Benjamin
- Show that the input value of the field "Change order sequence number" has the same value as the prior message and cannot be adjusted. Send a message to Benjamin.
- As Alexander drafted a response message, "Sequence Change order test message" to Ariana
- Show that the input value of the field "Change order sequence number" has the same value as the prior message and cannot be adjusted. Send the final message to Ariana.

****Expected Result:**** Sequence number is displayed as instructed

Test Case: TC96 - Scenario 11c Sequence number - change order D-B ****Pre-conditions:****

****Test Steps:****

- Transaction 11 Sequence Change order test organisation D (third sequence)
- As Doris drafted the message "Sequence Change order test message" to Benjamin

- Show that the input value of the field "Change order sequence number" is "70000" or higher (depending on how many transactions have been initiated) and cannot be adjusted.
- Send message to Benjamin
- As Benjamin drafted a response message, "Sequence Change order test message" to Doris
- Show that the input value of the field "Change order sequence number" has the same value as the prior message and cannot be adjusted. Send a message to Doris.

****Expected Result:**** Sequence number is displayed as instructed

Test Case: TC97 - Scenario 11d Sequence number - change order B-A

****Pre-conditions:****

****Test Steps:****

- Transaction 11 Sequence Change order test organisation B
- As Benjamin drafted the message "Sequence Change order test message" to Ariana
- Show that the input value of the field "Change order sequence number" is "60001" or higher (depending on how many transactions have been initiated) and cannot be adjusted.
- Send message to Ariana
- As Ariana drafts a response message to Benjamin
- Show that the input value of the field "Change order sequence number" has the same value as the prior message and cannot be adjusted. Send a message to Benjamin.

****Expected Result:**** Sequence number is displayed as instructed

Test Case: TC98 - Scenario 11e Sequence number - information document A

****Pre-conditions:****

****Test Steps:****

- Transaction 11 Sequence Information document test organisation A
- As Ariana drafted the message "Sequence Information document test message" to Benjamin
- Show that the input value of the field "Information document sequence number" is "Info CO-000000" or a higher number (depending on how many transactions have been initiated) and cannot be adjusted.
- Send message to Benjamin

****Expected Result:**** Sequence number is displayed as instructed

Test Case: TC99 - Scenario 11f Sequence number - information document B ****Pre-conditions:****

****Test Steps:****

- Transaction 11 Sequence Information document test organisation B
- As Benjamin drafted a message, "Sequence Information document test message" to Ariana
- Show that the input value of the field "Information document sequence number" is "Info CL-000000" or a higher number (depending on how many transactions have been initiated) and cannot be adjusted.
- Send message to Ariana

****Expected Result:**** Sequence number is displayed as instructed
Test Case: TC100 - Scenario 11g Sequence number - Approval document A ****Pre-conditions:****
****Test Steps:****

- Transaction 11 Sequence Approval document test organisation A
- As Ariana drafted the message "Sequence Approval document test message" to Benjamin
- Show that the input value of the field "Approval document sequence number" is "Document CO-00000" or a higher number (depending on how many transactions have been initiated) and cannot be adjusted.
- Send message to Benjamin

****Expected Result:**** Sequence number is displayed as instructed
Test Case: TC101 - Scenario 11h Sequence number - Approval document B ****Pre-conditions:****
****Test Steps:****

- Transaction 11 Sequence Approval document test organisation B
- As Benjamin drafted a message, "Sequence Approval document test message" to Ariana
- Show that the input value of the field "Information document sequence number" is "Document CL-00000" or a higher number (depending on how many transactions have been initiated) and cannot be adjusted.
- Send message to Ariana

****Expected Result:**** Sequence number is displayed as instructed
Test Case: TC102 - Scenario 11i Sequence number - Progress payment claim ****Pre-conditions:****
****Test Steps:****

- Transaction 11 Sequence Progress payment claim test
- As Ariana drafted the message "Sequence Progress payment claim test message" to Benjamin
- Show that the input value of the field "Progress payment claim sequence number" is "Payment 001" or a higher number (depending on how many transactions have been initiated) and cannot be adjusted.
- Send message to Benjamin

****Expected Result:****
Sequence number is displayed as instructed

Test Case: TC103 - Scenario 009a Revoke Use Case ****Pre-conditions:**** All users need the role Revoke role, but the role won't be assigned as Person-In-Role in PSB (Project-Specific-Message).
****Test Steps:****

- Transaction 009c Revoke: Use case
- In this testcase, messages are sent where a request to revoke will be allowed or won't be allowed.
- As Ariana sent a message "Request" to Benjamin in transaction type "Transaction 009c Revoke: Use case"
- As Benjamin replies to Ariana by sending a message "Decline"
- As Ariana shows, you can draft a message Request as a response to Benjamin. Don't save or send the message.
- Request to revoke message "Decline" that has already been sent is permitted because revokeAllowed = true (Mitt_48)

- As Benjamin requests to revoke the message "Decline". Give any input to the required string field 'Reason for revoke'
- As Ariana responds with "Revoke allow" to the request for revoke. String field 'Allowance explanation' is optional.
- As Benjamin shows, you can draft a message "Decline" or "Promise"
- As Benjamin sent a message "Promise" to Ariana
- As Benjamin sends a message "Declare" to Ariana
- Request to revoke a message "Declare" that has already been sent is not permitted because revokeAllowed = false (Mitt_47)
- As Benjamin shows, you can't request to revoke the message "Declare"
- As Ariana ends the transaction by sending either "Accept" or "Reject" to Benjamin.

****Expected Result:****

Request to revoke message "Decline" is permitted because revokeAllowed = true (Mitt_48). ///////////////
Request to revoke a message "Declare" that has already been sent is not permitted because revokeAllowed = false (Mitt_47)

Test Case: TC104 - Scenario 12 Signing **Pre-conditions:**

****Test Steps:****

- Show that a sent message is signed with xades

****Expected Result:****

HS 9

10 Conformity

In addition to items marked as non-normative, all diagrams, examples, and notes in this document are non-normative. Everything else in this document is normative.

Furthermore, all communication (if it takes place over the internet) between a VISI user and a VISI application must use HTTPS with the same specifications. Other security measures can be taken at the infrastructure level if desired. VISI communication via the internet may not be possible without an SSL security certificate, both for communication between SOAP servers and for communication between the user and the VISI web application.

10.1 Phasing out versions of the VISI Standard

10.1.1 Binding Directive

The management organisation of the VISI Standard supports the current and two previous versions of the VISI Standard for a maximum period of five years. The choice to support three versions over a period of five years is motivated by the facts that a) the maximum release frequency of a new version is once a year, but b) it also happens that no new version is released for several years.

For ongoing projects, stakeholders (users, consultants and software suppliers) can choose to continue using a version of the VISI Standard older than 5 years, but this version is **not** supported by the management organisation.

There is no choice for new projects. New projects must be started with the current (or one of the two previous versions) of the VISI Standard with a maximum period of five years.

10.1.2 Background

VISI is an open standard that appears on the ["Pas-toe-of-leg-uit-lijst"](<https://www.forumstandaardisatie.nl/open-standaarden/verplicht>) of the Forum for Standardisation (FvS). One of the management requirements of FvS is backwards compatibility, and during intake by FvS of the VISI Standard, the subject of backwards compatibility was added to the documentation. In the [change list](<https://github.com/nl-digigo/visi/wiki/2.-De-belangrijkste-wijzigingen-in-de-VISI-systematiek>) for version 1.4, backwards compatibility is defined as follows: The "backwards compatibility" of a new VISI standard means that the operation of the new VISI standard (and/or a framework according to this standard) has at least the same operation as the previous version of the VISI standard (and /or a framework according to this standard). With others, if functionality is lost or if behaviour changes, there is no backwards compatibility.

10.1.3 Practical situations

This note first describes what this means for stakeholders, and then how the phasing out of old versions can be handled. This concerns advice for active projects.

To apply the VISI Standard in practice, two components are crucial, namely 1) the framework and 2) the software. Both comply with/are certified for a certain version of the standard.

The impact of a new version (and the phasing out of a version) is elaborated based on three situations in which an organisation uses a combination of framework and software, where:

1. framework and software are old and the same version;
2. framework and software concern an old version and the same version, and the organisation will apply the latest version of the software;
3. framework and software concern an old and the same version, and the organisation will apply the latest version of the framework (in accordance with the current version of the VISI standard).

10.1.3.1 1. No upgrade - framework and software according to an old version of the VISI Standard

If the applied version of VISI Standard is not supported by the management organisation, ****no new**** projects may be started with the framework and software, but situation 2 or 3 applies. It is possible to continue existing, active projects with framework and software, but without support from a management organisation.

If the applied version of VISI Standard is supported by the management organisation, new projects may be started, and existing projects may be continued with the framework and software. No actions are necessary.

10.1.3.2 2. Upgrade framework - framework according to the current version of the VISI Standard

Upgrading the framework to the current version of the VISI Standard means that the software must also be upgraded to the certified version. New projects start with the upgraded framework and certified software.

When a new version of the VISI Standard is released, implementation guidelines are provided on what needs to be done in existing projects to switch (upgrade) to a framework in accordance with the current version of the VISI Standard.

The case that an existing project applies an unsupported version of the VISI Standard needs to be further elaborated. Questions that apply to this situation are: Should and/or can messages be upgraded to the current version? Or should a VISI archive (file export) be created? A sent message (with attachments), the confirmation and the associated framework must be kept and archived in the original format. Converting this sent information to a supported version of the VISI Standard is not simply possible and cannot be guaranteed without loss of information. For audit trail and legal validity, the information must be submitted in the original format. Archiving an existing project - which uses an unsupported version of the VISI Standard - provides a backup of the original data. In addition, you are free to convert the data to another format for other purposes. Consider converting XML to HTML for readability. However, the backup of the original data leads to disputes.

10.1.3.3 3. Upgrade software - software according to the current version of the VISI Standard

If, in addition to the current version, the software also supports the two previous versions (with a maximum period of five years), a software upgrade is sufficient - provided the framework conforms to a supported version of the VISI Standard.

If the framework does not conform to a supported version of the VISI Standard, the framework must be upgraded (see situation 2) to start new projects.

If the applied version of VISI Standard is supported by the management organisation, new projects may be started, and existing projects may be continued with the framework and software. No actions are necessary.

10.1.4 Certification and management

A software supplier is certified for one (1) version of the VISI standard. For new software suppliers, this will be the current version. This software is not certified for the two previous versions.

Of the three situations described, 1) is not important for the management organisation. Situation 2 represents the desired situation in which the management of the VISI Standard is designed. Situation 3 does not pose a problem.

Allowing old versions is necessary from an administrative perspective to ensure that users have a transition period and are not forced to immediately switch to the current version of the VISI Standard.

10.1.5 Implementation

When starting new projects, the framework and software must fit within one of the supported versions of the standard.

The versions of the VISI Standard that are no longer supported by the management organisation remain valid. The management organisation only supports the current and the two immediately preceding versions. Software suppliers may choose to support older versions.

It is important that the management organisations encourage stakeholders to switch to the current version. The five-year support period is ample to upgrade the existing framework and software.

Upgrading existing projects is a different story. Implementation guidelines are provided with the release of each new version of the VISI Standard. These implementation guidelines describe what should be done with the existing projects with regard to the framework and project-specific messages.

10.2 Version Management

The Guideline, including all appendices, was created in collaboration with the VISI Expert Committee. CROW is the author, maintainer and publisher of this document.

With the publication of this version, all previous versions of the Guideline have expired. VISI software may still exist based on previous versions.

The history is summarised in the table below.

10.3 Important changes in the VISI system

10.3.1 Introduction; Release Cycle

The VISI system is still evolving. Progressive insight, technical errors, wishes of users or software suppliers can all lead to proposals for improvement. The Expert Committee (VISI EC) deals with all improvement proposals and keeps a progressing status overview of which improvements are being implemented in which version of the VISI system.

The EC uses an annual release cycle from version 1.3. The management structure is available on the VISI website. We have now arrived at version 1.7. The technical elaboration of each improvement proposal is included in this Guideline and/or in one of the appendices. It is also indicated whether an improvement is normative or informative. This chapter explains the most important improvements.

This chapter briefly explains the changes in version 1.7 compared to version 1.6. Appendix 5 contains a content log of all changes, including changes in previous versions. From version 1.4, the EC works with an environment <https://github.com/bimloket/visi/wiki>. There, under 'Issues', the entire resolution of the issues in a certain version of the development process can be followed in detail. However, the latest, final version of a version is NOT on GitHub; it must be downloaded from the VISI website and/or the VISI toolbox.

10.3.2 Important changes in version 1.2

Below are the most important changes that have already been made in version 1.2 compared to the previous versions (these improvements are now an integral part of version 1.3). These points are further explained in Appendix 5.

- Transaction link: communication across multiple transactions.
- Reuse and blocking data elements.
- DateTime format.
- Replacement of people within a project (Successor; see also TC015 infinite loops).
- Add MITT MessageInTransactionType to message.

10.3.3 Changes in version 1.3 vs version 1.2

The numbering is in accordance with the status overview of the EC at the time.

Increase attachment capacity of VISI messages (M0001)

This concerns increasing the attachment capacity of the VISI messages to approx. 120 MB. This becomes combined with chunking for further optimisation.

Require subject field (M0002)

Making the subject field mandatory, so that no messages can be sent without a subject field.

Unique sequence number for each transaction (M0004)

Each transaction is given a unique identifier. A 'Universal Unique Identifier' (UUID) is used for this (for example, "ac0234d8-5c89-40c0-8afc-57cdbe9ac00c"). The Entity 'TransactionTemplate' in VISI Systematics Part 2 has been expanded with the mandatory attribute 'number' of type 'INTEGER', and the Entity 'OrganisationTemplate' has been expanded with an 'abbreviation' attribute. See Annex 3, paragraphs 1.13 and 1.7.

Authority to send messages on behalf of another user (M0006)

This concerns transparency when temporarily replacing a person, i.e. making it possible to 'authorise' people to send messages on their behalf. It can also be used to arrange that several people can send messages on behalf of the formal person fulfilling the role (for example, UAV Director). See appendix 3, section 1.8.

Ensure traceability of communication after changing framework (TC001)

This is done by changing the current namespace of the VISI messaging to a framework-specific namespace that is derived from the new namespace attribute of the ProjectType entity of the framework. The promoter software uses this data to generate the correct target namespace. Note: After the introduction of VISI Systematics version 1.3, VISI software will have to take into account that several versions of a framework can be active side by side.

Link appendix types to transactions (TC006)

It is now possible to indicate in a framework which AppendixType may be chosen. Standard and guideline for archiving VISI projects ((TC010) This concerns the requirements that the exchange and archiving format for (completed) VISI projects (the 'VISI archive') must meet (see Appendix 11).

Metaframework (TC013)

This is a transactional procedure for distributing a new version of a framework and/or a project-specific message. Through this 'meta-framework', the VISI system itself is used for managing VISI communication within a project. A project framework (for example, in the case of a long-term project) can now also be changed if there are still outstanding transactions (see Appendix 6).

Guideline for 'successor' (TC015)

This concerns a working method in which a user who fulfils a certain role in the VISI framework is succeeded (by means of a PersonInRole element) by another user. The link attribute 'successor' (in Systematics Part 1) serves this purpose. This prevents 'infinite loops' from now on (see also appendix 7).

Additional functional requirements (TC016)

In response to user wishes, several functions and requirements have been described (see appendix 10) that users of products with the VISI quality mark can expect from the product in question, as a minimum, when it comes to:

- assurance of the authenticity of VISI messages and associated files.
- presentation of the communication structure in a VISI framework.
- Reproduce the VISI communication afterwards.

Duplicate message types (TC022)

This concerns an adjustment to Systematics Part 2, which can be used to indicate explicitly whether a MessageInTransactionType concerns a message with which a new transaction can be started.

Boolean values (TC024)

It has been established that for Booleans the value '1' = 'True' and '0' = 'False', and both forms may be used. It also applies that 'no value' is equal to '0', or 'False', because within the VISI system, "no value" is also valid.

Optional Fields

By making a number of fields optional, frameworks (XML and xsd) can become more compact. A more compact and efficient framework results in increased scalability and speed of the VISI software and better readability. N.B. This improvement could not yet be implemented in the IDM-DIS (possibly in the F-DIS).

10.3.4 Changes in version 1.4 vs version 1.3

The numbering is in accordance with Codeplex.

Security of VISI communication (1003).

This concerns the requirement that VISI communication between SOAP servers always takes place via https and SSL. The use of HTTPS (TLS 1.0 or higher, with at least 128-bit encryption) for sending VISI messages and attachments is mandatory. All communication over the internet between a VISI user and a VISI application must use HTTPS with the same specifications.

Linking external and internal processes (1007)

This concerns the requirement that VISI must be able to facilitate that project partners can independently change the VISI communication within their own project organisation, i.e. that internal and external project processes can be bundled in one framework. In version 1.4, this problem definition is clearly documented (but no technical solution has yet been devised). The topic is a high priority on the list of points for improvement for version 1.5.

Use of 'DateTime' fields (1022)

This concerns a tightening of the quality mark test for support and correct use of this entity type (date + time) as an input field in VISI messages. Transfer roles from the standard (1024). This concerns being able to generate an export to the VISI archive format, and then an import of this VISI archive format, in order to transfer the responsibilities of a contractual partner who has left (e.g., after a bankruptcy).

Being able to enforce a certain order of messages to be sent (1025)

This involves limiting possible continuations of a transaction, assuming a certain state in the message flow, (1) so that a message may only be sent after one or more specific messages are present (already received or sent) in the transaction or its sub-transactions, i.e. a dependency between two (or more) already present messages, or (2) so that a message may only be sent as long as one or more specific messages are not yet present. (This can also be used to ensure that a message can only be sent once by referring to the message itself with this rule.

Automatically start a project (1026)

This concerns an extension of the 'Meta-framework' phenomenon so that a VISI project can be started in combination with a project-specific message from the meta-framework ('automatically'). The "meta-framework" phenomenon – which has existed since version 1.3 of VISI – is an overarching framework specific to each project, with which changes in the project framework itself or a project-specific message can be distributed among the participants of a VISI network. The meta-framework will now also include the 'Project Initiator' and 'Project Executor' role types.

(Impact on) backwards compatibility (1040)

Backwards compatibility means that the operation of a new version of the VISI Standard (and/or a framework according to this standard) has at least the same effect as the previous version. For each point of improvement, it is analysed and documented whether the technical solution has an impact on it. Backwards compatibility is not always obvious. If functionality is deliberately removed, or if behaviour is changed, there is no longer any question of backwards compatibility on that point.

10.3.5 Changes in version 1.6 vs version 1.4

Belongs to: 4.4.18

The numbering of issues is now according to GitHub.

Define number of rows in a table (1008)

For a ComplexElementType, a minimum and/or maximum number of times that the ComplexElementType may occur can optionally be defined in a framework.

```
1 ENTITY ComplexElementType;  
2 ...  
3 minOccurs : OPTIONAL INTEGER;  
4 maxOccurs : OPTIONAL INTEGER;  
5 ...  
6 END_ENTITY;
```

When nothing is entered in a framework (in accordance with version 1.6) at minOccurs and maxOccurs, the rules of the previous system versions apply;

For the ProjectType, MessageType, PersonType, AppendixType, and OrganisationType types, a Complex Element always occurs 0 or 1 time. For the Complex Element type, a complex Element always occurs from 0 to infinity.

Can make attachments mandatory per message type (1019) (see section 4.4.9

When a document has to be sent for acceptance, and the attachment is not included, it can be foreseen that the message will be rejected. Forcing that an attachment must be added can help. The impact of making an appendix mandatory is low. It enforces that the messages that need an attachment actually have it. In a number of cases, this can be an advantage for users because attachments are no longer forgotten.

The chosen solution is to indicate per MessageType whether an appendix is mandatory.

```
1 ENTITY MessageType;  
2 ...  
3 appendixMandatory: OPTIONAL BOOLEAN;  
4 ...  
5 END_ENTITY;
```

This solution is backwards compatible because the boolean has been made optional. Therefore, this option is not enforced.

ElementConditions for tables (1226)

Until version 1.6, there was no unequivocal agreement on which level an Element Condition should be set for a table. As a result, the operation in the software is difficult to predict in advance and can differ per software supplier. A table is created in a framework by adding a Complex Element to a Complex Element. The ambiguity lies in the Complex Element to which the element condition must be added. Is this the parent element or the child element? A new priority table has been included in Appendix 12 to remove this ambiguity.

Increase attachment size to 10GB (1236)

There is a limitation on the size of the attachments according to the VISI Standard. For VISI Standards version 1.0, 1.1 and 1.2, the maximum is 20MB per message, and for version 1.3 and 1.4, the maximum is 120MB per message. A maximum of 20MB or 120MB is not realistic in 2019. With the VISI Standard, BIM containers or delivery files can also be exchanged. These containers and delivery files can be/become very large, which is why the maximum has been increased to 10 GB.

10.3.6 Changes in version 1.7 vs version 1.6

Framework with non-ascii characters in list boxes and descriptions (enumerations & descriptions) (#68)

It used to be that messages with non-ASCII characters in picklists and descriptions were not displayed correctly to the recipient of a VISI message. To prevent this, it has been stated that in

- a) description of elements (SimpleElementType, ComplexElementType, MessageType, Transaction-Type)
- b) enumeration (UserDefinedType) non-ASCII characters must be displayed correctly

11 Release Policy

11.1 Governance bodies

The governance structure for the VISI release process consists of the User Committee, Expert Committee, and Steering Committee. Each committee has a distinct role and responsibility within the governance process. Figure 114 provides an overview of the governance structure and the interactions between the committees.

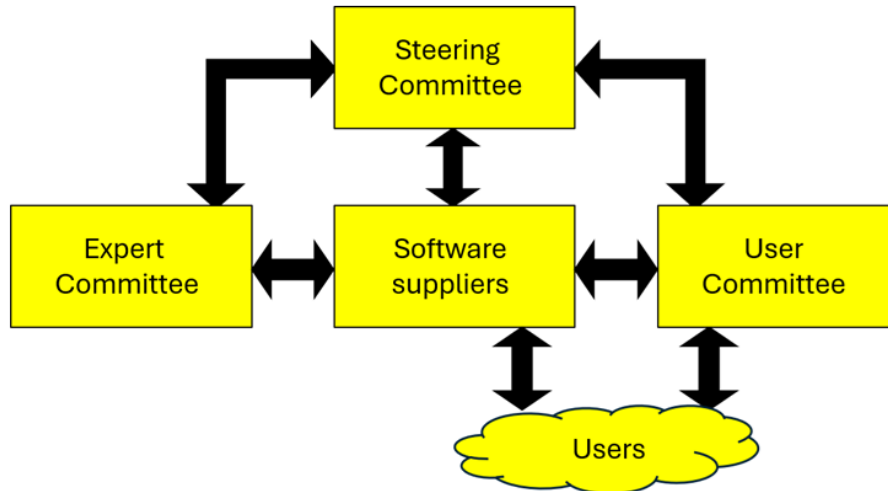


Figure 114: VISI governance

11.1.1 User Committee

Composition

The User Committee consists (UC) of members who represent the interests of VISI users. These members can be end users themselves, or suppliers who represent and act on behalf of multiple users.

Purpose

The User Committee is responsible for gathering information and feedback from the user community. Its purpose is to identify current challenges, understand user needs, and determine which changes or enhancements are required.

Proposal Definition and Requirements

The User Committee is responsible for defining the requirements for any new proposal. When a user proposes a new idea, it must first be translated into the requirements and a use case; this forms the proposal. Each proposal must be documented as a separate item, including a clear description, expected outcome, and a target deadline.

The User Committee also decides which proposal has priority to be worked out first.

All proposals are submitted to the Expert Committee.

Rejection

If the Expert Committee rejects the proposal due to lack of sufficient clarity, it is returned to the User Committee for revision. After the necessary changes have been made, the proposal can be resubmitted for review.

Acceptance and Validation

Once the Expert Committee has defined the functional solution, the User Committee is responsible for reviewing and verifying if it meets the requirements and use case as defined in proposal. If the functional solution is approved, the Expert Committee can continue to define the technical solution. When the User Committee rejects the functional solution, the Expert Committee has to revise it.

11.1.2 Expert Committee

Composition

The Expert Committee (EC) consists of people with extensive knowledge of VISI, its content, and its practical application. These members may include suppliers, but preferably also other experts who develop frameworks and therefore have a deeper understanding of VISI than the members of the User Committee.

The Expert Committee also includes the Product Owner, who is responsible for safeguarding the quality and integrity of the VISI standard.

Proposal Review

The Expert Committee reviews all proposals submitted by the User Committee. Its first responsibility is to determine whether a proposal is complete and sufficiently clear. If any ambiguities or missing details are identified, the proposal is returned for further clarification and refinement.

Feasibility Assessment

Once a proposal is fully understood, it is assessed whether the solution should be incorporated into the VISI standard or implemented within software solutions. The preferred option is inclusion in the standard, as this ensures consistent implementation across all software vendors. However, if a topic falls outside the scope of the VISI standard, it is handled within software implementations and is not included in the standard.

Functional Solution Definition

Once it is assessed as feasible, the Expert Committee develops a functional solution. The proposal is elaborated into a functional design and expected outcomes, providing a basis for the technical specification and implementation of the standard.

Functional Solution Review by User Committee

Once the functional solution is complete, it is submitted to the User Committee for review. The User Committee assesses whether the functional solution adequately addresses the approved proposal. If necessary, the Expert Committee revises the functional solution until it is considered sufficiently aligned with the proposal.

Functional Solution Review by Steering Committee

After approval by the User Committee, the functional solution is submitted to the Steering Committee for review. The Steering Committee evaluates the functional solution and decides whether it should be incorporated in the standard.

Technical Solution Definition

When the Steering Committee has decided to add the functional solution to the standard, the Expert Committee will start developing a technical solution. Based on the approved requirements and functional design, the committee prepares a detailed technical specification describing the implementation requirements needed to support the use case. This technical specification serves as the basis for the development of a proof of concept. One party in the Expert Committee is assigned to develop a proof of concept.

Proof of Concept Development

The proof of concept is used to validate the proposed solution, demonstrate its feasibility in a practical environment, and identify any issues or improvements before the solution is formally incorporated into the standard.

Review Of The Proof of Concept

The results of the proof of concept provide valuable feedback to the Expert Committee and help ensure that the final standard is robust, interoperable, and fit for purpose. When the proof of concept is done, the Expert Committee will review it. If the Expert Committee doesn't approve the proof of concept, it will be revised till its accepted.

Documentation Update

Once the proof of concept is accepted, the Expert Committee will update the VISI documentation based on the findings of the proof of concept. The proof of concept and the updated documentation together constitute the final solution, which is sent to the Steering Committee for approval. If the final solution is not accepted, it will be sent back to the Expert Committee for revision till it is accepted.

11.1.3 Steering Committee

Composition

The Steering Committee (SC) consists of members with a governance and strategic oversight role. The committee is responsible for ensuring that the VISI governance process operates effectively and in accordance with the established procedures.

Governance and Process Control

The Steering Committee oversees the overall change management process. It ensures that proposals have followed the required workflow, that all necessary reviews have been completed, and that the responsibilities of the various committees have been fulfilled.

The committee verifies that proposals are sufficiently documented, validated, and approved before they can be included in a release.

Committee Coordination

The Steering Committee maintains oversight of the interaction between the User Committee and the Expert Committee. It ensures that responsibilities are clearly assigned, that dependencies are managed, and that issues affecting multiple committees are addressed on time.

In addition, the committee monitors the relationships and dependencies between the various committees and ensures that any relevant matters affecting the overall process are properly considered. The Steering Committee is therefore a process-oriented committee, responsible for maintaining coordination and oversight throughout the governance process.

Release Approval and Planning

The steering committee is responsible for reviewing the functional solution, submitted by the Expert Committee, and decide if it belongs in the visi standard.

The Steering Committee is also responsible for the final approval of changes to the VISI standard. Once a proposal has completed the review, validation, and proof-of-concept phases, the committee determines whether it is ready for adoption.

The Steering Committee also decides in which release an approved change will be included.

11.2 Proof of Concept

Purpose

A Proof of Concept (PoC) is developed to demonstrate whether a proposed feature or change can be implemented and functions according to the requirements defined by the User Committee.

The objective of the PoC is to validate that the proposed solution can be implemented and that the specifications are sufficiently clear and complete.

Development

A member of the Expert Committee develops the PoC based on the approved use case, requirements, and expected outcome.

During the development of the PoC, any findings, ambiguities, or required clarifications are documented and shared with the Expert Committee.

Code Sharing

To support review and collaboration, the PoC code should be shared with the Expert Committee as far as possible.

Evaluation

Once completed, the PoC is submitted to the Expert Committee for evaluation.

The Steering Committee reviews the outcome of the PoC and determines whether the proposed change has been sufficiently validated.

Release Decision

Based on the evaluation of the PoC, the Steering Committee decides whether to include the proposed change in a future VISI release.

11.3 Release Process

Overview

Users can continuously submit ideas, wishes, and improvement proposals for the VISI Standard. DigiGO registers these proposals and ensures that they are processed through the governance structure consisting of the User Committee (UC), Expert Committee (EC), and Steering Committee (SC).

The scope and content of future VISI releases are determined based on proposals submitted by users, suppliers and advisors. Each proposal is treated as an individual change request and follows the review, validation, and approval process described in this document.

BPMN diagram

The BPMN diagrams below provide a high-level overview of the VISI release process. Together, they illustrate the interactions between the User Committee, Expert Committee, and Steering Committee, as well as the sequence of activities and decision points from proposal submission to release. The first diagram covers the proposal definition and functional design process, while the second diagram describes the technical solution, proof of concept, and release process.

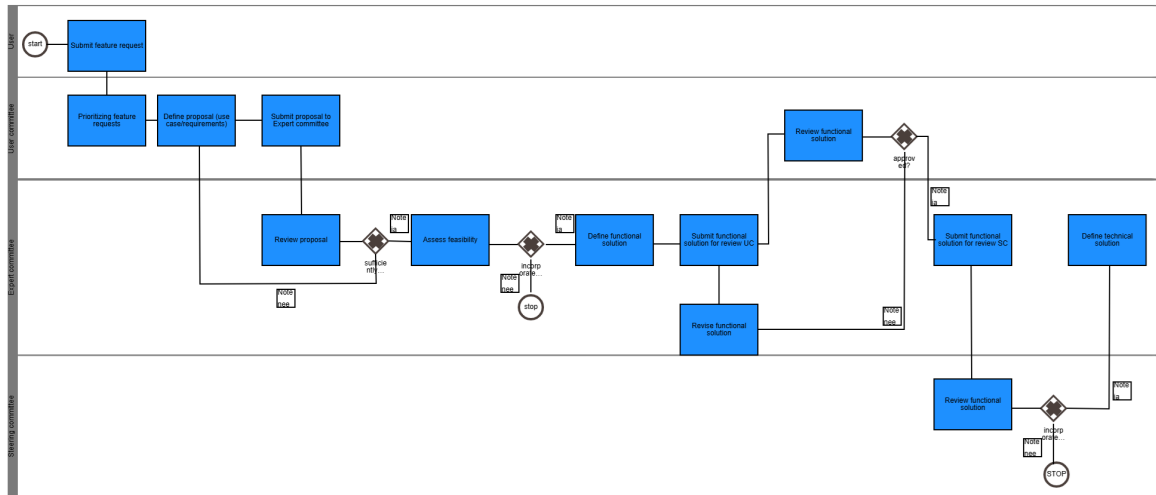


Figure 115: Proposal Definition and Functional Solution Process

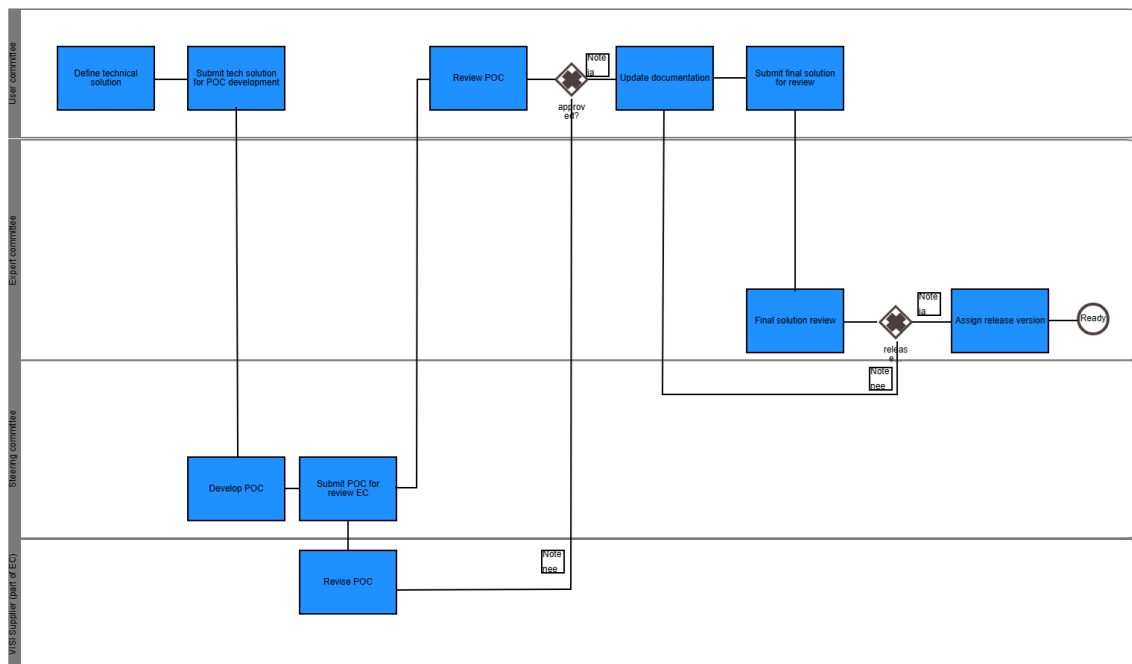


Figure 116: Technical Solution, Proof of Concept and Release Process

Step by step process

1. The User Committee gathers feedback from the user community and translates ideas into clearly documented use cases and requirements. Once sufficiently defined, proposals are submitted to the Expert Committee for assessment.

The state of the proposal now is: **Proposal Submitted**

2. The Expert Committee reviews each proposal and determines whether it is complete and sufficiently clear. If necessary, additional clarification is requested from the User Committee.

The state of the proposal now is: Proposal Reviewed

3. Once a proposal is fully understood, the Expert Committee determines whether the requested functionality belongs within the VISI Standard or should be implemented within software solutions.

The state of the proposal, if decided feasible, now is: Feasibility Assessed

The state of the proposal, if decided not feasible, now is: Proposal not feasible

4. When it is decided that the proposal belongs in the VISI standard, the Expert Committee defines a functional solution.

The state of the proposal now is: Functional Solution Defined

5. The User Committee reviews the functional solution and determines whether it meets the requirements defined in the use case. If necessary, additional clarification is requested from the Expert Committee.

The state of the proposal now is: User Committee Approved

6. The Steering Committee reviews the functional solution and determines if it's going to be added to the VISI standard or not.

The state of the proposal now is: Steering Committee Approved

When the Steering Committee rejects the functional solution, the state of proposal will change to: Technical Solution Rejected

7. When the Steering Committee accepts the functional solution, the Expert Committee defines a technical solution

The state of the proposal now is: Technical Solution Defined

8. The technical solution is submitted to one member of the Expert Committee, with the request to develop a PoC.

The state of the proposal now is: PoC developed

9. The findings of the PoC are reviewed by the Expert Committee and used to further refine the documentation, specifications, and test scenarios where necessary.

The state of the proposal now is: PoC Approved

10. Once the PoC is accepted, the documentation is updated by the Expert Committee.

The state of the proposal now is: Documentation Updated

11. After updating the Documentation, final solution is submitted to the Steering Committee for final approval. If necessary, additional clarification is requested from the Expert Committee.

The state of the proposal now is: Release Approved

12. When the Documentation is accepted, the Steering Committee will decide in which release it will be included.

The state of the proposal now is: Released

Unlike previous release processes, changes are no longer grouped into fixed annual release scopes. Instead, proposals are handled individually from analysis through implementation and validation

11.4 Process Constraints

Proof of Concept Development Constraint

Proofs of Concept (PoCs) cannot be developed simultaneously. While a PoC is in progress, no other PoC can be started. This means that PoCs must be developed one after another, even though they are not dependent on each other's results. As a result, delays in one PoC will postpone the start of the next PoC.

Proof of Concept Documentation Constraint

The documentation of a PoC can begin once the development of that PoC is completed and may continue during the development of the next PoC. However, a PoC cannot be considered fully completed until its documentation is finished. As a result, the development and documentation schedule must ensure that the documentation of a PoC is completed before the next PoC reaches completion.

A Index

A.1 Terms defined by this specification

Scenario name	RoleA	RoleB	RoleC	RoleB	RoleD	Organisation A
Transaction 001a Send message and reply	I	E				
Transaction 001b Same Init-Exec			I/E			
Transaction 001c Send message with response date	I	E				
Transaction 002 Multiple scenarios for basic functions	I	E				
Transaction 003a " SingleEscape	I	E				
Transaction 003b DoubleEscape	I	E				
Transaction 003c UTF-8	I	E				
Transaction 004a Serie	I	E				
Transaction 004b Serie		I	E			
Transaction 005a serie	I	E				
Transaction 005b Serie		I	E			
Transaction 005c Serie	I	E				
Transaction 005d Serie		I	E			
Transaction 006a Serie	I	E				
Transaction 006b Serie (error framework)	I		E			
Transaction 006c Serie			I	E		
Transaction 007a Element Conditions	I	E				
Transaction 007b Element Conditions		I	E			
Transaction 008a Promise: Obligatory Promise	I	E				
Transaction 008b Promise: Optional Promise	I	E				
Transaction 008c Promise: Repeating Promise	I	E				
Inhoud 2: vreemde tekens			I	E		
Transaction 009a Revoke						
Transaction 009b Revoke: Alternative process						
Transaction 009c Revoke: Use case	I	E				
Transaction 10 Sendafter AND-Condition First Subtransaction		I	E			
Transaction 10 Sendafter AND-Condition Main Transaction	I	E				
Transaction 10 Sendafter AND-Condition Second Subtransaction		I		E		
Transaction 10 Sendafter AND-Condition Third Subtransaction		I		E		
Transaction 11 Sequence Approval document test organisation A						
Transaction 11 Sequence Approval document test organisation B						
Transaction 11 Sequence Change order test organisation A						
Transaction 11 Sequence Change order test organisation A internal forward						
Transaction 11 Sequence Change order test organisation B						
Transaction 11 Sequence Change order test organisation B internal forward						
Transaction 11 Sequence Change order test organisation C (prepares for B)						
Transaction 11 Sequence Change order test organisation D (third sequence)						
Transaction 11 Sequence Information document test organisation A	I	E				
Transaction 11 Sequence Information document test organisation B	I	E				
Transaction 11 Sequence Progress payment claim test	I	E				
Transaction	I	E				
Transaction	I	E				
Transaction	I	E				
Transaction	I	E				

Table 19: RoleTypes in test framework

Author	Version	Datum	Status	Remarks
TEEC2/EC	1.7	01-09-2023	Future	Translation to English documentation to support internationalisation. Changes in 10.3.6.
	1.6	31-03-2016	Current	Changes in 10.3.5.
	1.4	31-03-2014	Current	Changes in 10.3.4.
CROW/EC	1.3	25-11-2011	Current	The entire guide has been adapted in accordance with version 1.3 of the VISI system. The Guidance now consists of a main document with a series of normative or informative annexes, each containing a specific requirement or guideline. Version 1.3 is aligned with the ISO/DIS 29481 Part 2 (Information Delivery Manual IDM). Changes in 10.3.3
Gobar/EC	1.2	31-10-2008	Expired	Changes in 10.3.2.
Gobar	1.1(a)		Expired	Still used in some cases.

Table 20:

Index

Message, 292

A.2 Terms defined by reference

B VISI Glossary

A specific jargon is used in the VISI system, in which concepts may have a slightly different meaning than the generally accepted ones.

B.1 Terms used in VISI standard

B.1.1 Message (Bericht)

Means of transferring information / exchanging data. A message is of a certain message type and is inextricably linked to a transaction. A message contains the data structure prescribed in the VISI framework. The message has a "header" that includes details of the involved roles, the transaction, and the work to which the transaction relates. The body of the message contains the information to be transferred consisting of completed data elements, attached documents and references to documents.

B.1.2 Message Category (Berichtcategorie)

Set of messages that can be classified in a certain category in the context of VISI based on corresponding characteristics, for example contractual basis, contractual change, operational, financial, escalation or quality.

B.1.3 Security Certificate (Beveiligingscertificaat)

A security certificate is intended to authenticate the data exchanged and ensure privacy. When communication takes place between server and user, this ensures that the data cannot be eavesdropped or falsified. The protocol is referred to as 'SSL' (Secure Sockets Layer). By means of cryptography and authentication, this protocol provides a secure connection to the Internet.

B.1.4 Complex (Data) Element (Complex (gegevens)element)

Type of data element used in the VISI framework to group (simple) data elements.

B.1.5 DEMO-method

Method for modelling business processes. DEMO stands for 'Dynamic Essential Modelling of Organisations'. The method was developed by prof.dr.ir. J.L.G. Dietz of Delft University of Technology. [see "Introductie tot DEMO", 1996, Dietz, J.L.G., Samson, Alphen aan den Rijn; ISBN 9789014053271]

B.1.6 Executor

Role performing a transaction.

B.1.7 Generic Framework (Generiek raamwerk)

A generic framework is managed and published by CROW. A generic framework is made for a specific application area, but can be tailored to a specific situation.

B.1.8 Initiator

Role that initiates a transaction.

B.1.9 Interaction Diagram (Interactiekaart)

See B.1.29.

B.1.10 Project Specific Framework (Projectspecifiek raamwerk)

Framework derived from a generic framework, which means that:

- the project-specific framework is first a subset of the generic framework;
- the project-specific framework can then receive adjustments and additions, if necessary, to meet the project-specific requirements.

B.1.11 Promotor

A computer program that translates a VISI framework (according to W3C technology) into a schedule of messages belonging to that framework. The Promoter is made available by CROW.

B.1.12 New promotor (#55)

Document rules that the promotor executes to create an XSD XSD is an easy standard (but not complete) XSD doesn't return clear errors Performance is not good with XSD from the promotor A central message validator

By properly handling error reporting, an external validator becomes redundant.

Promoter write out rules.

B.1.13 Framework validator (#10)

Has now been written by TEEC2

B.1.14 REST (#148)

Change communication to REST Confirmation will be a direct response to a message.

Er should be a REST service that is able to receive messages from software that does not fully implement a VISI framework.

B.1.15 TestServer

Investigate how we are going to conduct tests with software suppliers.

B.1.16 Framework (Raamwerk)

Is used in this context in the sense of 'VISI framework'; see B.1.32.

B.1.17 Role Description (Rolbeschrijving)

Description of the nature and responsibility per role, as defined in the VISI framework.

B.1.18 Role (Rol)

Participant in a transaction in the capacity of originator or executor.

B.1.19 Assigning Roles (Rolverdeling)

Allocation of (VISI) roles in a partnership of parties.

B.1.20 Cooperation Concept (Samenwerkingsconcept)

A composition of construction organisation model, contract form, tender form and project information provision to be agreed between (market) parties, with the aim of a project organisation and process handling in which risks and liability distribution are shared in an economically and socially responsible manner.

B.1.21 Simple (Data) Element (Simpel (gegevens)element)

Type of data element that forms the basis of the VISI framework containing the properties of a message. Simple data elements can be combined into complex data elements.

B.1.22 SSL

Secure Sockets Layer; see B.1.3

B.1.23 Transaction (Transactie)

Binding agreement between two roles that agree on what needs to be done. Each transaction has an initiator and an executor and goes through different transaction states.

B.1.24 Transaction Model (Transactieschema)

The overview of messages that can be sent in the context of a specific transaction. The transaction schedule naturally contains the name, description and desired outcome of the transaction.

B.1.25 Transaction State (Transactietoestand)

Stage in which a transaction is, recorded in a transaction schedule.

B.1.26 VISI

Original abbreviation of 'Creating conditions for the introduction of ICT standardisation in the construction industry' ('Voorwaarden scheppen voor de Invoering van Standaardisatie ICT in de bouw'). The term has become the acronym for an agreement system accepted by the Dutch construction sector for the digital exchange of formal communication.

B.1.27 VISI Communication (VISI-communicatie)

The digital exchange of information according to the VISI standard.

B.1.28 VISI-compatible software

Software from a VISI supplier that complies with the VISI System Guideline as published by CROW, and which has been provided with the VISI quality mark by CROW.

B.1.29 VISI interaction diagram (VISI-interactiekaart)

The schematic representation of a framework, intended to map the relevant roles and transactions for an existing process.

B.1.30 (VISI-certificate (VISI-keurmerk)

The quality mark assigned by CROW to the VISI-compatible software, including the associated (unique) registration number; the VISI quality mark guarantees that the relevant software product has been tested with a positive result for the VISI compatibility feature.

B.1.31 VISI Supplier (VISI-leverancier)

A supplier of VISI-compatible software.

B.1.32 VISI-Framework (VISI-raamwerk)

The whole of descriptions of roles, transactions, messages and data, recorded in accordance with the VISI system, which contain the agreements and/or communication that can and may be used in a project.

B.1.33 VISI-standard (VISI-standaard)

The open standard that comprises the whole of the VISI system and the published requirements and guidelines, as collected in the VISI System Guide.

B.1.34 VISI-system (VISI-systematiek)

The specification of the language used to describe VISI frameworks and create VISI messages. The VISI system consists of two parts: VISI frameworks and VISI messages.

B.1.35 VISI-website

<https://www.digigo.nu/standaarden/visi/>

B.1.36 W3C

World Wide Web Consortium (W3C) that provides standard formats for data exchange (such as XML, XSD and XSLT), which VISI uses.

B.1.37 XML

eXtensible Mark-up Language. The W3C standard file format for data exchange. XML files are readable text files with an ".xml" extension. VISI frameworks and messages are captured in XML. A good introduction and interactive course on XML can be found on the W3C website (www.w3schools.com).

B.1.38 XSD

eXtensible Schema Definition language. The W3C standard schema format for data exchange. XSD files are readable text files with an ".xsd" extension. XSD defines allowed building blocks of an XML file and can also define which values elements may have. XML files that comply with a certain XSD are 'valid'. The VISI system is recorded by means of an XSD file.

C References

C.1 Informative References

[uml]

OMG Unified Modeling Language. Open Management Group. OMG. 1 March 2015. Normative.
URL: <http://www.omg.org/spec/UML/>

[xml]

Extensible Markup Language (XML) 1.0 (Fifth Edition). Tim Bray; Jean Paoli; Michael Sperberg-McQueen; Eve Maler; François Yergeau et al. W3C. 26 November 2008. W3C Recommendation.
URL: <https://www.w3.org/TR/xml/>

C.2 FAQ

Below are some of the most important, frequently asked questions ('frequently asked questions') for the implementation of VISI. The complete (and most recent) list of FAQs can be consulted on the VISI website.

C.2.1 What is the first MessageType in a TransactionType?

The first MessageType is the MessageType that has no previous message within a TransactionType.

C.2.2 What is the last MessageType in a TransactionType?

The last MessageType within a transactionType is the MessageType that is not referenced by other MessageTypes.

C.2.3 If a message has been sent in a transaction, is it allowed to send another message?

Within a transaction, there is always one role to act. It is possible to send messages from another transaction (even if transactions are related to each other). So if a message is sent in a secondary transaction but there is also a MITT in the main transaction that may be sent based on a previous MITT, then this is simply possible.

C.2.4 How is the DATETIME data type handled when no time zone is specified?

The DATETIME data type refers to the "xsd:dateTime" definition (Instant of time (Gregorian calendar)). However, we apply one deviation to this: If no time zone is specified, UTC (=Z or +00:00) is assumed. UTC is almost equal to Greenwich Mean Time (GMT). However, GMT is a purely astronomical time. To compensate for the difference caused by the slowing of the Earth's rotation, leap seconds must be used. The difference is never more than a second and is therefore not important for most applications.

- The DATETIME datatype describes instances identified by the combination of a date and a time. Its value space is described as a combination of date and time of day in Chapter 5.4 of ISO 8601. Its lexical space is the extended format: '[⁻]CCYY-MM-DDThh:mm:ss[Z|(+|-)hh:mm]'
- The time zone may be specified as Z (UTC) or (+|-)hh:mm. Time zones that aren't specified are considered undetermined.
- Example: Valid values for xsd:dateTime include: 2001-10-26T21:32:52, 2001-10-26T21:32:52+02:00, 2001-10-26T19:32:52Z, 2001-10-26T19:32:52+00:00, -2001-10-26T21:32:52, or 2001-10-26T21:32:52.12679.

The following values are invalid: 2001-10-26 (all the parts must be specified), 2001-10-26T21:32 (all the parts must be specified), 2001-10-26T25:32:52+02:00 (the hours part—25—is out of range), or 01-10-26T21:32 (all the parts must be specified).

C.2.5 If I have promoted a framework and generated a message schema, do I have a guarantee that this framework will work in practice?

Promoting a framework only guarantees that the framework complies with the systematics and can be read in a VISI compatible software. This does not guarantee that a framework is usable in practice. For example, if the framework builder has not added a complex element to a message, this may not result in a desired situation in practice. This does not alter the fact that the framework is valid according to the systematics.

C.2.6 If I relate transactions to each other, is the traceability of communication always guaranteed?

As long as the VISI philosophy is followed, in which transactions always take place between two roles (initiator and executor) and the initiator always receives a message back from the executor, the communication remains traceable. In exceptional situations, however, it is possible to set up frameworks in such a way that traceability can be compromised. To prevent such situations, an example of such an exception is given below and then it is illustrated how it should be according to the VISI philosophy. If more of these types of situations are reported, they will be added to this FAQ.

The VISI systematics assumes that a transaction always takes place between two roles. When setting up a framework in which communication takes place over multiple transactions, this must be taken into account. In addition, a process that takes place over three transactions, for example, should also take place over the same transactions (important to keep communication traceable at a later stage). The example below is wrong, it is not wise to set up a framework in such a way.

The setup below is correct, the communication remains traceable.

When interpreting the diagrams, please note the following. The circles at the beginning and end indicate the beginning and end of a transaction respectively. The blocks are roles and the letters in the blocks represent a specific role. The codes in the arrows represent MessageTypes. For clarity, the transaction to which the MessageType belongs is also included in the code. M31T3 therefore stands for MessageType 31 from transaction 3.

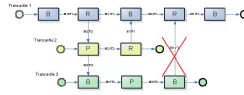


Figure 117:

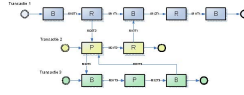


Figure 118:

C.2.7 7. How do I indicate in a framework whether a message is a start message or not?

Since VISI Standard version 1.2, a transaction can be linked to another transaction in a framework in a cascade form. This link takes place at message level. The start message of a subtransaction then has the message from the main transaction as a previous message (=‘previous’). However, the above definition conflicts with the unwritten rule that a message without a previous message (=‘previous’) is a start message of a transaction. Which message is the start message of a subtransaction is not clearly defined and cannot be determined for certain frameworks.

In practice, this has shown to lead to unclear situations, for example, if a transaction is a subtransaction of more than 1 transaction and can return to 2 or more main transactions at a single point. The green-colored transaction in figure 6 below meets these conditions.

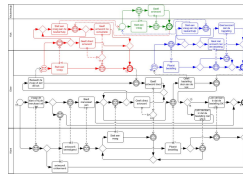


Figure 119:

If in the figure above the communication has gone from T1 (black) via T2 (red) to T4 (green), and the last message from T4 (green) has been sent, there is the choice to go back to T2 (red) or to go forward and start a new subtransaction T3 (blue). In this example it is visually clear to a human that this is not the intention in this case, but this is not possible for software.

In order to make this clear to software, an addition must be made to the framework to indicate whether certain messages are or are not a start message. The framework must then state that T3 (blue) can only be started from T1 (black) and NOT from T4 (green).

To this end, the ‘References’ of the entity ‘MessageInTransactionType’ in the VISI Systematics Part 1 (formerly Systematics I) have been expanded with the property ‘firstMessage’. This can be used to indicate whether a MessageInTransaction is a start message.

```

1 ENTITY MessageInTransactionType;
2   requiredNotify : INTEGER;
3   dateLamu : DATETIME;
4   userLamu : STRING;
5   received : BOOLEAN;
6   send : BOOLEAN;
7   state : STRING;
8   initiatorToExecutor : OPTIONAL BOOLEAN;
9   openSecondaryTransactionsAllowed : OPTIONAL BOOLEAN;
10  firstMessage : OPTIONAL BOOLEAN; // <—
11  message : MessageType;
12  previous : OPTIONAL SET [0:?] OF MessageInTransactionType;
13  transaction : TransactionType;

```

```
14     VISI Pagina 46 van 46
15     transactionPhase : OPTIONAL TransactionPhaseType;
16     group : GroupType;
17 END_ENTITY;
```

This new property is an optional Boolean ('True' or 'False'). If this Boolean is absent, the default value 'False' applies and a new transaction CANNOT be started with the relevant MessageInTransactionType.

If this Boolean is present and has the value 'True', a new transaction can be started with the MessageInTransactionType. In the example above, this Boolean must have the value 'True' for the start messages of T1, T2, T3 and T4.

If this is accidentally not set for T1, T1 may still be started, because the start message of T1 has no 'previous'. This condition always prevails over the Boolean.